

NSI

Terminale

Édition de travail 2026-2027

ÉDITION PROFESSEUR

01001110 01010011 01001001
01001110 01000101 01011000
01010101 01010011 00100001

NSI

Terminale

ÉDITION PROFESSEUR

Édition 2026-2027



NEXUS ♦ RÉUSSITE

Sommaire

1	1	Algorithmes sur les arbres et les graphes	1
	1.1	Taille, hauteur et parcours d'un arbre binaire	1
	1.2	Arbre binaire de recherche	2
	1.3	Parcours d'un graphe	3
	1.4	Cycle et chemin dans un graphe	4
	1.5	Diviser pour régner	5
	1.6	Programmation dynamique	6
	1.7	Recherche d'un motif : l'algorithme de Boyer-Moore	7
	1.8	Taille, hauteur et parcours d'un arbre binaire	8
	1.9	Arbre binaire de recherche	10
	1.10	Parcours d'un graphe	11
	1.11	Cycle et chemin dans un graphe	12
	1.12	Diviser pour régner	13
	1.13	Programmation dynamique	14
	1.14	Recherche d'un motif : l'algorithme de Boyer-Moore	15
	1.15	Taille, hauteur et parcours d'un arbre binaire	16
	1.16	Arbre binaire de recherche	18
	1.17	Parcours d'un graphe	19
	1.18	Cycle et chemin dans un graphe	20
	1.19	Diviser pour régner	21
	1.20	Programmation dynamique	22
2	2	Architectures matérielles, systèmes d'exploitation et réseaux	55
	2.1	Le système sur puce	55
	2.2	Processus : création, ordonnancement, interblocage	56
	2.3	Protocoles de routage	57
	2.4	Chiffrement symétrique et asymétrique	58
	2.5	Le système sur puce	59
	2.6	Processus : création, ordonnancement, interblocage	59
	2.7	Protocoles de routage	61
	2.8	Chiffrement symétrique et asymétrique	61
	2.9	Le système sur puce	62
	2.10	Processus : création, ordonnancement, interblocage	63
	2.11	Protocoles de routage	64

3	3	Bases de données relationnelles	85
	3.1	Le modèle relationnel	85
	3.2	Le système de gestion de bases de données	86
	3.3	Interroger une base : SELECT, FROM, WHERE	87
	3.4	Croiser les données : JOIN et ORDER BY	87
	3.5	Modifier une base : INSERT, UPDATE, DELETE	88
	3.6	Le modèle relationnel	89
	3.7	Le système de gestion de bases de données	90
	3.8	Interroger une base : SELECT, FROM, WHERE	91
	3.9	Croiser les données : JOIN et ORDER BY	92
	3.10	Modifier une base : INSERT, UPDATE, DELETE	92
	3.11	Le modèle relationnel	93
	3.12	Le système de gestion de bases de données	94
	3.13	Interroger une base : SELECT, FROM, WHERE	95
	3.14	Croiser les données : JOIN et ORDER BY	96
	3.15	Modifier une base : INSERT, UPDATE, DELETE	96
	3.16	Le modèle relationnel	97
	3.17	Le système de gestion de bases de données	98
4	4	Histoire de l'informatique	129
	4.1	Événements clés de l'histoire de l'informatique	129
	4.2	Évolution des rôles du logiciel et du matériel	129
	4.3	Événements clés de l'histoire de l'informatique	130
	4.4	Évolution des rôles du logiciel et du matériel	131
	4.5	Événements clés de l'histoire de l'informatique	139
	4.6	Évolution des rôles du logiciel et du matériel	140
5	5	Langages, récursivité et paradigmes de programmation	145
	5.1	Programme, donnée et calculabilité	145
	5.2	Écrire et analyser un programme récursif	146
	5.3	API, bibliothèques et modules	147
	5.4	Paradigmes de programmation	148
	5.5	Mise au point : causes typiques de bugs	149
	5.6	Programme, donnée et calculabilité	150
	5.7	Écrire et analyser un programme récursif	151
	5.8	API, bibliothèques et modules	152
	5.9	Paradigmes de programmation	153
	5.10	Mise au point : causes typiques de bugs	154
	5.11	Programme, donnée et calculabilité	155

5.12	Écrire et analyser un programme récursif	156
5.13	API, bibliothèques et modules	157
5.14	Paradigmes de programmation	158
5.15	Mise au point : causes typiques de bugs	159
5.16	Programme, donnée et calculabilité	160

6**6****Structures de données****195**

6.1	Interface et implémentation d'une structure de données	195
6.2	Définir et utiliser une classe en Python	196
6.3	Piles, files, et choix de structure adaptée	197
6.4	Arbres binaires	198
6.5	Graphes : modélisation et représentations	199
6.6	Interface et implémentation d'une structure de données	200
6.7	Définir et utiliser une classe en Python	201
6.8	Piles, files, et choix de structure adaptée	202
6.9	Arbres binaires	202
6.10	Graphes : modélisation et représentations	203
6.11	Interface et implémentation d'une structure de données	204
6.12	Définir et utiliser une classe en Python	206
6.13	Piles, files, et choix de structure adaptée	206
6.14	Arbres binaires	207
6.15	Graphes : modélisation et représentations	208
6.16	Interface et implémentation d'une structure de données	209
6.17	Définir et utiliser une classe en Python	210
6.18	Piles, files, et choix de structure adaptée	211
6.19	Arbres binaires	212

7**7****Conduire un projet informatique****249**

7.1	Conduire un projet informatique en Terminale	249
-----	--	-----

1 Algorithmes sur les arbres et les graphes

1.1 Taille, hauteur et parcours d'un arbre binaire

On représente un arbre binaire par des nœuds chaînés :

```
1 class Noeud:
2     def __init__(self, valeur, gauche=None, droite=None):
3         self.valeur = valeur
4         self.gauche = gauche
5         self.droite = droite
```

Un arbre vide est représenté par None.

DÉFINITION 1

La **taille** d'un arbre est son nombre de nœuds. La **hauteur** d'un arbre est la longueur du plus long chemin de la racine à une feuille : par convention, la hauteur de l'arbre vide est -1 , et la hauteur d'une feuille (nœud sans enfant) est 0 .

Exemple.

```
1 def taille(arbre):
2     if arbre is None:
3         return 0
4     return 1 + taille(arbre.gauche) + taille(arbre.droite)
5
6 def hauteur(arbre):
7     if arbre is None:
8         return -1
9     return 1 + max(hauteur(arbre.gauche), hauteur(arbre.droite))
```

◆ **PROPRIÉTÉ Parcours en profondeur** Un parcours **en profondeur** visite récursivement les sous-arbres avant de « remonter ». Trois ordres sont usuels selon la position de la racine :

- ▶ **préfixe** : racine, puis sous-arbre gauche, puis sous-arbre droit ;
- ▶ **infixe** : sous-arbre gauche, puis racine, puis sous-arbre droit ;
- ▶ **suffixe** : sous-arbre gauche, puis sous-arbre droit, puis racine.

Exemple.

```
1 def infixe(arbre):
2     if arbre is None:
3         return []
4     return infixe(arbre.gauche) + [arbre.valeur] + infixe(arbre.droite)
```

DÉFINITION 2

Un parcours **en largeur** (*Breadth-First Search*, BFS) visite les nœuds **niveau par niveau**, de la racine vers les feuilles. Il s'implémente avec une **file** : on enfile la racine, puis, tant que la file n'est pas vide, on défile un nœud, on le traite, et on enfile ses enfants non vides.

```

1 def largeur(arbre):
2     if arbre is None:
3         return []
4     resultat = []
5     file = [arbre]
6     while file:
7         noeud = file.pop(0)
8         resultat.append(noeud.valeur)
9         if noeud.gauche is not None:
10            file.append(noeud.gauche)
11            if noeud.droite is not None:
12                file.append(noeud.droite)
13    return resultat

```

⚠ ERREUR FRÉQUENTE

Confondre parcours en profondeur et en largeur. Un parcours en profondeur (préfixe/infixe/suffixe) descend au maximum dans un sous-arbre avant de passer au suivant ; un parcours en largeur traite **tous** les nœuds d'un niveau avant de passer au niveau suivant. Sur l'arbre Noeud(1, Noeud(2, Noeud(4), Noeud(5)), Noeud(3)), le parcours préfixe donne [1, 2, 4, 5, 3] alors que le parcours en largeur donne [1, 2, 3, 4, 5].

1.2 Arbre binaire de recherche

DÉFINITION 1

Un **arbre binaire de recherche** (ABR) est un arbre binaire dans lequel, pour chaque nœud, toutes les valeurs du sous-arbre gauche sont **inférieures** à la valeur du nœud, et toutes celles du sous-arbre droit lui sont **supérieures**.

Exemple. Rechercher une clé dans un ABR : on compare la clé cherchée à la racine, et on descend à gauche ou à droite selon le résultat.

```

1 class Noeud:
2     def __init__(self, valeur, gauche=None, droite=None):
3         self.valeur = valeur
4         self.gauche = gauche
5         self.droite = droite
6
7 def rechercher(arbre, cle):
8     if arbre is None:
9         return False
10    if cle == arbre.valeur:
11        return True
12    if cle < arbre.valeur:
13        return rechercher(arbre.gauche, cle)
14    return rechercher(arbre.droite, cle)

```

◆ **PROPRIÉTÉ Coût de la recherche** À chaque comparaison, la recherche élimine tout un sous-arbre : le coût de la recherche est proportionnel à la **hauteur** de l'arbre. Pour un arbre équilibré à n nœuds, cette hauteur est de l'ordre de $\log_2 n$: la recherche est très rapide. Pour un arbre dégénéré (ressemblant à une liste chaînée), elle peut coûter jusqu'à n comparaisons.

DÉFINITION 2

Insérer une clé dans un ABR consiste à descendre dans l'arbre comme pour une recherche, jusqu'à atteindre un sous-arbre vide, et à y placer la nouvelle clé sous forme de feuille : cela préserve la propriété d'ABR.

```
1 def inserer(arbre, cle):
2     if arbre is None:
3         return Noeud(cle)
4     if cle < arbre.valeur:
5         arbre.gauche = inserer(arbre.gauche, cle)
6     elif cle > arbre.valeur:
7         arbre.droite = inserer(arbre.droite, cle)
8     return arbre
```

⚠ ERREUR FRÉQUENTE

Oublier de réaffecter le résultat de inserer. Écrire simplement `inserer(arbre.gauche, cle)` sans faire `arbre.gauche = inserer(arbre.gauche, cle)` ne modifie rien lorsque `arbre.gauche` vaut `None` : un nouveau nœud est bien créé et renvoyé par la fonction, mais il n'est jamais rattaché à l'arbre existant.

1.3 Parcours d'un graphe

On représente un graphe par un dictionnaire associant à chaque sommet la liste de ses successeurs (voir le chapitre *Structures de données*).

DÉFINITION 1

Le parcours en **largeur d'abord** (*Breadth-First Search*, BFS) d'un graphe visite les sommets par distance croissante au sommet de départ, en utilisant une **file**. Contrairement à un arbre, un graphe peut contenir des **cycles** : il faut donc mémoriser les sommets déjà visités pour ne pas boucler indéfiniment.

Exemple.

```
1 def bfs(graphe, depart):
2     visites = [depart]
3     file = [depart]
4     while file:
5         sommet = file.pop(0)
6         for voisin in graphe[sommet]:
7             if voisin not in visites:
8                 visites.append(voisin)
9                 file.append(voisin)
```

```
10 return visites
```

DÉFINITION 2

Le parcours en **profondeur d'abord** (*Depth-First Search*, DFS) explore chaque branche aussi loin que possible avant de revenir en arrière (*backtrack*). Il s'implémente naturellement par **réursion** (la pile d'appels joue le rôle de la pile), ou explicitement avec une **pile**.

Exemple. Version réursive :

```
1 def dfs(graphe, sommet, visites=None):
2     if visites is None:
3         visites = []
4         visites.append(sommet)
5     for voisin in graphe[sommet]:
6         if voisin not in visites:
7             dfs(graphe, voisin, visites)
8     return visites
```

⚠ ERREUR FRÉQUENTE

Argument par défaut mutable. Écrire `def dfs(graphe, sommet, visites=[])` : au lieu de `visites=None` est un piège classique en Python : la liste par défaut est créée **une seule fois**, à la définition de la fonction, et **partagée** entre tous les appels qui ne fournissent pas explicitement `visites`. Un deuxième appel `dfs(graphe, 0)` repartirait alors avec les sommets déjà visités lors du premier appel.

◆ **PROPRIÉTÉ BFS et DFS : deux usages différents** Le BFS trouve le **plus court chemin** en nombre d'arêtes dans un graphe non pondéré (il visite par distance croissante). Le DFS est souvent plus simple à écrire et suffit pour explorer entièrement un graphe ou détecter sa connexité, mais ne garantit pas de trouver le chemin le plus court.

1.4 Cycle et chemin dans un graphe

DÉFINITION 1

Un **cycle** est un chemin qui part d'un sommet et y revient sans repasser deux fois par la même arête. Pour un graphe **non orienté**, on détecte un cycle lors d'un parcours (BFS ou DFS) si l'on rencontre un sommet déjà visité qui n'est **pas** le sommet dont on vient directement (son parent dans le parcours).

Exemple.

```
1 def a_un_cycle(graphe, sommet, visites, parent):
2     visites.append(sommet)
3     for voisin in graphe[sommet]:
4         if voisin not in visites:
5             if a_un_cycle(graphe, voisin, visites, sommet):
6                 return True
7     elif voisin != parent:
```

```

8     return True
9     return False

```

DÉFINITION 2

Chercher un chemin entre deux sommets A et B revient à effectuer un parcours (BFS ou DFS) depuis A en mémorisant, pour chaque sommet visité, le sommet depuis lequel il a été atteint ; on remonte ensuite cette chaîne depuis B pour reconstituer le chemin. **L'absence de chemin** doit être explicitement traitée : si B n'est jamais atteint, aucun chemin n'existe.

```

1 def chemin(graphe, depart, arrivee):
2     predecesseur = {depart: None}
3     file = [depart]
4     while file:
5         sommet = file.pop(0)
6         if sommet == arrivee:
7             break
8         for voisin in graphe[sommet]:
9             if voisin not in predecesseur:
10                predecesseur[voisin] = sommet
11                file.append(voisin)
12     if arrivee not in predecesseur:
13         return None
14     chemin_trouve = []
15     sommet = arrivee
16     while sommet is not None:
17         chemin_trouve.append(sommet)
18         sommet = predecesseur[sommet]
19     chemin_trouve.reverse()
20     return chemin_trouve

```

⚠ ERREUR FRÉQUENTE

Ne pas traiter le cas où aucun chemin n'existe. Si l'on oublie de tester `if arrivee not in predecesseur`, la reconstruction du chemin lève une `KeyError` lorsque le sommet d'arrivée n'a jamais été atteint (graphe non connexe). Un algorithme de recherche de chemin correct doit toujours prévoir ce cas et renvoyer une valeur explicite (ici `None`) plutôt que de planter.

1.5 Diviser pour régner

DÉFINITION 1

La méthode **diviser pour régner** résout un problème en trois étapes : **diviser** l'instance en sous-instances plus petites de même nature, **régner** en résolvant récursivement chaque sous-instance, puis **combinaison** des solutions partielles pour obtenir la solution du problème initial.

Exemple.

```

1 def fusionner(gauche, droite):
2     resultat = []
3     i, j = 0, 0
4     while i < len(gauche) and j < len(droite):

```



```
5     if gauche[i] <= droite[j]:
6         resultat.append(gauche[i])
7         i += 1
8     else:
9         resultat.append(droite[j])
10        j += 1
11    return resultat + gauche[i:] + droite[j:]
12
13 def tri_fusion(liste):
14     if len(liste) <= 1:
15         return liste
16     milieu = len(liste) // 2
17     gauche = tri_fusion(liste[:milieu])
18     droite = tri_fusion(liste[milieu:])
19     return fusionner(gauche, droite)
```

◆ **PROPRIÉTÉ Coût du tri fusion** À chaque niveau de récursion, la liste est divisée en deux, ce qui donne $\log_2 n$ niveaux ; à chaque niveau, la fusion de toutes les sous-listes coûte n comparaisons au total. Le coût total du tri fusion est donc de l'ordre de $n \log_2 n$, bien meilleur que les n^2 comparaisons d'un tri par sélection ou par insertion sur de grandes listes.

⚠ ERREUR FRÉQUENTE

Oublier le cas de base. Sans le test `if len(liste) <= 1: return liste`, la récursion ne s'arrête jamais : diviser une liste vide donnerait deux listes vides, et l'appel récursif se répéterait indéfiniment (ou lèverait une erreur de profondeur de récursion maximale atteinte).

+ POUR ALLER PLUS LOIN

La **rotation d'image** est un autre exemple classique de diviser pour régner : on découpe l'image en quadrants, on fait pivoter chaque quadrant récursivement, puis on réassemble le résultat en permutant les quadrants entre eux.

1.6 Programmation dynamique

DÉFINITION 1

La **programmation dynamique** résout un problème en le décomposant en sous-problèmes, comme diviser pour régner, mais s'applique lorsque ces sous-problèmes se **recoupent** (les mêmes sous-problèmes réapparaissent plusieurs fois). On **mémore** (*mémoïsation*) le résultat de chaque sous-problème déjà résolu pour ne jamais le recalculer.

Exemple. Version naïve, sans mémoïsation : recalcule plusieurs fois les mêmes sous-sommes.

```
1 def rendu_naif(pieces, somme):
2     if somme == 0:
3         return 0
4     meilleur = None
5     for p in pieces:
6         if p <= somme:
7             candidat = 1 + rendu_naif(pieces, somme - p)
```

```

8         if meilleur is None or candidat < meilleur:
9             meilleur = candidat
10    return meilleur

```

◆ **PROPRIÉTÉ Mémoïsation** On stocke chaque résultat déjà calculé dans un **dictionnaire** (somme → nombre minimal de pièces) : avant de calculer, on vérifie si le résultat est déjà connu ; sinon, on le calcule puis on l'enregistre avant de le renvoyer.

```

1 def rendu_memo(pieces, somme, memo=None):
2     if memo is None:
3         memo = {0: 0}
4     if somme in memo:
5         return memo[somme]
6     meilleur = None
7     for p in pieces:
8         if p <= somme:
9             candidat = 1 + rendu_memo(pieces, somme - p, memo)
10            if meilleur is None or candidat < meilleur:
11                meilleur = candidat
12    memo[somme] = meilleur
13    return meilleur

```

⚠ ERREUR FRÉQUENTE

Argument memo par défaut mutable partagé entre appels. Comme pour un graphe, écrire `memo={0: 0}` directement dans la signature créerait un dictionnaire partagé et corrompu entre appels indépendants ; on utilise `memo=None` puis on l'initialise à l'intérieur de la fonction.

◆ **PROPRIÉTÉ Gain apporté par la mémoïsation** Sans mémoïsation, le nombre d'appels récurifs croît de façon **exponentielle** avec la somme à rendre (les mêmes sous-sommes sont recalculées un grand nombre de fois). Avec mémoïsation, chaque sous-somme n'est calculée **qu'une seule fois** : le coût devient proportionnel au nombre de sous-problèmes distincts (ici, la valeur de la somme), au prix d'un espace mémoire supplémentaire pour stocker le dictionnaire.

1.7 Recherche d'un motif : l'algorithme de Boyer-Moore

DÉFINITION 1

Rechercher un **motif** (chaîne courte) dans un **texte** (chaîne longue) consiste à trouver la ou les positions où le motif apparaît dans le texte. La méthode **naïve** teste, pour chaque position du texte, si le motif y correspond caractère par caractère.

```

1 def recherche_naive(texte, motif):
2     positions = []
3     for i in range(len(texte) - len(motif) + 1):
4         if texte[i:i + len(motif)] == motif:
5             positions.append(i)
6     return positions

```

◆ **PROPRIÉTÉ Principe de Boyer-Moore** L'algorithme de Boyer-Moore compare le motif au texte de **droite à gauche**, et se déplace en s'appuyant sur un **prétraitement** du motif : lorsqu'un caractère du texte ne correspond pas, on utilise la position de ce caractère dans le motif (règle du « mauvais caractère ») pour décaler le motif directement à la position suivante où une correspondance est encore possible, au lieu d'avancer d'une seule case comme la méthode naïve.

Exemple. Règle simplifiée du mauvais caractère (décalage fondé sur la dernière occurrence de chaque caractère dans le motif) :

```

1 def table_decalage(motif):
2     table = {}
3     for i, c in enumerate(motif):
4         table[c] = i
5     return table
6
7 def boyer_moore_simplifie(texte, motif):
8     table = table_decalage(motif)
9     n, m = len(texte), len(motif)
10    positions = []
11    i = 0
12    while i <= n - m:
13        j = m - 1
14        while j >= 0 and texte[i + j] == motif[j]:
15            j -= 1
16        if j < 0:
17            positions.append(i)
18            i += 1
19        else:
20            decalage_c = j - table.get(texte[i + j], -1)
21            i += max(1, decalage_c)
22    return positions

```

⚠ ERREUR FRÉQUENTE

Croire que Boyer-Moore compare toujours de gauche à droite comme la méthode naïve. C'est précisément l'inverse : c'est en comparant le motif au texte **de droite à gauche**, tout en avançant le motif de gauche à droite le long du texte, que l'algorithme peut exploiter un désaccord précoce (sur le dernier caractère comparé) pour décaler le motif de plusieurs positions d'un coup.

+ POUR ALLER PLUS LOIN

Le programme ne demande pas l'étude formelle du coût de Boyer-Moore, qui est délicate (le meilleur cas est sous-linéaire, mais certaines variantes du pire cas restent quadratiques sans une règle de décalage supplémentaire, dite du « bon suffixe », non traitée ici).

1.8 Taille, hauteur et parcours d'un arbre binaire

On représente un arbre binaire par des nœuds chaînés :

```

1 class Noeud:
2     def __init__(self, valeur, gauche=None, droite=None):

```

```

3     self.valeur = valeur
4     self.gauche = gauche
5     self.droite = droite

```

Un arbre vide est représenté par None.

DÉFINITION 1

La **taille** d'un arbre est son nombre de nœuds. La **hauteur** d'un arbre est la longueur du plus long chemin de la racine à une feuille : par convention, la hauteur de l'arbre vide est -1 , et la hauteur d'une feuille (nœud sans enfant) est 0 .

Exemple.

```

1 def taille(arbre):
2     if arbre is None:
3         return 0
4     return 1 + taille(arbre.gauche) + taille(arbre.droite)
5
6 def hauteur(arbre):
7     if arbre is None:
8         return -1
9     return 1 + max(hauteur(arbre.gauche), hauteur(arbre.droite))

```

◆ **PROPRIÉTÉ Parcours en profondeur** Un parcours **en profondeur** visite récursivement les sous-arbres avant de « remonter ». Trois ordres sont usuels selon la position de la racine :

- ▶ **préfixe** : racine, puis sous-arbre gauche, puis sous-arbre droit ;
- ▶ **infixe** : sous-arbre gauche, puis racine, puis sous-arbre droit ;
- ▶ **suffixe** : sous-arbre gauche, puis sous-arbre droit, puis racine.

Exemple.

```

1 def infixe(arbre):
2     if arbre is None:
3         return []
4     return infixe(arbre.gauche) + [arbre.valeur] + infixe(arbre.droite)

```

DÉFINITION 2

Un parcours **en largeur** (*Breadth-First Search*, BFS) visite les nœuds **niveau par niveau**, de la racine vers les feuilles. Il s'implémente avec une **file** : on enfile la racine, puis, tant que la file n'est pas vide, on défile un nœud, on le traite, et on enfile ses enfants non vides.

```

1 def largeur(arbre):
2     if arbre is None:
3         return []
4     resultat = []
5     file = [arbre]
6     while file:
7         noeud = file.pop(0)
8         resultat.append(noeud.valeur)
9         if noeud.gauche is not None:

```

```

10     file.append(noeud.gauche)
11     if noeud.droite is not None:
12         file.append(noeud.droite)
13     return resultat

```

⚠ ERREUR FRÉQUENTE

Confondre parcours en profondeur et en largeur. Un parcours en profondeur (préfixe/infixe/suffixe) descend au maximum dans un sous-arbre avant de passer au suivant ; un parcours en largeur traite **tous** les nœuds d'un niveau avant de passer au niveau suivant. Sur l'arbre Noeud(1, Noeud(2, Noeud(4), Noeud(5)), Noeud(3)), le parcours préfixe donne [1, 2, 4, 5, 3] alors que le parcours en largeur donne [1, 2, 3, 4, 5].

1.9 Arbre binaire de recherche

DÉFINITION 1

Un **arbre binaire de recherche** (ABR) est un arbre binaire dans lequel, pour chaque nœud, toutes les valeurs du sous-arbre gauche sont **inférieures** à la valeur du nœud, et toutes celles du sous-arbre droit lui sont **supérieures**.

Exemple. Rechercher une clé dans un ABR : on compare la clé cherchée à la racine, et on descend à gauche ou à droite selon le résultat.

```

1 class Noeud:
2     def __init__(self, valeur, gauche=None, droite=None):
3         self.valeur = valeur
4         self.gauche = gauche
5         self.droite = droite
6
7     def rechercher(arbre, cle):
8         if arbre is None:
9             return False
10        if cle == arbre.valeur:
11            return True
12        if cle < arbre.valeur:
13            return rechercher(arbre.gauche, cle)
14        return rechercher(arbre.droite, cle)

```

◆ **PROPRIÉTÉ Coût de la recherche** À chaque comparaison, la recherche élimine tout un sous-arbre : le coût de la recherche est proportionnel à la **hauteur** de l'arbre. Pour un arbre équilibré à n nœuds, cette hauteur est de l'ordre de $\log_2 n$: la recherche est très rapide. Pour un arbre dégénéré (ressemblant à une liste chaînée), elle peut coûter jusqu'à n comparaisons.

DÉFINITION 2

Insérer une clé dans un ABR consiste à descendre dans l'arbre comme pour une recherche, jusqu'à atteindre un sous-arbre vide, et à y placer la nouvelle clé sous forme de feuille : cela préserve la propriété d'ABR.

```

1 def inserer(arbre, cle):
2     if arbre is None:
3         return Noeud(cle)
4     if cle < arbre.valeur:
5         arbre.gauche = inserer(arbre.gauche, cle)
6     elif cle > arbre.valeur:
7         arbre.droite = inserer(arbre.droite, cle)
8     return arbre

```

⚠ ERREUR FRÉQUENTE

Oublier de réaffecter le résultat de `inserer`. Écrire simplement `inserer(arbre.gauche, cle)` sans faire `arbre.gauche = inserer(arbre.gauche, cle)` ne modifie rien lorsque `arbre.gauche` vaut `None` : un nouveau nœud est bien créé et renvoyé par la fonction, mais il n'est jamais rattaché à l'arbre existant.

1.10 Parcours d'un graphe

On représente un graphe par un dictionnaire associant à chaque sommet la liste de ses successeurs (voir le chapitre *Structures de données*).

DÉFINITION 1

Le parcours en **largeur d'abord** (*Breadth-First Search*, BFS) d'un graphe visite les sommets par distance croissante au sommet de départ, en utilisant une **file**. Contrairement à un arbre, un graphe peut contenir des **cycles** : il faut donc mémoriser les sommets déjà visités pour ne pas boucler indéfiniment.

Exemple.

```

1 def bfs(graphe, depart):
2     visites = [depart]
3     file = [depart]
4     while file:
5         sommet = file.pop(0)
6         for voisin in graphe[sommet]:
7             if voisin not in visites:
8                 visites.append(voisin)
9                 file.append(voisin)
10    return visites

```

DÉFINITION 2

Le parcours en **profondeur d'abord** (*Depth-First Search*, DFS) explore chaque branche aussi loin que possible avant de revenir en arrière (*backtrack*). Il s'implémente naturellement par **réursion** (la pile d'appels joue le rôle de la pile), ou explicitement avec une **pile**.

Exemple. Version récursive :

```

1 def dfs(graphe, sommet, visites=None):
2     if visites is None:
3         visites = []

```

```

4     visites.append(sommet)
5     for voisin in graphe[sommet]:
6         if voisin not in visites:
7             dfs(graphe, voisin, visites)
8     return visites

```

⚠ ERREUR FRÉQUENTE

Argument par défaut mutable. Écrire `def dfs(graphe, sommet, visites=[])`: au lieu de `visites=None` est un piège classique en Python : la liste par défaut est créée **une seule fois**, à la définition de la fonction, et **partagée** entre tous les appels qui ne fournissent pas explicitement `visites`. Un deuxième appel `dfs(graphe, 0)` repartirait alors avec les sommets déjà visités lors du premier appel.

◆ **PROPRIÉTÉ BFS et DFS : deux usages différents** Le BFS trouve le **plus court chemin** en nombre d'arêtes dans un graphe non pondéré (il visite par distance croissante). Le DFS est souvent plus simple à écrire et suffit pour explorer entièrement un graphe ou détecter sa connexité, mais ne garantit pas de trouver le chemin le plus court.

1.11 Cycle et chemin dans un graphe

DÉFINITION 1

Un **cycle** est un chemin qui part d'un sommet et y revient sans repasser deux fois par la même arête. Pour un graphe **non orienté**, on détecte un cycle lors d'un parcours (BFS ou DFS) si l'on rencontre un sommet déjà visité qui n'est **pas** le sommet dont on vient directement (son parent dans le parcours).

Exemple.

```

1 def a_un_cycle(graphe, sommet, visites, parent):
2     visites.append(sommet)
3     for voisin in graphe[sommet]:
4         if voisin not in visites:
5             if a_un_cycle(graphe, voisin, visites, sommet):
6                 return True
7         elif voisin != parent:
8             return True
9     return False

```

DÉFINITION 2

Chercher un chemin entre deux sommets A et B revient à effectuer un parcours (BFS ou DFS) depuis A en mémorisant, pour chaque sommet visité, le sommet depuis lequel il a été atteint ; on remonte ensuite cette chaîne depuis B pour reconstituer le chemin. **L'absence de chemin** doit être explicitement traitée : si B n'est jamais atteint, aucun chemin n'existe.

```

1 def chemin(graphe, depart, arrivee):
2     predecesseur = {depart: None}

```

```

3 file = [depart]
4 while file:
5     sommet = file.pop(0)
6     if sommet == arrivee:
7         break
8     for voisin in graphe[sommet]:
9         if voisin not in predecesseur:
10            predecesseur[voisin] = sommet
11            file.append(voisin)
12 if arrivee not in predecesseur:
13     return None
14 chemin_trouve = []
15 sommet = arrivee
16 while sommet is not None:
17     chemin_trouve.append(sommet)
18     sommet = predecesseur[sommet]
19 chemin_trouve.reverse()
20 return chemin_trouve

```

⚠ ERREUR FRÉQUENTE

Ne pas traiter le cas où aucun chemin n'existe. Si l'on oublie de tester if arrivee not in predecesseur, la reconstruction du chemin lève une KeyError lorsque le sommet d'arrivée n'a jamais été atteint (graphe non connexe). Un algorithme de recherche de chemin correct doit toujours prévoir ce cas et renvoyer une valeur explicite (ici None) plutôt que de planter.

1.12 Diviser pour régner

DÉFINITION 1

La méthode **diviser pour régner** résout un problème en trois étapes : **diviser** l'instance en sous-instances plus petites de même nature, **régner** en résolvant récursivement chaque sous-instance, puis **combinaison** des solutions partielles pour obtenir la solution du problème initial.

Exemple.

```

1 def fusionner(gauche, droite):
2     resultat = []
3     i, j = 0, 0
4     while i < len(gauche) and j < len(droite):
5         if gauche[i] <= droite[j]:
6             resultat.append(gauche[i])
7             i += 1
8         else:
9             resultat.append(droite[j])
10            j += 1
11     return resultat + gauche[i:] + droite[j:]
12
13 def tri_fusion(liste):
14     if len(liste) <= 1:
15         return liste
16     milieu = len(liste) // 2
17     gauche = tri_fusion(liste[:milieu])
18     droite = tri_fusion(liste[milieu:])
19     return fusionner(gauche, droite)

```

◆ **PROPRIÉTÉ Coût du tri fusion** À chaque niveau de récursion, la liste est divisée en deux, ce qui donne $\log_2 n$ niveaux ; à chaque niveau, la fusion de toutes les sous-listes coûte n comparaisons au total. Le coût total du tri fusion est donc de l'ordre de $n \log_2 n$, bien meilleur que les n^2 comparaisons d'un tri par sélection ou par insertion sur de grandes listes.

⚠ ERREUR FRÉQUENTE

Oublier le cas de base. Sans le test `if len(liste) <= 1: return liste`, la récursion ne s'arrête jamais : diviser une liste vide donnerait deux listes vides, et l'appel récursif se répéterait indéfiniment (ou lèverait une erreur de profondeur de récursion maximale atteinte).

+ POUR ALLER PLUS LOIN

La **rotation d'image** est un autre exemple classique de diviser pour régner : on découpe l'image en quadrants, on fait pivoter chaque quadrant récursivement, puis on réassemble le résultat en permutant les quadrants entre eux.

1.13 Programmation dynamique

DÉFINITION 1

La **programmation dynamique** résout un problème en le décomposant en sous-problèmes, comme diviser pour régner, mais s'applique lorsque ces sous-problèmes se **recoupent** (les mêmes sous-problèmes réapparaissent plusieurs fois). On **mémorise** (*mémoïsation*) le résultat de chaque sous-problème déjà résolu pour ne jamais le recalculer.

Exemple. Version naïve, sans mémoïsation : recalcule plusieurs fois les mêmes sous-sommes.

```
1 def rendu_naif(pieces, somme):
2     if somme == 0:
3         return 0
4     meilleur = None
5     for p in pieces:
6         if p <= somme:
7             candidat = 1 + rendu_naif(pieces, somme - p)
8             if meilleur is None or candidat < meilleur:
9                 meilleur = candidat
10    return meilleur
```

◆ **PROPRIÉTÉ Mémoïsation** On stocke chaque résultat déjà calculé dans un **dictionnaire** (`somme` → nombre minimal de pièces) : avant de calculer, on vérifie si le résultat est déjà connu ; sinon, on le calcule puis on l'enregistre avant de le renvoyer.

```
1 def rendu_memo(pieces, somme, memo=None):
2     if memo is None:
3         memo = {0: 0}
4     if somme in memo:
5         return memo[somme]
6     meilleur = None
```

```

7   for p in pieces:
8       if p <= somme:
9           candidat = 1 + rendu_memo(pieces, somme - p, memo)
10          if meilleur is None or candidat < meilleur:
11              meilleur = candidat
12          memo[somme] = meilleur
13      return meilleur

```

⚠ ERREUR FRÉQUENTE

Argument memo par défaut mutable partagé entre appels. Comme pour un graphe, écrire `memo={0: 0}` directement dans la signature créerait un dictionnaire partagé et corrompu entre appels indépendants ; on utilise `memo=None` puis on l'initialise à l'intérieur de la fonction.

◆ **PROPRIÉTÉ Gain apporté par la mémorisation** Sans mémorisation, le nombre d'appels récur­sifs croît de façon **exponentielle** avec la somme à rendre (les mêmes sous-sommes sont recalculées un grand nombre de fois). Avec mémorisation, chaque sous-somme n'est calculée **qu'une seule fois** : le coût devient proportionnel au nombre de sous-problèmes distincts (ici, la valeur de la somme), au prix d'un espace mémoire supplémentaire pour stocker le dictionnaire.

1.14 Recherche d'un motif : l'algorithme de Boyer-Moore

DÉFINITION 1

Rechercher un **motif** (chaîne courte) dans un **texte** (chaîne longue) consiste à trouver la ou les positions où le motif apparaît dans le texte. La méthode **naïve** teste, pour chaque position du texte, si le motif y correspond caractère par caractère.

```

1   def recherche_naive(texte, motif):
2       positions = []
3       for i in range(len(texte) - len(motif) + 1):
4           if texte[i:i + len(motif)] == motif:
5               positions.append(i)
6       return positions

```

◆ **PROPRIÉTÉ Principe de Boyer-Moore** L'algorithme de Boyer-Moore compare le motif au texte de **droite à gauche**, et se déplace en s'appuyant sur un **prétraitement** du motif : lorsqu'un caractère du texte ne correspond pas, on utilise la position de ce caractère dans le motif (règle du « mauvais caractère ») pour décaler le motif directement à la position suivante où une correspondance est encore possible, au lieu d'avancer d'une seule case comme la méthode naïve.

Exemple. Règle simplifiée du mauvais caractère (décalage fondé sur la dernière occurrence de chaque caractère dans le motif) :

```

1   def table_decalage(motif):
2       table = {}
3       for i, c in enumerate(motif):
4           table[c] = i
5       return table

```



```
6
7 def boyer_moore_simplifie(texte, motif):
8     table = table_decalage(motif)
9     n, m = len(texte), len(motif)
10    positions = []
11    i = 0
12    while i <= n - m:
13        j = m - 1
14        while j >= 0 and texte[i + j] == motif[j]:
15            j -= 1
16        if j < 0:
17            positions.append(i)
18            i += 1
19        else:
20            decalage_c = j - table.get(texte[i + j], -1)
21            i += max(1, decalage_c)
22    return positions
```

⚠ ERREUR FRÉQUENTE

Croire que Boyer-Moore compare toujours de gauche à droite comme la méthode naïve. C'est précisément l'inverse : c'est en comparant le motif au texte **de droite à gauche**, tout en avançant le motif de gauche à droite le long du texte, que l'algorithme peut exploiter un désaccord précoce (sur le dernier caractère comparé) pour décaler le motif de plusieurs positions d'un coup.

+ POUR ALLER PLUS LOIN

Le programme ne demande pas l'étude formelle du coût de Boyer-Moore, qui est délicate (le meilleur cas est sous-linéaire, mais certaines variantes du pire cas restent quadratiques sans une règle de décalage supplémentaire, dite du « bon suffixe », non traitée ici).

1.15 Taille, hauteur et parcours d'un arbre binaire

On représente un arbre binaire par des nœuds chaînés :

```
1 class Noeud:
2     def __init__(self, valeur, gauche=None, droite=None):
3         self.valeur = valeur
4         self.gauche = gauche
5         self.droite = droite
```

Un arbre vide est représenté par None.

DÉFINITION 1

La **taille** d'un arbre est son nombre de nœuds. La **hauteur** d'un arbre est la longueur du plus long chemin de la racine à une feuille : par convention, la hauteur de l'arbre vide est -1 , et la hauteur d'une feuille (nœud sans enfant) est 0 .

Exemple.

```
1 def taille(arbre):
```

```

2     if arbre is None:
3         return 0
4     return 1 + taille(arbre.gauche) + taille(arbre.droite)
5
6 def hauteur(arbre):
7     if arbre is None:
8         return -1
9     return 1 + max(hauteur(arbre.gauche), hauteur(arbre.droite))

```

◆ **PROPRIÉTÉ Parcours en profondeur** Un parcours **en profondeur** visite récursivement les sous-arbres avant de « remonter ». Trois ordres sont usuels selon la position de la racine :

- ▶ **préfixe** : racine, puis sous-arbre gauche, puis sous-arbre droit ;
- ▶ **infixe** : sous-arbre gauche, puis racine, puis sous-arbre droit ;
- ▶ **suffixe** : sous-arbre gauche, puis sous-arbre droit, puis racine.

Exemple.

```

1 def infixe(arbre):
2     if arbre is None:
3         return []
4     return infixe(arbre.gauche) + [arbre.valeur] + infixe(arbre.droite)

```

DÉFINITION 2

Un parcours **en largeur** (*Breadth-First Search*, BFS) visite les nœuds **niveau par niveau**, de la racine vers les feuilles. Il s'implémente avec une **file** : on enfile la racine, puis, tant que la file n'est pas vide, on défile un nœud, on le traite, et on enfile ses enfants non vides.

```

1 def largeur(arbre):
2     if arbre is None:
3         return []
4     resultat = []
5     file = [arbre]
6     while file:
7         noeud = file.pop(0)
8         resultat.append(noeud.valeur)
9         if noeud.gauche is not None:
10            file.append(noeud.gauche)
11        if noeud.droite is not None:
12            file.append(noeud.droite)
13    return resultat

```

⚠ ERREUR FRÉQUENTE

Confondre parcours en profondeur et en largeur. Un parcours en profondeur (préfixe/infixe/suffixe) descend au maximum dans un sous-arbre avant de passer au suivant ; un parcours en largeur traite **tous** les nœuds d'un niveau avant de passer au niveau suivant. Sur l'arbre Noeud (1, Noeud(2, Noeud(4), Noeud(5)), Noeud(3)), le parcours préfixe donne [1, 2, 4, 5, 3] alors que le parcours en largeur donne [1, 2, 3, 4, 5].

1.16 Arbre binaire de recherche

DÉFINITION 1

Un **arbre binaire de recherche** (ABR) est un arbre binaire dans lequel, pour chaque nœud, toutes les valeurs du sous-arbre gauche sont **inférieures** à la valeur du nœud, et toutes celles du sous-arbre droit lui sont **supérieures**.

Exemple. Rechercher une clé dans un ABR : on compare la clé cherchée à la racine, et on descend à gauche ou à droite selon le résultat.

```

1 class Noeud:
2     def __init__(self, valeur, gauche=None, droite=None):
3         self.valeur = valeur
4         self.gauche = gauche
5         self.droite = droite
6
7     def rechercher(arbre, cle):
8         if arbre is None:
9             return False
10        if cle == arbre.valeur:
11            return True
12        if cle < arbre.valeur:
13            return rechercher(arbre.gauche, cle)
14        return rechercher(arbre.droite, cle)

```

◆ **PROPRIÉTÉ Coût de la recherche** À chaque comparaison, la recherche élimine tout un sous-arbre : le coût de la recherche est proportionnel à la **hauteur** de l'arbre. Pour un arbre équilibré à n nœuds, cette hauteur est de l'ordre de $\log_2 n$: la recherche est très rapide. Pour un arbre dégénéré (ressemblant à une liste chaînée), elle peut coûter jusqu'à n comparaisons.

DÉFINITION 2

Insérer une clé dans un ABR consiste à descendre dans l'arbre comme pour une recherche, jusqu'à atteindre un sous-arbre vide, et à y placer la nouvelle clé sous forme de feuille : cela préserve la propriété d'ABR.

```

1 def inserer(arbre, cle):
2     if arbre is None:
3         return Noeud(cle)
4     if cle < arbre.valeur:
5         arbre.gauche = inserer(arbre.gauche, cle)
6     elif cle > arbre.valeur:
7         arbre.droite = inserer(arbre.droite, cle)
8     return arbre

```

⚠ ERREUR FRÉQUENTE

Oublier de réaffecter le résultat de inserer. Écrire simplement `inserer(arbre.gauche, cle)` sans faire `arbre.gauche = inserer(arbre.gauche, cle)` ne modifie rien lorsque `arbre.gauche`

vaut `None` : un nouveau nœud est bien créé et renvoyé par la fonction, mais il n'est jamais rattaché à l'arbre existant.

1.17 Parcours d'un graphe

On représente un graphe par un dictionnaire associant à chaque sommet la liste de ses successeurs (voir le chapitre *Structures de données*).

DÉFINITION 1

Le parcours en **largeur d'abord** (*Breadth-First Search*, BFS) d'un graphe visite les sommets par distance croissante au sommet de départ, en utilisant une **file**. Contrairement à un arbre, un graphe peut contenir des **cycles** : il faut donc mémoriser les sommets déjà visités pour ne pas boucler indéfiniment.

Exemple.

```
1 def bfs(graphe, depart):
2     visites = [depart]
3     file = [depart]
4     while file:
5         sommet = file.pop(0)
6         for voisin in graphe[sommet]:
7             if voisin not in visites:
8                 visites.append(voisin)
9                 file.append(voisin)
10    return visites
```

DÉFINITION 2

Le parcours en **profondeur d'abord** (*Depth-First Search*, DFS) explore chaque branche aussi loin que possible avant de revenir en arrière (*backtrack*). Il s'implémente naturellement par **réursion** (la pile d'appels joue le rôle de la pile), ou explicitement avec une **pile**.

Exemple. Version récursive :

```
1 def dfs(graphe, sommet, visites=None):
2     if visites is None:
3         visites = []
4     visites.append(sommet)
5     for voisin in graphe[sommet]:
6         if voisin not in visites:
7             dfs(graphe, voisin, visites)
8     return visites
```

⚠ ERREUR FRÉQUENTE

Argument par défaut mutable. Écrire `def dfs(graphe, sommet, visites=[])` : au lieu de `visites=None` est un piège classique en Python : la liste par défaut est créée **une seule fois**, à la

définition de la fonction, et **partagée** entre tous les appels qui ne fournissent pas explicitement visites. Un deuxième appel `dfs(graphe, 0)` repartirait alors avec les sommets déjà visités lors du premier appel.

◆ **PROPRIÉTÉ BFS et DFS : deux usages différents** Le BFS trouve le **plus court chemin** en nombre d'arêtes dans un graphe non pondéré (il visite par distance croissante). Le DFS est souvent plus simple à écrire et suffit pour explorer entièrement un graphe ou détecter sa connexité, mais ne garantit pas de trouver le chemin le plus court.

1.18 Cycle et chemin dans un graphe

DÉFINITION 1

Un **cycle** est un chemin qui part d'un sommet et y revient sans repasser deux fois par la même arête. Pour un graphe **non orienté**, on détecte un cycle lors d'un parcours (BFS ou DFS) si l'on rencontre un sommet déjà visité qui n'est **pas** le sommet dont on vient directement (son parent dans le parcours).

Exemple.

```
1 def a_un_cycle(graphe, sommet, visites, parent):
2     visites.append(sommet)
3     for voisin in graphe[sommet]:
4         if voisin not in visites:
5             if a_un_cycle(graphe, voisin, visites, sommet):
6                 return True
7         elif voisin != parent:
8             return True
9     return False
```

DÉFINITION 2

Chercher un chemin entre deux sommets *A* et *B* revient à effectuer un parcours (BFS ou DFS) depuis *A* en mémorisant, pour chaque sommet visité, le sommet depuis lequel il a été atteint ; on remonte ensuite cette chaîne depuis *B* pour reconstituer le chemin. **L'absence de chemin** doit être explicitement traitée : si *B* n'est jamais atteint, aucun chemin n'existe.

```
1 def chemin(graphe, depart, arrivee):
2     predecesseur = {depart: None}
3     file = [depart]
4     while file:
5         sommet = file.pop(0)
6         if sommet == arrivee:
7             break
8         for voisin in graphe[sommet]:
9             if voisin not in predecesseur:
10                predecesseur[voisin] = sommet
11                file.append(voisin)
12     if arrivee not in predecesseur:
13         return None
```

```

14 chemin_trouve = []
15 sommet = arrivee
16 while sommet is not None:
17     chemin_trouve.append(sommet)
18     sommet = predecesseur[sommet]
19 chemin_trouve.reverse()
20 return chemin_trouve

```

⚠ ERREUR FRÉQUENTE

Ne pas traiter le cas où aucun chemin n'existe. Si l'on oublie de tester `if arrivee not in predecesseur`, la reconstruction du chemin lève une `KeyError` lorsque le sommet d'arrivée n'a jamais été atteint (graphe non connexe). Un algorithme de recherche de chemin correct doit toujours prévoir ce cas et renvoyer une valeur explicite (ici `None`) plutôt que de planter.

1.19 Diviser pour régner

DÉFINITION 1

La méthode **diviser pour régner** résout un problème en trois étapes : **diviser** l'instance en sous-instances plus petites de même nature, **régner** en résolvant récursivement chaque sous-instance, puis **combinaison** des solutions partielles pour obtenir la solution du problème initial.

Exemple.

```

1 def fusionner(gauche, droite):
2     resultat = []
3     i, j = 0, 0
4     while i < len(gauche) and j < len(droite):
5         if gauche[i] <= droite[j]:
6             resultat.append(gauche[i])
7             i += 1
8         else:
9             resultat.append(droite[j])
10            j += 1
11     return resultat + gauche[i:] + droite[j:]
12
13 def tri_fusion(liste):
14     if len(liste) <= 1:
15         return liste
16     milieu = len(liste) // 2
17     gauche = tri_fusion(liste[:milieu])
18     droite = tri_fusion(liste[milieu:])
19     return fusionner(gauche, droite)

```

◆ **PROPRIÉTÉ Coût du tri fusion** À chaque niveau de récursion, la liste est divisée en deux, ce qui donne $\log_2 n$ niveaux ; à chaque niveau, la fusion de toutes les sous-listes coûte n comparaisons au total. Le coût total du tri fusion est donc de l'ordre de $n \log_2 n$, bien meilleur que les n^2 comparaisons d'un tri par sélection ou par insertion sur de grandes listes.

⚠ ERREUR FRÉQUENTE

Oublier le cas de base. Sans le test `if len(liste) <= 1: return liste`, la récursion ne s'arrête jamais : diviser une liste vide donnerait deux listes vides, et l'appel récursif se répéterait indéfiniment (ou lèverait une erreur de profondeur de récursion maximale atteinte).

+ POUR ALLER PLUS LOIN

La **rotation d'image** est un autre exemple classique de diviser pour régner : on découpe l'image en quadrants, on fait pivoter chaque quadrant récursivement, puis on réassemble le résultat en permutant les quadrants entre eux.

1.20 Programmation dynamique

DÉFINITION 1

La **programmation dynamique** résout un problème en le décomposant en sous-problèmes, comme diviser pour régner, mais s'applique lorsque ces sous-problèmes se **recoupent** (les mêmes sous-problèmes réapparaissent plusieurs fois). On **mémore** (*mémoïsation*) le résultat de chaque sous-problème déjà résolu pour ne jamais le recalculer.

Exemple. Version naïve, sans mémoïsation : recalcule plusieurs fois les mêmes sous-sommes.

```

1 def rendu_naif(pieces, somme):
2     if somme == 0:
3         return 0
4     meilleur = None
5     for p in pieces:
6         if p <= somme:
7             candidat = 1 + rendu_naif(pieces, somme - p)
8             if meilleur is None or candidat < meilleur:
9                 meilleur = candidat
10    return meilleur

```

◆ **PROPRIÉTÉ Mémoïsation** On stocke chaque résultat déjà calculé dans un **dictionnaire** (`somme` → nombre minimal de pièces) : avant de calculer, on vérifie si le résultat est déjà connu ; sinon, on le calcule puis on l'enregistre avant de le renvoyer.

```

1 def rendu_memo(pieces, somme, memo=None):
2     if memo is None:
3         memo = {0: 0}
4     if somme in memo:
5         return memo[somme]
6     meilleur = None
7     for p in pieces:
8         if p <= somme:
9             candidat = 1 + rendu_memo(pieces, somme - p, memo)
10            if meilleur is None or candidat < meilleur:
11                meilleur = candidat
12    memo[somme] = meilleur
13    return meilleur

```

⚠ ERREUR FRÉQUENTE

Argument memo par défaut mutable partagé entre appels. Comme pour un graphe, écrire `memo={0: 0}` directement dans la signature créerait un dictionnaire partagé et corrompu entre appels indépendants ; on utilise `memo=None` puis on l'initialise à l'intérieur de la fonction.

◆ **PROPRIÉTÉ Gain apporté par la mémorisation** Sans mémorisation, le nombre d'appels récurifs croît de façon **exponentielle** avec la somme à rendre (les mêmes sous-sommes sont recalculées un grand nombre de fois). Avec mémorisation, chaque sous-somme n'est calculée **qu'une seule fois** : le coût devient proportionnel au nombre de sous-problèmes distincts (ici, la valeur de la somme), au prix d'un espace mémoire supplémentaire pour stocker le dictionnaire.

Méthodes

► Méthode directe de travail sur le sujet.

Exercice 1 ◆

Écrire une fonction récursive `compter_feuilles(arbre)` qui renvoie le nombre de **feuilles** (nœuds sans enfant) d'un arbre binaire représenté avec la classe `Noeud` du cours. Tester sur l'arbre `Noeud(1, Noeud(2, Noeud(4), Noeud(5)), Noeud(3))`, qui possède 3 feuilles.

Exercice 2 ◆

En utilisant les fonctions `inserer` et `rechercher` du cours, construire un arbre binaire de recherche à partir des valeurs 7, 3, 9, 1, 5 insérées dans cet ordre. Vérifier ensuite que 5 est présent dans l'arbre et que 4 n'y est pas.

Exercice 3 ◆◆

On modélise un petit réseau de métro par le graphe (non orienté) suivant, où chaque station est reliée à ses stations directement accessibles :

```
1 metro = {
2     "A": ["B"],
3     "B": ["A", "C", "D"],
4     "C": ["B", "E"],
5     "D": ["B", "E"],
6     "E": ["C", "D"],
7 }
```

En utilisant la fonction `chemin` du cours (recherche de chemin par BFS), déterminer le trajet trouvé de "A" à "E". Combien de stations ce trajet compte-t-il (départ et arrivée compris) ?

Exercice 4 ◆◆

La suite de Fibonacci est définie par $F_0 = 0$, $F_1 = 1$, et $F_n = F_{n-1} + F_{n-2}$ pour $n \geq 2$. Une version récursive naïve de son calcul recalcule un très grand nombre de fois les mêmes valeurs.

1. Écrire une fonction `fib_memo(n, memo=None)` qui calcule F_n par **programmation dynamique** (mémorisation), en s'inspirant de `rendu_memo` du cours.
2. Vérifier que $F_0, F_1, \dots, F_9$ vaut 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, et que $F_{30} = 832\,040$.

Exercice 5

Écrire, selon le principe **diviser pour régner** (comme le tri fusion du cours), une fonction récursive `maximum_dpr(liste)` qui renvoie le plus grand élément d'une liste non vide : diviser la liste en deux moitiés, trouver récursivement le maximum de chaque moitié, puis combiner en renvoyant le plus grand des deux. Quel est le cas de base ?

Exercice 6

Écrire une fonction récursive `compter_feuilles(arbre)` qui renvoie le nombre de **feuilles** (nœuds sans enfant) d'un arbre binaire représenté avec la classe `Noeud` du cours. Tester sur l'arbre `Noeud(1, Noeud(2, Noeud(4), Noeud(5)), Noeud(3))`, qui possède 3 feuilles.

Exercice 7

En utilisant les fonctions `insérer` et `rechercher` du cours, construire un arbre binaire de recherche à partir des valeurs 7, 3, 9, 1, 5 insérées dans cet ordre. Vérifier ensuite que 5 est présent dans l'arbre et que 4 n'y est pas.

Exercice 8

On modélise un petit réseau de métro par le graphe (non orienté) suivant, où chaque station est reliée à ses stations directement accessibles :

```
1 metro = {
2     "A": ["B"],
3     "B": ["A", "C", "D"],
4     "C": ["B", "E"],
5     "D": ["B", "E"],
6     "E": ["C", "D"],
7 }
```

En utilisant la fonction `chemin` du cours (recherche de chemin par BFS), déterminer le trajet trouvé de "A" à "E". Combien de stations ce trajet compte-t-il (départ et arrivée compris) ?

Exercice 9

La suite de Fibonacci est définie par $F_0 = 0$, $F_1 = 1$, et $F_n = F_{n-1} + F_{n-2}$ pour $n \geq 2$. Une version récursive naïve de son calcul recalcule un très grand nombre de fois les mêmes valeurs.

1. Écrire une fonction `fib_memo(n, memo=None)` qui calcule F_n par **programmation dynamique** (mémoïsation), en s'inspirant de `rendu_memo` du cours.
2. Vérifier que $F_0, F_1, \dots, F_9$ vaut 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, et que $F_{30} = 832\,040$.

Exercice 10

Écrire, selon le principe **diviser pour régner** (comme le tri fusion du cours), une fonction récursive `maximum_dpr(liste)` qui renvoie le plus grand élément d'une liste non vide : diviser la liste en deux moitiés, trouver récursivement le maximum de chaque moitié, puis combiner en renvoyant le plus grand des deux. Quel est le cas de base ?

Exercice 11

Écrire une fonction récursive `compter_feuilles(arbre)` qui renvoie le nombre de **feuilles** (nœuds sans enfant) d'un arbre binaire représenté avec la classe `Noeud` du cours. Tester sur l'arbre `Noeud(1, Noeud(2, Noeud(4), Noeud(5)), Noeud(3))`, qui possède 3 feuilles.

Exercice 12 ◆

En utilisant les fonctions `insérer` et `rechercher` du cours, construire un arbre binaire de recherche à partir des valeurs 7, 3, 9, 1, 5 insérées dans cet ordre. Vérifier ensuite que 5 est présent dans l'arbre et que 4 n'y est pas.

Exercice 13 ◆◆

On modélise un petit réseau de métro par le graphe (non orienté) suivant, où chaque station est reliée à ses stations directement accessibles :

```

1 metro = {
2     "A": ["B"],
3     "B": ["A", "C", "D"],
4     "C": ["B", "E"],
5     "D": ["B", "E"],
6     "E": ["C", "D"],
7 }
```

En utilisant la fonction `chemin` du cours (recherche de chemin par BFS), déterminer le trajet trouvé de "A" à "E". Combien de stations ce trajet compte-t-il (départ et arrivée compris) ?

Exercice 14 ◆◆

La suite de Fibonacci est définie par $F_0 = 0$, $F_1 = 1$, et $F_n = F_{n-1} + F_{n-2}$ pour $n \geq 2$. Une version récursive naïve de son calcul recalcule un très grand nombre de fois les mêmes valeurs.

1. Écrire une fonction `fib_memo(n, memo=None)` qui calcule F_n par **programmation dynamique** (mémoïsation), en s'inspirant de `rendu_memo` du cours.
2. Vérifier que $F_0, F_1, \dots, F_9$ vaut 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, et que $F_{30} = 832\,040$.

Exercice 15 ◆◆

Écrire, selon le principe **diviser pour régner** (comme le tri fusion du cours), une fonction récursive `maximum_dpr(liste)` qui renvoie le plus grand élément d'une liste non vide : diviser la liste en deux moitiés, trouver récursivement le maximum de chaque moitié, puis combiner en renvoyant le plus grand des deux. Quel est le cas de base ?

Exercice 16 ◆

Écrire une fonction récursive `compter_feuilles(arbre)` qui renvoie le nombre de **feuilles** (nœuds sans enfant) d'un arbre binaire représenté avec la classe `Noeud` du cours. Tester sur l'arbre `Noeud(1, Noeud(2, Noeud(4), Noeud(5)), Noeud(3))`, qui possède 3 feuilles.

Exercice 17 ◆

En utilisant les fonctions `insérer` et `rechercher` du cours, construire un arbre binaire de recherche à partir des valeurs 7, 3, 9, 1, 5 insérées dans cet ordre. Vérifier ensuite que 5 est présent dans l'arbre et que 4 n'y est pas.

Exercice 18 ◆◆

On modélise un petit réseau de métro par le graphe (non orienté) suivant, où chaque station est reliée à ses stations directement accessibles :

```

1 metro = {
2     "A": ["B"],
3     "B": ["A", "C", "D"],
4     "C": ["B", "E"],
```

```

5     "D": ["B", "E"],
6     "E": ["C", "D"],
7 }

```

En utilisant la fonction `chemin` du cours (recherche de chemin par BFS), déterminer le trajet trouvé de "A" à "E". Combien de stations ce trajet compte-t-il (départ et arrivée compris) ?

Exercice 19 ♦♦

La suite de Fibonacci est définie par $F_0 = 0$, $F_1 = 1$, et $F_n = F_{n-1} + F_{n-2}$ pour $n \geq 2$. Une version récursive naïve de son calcul recalcule un très grand nombre de fois les mêmes valeurs.

1. Écrire une fonction `fib_memo(n, memo=None)` qui calcule F_n par **programmation dynamique** (mémoïsation), en s'inspirant de `rendu_memo` du cours.
2. Vérifier que $F_0, F_1, \dots, F_9$ vaut 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, et que $F_{30} = 832\,040$.

Exercice 20 ♦♦

Écrire, selon le principe **diviser pour régner** (comme le tri fusion du cours), une fonction récursive `maximum_dpr(liste)` qui renvoie le plus grand élément d'une liste non vide : diviser la liste en deux moitiés, trouver récursivement le maximum de chaque moitié, puis combiner en renvoyant le plus grand des deux. Quel est le cas de base ?

Exercice 21 ♦

Écrire une fonction récursive `compter_feuilles(arbre)` qui renvoie le nombre de **feuilles** (nœuds sans enfant) d'un arbre binaire représenté avec la classe `Noeud` du cours. Tester sur l'arbre `Noeud(1, Noeud(2, Noeud(4), Noeud(5)), Noeud(3))`, qui possède 3 feuilles.

Exercice 22 ♦

En utilisant les fonctions `insérer` et `rechercher` du cours, construire un arbre binaire de recherche à partir des valeurs 7, 3, 9, 1, 5 insérées dans cet ordre. Vérifier ensuite que 5 est présent dans l'arbre et que 4 n'y est pas.

Exercice 23 ♦♦

On modélise un petit réseau de métro par le graphe (non orienté) suivant, où chaque station est reliée à ses stations directement accessibles :

```

1 metro = {
2     "A": ["B"],
3     "B": ["A", "C", "D"],
4     "C": ["B", "E"],
5     "D": ["B", "E"],
6     "E": ["C", "D"],
7 }

```

En utilisant la fonction `chemin` du cours (recherche de chemin par BFS), déterminer le trajet trouvé de "A" à "E". Combien de stations ce trajet compte-t-il (départ et arrivée compris) ?

Exercice 24 ♦♦

La suite de Fibonacci est définie par $F_0 = 0$, $F_1 = 1$, et $F_n = F_{n-1} + F_{n-2}$ pour $n \geq 2$. Une version récursive naïve de son calcul recalcule un très grand nombre de fois les mêmes valeurs.

1. Écrire une fonction `fib_memo(n, memo=None)` qui calcule F_n par **programmation dynamique** (mémoïsation), en s'inspirant de `rendu_memo` du cours.
2. Vérifier que $F_0, F_1, \dots, F_9$ vaut 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, et que $F_{30} = 832\,040$.

Exercice 25 ♦♦

Écrire, selon le principe **diviser pour régner** (comme le tri fusion du cours), une fonction récursive `maximum_dpr(liste)` qui renvoie le plus grand élément d'une liste non vide : diviser la liste en deux moitiés, trouver récursivement le maximum de chaque moitié, puis combiner en renvoyant le plus grand des deux. Quel est le cas de base ?

Exercice 26 ♦

Écrire une fonction récursive `compter_feuilles(arbre)` qui renvoie le nombre de **feuilles** (nœuds sans enfant) d'un arbre binaire représenté avec la classe `Noeud` du cours. Tester sur l'arbre `Noeud(1, Noeud(2, Noeud(4), Noeud(5)), Noeud(3))`, qui possède 3 feuilles.

Exercice 27 ♦

En utilisant les fonctions `inserer` et `rechercher` du cours, construire un arbre binaire de recherche à partir des valeurs 7, 3, 9, 1, 5 insérées dans cet ordre. Vérifier ensuite que 5 est présent dans l'arbre et que 4 n'y est pas.

Exercice 28 ♦♦

On modélise un petit réseau de métro par le graphe (non orienté) suivant, où chaque station est reliée à ses stations directement accessibles :

```

1 metro = {
2     "A": ["B"],
3     "B": ["A", "C", "D"],
4     "C": ["B", "E"],
5     "D": ["B", "E"],
6     "E": ["C", "D"],
7 }
```

En utilisant la fonction `chemin` du cours (recherche de chemin par BFS), déterminer le trajet trouvé de "A" à "E". Combien de stations ce trajet compte-t-il (départ et arrivée compris) ?

Exercice 29 ♦♦

La suite de Fibonacci est définie par $F_0 = 0$, $F_1 = 1$, et $F_n = F_{n-1} + F_{n-2}$ pour $n \geq 2$. Une version récursive naïve de son calcul recalcule un très grand nombre de fois les mêmes valeurs.

1. Écrire une fonction `fib_memo(n, memo=None)` qui calcule F_n par **programmation dynamique** (mémoïsation), en s'inspirant de `rendu_memo` du cours.
2. Vérifier que $F_0, F_1, \dots, F_9$ vaut 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, et que $F_{30} = 832\,040$.

Exercice 30 ♦♦

Écrire, selon le principe **diviser pour régner** (comme le tri fusion du cours), une fonction récursive `maximum_dpr(liste)` qui renvoie le plus grand élément d'une liste non vide : diviser la liste en deux moitiés, trouver récursivement le maximum de chaque moitié, puis combiner en renvoyant le plus grand des deux. Quel est le cas de base ?

Exercice 31 ♦

Écrire une fonction récursive `compter_feuilles(arbre)` qui renvoie le nombre de **feuilles** (nœuds sans enfant) d'un arbre binaire représenté avec la classe `Noeud` du cours. Tester sur l'arbre `Noeud(1, Noeud(2, Noeud(4), Noeud(5)), Noeud(3))`, qui possède 3 feuilles.

Exercice 32

En utilisant les fonctions `insérer` et `rechercher` du cours, construire un arbre binaire de recherche à partir des valeurs 7, 3, 9, 1, 5 insérées dans cet ordre. Vérifier ensuite que 5 est présent dans l'arbre et que 4 n'y est pas.

Exercice 33

On modélise un petit réseau de métro par le graphe (non orienté) suivant, où chaque station est reliée à ses stations directement accessibles :

```
1 metro = {
2     "A": ["B"],
3     "B": ["A", "C", "D"],
4     "C": ["B", "E"],
5     "D": ["B", "E"],
6     "E": ["C", "D"],
7 }
```

En utilisant la fonction `chemin` du cours (recherche de chemin par BFS), déterminer le trajet trouvé de "A" à "E". Combien de stations ce trajet compte-t-il (départ et arrivée compris) ?

Exercice 34

La suite de Fibonacci est définie par $F_0 = 0$, $F_1 = 1$, et $F_n = F_{n-1} + F_{n-2}$ pour $n \geq 2$. Une version récursive naïve de son calcul recalcule un très grand nombre de fois les mêmes valeurs.

1. Écrire une fonction `fib_memo(n, memo=None)` qui calcule F_n par **programmation dynamique** (mémoïsation), en s'inspirant de `rendu_memo` du cours.
2. Vérifier que $F_0, F_1, \dots, F_9$ vaut 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, et que $F_{30} = 832\,040$.

Exercice 35

Écrire, selon le principe **diviser pour régner** (comme le tri fusion du cours), une fonction récursive `maximum_dpr(liste)` qui renvoie le plus grand élément d'une liste non vide : diviser la liste en deux moitiés, trouver récursivement le maximum de chaque moitié, puis combiner en renvoyant le plus grand des deux. Quel est le cas de base ?

Exercice 36

Écrire une fonction récursive `compter_feuilles(arbre)` qui renvoie le nombre de **feuilles** (nœuds sans enfant) d'un arbre binaire représenté avec la classe `Noeud` du cours. Tester sur l'arbre `Noeud(1, Noeud(2, Noeud(4), Noeud(5)), Noeud(3))`, qui possède 3 feuilles.

Exercice 37

En utilisant les fonctions `insérer` et `rechercher` du cours, construire un arbre binaire de recherche à partir des valeurs 7, 3, 9, 1, 5 insérées dans cet ordre. Vérifier ensuite que 5 est présent dans l'arbre et que 4 n'y est pas.

Exercice 38

On modélise un petit réseau de métro par le graphe (non orienté) suivant, où chaque station est reliée à ses stations directement accessibles :

```
1 metro = {
2     "A": ["B"],
3     "B": ["A", "C", "D"],
4     "C": ["B", "E"],
```

```

5  "D": ["B", "E"],
6  "E": ["C", "D"],
7  }

```

En utilisant la fonction `chemin` du cours (recherche de chemin par BFS), déterminer le trajet trouvé de "A" à "E". Combien de stations ce trajet compte-t-il (départ et arrivée compris) ?

Exercice 39 ♦♦

La suite de Fibonacci est définie par $F_0 = 0$, $F_1 = 1$, et $F_n = F_{n-1} + F_{n-2}$ pour $n \geq 2$. Une version récursive naïve de son calcul recalcule un très grand nombre de fois les mêmes valeurs.

1. Écrire une fonction `fib_memo(n, memo=None)` qui calcule F_n par **programmation dynamique** (mémoïsation), en s'inspirant de `rendu_memo` du cours.
2. Vérifier que $F_0, F_1, \dots, F_9$ vaut 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, et que $F_{30} = 832\,040$.

Exercice 40 ♦♦

Écrire, selon le principe **diviser pour régner** (comme le tri fusion du cours), une fonction récursive `maximum_dpr(liste)` qui renvoie le plus grand élément d'une liste non vide : diviser la liste en deux moitiés, trouver récursivement le maximum de chaque moitié, puis combiner en renvoyant le plus grand des deux. Quel est le cas de base ?

Exercice 41 ♦

Écrire une fonction récursive `compter_feuilles(arbre)` qui renvoie le nombre de **feuilles** (nœuds sans enfant) d'un arbre binaire représenté avec la classe `Noeud` du cours. Tester sur l'arbre `Noeud(1, Noeud(2, Noeud(4), Noeud(5)), Noeud(3))`, qui possède 3 feuilles.

Exercice 42 ♦

En utilisant les fonctions `insérer` et `rechercher` du cours, construire un arbre binaire de recherche à partir des valeurs 7, 3, 9, 1, 5 insérées dans cet ordre. Vérifier ensuite que 5 est présent dans l'arbre et que 4 n'y est pas.

Exercice 43 ♦♦

On modélise un petit réseau de métro par le graphe (non orienté) suivant, où chaque station est reliée à ses stations directement accessibles :

```

1 metro = {
2     "A": ["B"],
3     "B": ["A", "C", "D"],
4     "C": ["B", "E"],
5     "D": ["B", "E"],
6     "E": ["C", "D"],
7 }

```

En utilisant la fonction `chemin` du cours (recherche de chemin par BFS), déterminer le trajet trouvé de "A" à "E". Combien de stations ce trajet compte-t-il (départ et arrivée compris) ?

Exercice 44 ♦♦

La suite de Fibonacci est définie par $F_0 = 0$, $F_1 = 1$, et $F_n = F_{n-1} + F_{n-2}$ pour $n \geq 2$. Une version récursive naïve de son calcul recalcule un très grand nombre de fois les mêmes valeurs.

1. Écrire une fonction `fib_memo(n, memo=None)` qui calcule F_n par **programmation dynamique** (mémoïsation), en s'inspirant de `rendu_memo` du cours.
2. Vérifier que $F_0, F_1, \dots, F_9$ vaut 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, et que $F_{30} = 832\,040$.

Exercice 45 ♦♦

Écrire, selon le principe **diviser pour régner** (comme le tri fusion du cours), une fonction récursive `maximum_dpr(liste)` qui renvoie le plus grand élément d'une liste non vide : diviser la liste en deux moitiés, trouver récursivement le maximum de chaque moitié, puis combiner en renvoyant le plus grand des deux. Quel est le cas de base ?

Exercice 46 ♦

Écrire une fonction récursive `compter_feuilles(arbre)` qui renvoie le nombre de **feuilles** (nœuds sans enfant) d'un arbre binaire représenté avec la classe `Noeud` du cours. Tester sur l'arbre `Noeud(1, Noeud(2, Noeud(4), Noeud(5)), Noeud(3))`, qui possède 3 feuilles.

Exercice 47 ♦

En utilisant les fonctions `insérer` et `rechercher` du cours, construire un arbre binaire de recherche à partir des valeurs 7, 3, 9, 1, 5 insérées dans cet ordre. Vérifier ensuite que 5 est présent dans l'arbre et que 4 n'y est pas.

Exercice 48 ♦♦

On modélise un petit réseau de métro par le graphe (non orienté) suivant, où chaque station est reliée à ses stations directement accessibles :

```
1 metro = {
2     "A": ["B"],
3     "B": ["A", "C", "D"],
4     "C": ["B", "E"],
5     "D": ["B", "E"],
6     "E": ["C", "D"],
7 }
```

En utilisant la fonction `chemin` du cours (recherche de chemin par BFS), déterminer le trajet trouvé de "A" à "E". Combien de stations ce trajet compte-t-il (départ et arrivée compris) ?

Exercice 49 ♦♦

La suite de Fibonacci est définie par $F_0 = 0$, $F_1 = 1$, et $F_n = F_{n-1} + F_{n-2}$ pour $n \geq 2$. Une version récursive naïve de son calcul recalcule un très grand nombre de fois les mêmes valeurs.

1. Écrire une fonction `fib_memo(n, memo=None)` qui calcule F_n par **programmation dynamique** (mémoïsation), en s'inspirant de `rendu_memo` du cours.
2. Vérifier que $F_0, F_1, \dots, F_9$ vaut 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, et que $F_{30} = 832\,040$.

Exercice 50 ♦♦

Écrire, selon le principe **diviser pour régner** (comme le tri fusion du cours), une fonction récursive `maximum_dpr(liste)` qui renvoie le plus grand élément d'une liste non vide : diviser la liste en deux moitiés, trouver récursivement le maximum de chaque moitié, puis combiner en renvoyant le plus grand des deux. Quel est le cas de base ?

ALGORITHMES SUR LES ARBRES ET LES GRAPHS — QCM D'AUTO-ÉVALUATION

Pour chaque question, une seule réponse est exacte.

Capacité C1

1. [Q1] La taille d'un arbre binaire se calcule récursivement par :

- A. $1 + \text{taille}(\text{gauche}) + \text{taille}(\text{droite})$, l'arbre vide ayant pour taille 0.
- B. $1 + \max(\text{taille}(\text{gauche}), \text{taille}(\text{droite}))$.
- C. le nombre de feuilles seulement.
- D. $\text{taille}(\text{gauche}) \times \text{taille}(\text{droite})$.

Capacité C2

2. [Q2] Selon la convention du cours, la hauteur d'un arbre réduit à un seul nœud vaut :

- A. 0.
- B. -1.
- C. 1.
- D. indéfinie.

Capacité C3

3. [Q3] Le parcours INFIXE d'un arbre binaire visite :

- A. la racine, puis le sous-arbre gauche, puis le sous-arbre droit.
- B. le sous-arbre gauche, puis la racine, puis le sous-arbre droit.
- C. le sous-arbre gauche, puis le sous-arbre droit, puis la racine.
- D. les nœuds niveau par niveau, de haut en bas.

Capacité C4

4. [Q4] Le parcours en largeur d'un arbre s'appuie sur :

- A. une pile.
- B. un dictionnaire, et rien d'autre.
- C. une file.
- D. aucune structure auxiliaire.

Capacité C5

5. [Q5] Dans un arbre binaire de recherche, la recherche d'une clé plus petite que celle de la racine se poursuit :

- A. en repartant de la racine.
- B. dans les deux sous-arbres, pour être sûr.
- C. dans le sous-arbre droit uniquement.
- D. dans le sous-arbre gauche uniquement.

Capacité C6

6. [Q6] Insérer successivement les clés 1, 2, 3, 4 dans un arbre binaire de recherche vide produit :

- A. un arbre dégénéré, de hauteur 3, semblable à une liste chaînée.
- B. un arbre parfaitement équilibré de hauteur 1.
- C. un arbre où 4 devient la racine.
- D. une erreur, les clés devant être insérées dans le désordre.

Capacité C7

7. [Q7] Le parcours en largeur d'un graphe, à partir d'un sommet de départ, visite les sommets :

- A. dans l'ordre croissant de leur numéro.
- B. par distance croissante au sommet de départ, comptée en nombre d'arêtes.
- C. en descendant d'abord le plus loin possible le long d'un chemin.
- D. dans un ordre qui dépend du hasard.

Capacité C8

8. [Q8] Le parcours en profondeur d'un graphe s'implémente naturellement :

- A. après un tri préalable des sommets.
- B. avec une file, et elle seule.
- C. par récursion, ou au moyen d'une pile explicite.
- D. par une recherche dichotomique.

Capacité C9

9. [Q9] Lors du parcours d'un graphe non orienté, on détecte un cycle quand on rencontre un sommet déjà visité qui :

- A. est le sommet de départ.
- B. possède le plus grand nombre de voisins.
- C. n'a aucun voisin.
- D. n'est pas le parent direct dans le parcours.

Capacité C10

10. [Q10] Pour reconstituer le chemin trouvé entre deux sommets par un parcours, il faut :
- A. mémoriser, pour chaque sommet visité, celui depuis lequel on l'a atteint.
 - B. relancer le parcours depuis l'arrivée.
 - C. trier les sommets par distance.
 - D. conserver uniquement la liste des sommets visités, dans l'ordre.

Capacité C11

11. [Q11] La méthode « diviser pour régner » consiste à :
- A. traiter les données une par une, du début à la fin.
 - B. découper le problème en sous-problèmes de même nature, les résoudre, puis combiner leurs solutions.
 - C. essayer toutes les solutions possibles avant de choisir.
 - D. mémoriser les résultats déjà calculés pour ne pas les recalculer.

Capacité C12

12. [Q12] La programmation dynamique est particulièrement utile lorsque :
- A. les sous-problèmes ne se recoupent jamais.
 - B. le problème n'admet aucune formulation récursive.
 - C. les mêmes sous-problèmes réapparaissent plusieurs fois.
 - D. il n'existe qu'un seul sous-problème.

Capacité C13

13. [Q13] À la différence de la recherche naïve, l'algorithme de Boyer-Moore compare le motif au texte :
- A. dans un ordre tiré au hasard.
 - B. en ignorant complètement le texte.
 - C. sur le seul premier caractère.
 - D. de droite à gauche, en s'appuyant sur un prétraitement du motif.

Clé de correction — réservée au professeur

Question	Capacité	Réponse exacte
Q1	C1	A
Q2	C2	A
Q3	C3	B
Q4	C4	C
Q5	C5	D
Q6	C6	A
Q7	C7	B
Q8	C8	C
Q9	C9	D
Q10	C10	A
Q11	C11	B
Q12	C12	C
Q13	C13	D

Diagnostic des réponses erronées

► Q1

- **B** — Cette formule est celle de la HAUTEUR : le maximum ne retient qu'une branche, alors que la taille doit additionner tous les nœuds. *Renvoi : C1, taille d'un arbre.*
- **C** — Les nœuds internes appartiennent aussi à l'arbre ; les omettre sous-estime la taille. *Renvoi : C1, taille d'un arbre.*
- **D** — Un produit s'annulerait dès qu'un sous-arbre est vide, et donnerait 0 pour un arbre pourtant non vide. *Renvoi : C1, taille d'un arbre.*

► Q2

- **B** — -1 est la hauteur de l'arbre VIDE. Un arbre à un nœud n'est pas vide. *Renvoi : C2, hauteur d'un arbre.*
- **C** — L'élève compte les niveaux au lieu des arêtes. La hauteur est la longueur du plus long chemin de la racine à une feuille : ici, ce chemin est vide. *Renvoi : C2, hauteur d'un arbre.*
- **D** — La définition récursive couvre ce cas sans ambiguïté, à partir de la hauteur -1 de l'arbre vide. *Renvoi : C2, hauteur d'un arbre.*

► Q3

- **A** — C'est le parcours PRÉFIXE : la racine y est visitée avant ses deux sous-arbres. *Renvoi : C3, parcours infixe, préfixe, suffixe.*
- **C** — C'est le parcours SUFFIXE : la racine y est visitée en dernier. *Renvoi : C3, parcours infixe, préfixe, suffixe.*
- **D** — C'est le parcours en LARGEUR, qui n'appartient pas à la famille des trois parcours en profondeur. *Renvoi : C3, parcours en profondeur et en largeur.*

► Q4

- **A** — La pile produit un parcours en PROFONDEUR : elle ressort d'abord le dernier nœud rencontré, donc le plus profond. *Renvoi : C4, parcours en largeur.*
- **B** — Un dictionnaire peut mémoriser les nœuds vus, mais il n'impose aucun ordre de traitement, qui est ici l'essentiel. *Renvoi : C4, parcours en largeur.*
- **D** — Sans structure d'attente, il est impossible de mémoriser les nœuds d'un niveau pendant qu'on le traite. *Renvoi : C4, parcours en largeur.*

► Q5

- **A** — Recommencer à la racine boucle indéfiniment : la recherche doit descendre d'un niveau à chaque comparaison. *Renvoi : C5, recherche dans un ABR.*
- **B** — Explorer les deux côtés reviendrait à parcourir tout l'arbre : c'est précisément ce que la propriété de l'ABR permet d'éviter. *Renvoi : C5, recherche dans un ABR.*
- **C** — L'élève inverse la comparaison. Le sous-arbre droit ne contient que des clés supérieures à la racine. *Renvoi : C5, recherche dans un ABR.*

► Q6

- B — L'insertion ne rééquilibre rien : chaque clé étant supérieure à la précédente, elle part toujours à droite. *Renvoi : C6, insertion dans un ABR.*
- C — La racine est fixée par la PREMIÈRE clé insérée, ici 1, et ne change plus. *Renvoi : C6, insertion dans un ABR.*
- D — L'insertion en ordre trié est parfaitement licite ; elle est seulement le pire cas pour la hauteur obtenue. *Renvoi : C6, insertion dans un ABR.*

► Q7

- A — La numérotation des sommets n'a aucune influence : seule la structure des arêtes gouverne l'ordre de visite. *Renvoi : C7, parcours en largeur d'un graphe.*
- C — C'est le parcours en PROFONDEUR. Le parcours en largeur épuise un niveau entier avant de passer au suivant. *Renvoi : C7, largeur et profondeur.*
- D — Le parcours est déterministe une fois fixé l'ordre des voisins dans la structure de données. *Renvoi : C7, parcours en largeur d'un graphe.*

► Q8

- A — Aucun tri n'est nécessaire ni même défini : un graphe n'est pas une structure ordonnée. *Renvoi : C8, parcours en profondeur.*
- B — La file donne le parcours en LARGEUR : c'est le choix de la structure d'attente qui distingue les deux parcours. *Renvoi : C8, parcours en profondeur.*
- D — La dichotomie suppose un ordre total sur les éléments, dont un graphe est dépourvu. *Renvoi : C8, parcours en profondeur.*

► Q9

- A — Un cycle peut se refermer sur n'importe quel sommet, pas seulement sur celui d'où l'on est parti. *Renvoi : C9, détection de cycle.*
- B — Le degré d'un sommet ne dit rien sur l'existence d'un cycle : un sommet de degré élevé peut appartenir à un arbre. *Renvoi : C9, détection de cycle.*
- C — Un sommet isolé ne peut appartenir à aucun cycle, faute d'arête pour y entrer. *Renvoi : C9, détection de cycle.*

► Q10

- B — Un second parcours redonnerait l'existence d'un chemin, mais pas celui qui vient d'être emprunté. *Renvoi : C10, reconstitution d'un chemin.*
- C — Connaître les distances ne suffit pas : il manque le lien qui relie chaque sommet à son prédécesseur. *Renvoi : C10, reconstitution d'un chemin.*
- D — L'ordre de visite mêle toutes les branches explorées ; il ne distingue pas celles qui mènent au but de celles qui ont été abandonnées. *Renvoi : C10, reconstitution d'un chemin.*

► Q11

- A — L'élève décrit un parcours séquentiel, sans aucun découpage récursif. *Renvoi : C11, diviser pour régner.*
- C — C'est la recherche exhaustive, que la méthode cherche justement à éviter. *Renvoi : C11, diviser pour régner.*
- D — La mémorisation caractérise la PROGRAMMATION DYNAMIQUE. Elle s'ajoute parfois au découpage, mais ne le définit pas. *Renvoi : C11, diviser pour régner et programmation dynamique.*

► Q12

- A — Sans recouvrement, il n'y a rien à réutiliser : un simple « diviser pour régner » suffit. *Renvoi : C12, programmation dynamique.*
- B — C'est l'inverse : la méthode part d'une relation de récurrence, qu'elle évalue en mémorisant les résultats. *Renvoi : C12, programmation dynamique.*
- D — Avec un unique sous-problème, il n'y a aucun recalcul à éviter et donc aucun gain. *Renvoi : C12, programmation dynamique.*

► Q13

- A — L'ordre est au contraire fixé et essentiel : c'est la comparaison depuis la fin du motif qui autorise les grands décalages. *Renvoi : C13, algorithme de Boyer-Moore.*

- **B** — Le texte est évidemment lu ; ce que l'algorithme économise, ce sont des comparaisons, grâce aux décalages qu'il peut anticiper. *Renvoi : C13, algorithme de Boyer-Moore.*
- **C** — Une comparaison sur un seul caractère ne prouverait rien : c'est le motif entier qui doit être reconnu. *Renvoi : C13, algorithme de Boyer-Moore.*

Corrigé

Q1. L'arbre a 5 nœuds (10, 5, 15, 2, 7) : taille = 5. Le plus long chemin racine-feuille est $10 \rightarrow 5 \rightarrow 2$ (ou $10 \rightarrow 5 \rightarrow 7$), soit 2 arêtes : hauteur = 2.

Q2.

```
1 def est_feuille(arbre):
2     return arbre is not None and arbre.gauche is None and arbre.droite is None
```

Corrigé

Q1. L'ABR obtenu a pour racine 10, sous-arbre gauche enraciné en 5 (avec 3 à gauche et 7 à droite), et sous-arbre droit enraciné en 15 (avec 12 à gauche).

Q2.

```
1 def minimum(arbre):
2     while arbre.gauche is not None:
3         arbre = arbre.gauche
4     return arbre.valeur
```

Dans un ABR, chaque descente à gauche mène vers des valeurs plus petites ; la plus petite valeur est donc atteinte lorsqu'il n'y a plus de sous-arbre gauche.

Évaluation A — Arbres binaires

Durée : 45 minutes. Ordinateur avec interpréteur Python autorisé.

Barème indicatif : Ex. 1 : 8 pts (taille, hauteur) ; Ex. 2 : 12 pts (arbre de recherche)

Exercice 1 — Taille et hauteur

(8 points)

On donne la classe Noeud du cours (attributs valeur, gauche, droite) et l'arbre :

```
1 arbre = Noeud(10, Noeud(5, Noeud(2), Noeud(7)), Noeud(15))
```

- (4 pts) Sans exécuter de code, donner la taille et la hauteur de cet arbre en justifiant (convention : hauteur d'une feuille = 0).
- (4 pts) Écrire une fonction `est_feuille(arbre)` qui renvoie True si et seulement si `arbre` est un nœud sans aucun enfant.

Exercice 2 — Arbre binaire de recherche

(12 points)

- (6 pts) En utilisant `insérer` du cours, construire un ABR à partir des valeurs 10, 5, 15, 3, 7, 12 insérées dans cet ordre.
- (6 pts) Écrire une fonction `minimum(arbre)` qui renvoie la plus petite valeur d'un ABR non vide (indication : la plus petite valeur d'un ABR se trouve toujours tout en bas à gauche).

Corrigé

Q1. Depuis "P" : on enfile "P", on le défile et on enfile ses voisins "Q" puis "R" ; on défile "Q", son voisin "S" n'est pas encore visité, on l'enfile ; on défile "R", son voisin "S" est déjà visité. Ordre de visite : P, Q, R, S.

Q2. Oui : $P \rightarrow Q \rightarrow S \rightarrow R \rightarrow P$ est un cycle (chaque arête utilisée existe bien dans le graphe, et on revient au sommet de départ sans repasser par une même arête).

Corrigé

Q1. Diviser (découper le problème en sous-instances plus petites de même nature), régner (résoudre récursivement chaque sous-instance), combiner (assembler les solutions partielles en la solution globale).

Q2.

```
1 def somme_dpr(liste):
2     if len(liste) == 1:
3         return liste[0]
4     milieu = len(liste) // 2
5     return somme_dpr(liste[:milieu]) + somme_dpr(liste[milieu:])
```

Évaluation B — Graphes et diviser pour régner

Durée : 45 minutes. Ordinateur avec interpréteur Python autorisé.

Barème indicatif : Ex. 1 : 10 pts (parcours et cycle); Ex. 2 : 10 pts (diviser pour régner)

Exercice 1 — Parcours et cycle

(10 points)

On donne le graphe non orienté :

```
1 graphe = {
2     "P": ["Q", "R"],
3     "Q": ["P", "S"],
4     "R": ["P", "S"],
5     "S": ["Q", "R"],
6 }
```

- (5 pts) Donner l'ordre de visite des sommets par un parcours en largeur (BFS) depuis "P", en supposant que les voisins sont explorés dans l'ordre où ils apparaissent dans la liste.
- (5 pts) Ce graphe contient-il un cycle ? Justifier en citant un cycle si oui.

Exercice 2 — Diviser pour régner

(10 points)

- (4 pts) Rappeler les trois étapes de la méthode diviser pour régner.
- (6 pts) Écrire une fonction récursive `somme_dpr(liste)` qui calcule la somme des éléments d'une liste non vide par diviser pour régner (diviser en deux moitiés, additionner récursivement, puis combiner par addition).

Exercice 51

Une fonction de mémoïsation est écrite ainsi :

```
1 def f(n, memo={}):
2     if n in memo:
```

```

3     return memo[n]
4     memo[n] = n * n
5     return memo[n]

```

On appelle $f(3)$, puis $f(2, \{\})$ (avec un dictionnaire frais), puis $f(3)$ à nouveau. Le deuxième appel à $f(3)$ recalcule-t-il 3×3 , ou retrouve-t-il directement le résultat mémorisé du tout premier appel ? Pourquoi ce comportement peut-il surprendre ?

Corrigé

Le deuxième appel à $f(3)$ retrouve directement 9 dans memo, sans recalcul : le dictionnaire par défaut $\text{memo}=\{\}$ est créé **une seule fois**, à la définition de la fonction, et **partagé** par tous les appels qui ne fournissent pas explicitement leur propre dictionnaire. C'est surprenant car on pourrait croire, par analogie avec d'autres langages, que $\text{memo}=\{\}$ recrée un dictionnaire vide à **chaque** appel – ce n'est pas le cas en Python. C'est pourquoi le cours utilise systématiquement $\text{memo}=\text{None}$ suivi d'une initialisation à l'intérieur de la fonction.

Exercice 52

Une fonction de mémorisation est écrite ainsi :

```

1 def f(n, memo={}):
2     if n in memo:
3         return memo[n]
4     memo[n] = n * n
5     return memo[n]

```

On appelle $f(3)$, puis $f(2, \{\})$ (avec un dictionnaire frais), puis $f(3)$ à nouveau. Le deuxième appel à $f(3)$ recalcule-t-il 3×3 , ou retrouve-t-il directement le résultat mémorisé du tout premier appel ? Pourquoi ce comportement peut-il surprendre ?

Corrigé

Le deuxième appel à $f(3)$ retrouve directement 9 dans memo, sans recalcul : le dictionnaire par défaut $\text{memo}=\{\}$ est créé **une seule fois**, à la définition de la fonction, et **partagé** par tous les appels qui ne fournissent pas explicitement leur propre dictionnaire. C'est surprenant car on pourrait croire, par analogie avec d'autres langages, que $\text{memo}=\{\}$ recrée un dictionnaire vide à **chaque** appel – ce n'est pas le cas en Python. C'est pourquoi le cours utilise systématiquement $\text{memo}=\text{None}$ suivi d'une initialisation à l'intérieur de la fonction.

Exercice 53

Une fonction de mémorisation est écrite ainsi :

```

1 def f(n, memo={}):
2     if n in memo:
3         return memo[n]
4     memo[n] = n * n
5     return memo[n]

```

On appelle $f(3)$, puis $f(2, \{\})$ (avec un dictionnaire frais), puis $f(3)$ à nouveau. Le deuxième appel à $f(3)$ recalcule-t-il 3×3 , ou retrouve-t-il directement le résultat mémorisé du tout premier appel ? Pourquoi ce comportement peut-il surprendre ?

Corrigé

Le deuxième appel à $f(3)$ retrouve directement 9 dans memo, sans recalcul : le dictionnaire par défaut $\text{memo}=\{\}$ est créé **une seule fois**, à la définition de la fonction, et **partagé** par tous les appels qui ne fournissent pas explicitement leur propre dictionnaire. C'est surprenant car on pourrait croire,

par analogie avec d'autres langages, que `memo={}` recrée un dictionnaire vide à **chaque** appel – ce n'est pas le cas en Python. C'est pourquoi le cours utilise systématiquement `memo=None` suivi d'une initialisation à l'intérieur de la fonction.

Exercice 54

Une fonction de mémorisation est écrite ainsi :

```
1 def f(n, memo={}):
2     if n in memo:
3         return memo[n]
4     memo[n] = n * n
5     return memo[n]
```

On appelle `f(3)`, puis `f(2, {})` (avec un dictionnaire frais), puis `f(3)` à nouveau. Le deuxième appel à `f(3)` recalcule-t-il 3×3 , ou retrouve-t-il directement le résultat mémorisé du tout premier appel ? Pourquoi ce comportement peut-il surprendre ?

Corrigé

Le deuxième appel à `f(3)` retrouve directement 9 dans `memo`, sans recalcul : le dictionnaire par défaut `memo={}` est créé **une seule fois**, à la définition de la fonction, et **partagé** par tous les appels qui ne fournissent pas explicitement leur propre dictionnaire. C'est surprenant car on pourrait croire, par analogie avec d'autres langages, que `memo={}` recrée un dictionnaire vide à **chaque** appel – ce n'est pas le cas en Python. C'est pourquoi le cours utilise systématiquement `memo=None` suivi d'une initialisation à l'intérieur de la fonction.

Exercice 55

Une fonction de mémorisation est écrite ainsi :

```
1 def f(n, memo={}):
2     if n in memo:
3         return memo[n]
4     memo[n] = n * n
5     return memo[n]
```

On appelle `f(3)`, puis `f(2, {})` (avec un dictionnaire frais), puis `f(3)` à nouveau. Le deuxième appel à `f(3)` recalcule-t-il 3×3 , ou retrouve-t-il directement le résultat mémorisé du tout premier appel ? Pourquoi ce comportement peut-il surprendre ?

Corrigé

Le deuxième appel à `f(3)` retrouve directement 9 dans `memo`, sans recalcul : le dictionnaire par défaut `memo={}` est créé **une seule fois**, à la définition de la fonction, et **partagé** par tous les appels qui ne fournissent pas explicitement leur propre dictionnaire. C'est surprenant car on pourrait croire, par analogie avec d'autres langages, que `memo={}` recrée un dictionnaire vide à **chaque** appel – ce n'est pas le cas en Python. C'est pourquoi le cours utilise systématiquement `memo=None` suivi d'une initialisation à l'intérieur de la fonction.

Exercice 56

Une fonction de mémorisation est écrite ainsi :

```
1 def f(n, memo={}):
2     if n in memo:
3         return memo[n]
4     memo[n] = n * n
5     return memo[n]
```

On appelle $f(3)$, puis $f(2, \{\})$ (avec un dictionnaire frais), puis $f(3)$ à nouveau. Le deuxième appel à $f(3)$ recalcule-t-il 3×3 , ou retrouve-t-il directement le résultat mémorisé du tout premier appel ? Pourquoi ce comportement peut-il surprendre ?

Corrigé

Le deuxième appel à $f(3)$ retrouve directement 9 dans memo, sans recalcul : le dictionnaire par défaut `memo={}` est créé **une seule fois**, à la définition de la fonction, et **partagé** par tous les appels qui ne fournissent pas explicitement leur propre dictionnaire. C'est surprenant car on pourrait croire, par analogie avec d'autres langages, que `memo={}` recrée un dictionnaire vide à **chaque** appel – ce n'est pas le cas en Python. C'est pourquoi le cours utilise systématiquement `memo=None` suivi d'une initialisation à l'intérieur de la fonction.

Exercice 57

Une fonction de mémorisation est écrite ainsi :

```
1 def f(n, memo={}):
2     if n in memo:
3         return memo[n]
4     memo[n] = n * n
5     return memo[n]
```

On appelle $f(3)$, puis $f(2, \{\})$ (avec un dictionnaire frais), puis $f(3)$ à nouveau. Le deuxième appel à $f(3)$ recalcule-t-il 3×3 , ou retrouve-t-il directement le résultat mémorisé du tout premier appel ? Pourquoi ce comportement peut-il surprendre ?

Corrigé

Le deuxième appel à $f(3)$ retrouve directement 9 dans memo, sans recalcul : le dictionnaire par défaut `memo={}` est créé **une seule fois**, à la définition de la fonction, et **partagé** par tous les appels qui ne fournissent pas explicitement leur propre dictionnaire. C'est surprenant car on pourrait croire, par analogie avec d'autres langages, que `memo={}` recrée un dictionnaire vide à **chaque** appel – ce n'est pas le cas en Python. C'est pourquoi le cours utilise systématiquement `memo=None` suivi d'une initialisation à l'intérieur de la fonction.

Exercice 58

Une fonction de mémorisation est écrite ainsi :

```
1 def f(n, memo={}):
2     if n in memo:
3         return memo[n]
4     memo[n] = n * n
5     return memo[n]
```

On appelle $f(3)$, puis $f(2, \{\})$ (avec un dictionnaire frais), puis $f(3)$ à nouveau. Le deuxième appel à $f(3)$ recalcule-t-il 3×3 , ou retrouve-t-il directement le résultat mémorisé du tout premier appel ? Pourquoi ce comportement peut-il surprendre ?

Corrigé

Le deuxième appel à $f(3)$ retrouve directement 9 dans memo, sans recalcul : le dictionnaire par défaut `memo={}` est créé **une seule fois**, à la définition de la fonction, et **partagé** par tous les appels qui ne fournissent pas explicitement leur propre dictionnaire. C'est surprenant car on pourrait croire, par analogie avec d'autres langages, que `memo={}` recrée un dictionnaire vide à **chaque** appel – ce n'est pas le cas en Python. C'est pourquoi le cours utilise systématiquement `memo=None` suivi d'une initialisation à l'intérieur de la fonction.

Exercice 59

Une fonction de mémoïsation est écrite ainsi :

```
1 def f(n, memo={}):
2     if n in memo:
3         return memo[n]
4     memo[n] = n * n
5     return memo[n]
```

On appelle $f(3)$, puis $f(2, \{\})$ (avec un dictionnaire frais), puis $f(3)$ à nouveau. Le deuxième appel à $f(3)$ recalcule-t-il 3×3 , ou retrouve-t-il directement le résultat mémorisé du tout premier appel ? Pourquoi ce comportement peut-il surprendre ?

Corrigé

Le deuxième appel à $f(3)$ retrouve directement 9 dans memo, sans recalcul : le dictionnaire par défaut `memo={}` est créé **une seule fois**, à la définition de la fonction, et **partagé** par tous les appels qui ne fournissent pas explicitement leur propre dictionnaire. C'est surprenant car on pourrait croire, par analogie avec d'autres langages, que `memo={}` recrée un dictionnaire vide à **chaque** appel – ce n'est pas le cas en Python. C'est pourquoi le cours utilise systématiquement `memo=None` suivi d'une initialisation à l'intérieur de la fonction.

Exercice 60

Une fonction de mémoïsation est écrite ainsi :

```
1 def f(n, memo={}):
2     if n in memo:
3         return memo[n]
4     memo[n] = n * n
5     return memo[n]
```

On appelle $f(3)$, puis $f(2, \{\})$ (avec un dictionnaire frais), puis $f(3)$ à nouveau. Le deuxième appel à $f(3)$ recalcule-t-il 3×3 , ou retrouve-t-il directement le résultat mémorisé du tout premier appel ? Pourquoi ce comportement peut-il surprendre ?

Corrigé

Le deuxième appel à $f(3)$ retrouve directement 9 dans memo, sans recalcul : le dictionnaire par défaut `memo={}` est créé **une seule fois**, à la définition de la fonction, et **partagé** par tous les appels qui ne fournissent pas explicitement leur propre dictionnaire. C'est surprenant car on pourrait croire, par analogie avec d'autres langages, que `memo={}` recrée un dictionnaire vide à **chaque** appel – ce n'est pas le cas en Python. C'est pourquoi le cours utilise systématiquement `memo=None` suivi d'une initialisation à l'intérieur de la fonction.

Exercice 61

Une fonction de mémoïsation est écrite ainsi :

```
1 def f(n, memo={}):
2     if n in memo:
3         return memo[n]
4     memo[n] = n * n
5     return memo[n]
```

On appelle $f(3)$, puis $f(2, \{\})$ (avec un dictionnaire frais), puis $f(3)$ à nouveau. Le deuxième appel à $f(3)$ recalcule-t-il 3×3 , ou retrouve-t-il directement le résultat mémorisé du tout premier appel ? Pourquoi ce comportement peut-il surprendre ?

Corrigé

Le deuxième appel à $f(3)$ retrouve directement 9 dans memo, sans recalcul : le dictionnaire par défaut $\text{memo}=\{\}$ est créé **une seule fois**, à la définition de la fonction, et **partagé** par tous les appels qui ne fournissent pas explicitement leur propre dictionnaire. C'est surprenant car on pourrait croire, par analogie avec d'autres langages, que $\text{memo}=\{\}$ recrée un dictionnaire vide à **chaque** appel – ce n'est pas le cas en Python. C'est pourquoi le cours utilise systématiquement $\text{memo}=\text{None}$ suivi d'une initialisation à l'intérieur de la fonction.

Exercice 62

Une fonction de mémorisation est écrite ainsi :

```
1 def f(n, memo={}):
2     if n in memo:
3         return memo[n]
4     memo[n] = n * n
5     return memo[n]
```

On appelle $f(3)$, puis $f(2, \{\})$ (avec un dictionnaire frais), puis $f(3)$ à nouveau. Le deuxième appel à $f(3)$ recalcule-t-il 3×3 , ou retrouve-t-il directement le résultat mémorisé du tout premier appel ? Pourquoi ce comportement peut-il surprendre ?

Corrigé

Le deuxième appel à $f(3)$ retrouve directement 9 dans memo, sans recalcul : le dictionnaire par défaut $\text{memo}=\{\}$ est créé **une seule fois**, à la définition de la fonction, et **partagé** par tous les appels qui ne fournissent pas explicitement leur propre dictionnaire. C'est surprenant car on pourrait croire, par analogie avec d'autres langages, que $\text{memo}=\{\}$ recrée un dictionnaire vide à **chaque** appel – ce n'est pas le cas en Python. C'est pourquoi le cours utilise systématiquement $\text{memo}=\text{None}$ suivi d'une initialisation à l'intérieur de la fonction.

Exercice 63

Une fonction de mémorisation est écrite ainsi :

```
1 def f(n, memo={}):
2     if n in memo:
3         return memo[n]
4     memo[n] = n * n
5     return memo[n]
```

On appelle $f(3)$, puis $f(2, \{\})$ (avec un dictionnaire frais), puis $f(3)$ à nouveau. Le deuxième appel à $f(3)$ recalcule-t-il 3×3 , ou retrouve-t-il directement le résultat mémorisé du tout premier appel ? Pourquoi ce comportement peut-il surprendre ?

Corrigé

Le deuxième appel à $f(3)$ retrouve directement 9 dans memo, sans recalcul : le dictionnaire par défaut $\text{memo}=\{\}$ est créé **une seule fois**, à la définition de la fonction, et **partagé** par tous les appels qui ne fournissent pas explicitement leur propre dictionnaire. C'est surprenant car on pourrait croire, par analogie avec d'autres langages, que $\text{memo}=\{\}$ recrée un dictionnaire vide à **chaque** appel – ce n'est pas le cas en Python. C'est pourquoi le cours utilise systématiquement $\text{memo}=\text{None}$ suivi d'une initialisation à l'intérieur de la fonction.

Exercice 64

Une fonction de mémorisation est écrite ainsi :

```
1 def f(n, memo={}):
2     if n in memo:
3         return memo[n]
4     memo[n] = n * n
5     return memo[n]
```

On appelle $f(3)$, puis $f(2, \{\})$ (avec un dictionnaire frais), puis $f(3)$ à nouveau. Le deuxième appel à $f(3)$ recalcule-t-il 3×3 , ou retrouve-t-il directement le résultat mémorisé du tout premier appel ? Pourquoi ce comportement peut-il surprendre ?

Corrigé

Le deuxième appel à $f(3)$ retrouve directement 9 dans memo, sans recalcul : le dictionnaire par défaut `memo={}` est créé **une seule fois**, à la définition de la fonction, et **partagé** par tous les appels qui ne fournissent pas explicitement leur propre dictionnaire. C'est surprenant car on pourrait croire, par analogie avec d'autres langages, que `memo={}` recrée un dictionnaire vide à **chaque** appel – ce n'est pas le cas en Python. C'est pourquoi le cours utilise systématiquement `memo=None` suivi d'une initialisation à l'intérieur de la fonction.

Corrigé

```
1 def compter_feuilles(arbre):
2     if arbre is None:
3         return 0
4     if arbre.gauche is None and arbre.droite is None:
5         return 1
6     return compter_feuilles(arbre.gauche) + compter_feuilles(arbre.droite)
```

Cas de base : l'arbre vide n'a aucune feuille. Un nœud sans aucun enfant est lui-même une feuille (on renvoie 1, sans appel récursif supplémentaire). Sinon, on additionne le nombre de feuilles des deux sous-arbres.

Corrigé

```
1 abr = None
2 for v in (7, 3, 9, 1, 5):
3     abr = inserer(abr, v)
4
5 print(rechercher(abr, 5)) # True
6 print(rechercher(abr, 4)) # False
```

L'arbre construit a pour racine 7, sous-arbre gauche enraciné en 3 (lui-même avec 1 à gauche et 5 à droite), et sous-arbre droit réduit à 9. La recherche de 5 descend $7 \rightarrow 3 \rightarrow 5$ (trouvé) ; celle de 4 descend $7 \rightarrow 3 \rightarrow 5 \rightarrow$ sous-arbre gauche vide (non trouvé).

Corrigé

```
1 trajet = chemin(metro, "A", "E")
2 print(trajet) # ['A', 'B', 'C', 'E']
```

Le BFS depuis "A" atteint "B" en premier (distance 1), puis "C" et "D" (distance 2, "C" découvert avant "D" car il apparaît en premier dans la liste des voisins de "B"), puis "E" (distance 3, découvert depuis "C"). Le trajet reconstruit est $A \rightarrow B \rightarrow C \rightarrow E$: il compte 4 stations (départ et arrivée compris).

Corrigé

```

1 def fib_memo(n, memo=None):
2     if memo is None:
3         memo = {0: 0, 1: 1}
4     if n in memo:
5         return memo[n]
6     memo[n] = fib_memo(n - 1, memo) + fib_memo(n - 2, memo)
7     return memo[n]

```

Comme `rendu_memo`, les cas de base ($n = 0$ et $n = 1$) sont préchargés dans `memo`. Sans mémorisation, le nombre d'appels récursifs pour calculer F_{30} dépasse le million ; avec mémorisation, chaque valeur $F_0, \dots, F_{30}$ n'est calculée qu'une seule fois.

Corrigé

```

1 def maximum_dpr(liste):
2     if len(liste) == 1:
3         return liste[0]
4     milieu = len(liste) // 2
5     max_gauche = maximum_dpr(liste[:milieu])
6     max_droite = maximum_dpr(liste[milieu:])
7     return max(max_gauche, max_droite)

```

Le cas de base est une liste réduite à un seul élément : son maximum est cet élément lui-même, sans appel récursif. On suppose la liste non vide en entrée (une liste vide n'a pas de maximum).

Corrigé

```

1 def compter_feuilles(arbre):
2     if arbre is None:
3         return 0
4     if arbre.gauche is None and arbre.droite is None:
5         return 1
6     return compter_feuilles(arbre.gauche) + compter_feuilles(arbre.droite)

```

Cas de base : l'arbre vide n'a aucune feuille. Un nœud sans aucun enfant est lui-même une feuille (on renvoie 1, sans appel récursif supplémentaire). Sinon, on additionne le nombre de feuilles des deux sous-arbres.

Corrigé

```

1 abr = None
2 for v in (7, 3, 9, 1, 5):
3     abr = inserer(abr, v)
4
5 print(rechercher(abr, 5)) # True
6 print(rechercher(abr, 4)) # False

```

L'arbre construit a pour racine 7, sous-arbre gauche enraciné en 3 (lui-même avec 1 à gauche et 5 à droite), et sous-arbre droit réduit à 9. La recherche de 5 descend $7 \rightarrow 3 \rightarrow 5$ (trouvé) ; celle de 4 descend $7 \rightarrow 3 \rightarrow 5 \rightarrow$ sous-arbre gauche vide (non trouvé).

Corrigé

```

1 trajet = chemin(metro, "A", "E")
2 print(trajet) # ['A', 'B', 'C', 'E']

```

Le BFS depuis "A" atteint "B" en premier (distance 1), puis "C" et "D" (distance 2, "C" découvert avant "D" car il apparaît en premier dans la liste des voisins de "B"), puis "E" (distance 3, découvert depuis "C"). Le trajet reconstruit est $A \rightarrow B \rightarrow C \rightarrow E$: il compte 4 stations (départ et arrivée compris).

Corrigé

```
1 def fib_memo(n, memo=None):
2     if memo is None:
3         memo = {0: 0, 1: 1}
4     if n in memo:
5         return memo[n]
6     memo[n] = fib_memo(n - 1, memo) + fib_memo(n - 2, memo)
7     return memo[n]
```

Comme rendu_memo, les cas de base ($n = 0$ et $n = 1$) sont préchargés dans memo. Sans mémorisation, le nombre d'appels récursifs pour calculer F_{30} dépasse le million ; avec mémorisation, chaque valeur $F_0, \dots, F_{30}$ n'est calculée qu'une seule fois.

Corrigé

```
1 def maximum_dpr(liste):
2     if len(liste) == 1:
3         return liste[0]
4     milieu = len(liste) // 2
5     max_gauche = maximum_dpr(liste[:milieu])
6     max_droite = maximum_dpr(liste[milieu:])
7     return max(max_gauche, max_droite)
```

Le cas de base est une liste réduite à un seul élément : son maximum est cet élément lui-même, sans appel récursif. On suppose la liste non vide en entrée (une liste vide n'a pas de maximum).

Corrigé

```
1 def compter_feuilles(arbre):
2     if arbre is None:
3         return 0
4     if arbre.gauche is None and arbre.droite is None:
5         return 1
6     return compter_feuilles(arbre.gauche) + compter_feuilles(arbre.droite)
```

Cas de base : l'arbre vide n'a aucune feuille. Un nœud sans aucun enfant est lui-même une feuille (on renvoie 1, sans appel récursif supplémentaire). Sinon, on additionne le nombre de feuilles des deux sous-arbres.

Corrigé

```
1 abr = None
2 for v in (7, 3, 9, 1, 5):
3     abr = inserer(abr, v)
4
5 print(rechercher(abr, 5)) # True
6 print(rechercher(abr, 4)) # False
```

L'arbre construit a pour racine 7, sous-arbre gauche enraciné en 3 (lui-même avec 1 à gauche et 5 à droite), et sous-arbre droit réduit à 9. La recherche de 5 descend $7 \rightarrow 3 \rightarrow 5$ (trouvé) ; celle de 4 descend $7 \rightarrow 3 \rightarrow 5 \rightarrow$ sous-arbre gauche vide (non trouvé).

Corrigé

```

1 trajet = chemin(metro, "A", "E")
2 print(trajet) # ['A', 'B', 'C', 'E']

```

Le BFS depuis "A" atteint "B" en premier (distance 1), puis "C" et "D" (distance 2, "C" découvert avant "D" car il apparaît en premier dans la liste des voisins de "B"), puis "E" (distance 3, découvert depuis "C"). Le trajet reconstruit est $A \rightarrow B \rightarrow C \rightarrow E$: il compte 4 stations (départ et arrivée compris).

Corrigé

```

1 def fib_memo(n, memo=None):
2     if memo is None:
3         memo = {0: 0, 1: 1}
4     if n in memo:
5         return memo[n]
6     memo[n] = fib_memo(n - 1, memo) + fib_memo(n - 2, memo)
7     return memo[n]

```

Comme rendu_memo, les cas de base ($n = 0$ et $n = 1$) sont préchargés dans memo. Sans mémorisation, le nombre d'appels récur­sifs pour calculer F_{30} dépasse le million ; avec mémorisation, chaque valeur $F_0, \dots, F_{30}$ n'est calculée qu'une seule fois.

Corrigé

```

1 def maximum_dpr(liste):
2     if len(liste) == 1:
3         return liste[0]
4     milieu = len(liste) // 2
5     max_gauche = maximum_dpr(liste[:milieu])
6     max_droite = maximum_dpr(liste[milieu:])
7     return max(max_gauche, max_droite)

```

Le cas de base est une liste réduite à un seul élément : son maximum est cet élément lui-même, sans appel récur­sif. On suppose la liste non vide en entrée (une liste vide n'a pas de maximum).

Corrigé

```

1 def compter_feuilles(arbre):
2     if arbre is None:
3         return 0
4     if arbre.gauche is None and arbre.droite is None:
5         return 1
6     return compter_feuilles(arbre.gauche) + compter_feuilles(arbre.droite)

```

Cas de base : l'arbre vide n'a aucune feuille. Un nœud sans aucun enfant est lui-même une feuille (on renvoie 1, sans appel récur­sif supplémentaire). Sinon, on additionne le nombre de feuilles des deux sous-arbres.

Corrigé

```

1 abr = None
2 for v in (7, 3, 9, 1, 5):
3     abr = inserer(abr, v)
4
5 print(rechercher(abr, 5)) # True
6 print(rechercher(abr, 4)) # False

```

L'arbre construit a pour racine 7, sous-arbre gauche enraciné en 3 (lui-même avec 1 à gauche et 5 à droite), et sous-arbre droit réduit à 9. La recherche de 5 descend $7 \rightarrow 3 \rightarrow 5$ (trouvé) ; celle de 4 descend $7 \rightarrow 3 \rightarrow 5 \rightarrow$ sous-arbre gauche vide (non trouvé).

Corrigé

```
1 trajet = chemin(metro, "A", "E")
2 print(trajet) # ['A', 'B', 'C', 'E']
```

Le BFS depuis "A" atteint "B" en premier (distance 1), puis "C" et "D" (distance 2, "C" découvert avant "D" car il apparaît en premier dans la liste des voisins de "B"), puis "E" (distance 3, découvert depuis "C"). Le trajet reconstruit est $A \rightarrow B \rightarrow C \rightarrow E$: il compte 4 stations (départ et arrivée compris).

Corrigé

```
1 def fib_memo(n, memo=None):
2     if memo is None:
3         memo = {0: 0, 1: 1}
4     if n in memo:
5         return memo[n]
6     memo[n] = fib_memo(n - 1, memo) + fib_memo(n - 2, memo)
7     return memo[n]
```

Comme `rendu_memo`, les cas de base ($n = 0$ et $n = 1$) sont préchargés dans `memo`. Sans mémorisation, le nombre d'appels récursifs pour calculer F_{30} dépasse le million ; avec mémorisation, chaque valeur $F_0, \dots, F_{30}$ n'est calculée qu'une seule fois.

Corrigé

```
1 def maximum_dpr(liste):
2     if len(liste) == 1:
3         return liste[0]
4     milieu = len(liste) // 2
5     max_gauche = maximum_dpr(liste[:milieu])
6     max_droite = maximum_dpr(liste[milieu:])
7     return max(max_gauche, max_droite)
```

Le cas de base est une liste réduite à un seul élément : son maximum est cet élément lui-même, sans appel récursif. On suppose la liste non vide en entrée (une liste vide n'a pas de maximum).

Corrigé

```
1 def compter_feuilles(arbre):
2     if arbre is None:
3         return 0
4     if arbre.gauche is None and arbre.droite is None:
5         return 1
6     return compter_feuilles(arbre.gauche) + compter_feuilles(arbre.droite)
```

Cas de base : l'arbre vide n'a aucune feuille. Un nœud sans aucun enfant est lui-même une feuille (on renvoie 1, sans appel récursif supplémentaire). Sinon, on additionne le nombre de feuilles des deux sous-arbres.

Corrigé

```

1 abr = None
2 for v in (7, 3, 9, 1, 5):
3     abr = inserer(abr, v)
4
5 print(rechercher(abr, 5)) # True
6 print(rechercher(abr, 4)) # False

```

L'arbre construit a pour racine 7, sous-arbre gauche enraciné en 3 (lui-même avec 1 à gauche et 5 à droite), et sous-arbre droit réduit à 9. La recherche de 5 descend $7 \rightarrow 3 \rightarrow 5$ (trouvé) ; celle de 4 descend $7 \rightarrow 3 \rightarrow 5 \rightarrow$ sous-arbre gauche vide (non trouvé).

Corrigé

```

1 trajet = chemin(metro, "A", "E")
2 print(trajet) # ['A', 'B', 'C', 'E']

```

Le BFS depuis "A" atteint "B" en premier (distance 1), puis "C" et "D" (distance 2, "C" découvert avant "D" car il apparaît en premier dans la liste des voisins de "B"), puis "E" (distance 3, découvert depuis "C"). Le trajet reconstruit est $A \rightarrow B \rightarrow C \rightarrow E$: il compte 4 stations (départ et arrivée compris).

Corrigé

```

1 def fib_memo(n, memo=None):
2     if memo is None:
3         memo = {0: 0, 1: 1}
4     if n in memo:
5         return memo[n]
6     memo[n] = fib_memo(n - 1, memo) + fib_memo(n - 2, memo)
7     return memo[n]

```

Comme `rendu_memo`, les cas de base ($n = 0$ et $n = 1$) sont préchargés dans `memo`. Sans mémorisation, le nombre d'appels récursifs pour calculer F_{30} dépasse le million ; avec mémorisation, chaque valeur $F_0, \dots, F_{30}$ n'est calculée qu'une seule fois.

Corrigé

```

1 def maximum_dpr(liste):
2     if len(liste) == 1:
3         return liste[0]
4     milieu = len(liste) // 2
5     max_gauche = maximum_dpr(liste[:milieu])
6     max_droite = maximum_dpr(liste[milieu:])
7     return max(max_gauche, max_droite)

```

Le cas de base est une liste réduite à un seul élément : son maximum est cet élément lui-même, sans appel récursif. On suppose la liste non vide en entrée (une liste vide n'a pas de maximum).

Corrigé

```

1 def compter_feuilles(arbre):
2     if arbre is None:
3         return 0
4     if arbre.gauche is None and arbre.droite is None:
5         return 1
6     return compter_feuilles(arbre.gauche) + compter_feuilles(arbre.droite)

```

Cas de base : l'arbre vide n'a aucune feuille. Un nœud sans aucun enfant est lui-même une feuille (on renvoie 1, sans appel récursif supplémentaire). Sinon, on additionne le nombre de feuilles des deux sous-arbres.

Corrigé

```
1 abr = None
2 for v in (7, 3, 9, 1, 5):
3     abr = inserer(abr, v)
4
5 print(rechercher(abr, 5)) # True
6 print(rechercher(abr, 4)) # False
```

L'arbre construit a pour racine 7, sous-arbre gauche enraciné en 3 (lui-même avec 1 à gauche et 5 à droite), et sous-arbre droit réduit à 9. La recherche de 5 descend $7 \rightarrow 3 \rightarrow 5$ (trouvé) ; celle de 4 descend $7 \rightarrow 3 \rightarrow 5 \rightarrow$ sous-arbre gauche vide (non trouvé).

Corrigé

```
1 trajet = chemin(metro, "A", "E")
2 print(trajet) # ['A', 'B', 'C', 'E']
```

Le BFS depuis "A" atteint "B" en premier (distance 1), puis "C" et "D" (distance 2, "C" découvert avant "D" car il apparaît en premier dans la liste des voisins de "B"), puis "E" (distance 3, découvert depuis "C"). Le trajet reconstruit est $A \rightarrow B \rightarrow C \rightarrow E$: il compte 4 stations (départ et arrivée compris).

Corrigé

```
1 def fib_memo(n, memo=None):
2     if memo is None:
3         memo = {0: 0, 1: 1}
4     if n in memo:
5         return memo[n]
6     memo[n] = fib_memo(n - 1, memo) + fib_memo(n - 2, memo)
7     return memo[n]
```

Comme `rendu_memo`, les cas de base ($n = 0$ et $n = 1$) sont préchargés dans `memo`. Sans mémoïsation, le nombre d'appels récursifs pour calculer F_{30} dépasse le million ; avec mémoïsation, chaque valeur $F_0, \dots, F_{30}$ n'est calculée qu'une seule fois.

Corrigé

```
1 def maximum_dpr(liste):
2     if len(liste) == 1:
3         return liste[0]
4     milieu = len(liste) // 2
5     max_gauche = maximum_dpr(liste[:milieu])
6     max_droite = maximum_dpr(liste[milieu:])
7     return max(max_gauche, max_droite)
```

Le cas de base est une liste réduite à un seul élément : son maximum est cet élément lui-même, sans appel récursif. On suppose la liste non vide en entrée (une liste vide n'a pas de maximum).

Corrigé

```

1 def compter_feuilles(arbre):
2     if arbre is None:
3         return 0
4     if arbre.gauche is None and arbre.droite is None:
5         return 1
6     return compter_feuilles(arbre.gauche) + compter_feuilles(arbre.droite)

```

Cas de base : l'arbre vide n'a aucune feuille. Un nœud sans aucun enfant est lui-même une feuille (on renvoie 1, sans appel récursif supplémentaire). Sinon, on additionne le nombre de feuilles des deux sous-arbres.

Corrigé

```

1 abr = None
2 for v in (7, 3, 9, 1, 5):
3     abr = inserer(abr, v)
4
5 print(rechercher(abr, 5)) # True
6 print(rechercher(abr, 4)) # False

```

L'arbre construit a pour racine 7, sous-arbre gauche enraciné en 3 (lui-même avec 1 à gauche et 5 à droite), et sous-arbre droit réduit à 9. La recherche de 5 descend $7 \rightarrow 3 \rightarrow 5$ (trouvé) ; celle de 4 descend $7 \rightarrow 3 \rightarrow 5 \rightarrow$ sous-arbre gauche vide (non trouvé).

Corrigé

```

1 trajet = chemin(metro, "A", "E")
2 print(trajet) # ['A', 'B', 'C', 'E']

```

Le BFS depuis "A" atteint "B" en premier (distance 1), puis "C" et "D" (distance 2, "C" découvert avant "D" car il apparaît en premier dans la liste des voisins de "B"), puis "E" (distance 3, découvert depuis "C"). Le trajet reconstruit est $A \rightarrow B \rightarrow C \rightarrow E$: il compte 4 stations (départ et arrivée compris).

Corrigé

```

1 def fib_memo(n, memo=None):
2     if memo is None:
3         memo = {0: 0, 1: 1}
4     if n in memo:
5         return memo[n]
6     memo[n] = fib_memo(n - 1, memo) + fib_memo(n - 2, memo)
7     return memo[n]

```

Comme `rendu_memo`, les cas de base ($n = 0$ et $n = 1$) sont préchargés dans `memo`. Sans mémorisation, le nombre d'appels récursifs pour calculer F_{30} dépasse le million ; avec mémorisation, chaque valeur $F_0, \dots, F_{30}$ n'est calculée qu'une seule fois.

Corrigé

```

1 def maximum_dpr(liste):
2     if len(liste) == 1:
3         return liste[0]
4     milieu = len(liste) // 2
5     max_gauche = maximum_dpr(liste[:milieu])
6     max_droite = maximum_dpr(liste[milieu:])
7     return max(max_gauche, max_droite)

```

Le cas de base est une liste réduite à un seul élément : son maximum est cet élément lui-même, sans appel récursif. On suppose la liste non vide en entrée (une liste vide n'a pas de maximum).

Corrigé

```
1 def compter_feuilles(arbre):
2     if arbre is None:
3         return 0
4     if arbre.gauche is None and arbre.droite is None:
5         return 1
6     return compter_feuilles(arbre.gauche) + compter_feuilles(arbre.droite)
```

Cas de base : l'arbre vide n'a aucune feuille. Un nœud sans aucun enfant est lui-même une feuille (on renvoie 1, sans appel récursif supplémentaire). Sinon, on additionne le nombre de feuilles des deux sous-arbres.

Corrigé

```
1 abr = None
2 for v in (7, 3, 9, 1, 5):
3     abr = inserer(abr, v)
4
5 print(rechercher(abr, 5)) # True
6 print(rechercher(abr, 4)) # False
```

L'arbre construit a pour racine 7, sous-arbre gauche enraciné en 3 (lui-même avec 1 à gauche et 5 à droite), et sous-arbre droit réduit à 9. La recherche de 5 descend $7 \rightarrow 3 \rightarrow 5$ (trouvé) ; celle de 4 descend $7 \rightarrow 3 \rightarrow 5 \rightarrow$ sous-arbre gauche vide (non trouvé).

Corrigé

```
1 trajet = chemin(metro, "A", "E")
2 print(trajet) # ['A', 'B', 'C', 'E']
```

Le BFS depuis "A" atteint "B" en premier (distance 1), puis "C" et "D" (distance 2, "C" découvert avant "D" car il apparaît en premier dans la liste des voisins de "B"), puis "E" (distance 3, découvert depuis "C"). Le trajet reconstruit est $A \rightarrow B \rightarrow C \rightarrow E$: il compte 4 stations (départ et arrivée compris).

Corrigé

```
1 def fib_memo(n, memo=None):
2     if memo is None:
3         memo = {0: 0, 1: 1}
4     if n in memo:
5         return memo[n]
6     memo[n] = fib_memo(n - 1, memo) + fib_memo(n - 2, memo)
7     return memo[n]
```

Comme `rendu_memo`, les cas de base ($n = 0$ et $n = 1$) sont préchargés dans `memo`. Sans mémorisation, le nombre d'appels récursifs pour calculer F_{30} dépasse le million ; avec mémorisation, chaque valeur $F_0, \dots, F_{30}$ n'est calculée qu'une seule fois.

Corrigé

```
1 def maximum_dpr(liste):
2     if len(liste) == 1:
```

```

3     return liste[0]
4     milieu = len(liste) // 2
5     max_gauche = maximum_dpr(liste[:milieu])
6     max_droite = maximum_dpr(liste[milieu:])
7     return max(max_gauche, max_droite)

```

Le cas de base est une liste réduite à un seul élément : son maximum est cet élément lui-même, sans appel récursif. On suppose la liste non vide en entrée (une liste vide n'a pas de maximum).

Corrigé

```

1 def compter_feuilles(arbre):
2     if arbre is None:
3         return 0
4     if arbre.gauche is None and arbre.droite is None:
5         return 1
6     return compter_feuilles(arbre.gauche) + compter_feuilles(arbre.droite)

```

Cas de base : l'arbre vide n'a aucune feuille. Un nœud sans aucun enfant est lui-même une feuille (on renvoie 1, sans appel récursif supplémentaire). Sinon, on additionne le nombre de feuilles des deux sous-arbres.

Corrigé

```

1 abr = None
2 for v in (7, 3, 9, 1, 5):
3     abr = inserer(abr, v)
4
5 print(rechercher(abr, 5)) # True
6 print(rechercher(abr, 4)) # False

```

L'arbre construit a pour racine 7, sous-arbre gauche enraciné en 3 (lui-même avec 1 à gauche et 5 à droite), et sous-arbre droit réduit à 9. La recherche de 5 descend $7 \rightarrow 3 \rightarrow 5$ (trouvé) ; celle de 4 descend $7 \rightarrow 3 \rightarrow 5 \rightarrow$ sous-arbre gauche vide (non trouvé).

Corrigé

```

1 trajet = chemin(metro, "A", "E")
2 print(trajet) # ['A', 'B', 'C', 'E']

```

Le BFS depuis "A" atteint "B" en premier (distance 1), puis "C" et "D" (distance 2, "C" découvert avant "D" car il apparaît en premier dans la liste des voisins de "B"), puis "E" (distance 3, découvert depuis "C"). Le trajet reconstruit est $A \rightarrow B \rightarrow C \rightarrow E$: il compte 4 stations (départ et arrivée compris).

Corrigé

```

1 def fib_memo(n, memo=None):
2     if memo is None:
3         memo = {0: 0, 1: 1}
4     if n in memo:
5         return memo[n]
6     memo[n] = fib_memo(n - 1, memo) + fib_memo(n - 2, memo)
7     return memo[n]

```

Comme rendu_memo, les cas de base ($n = 0$ et $n = 1$) sont préchargés dans memo. Sans mémorisation, le nombre d'appels récursifs pour calculer F_{30} dépasse le million ; avec mémorisation, chaque valeur $F_0, \dots, F_{30}$ n'est calculée qu'une seule fois.

Corrigé

```

1 def maximum_dpr(liste):
2     if len(liste) == 1:
3         return liste[0]
4     milieu = len(liste) // 2
5     max_gauche = maximum_dpr(liste[:milieu])
6     max_droite = maximum_dpr(liste[milieu:])
7     return max(max_gauche, max_droite)

```

Le cas de base est une liste réduite à un seul élément : son maximum est cet élément lui-même, sans appel récursif. On suppose la liste non vide en entrée (une liste vide n'a pas de maximum).

Corrigé

```

1 def compter_feuilles(arbre):
2     if arbre is None:
3         return 0
4     if arbre.gauche is None and arbre.droite is None:
5         return 1
6     return compter_feuilles(arbre.gauche) + compter_feuilles(arbre.droite)

```

Cas de base : l'arbre vide n'a aucune feuille. Un nœud sans aucun enfant est lui-même une feuille (on renvoie 1, sans appel récursif supplémentaire). Sinon, on additionne le nombre de feuilles des deux sous-arbres.

Corrigé

```

1 abr = None
2 for v in (7, 3, 9, 1, 5):
3     abr = inserer(abr, v)
4
5 print(rechercher(abr, 5)) # True
6 print(rechercher(abr, 4)) # False

```

L'arbre construit a pour racine 7, sous-arbre gauche enraciné en 3 (lui-même avec 1 à gauche et 5 à droite), et sous-arbre droit réduit à 9. La recherche de 5 descend $7 \rightarrow 3 \rightarrow 5$ (trouvé) ; celle de 4 descend $7 \rightarrow 3 \rightarrow 5 \rightarrow$ sous-arbre gauche vide (non trouvé).

Corrigé

```

1 trajet = chemin(metro, "A", "E")
2 print(trajet) # ['A', 'B', 'C', 'E']

```

Le BFS depuis "A" atteint "B" en premier (distance 1), puis "C" et "D" (distance 2, "C" découvert avant "D" car il apparaît en premier dans la liste des voisins de "B"), puis "E" (distance 3, découvert depuis "C"). Le trajet reconstruit est $A \rightarrow B \rightarrow C \rightarrow E$: il compte 4 stations (départ et arrivée compris).

Corrigé

```

1 def fib_memo(n, memo=None):
2     if memo is None:
3         memo = {0: 0, 1: 1}
4     if n in memo:
5         return memo[n]
6     memo[n] = fib_memo(n - 1, memo) + fib_memo(n - 2, memo)
7     return memo[n]

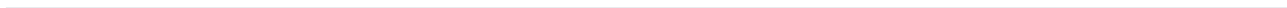
```

Comme `rendu_memo`, les cas de base ($n = 0$ et $n = 1$) sont préchargés dans `memo`. Sans mémorisation, le nombre d'appels récursifs pour calculer F_{30} dépasse le million ; avec mémorisation, chaque valeur $F_0, \dots, F_{30}$ n'est calculée qu'une seule fois.

Corrigé

```
1 def maximum_dpr(liste):
2     if len(liste) == 1:
3         return liste[0]
4     milieu = len(liste) // 2
5     max_gauche = maximum_dpr(liste[:milieu])
6     max_droite = maximum_dpr(liste[milieu:])
7     return max(max_gauche, max_droite)
```

Le cas de base est une liste réduite à un seul élément : son maximum est cet élément lui-même, sans appel récursif. On suppose la liste non vide en entrée (une liste vide n'a pas de maximum).



2 Architectures matérielles, systèmes d'exploitation et réseaux

2.1 Le système sur puce

DÉFINITION 1

Un **système sur puce** (*System on Chip*, SoC) intègre sur un unique circuit intégré plusieurs composants qui, historiquement, occupaient des circuits séparés : un ou plusieurs **processeurs**, de la **mémoire**, des **interfaces** de communication (Wi-Fi, Bluetooth, USB...), et parfois des **accélérateurs** spécialisés (traitement graphique, calcul pour l'intelligence artificielle, traitement du signal audio).

◆ **PROPRIÉTÉ Avantages de l'intégration** Intégrer plusieurs composants sur une même puce présente trois avantages principaux :

- ▶ **compacité** : un seul circuit remplace plusieurs cartes séparées, ce qui réduit la taille du produit final ;
- ▶ **consommation énergétique réduite** : les distances entre composants, très courtes sur une même puce, réduisent les pertes électriques liées à la communication entre eux ;
- ▶ **coût de fabrication** : produire une seule puce en grande série revient souvent moins cher qu'assembler plusieurs composants distincts.

⚠ ERREUR FRÉQUENTE

Croire que l'intégration n'a que des avantages. L'inconvénient principal d'un système sur puce est la perte de **modularité** : si un seul composant est défaillant ou devient obsolète (par exemple la mémoire), c'est toute la puce qu'il faut remplacer, alors que des composants séparés auraient pu être changés individuellement.

✦ POUR ALLER PLUS LOIN

Certains systèmes sur puce embarquent également des **FPGA** (circuits reconfigurables) pour adapter certains traitements sans changer de puce physique, ou des zones mémoire dédiées à la sécurité (*secure enclave*) isolées du reste du système.

2.2 Processus : création, ordonnancement, interblocage

DÉFINITION 1

Un **processus** est un programme en cours d'exécution, avec son propre espace mémoire et son propre état (registres, pile d'appels). La **création** d'un processus consiste, pour le système d'exploitation, à lui allouer de la mémoire, à charger le code du programme, et à l'inscrire dans la liste des processus à exécuter.

DÉFINITION 2

Un seul processeur ne peut exécuter qu'une instruction à la fois : quand plusieurs processus doivent s'exécuter « en même temps », l'**ordonnanceur** du système d'exploitation leur alloue le processeur à tour de rôle, pendant de courtes tranches de temps, donnant l'illusion d'une exécution simultanée.

Exemple. La politique **tourniquet** (*round-robin*) alloue à chaque processus une tranche de temps fixe, dans l'ordre, en boucle :

```
1 def tourniquet(processus, tranche, duree_totale):
2     ordre_execution = []
3     file = list(processus)
4     temps_restant = dict(duree_totale)
5     while file:
6         p = file.pop(0)
7         execute = min(tranche, temps_restant[p])
8         ordre_execution.append((p, execute))
9         temps_restant[p] -= execute
10        if temps_restant[p] > 0:
11            file.append(p)
12    return ordre_execution
```

⚠ ERREUR FRÉQUENTE

Confondre concurrence et parallélisme. Sur un processeur à un seul cœur, plusieurs processus s'exécutent en **concurrence** (à tour de rôle, jamais réellement simultanément) : c'est l'ordonnancement rapide qui donne l'illusion du parallélisme. Sur un processeur multi-cœurs, plusieurs processus peuvent en revanche s'exécuter en **parallélisme** réel, un par cœur.

DÉFINITION 3

Un **interblocage** (*deadlock*) survient lorsque plusieurs processus attendent chacun une ressource retenue par un autre processus du même groupe, formant un cycle d'attente dont aucun ne peut sortir seul.

Exemple. Deux processus, chacun retenant une ressource et attendant celle de l'autre :

```
1 # Processus P1 : retient R1, attend R2
2 # Processus P2 : retient R2, attend R1
3 attente = {"P1": "R2", "P2": "R1"}
4 retenue_par = {"R1": "P1", "R2": "P2"}
```

◆ **PROPRIÉTÉ Mise en évidence par un graphe d'attente** Un interblocage se traduit par un **cycle** dans le graphe orienté où chaque processus pointe vers la ressource qu'il attend, et chaque ressource pointe vers le processus qui la retient. On retrouve ainsi la détection de cycle vue dans le chapitre *Algorithmique*, appliquée cette fois à un graphe de dépendances entre processus et ressources.

2.3 Protocoles de routage

DÉFINITION 1

Un réseau se modélise par un **graphe** dont les sommets sont des routeurs et les arêtes des liaisons directes. **Router** un paquet consiste à choisir, à chaque routeur, la liaison à emprunter pour progresser vers la destination. Le **protocole de routage** détermine comment cette route est calculée.

◆ **PROPRIÉTÉ RIP : minimiser le nombre de sauts** Le protocole **RIP** (*Routing Information Protocol*) choisit la route qui minimise le **nombre de sauts** (de routeurs traversés), sans tenir compte de la capacité ou de la congestion des liaisons. C'est exactement ce que calcule un parcours en largeur (BFS) sur le graphe du réseau.

◆ **PROPRIÉTÉ OSPF : minimiser un coût** Le protocole **OSPF** (*Open Shortest Path First*) associe un **coût** à chaque liaison (reflétant par exemple sa bande passante) et choisit la route qui minimise la somme des coûts, pas le nombre de sauts. Une route avec plus de sauts mais de meilleure capacité peut ainsi être préférée à une route plus courte mais plus lente.

Exemple. Sur ce réseau, la route au moins de sauts (RIP) et la route au moindre coût (OSPF) diffèrent :

```
1 reseau_couts = {
2   "A": {"B": 1, "C": 5},
3   "B": {"A": 1, "D": 1},
4   "C": {"A": 5, "D": 1},
5   "D": {"B": 1, "C": 1},
6 }
```

De A vers D : la route A-C-D compte 2 sauts (comme A-B-D), mais coûte $5 + 1 = 6$, alors que A-B-D compte aussi 2 sauts pour un coût de $1 + 1 = 2$: ici les deux critères coïncident. En revanche, si l'on ajoute une liaison directe coûteuse A-D de coût 3, RIP choisirait ce trajet à **un seul saut**, alors qu'OSPF préférerait toujours A-B-D (coût $2 < 3$).

⚠ ERREUR FRÉQUENTE

Croire qu'un protocole de routage choisit toujours le chemin « le plus court » au sens intuitif. Ce que minimise le protocole dépend entièrement de sa **métrique** : RIP minimise le nombre de sauts (route A-D directe préférée dans l'exemple), OSPF minimise un coût cumulé (route A-B-D préférée si son coût total est plus faible), même si cette dernière traverse davantage de routeurs.

2.4 Chiffrement symétrique et asymétrique

DÉFINITION 1

Le **chiffrement symétrique** utilise une **unique clé secrète**, partagée à l'avance entre l'émetteur et le destinataire, pour chiffrer et déchiffrer un message. Il est rapide, mais suppose que les deux parties aient pu échanger la clé de façon sûre au préalable.

Exemple. Un chiffrement symétrique très simplifié par **XOR** avec une clé répétée (principe illustratif, non utilisé tel quel en pratique) :

```
1 def chiffrer_xor(message, cle):
2     return bytes(o ^ cle[i % len(cle)] for i, o in enumerate(message))
3
4 def déchiffrer_xor(chiffre, cle):
5     return chiffrer_xor(chiffre, cle) # XOR est sa propre inverse
```

DÉFINITION 2

Le **chiffrement asymétrique** utilise une **paire de clés** liées mathématiquement : une **clé publique**, diffusée librement, et une **clé privée**, gardée secrète. Ce qui est chiffré avec la clé publique ne peut être déchiffré qu'avec la clé privée correspondante. Il est plus lent que le chiffrement symétrique, mais ne nécessite **aucun** échange préalable de secret.

◆ **PROPRIÉTÉ Sécuriser HTTPS : le meilleur des deux mondes** Une connexion **HTTPS** combine les deux : au début de la connexion, le client et le serveur utilisent un protocole **asymétrique** pour échanger, en toute sécurité sur un réseau public, une **clé symétrique** temporaire (dite *clé de session*). Toute la suite de l'échange (les pages web, les données du formulaire...) est ensuite chiffrée avec cette clé symétrique, beaucoup plus rapide à utiliser pour de gros volumes de données.

⚠ ERREUR FRÉQUENTE

Croire que HTTPS chiffre tout avec le protocole asymétrique de bout en bout. Ce serait beaucoup trop lent pour des échanges volumineux : le chiffrement asymétrique sert uniquement à échanger la clé symétrique de session au tout début de la connexion, puis c'est le chiffrement symétrique, bien plus rapide, qui protège le reste des données échangées.

+ POUR ALLER PLUS LOIN

Le chiffrement asymétrique repose sur des problèmes mathématiques réputés difficiles à inverser sans la clé privée (par exemple, la factorisation de grands nombres entiers pour l'algorithme RSA). Ce fondement mathématique est étudié plus en détail dans le programme de mathématiques expertes (arithmétique).

2.5 Le système sur puce

DÉFINITION 1

Un **système sur puce** (*System on Chip*, SoC) intègre sur un unique circuit intégré plusieurs composants qui, historiquement, occupaient des circuits séparés : un ou plusieurs **processeurs**, de la **mémoire**, des **interfaces** de communication (Wi-Fi, Bluetooth, USB...), et parfois des **accélérateurs** spécialisés (traitement graphique, calcul pour l'intelligence artificielle, traitement du signal audio).

◆ **PROPRIÉTÉ Avantages de l'intégration** Intégrer plusieurs composants sur une même puce présente trois avantages principaux :

- ▶ **compacité** : un seul circuit remplace plusieurs cartes séparées, ce qui réduit la taille du produit final ;
- ▶ **consommation énergétique réduite** : les distances entre composants, très courtes sur une même puce, réduisent les pertes électriques liées à la communication entre eux ;
- ▶ **coût de fabrication** : produire une seule puce en grande série revient souvent moins cher qu'assembler plusieurs composants distincts.

⚠ ERREUR FRÉQUENTE

Croire que l'intégration n'a que des avantages. L'inconvénient principal d'un système sur puce est la perte de **modularité** : si un seul composant est défaillant ou devient obsolète (par exemple la mémoire), c'est toute la puce qu'il faut remplacer, alors que des composants séparés auraient pu être changés individuellement.

+ POUR ALLER PLUS LOIN

Certains systèmes sur puce embarquent également des **FPGA** (circuits reconfigurables) pour adapter certains traitements sans changer de puce physique, ou des zones mémoire dédiées à la sécurité (*secure enclave*) isolées du reste du système.

2.6 Processus : création, ordonnancement, interblocage

DÉFINITION 1

Un **processus** est un programme en cours d'exécution, avec son propre espace mémoire et son propre état (registres, pile d'appels). La **création** d'un processus consiste, pour le système d'exploitation, à lui allouer de la mémoire, à charger le code du programme, et à l'inscrire dans la liste des processus à exécuter.

DÉFINITION 2

Un seul processeur ne peut exécuter qu'une instruction à la fois : quand plusieurs processus doivent s'exécuter « en même temps », l'**ordonnanceur** du système d'exploitation leur alloue

le processeur à tour de rôle, pendant de courtes tranches de temps, donnant l'illusion d'une exécution simultanée.

Exemple. La politique **tourniquet** (*round-robin*) alloue à chaque processus une tranche de temps fixe, dans l'ordre, en boucle :

```
1 def tourniquet(processus, tranche, duree_totale):
2     ordre_execution = []
3     file = list(processus)
4     temps_restant = dict(duree_totale)
5     while file:
6         p = file.pop(0)
7         execute = min(tranche, temps_restant[p])
8         ordre_execution.append((p, execute))
9         temps_restant[p] -= execute
10        if temps_restant[p] > 0:
11            file.append(p)
12    return ordre_execution
```

⚠ ERREUR FRÉQUENTE

Confondre concurrence et parallélisme. Sur un processeur à un seul cœur, plusieurs processus s'exécutent en **concurrence** (à tour de rôle, jamais réellement simultanément) : c'est l'ordonnancement rapide qui donne l'illusion du parallélisme. Sur un processeur multi-cœurs, plusieurs processus peuvent en revanche s'exécuter en **parallélisme** réel, un par cœur.

DÉFINITION 3

Un **interblocage** (*deadlock*) survient lorsque plusieurs processus attendent chacun une ressource retenue par un autre processus du même groupe, formant un cycle d'attente dont aucun ne peut sortir seul.

Exemple. Deux processus, chacun retenant une ressource et attendant celle de l'autre :

```
1 # Processus P1 : retient R1, attend R2
2 # Processus P2 : retient R2, attend R1
3 attente = {"P1": "R2", "P2": "R1"}
4 retenue_par = {"R1": "P1", "R2": "P2"}
```

♦ **PROPRIÉTÉ Mise en évidence par un graphe d'attente** Un interblocage se traduit par un **cycle** dans le graphe orienté où chaque processus pointe vers la ressource qu'il attend, et chaque ressource pointe vers le processus qui la retient. On retrouve ainsi la détection de cycle vue dans le chapitre *Algorithmique*, appliquée cette fois à un graphe de dépendances entre processus et ressources.

2.7 Protocoles de routage

DÉFINITION 1

Un réseau se modélise par un **graphe** dont les sommets sont des routeurs et les arêtes des liaisons directes. **Router** un paquet consiste à choisir, à chaque routeur, la liaison à emprunter pour progresser vers la destination. Le **protocole de routage** détermine comment cette route est calculée.

◆ **PROPRIÉTÉ RIP : minimiser le nombre de sauts** Le protocole **RIP** (*Routing Information Protocol*) choisit la route qui minimise le **nombre de sauts** (de routeurs traversés), sans tenir compte de la capacité ou de la congestion des liaisons. C'est exactement ce que calcule un parcours en largeur (BFS) sur le graphe du réseau.

◆ **PROPRIÉTÉ OSPF : minimiser un coût** Le protocole **OSPF** (*Open Shortest Path First*) associe un **coût** à chaque liaison (reflétant par exemple sa bande passante) et choisit la route qui minimise la somme des coûts, pas le nombre de sauts. Une route avec plus de sauts mais de meilleure capacité peut ainsi être préférée à une route plus courte mais plus lente.

Exemple. Sur ce réseau, la route au moins de sauts (RIP) et la route au moindre coût (OSPF) diffèrent :

```
1 reseau_couts = {
2   "A": {"B": 1, "C": 5},
3   "B": {"A": 1, "D": 1},
4   "C": {"A": 5, "D": 1},
5   "D": {"B": 1, "C": 1},
6 }
```

De A vers D : la route A-C-D compte 2 sauts (comme A-B-D), mais coûte $5 + 1 = 6$, alors que A-B-D compte aussi 2 sauts pour un coût de $1 + 1 = 2$: ici les deux critères coïncident. En revanche, si l'on ajoute une liaison directe coûteuse A-D de coût 3, RIP choisirait ce trajet à **un seul saut**, alors qu'OSPF préférerait toujours A-B-D (coût $2 < 3$).

⚠ ERREUR FRÉQUENTE

Croire qu'un protocole de routage choisit toujours le chemin « le plus court » au sens intuitif. Ce que minimise le protocole dépend entièrement de sa **métrique** : RIP minimise le nombre de sauts (route A-D directe préférée dans l'exemple), OSPF minimise un coût cumulé (route A-B-D préférée si son coût total est plus faible), même si cette dernière traverse davantage de routeurs.

2.8 Chiffrement symétrique et asymétrique

DÉFINITION 1

Le **chiffrement symétrique** utilise une **unique clé secrète**, partagée à l'avance entre l'émetteur et le destinataire, pour chiffrer et déchiffrer un message. Il est rapide, mais suppose que les deux parties aient pu échanger la clé de façon sûre au préalable.

Exemple. Un chiffrement symétrique très simplifié par **XOR** avec une clé répétée (principe illustratif, non utilisé tel quel en pratique) :

```
1 def chiffrer_xor(message, cle):
2     return bytes(o ^ cle[i % len(cle)] for i, o in enumerate(message))
3
4 def dechiffrer_xor(chiffre, cle):
5     return chiffrer_xor(chiffre, cle) # XOR est sa propre inverse
```

DÉFINITION 2

Le **chiffrement asymétrique** utilise une **paire de clés** liées mathématiquement : une **clé publique**, diffusée librement, et une **clé privée**, gardée secrète. Ce qui est chiffré avec la clé publique ne peut être déchiffré qu'avec la clé privée correspondante. Il est plus lent que le chiffrement symétrique, mais ne nécessite **aucun** échange préalable de secret.

◆ **PROPRIÉTÉ Sécuriser HTTPS : le meilleur des deux mondes** Une connexion **HTTPS** combine les deux : au début de la connexion, le client et le serveur utilisent un protocole **asymétrique** pour échanger, en toute sécurité sur un réseau public, une **clé symétrique** temporaire (dite *clé de session*). Toute la suite de l'échange (les pages web, les données du formulaire...) est ensuite chiffrée avec cette clé symétrique, beaucoup plus rapide à utiliser pour de gros volumes de données.

⚠ ERREUR FRÉQUENTE

Croire que HTTPS chiffre tout avec le protocole asymétrique de bout en bout. Ce serait beaucoup trop lent pour des échanges volumineux : le chiffrement asymétrique sert uniquement à échanger la clé symétrique de session au tout début de la connexion, puis c'est le chiffrement symétrique, bien plus rapide, qui protège le reste des données échangées.

+ POUR ALLER PLUS LOIN

Le chiffrement asymétrique repose sur des problèmes mathématiques réputés difficiles à inverser sans la clé privée (par exemple, la factorisation de grands nombres entiers pour l'algorithme RSA). Ce fondement mathématique est étudié plus en détail dans le programme de mathématiques expertes (arithmétique).

2.9 Le système sur puce

DÉFINITION 1

Un **système sur puce** (*System on Chip*, SoC) intègre sur un unique circuit intégré plusieurs composants qui, historiquement, occupaient des circuits séparés : un ou plusieurs **processeurs**, de la **mémoire**, des **interfaces** de communication (Wi-Fi, Bluetooth, USB...), et parfois des **accélérateurs** spécialisés (traitement graphique, calcul pour l'intelligence artificielle, traitement du signal audio).

◆ **PROPRIÉTÉ Avantages de l'intégration** Intégrer plusieurs composants sur une même puce présente trois avantages principaux :

- ▶ **compacité** : un seul circuit remplace plusieurs cartes séparées, ce qui réduit la taille du produit final ;
- ▶ **consommation énergétique réduite** : les distances entre composants, très courtes sur une même puce, réduisent les pertes électriques liées à la communication entre eux ;
- ▶ **coût de fabrication** : produire une seule puce en grande série revient souvent moins cher qu'assembler plusieurs composants distincts.

⚠ ERREUR FRÉQUENTE

Croire que l'intégration n'a que des avantages. L'inconvénient principal d'un système sur puce est la perte de **modularité** : si un seul composant est défaillant ou devient obsolète (par exemple la mémoire), c'est toute la puce qu'il faut remplacer, alors que des composants séparés auraient pu être changés individuellement.

✦ POUR ALLER PLUS LOIN

Certains systèmes sur puce embarquent également des **FPGA** (circuits reconfigurables) pour adapter certains traitements sans changer de puce physique, ou des zones mémoire dédiées à la sécurité (*secure enclave*) isolées du reste du système.

2.10 Processus : création, ordonnancement, interblocage

DÉFINITION 1

Un **processus** est un programme en cours d'exécution, avec son propre espace mémoire et son propre état (registres, pile d'appels). La **création** d'un processus consiste, pour le système d'exploitation, à lui allouer de la mémoire, à charger le code du programme, et à l'inscrire dans la liste des processus à exécuter.

DÉFINITION 2

Un seul processeur ne peut exécuter qu'une instruction à la fois : quand plusieurs processus doivent s'exécuter « en même temps », l'**ordonnanceur** du système d'exploitation leur alloue le processeur à tour de rôle, pendant de courtes tranches de temps, donnant l'illusion d'une exécution simultanée.

Exemple. La politique **tourniquet** (*round-robin*) alloue à chaque processus une tranche de temps fixe, dans l'ordre, en boucle :

```
1 def tourniquet(processus, tranche, duree_totale):
2     ordre_execution = []
3     file = list(processus)
4     temps_restant = dict(duree_totale)
5     while file:
6         p = file.pop(0)
7         execute = min(tranche, temps_restant[p])
8         ordre_execution.append((p, execute))
```

```

9     temps_restant[p] -= execute
10    if temps_restant[p] > 0:
11        file.append(p)
12    return ordre_execution

```

⚠ ERREUR FRÉQUENTE

Confondre concurrence et parallélisme. Sur un processeur à un seul cœur, plusieurs processus s'exécutent en **concurrence** (à tour de rôle, jamais réellement simultanément) : c'est l'ordonnancement rapide qui donne l'illusion du parallélisme. Sur un processeur multi-cœurs, plusieurs processus peuvent en revanche s'exécuter en **parallélisme** réel, un par cœur.

DÉFINITION 3

Un **interblocage** (*deadlock*) survient lorsque plusieurs processus attendent chacun une ressource retenue par un autre processus du même groupe, formant un cycle d'attente dont aucun ne peut sortir seul.

Exemple. Deux processus, chacun retenant une ressource et attendant celle de l'autre :

```

1 # Processus P1 : retient R1, attend R2
2 # Processus P2 : retient R2, attend R1
3 attente = {"P1": "R2", "P2": "R1"}
4 retenue_par = {"R1": "P1", "R2": "P2"}

```

◆ **PROPRIÉTÉ Mise en évidence par un graphe d'attente** Un interblocage se traduit par un **cycle** dans le graphe orienté où chaque processus pointe vers la ressource qu'il attend, et chaque ressource pointe vers le processus qui la retient. On retrouve ainsi la détection de cycle vue dans le chapitre *Algorithmique*, appliquée cette fois à un graphe de dépendances entre processus et ressources.

2.11 Protocoles de routage

DÉFINITION 1

Un réseau se modélise par un **graphe** dont les sommets sont des routeurs et les arêtes des liaisons directes. **Router** un paquet consiste à choisir, à chaque routeur, la liaison à emprunter pour progresser vers la destination. Le **protocole de routage** détermine comment cette route est calculée.

◆ **PROPRIÉTÉ RIP : minimiser le nombre de sauts** Le protocole **RIP** (*Routing Information Protocol*) choisit la route qui minimise le **nombre de sauts** (de routeurs traversés), sans tenir compte de la capacité ou de la congestion des liaisons. C'est exactement ce que calcule un parcours en largeur (BFS) sur le graphe du réseau.

◆ **PROPRIÉTÉ OSPF : minimiser un coût** Le protocole **OSPF** (*Open Shortest Path First*) associe un **coût** à chaque liaison (reflétant par exemple sa bande passante) et choisit la route qui minimise la somme des coûts, pas le nombre de sauts. Une route avec plus de sauts mais de meilleure capacité peut ainsi être préférée à une route plus courte mais plus lente.

Exemple. Sur ce réseau, la route au moins de sauts (RIP) et la route au moindre coût (OSPF) diffèrent :

```
1 reseau_couts = {
2     "A": {"B": 1, "C": 5},
3     "B": {"A": 1, "D": 1},
4     "C": {"A": 5, "D": 1},
5     "D": {"B": 1, "C": 1},
6 }
```

De A vers D : la route A-C-D compte 2 sauts (comme A-B-D), mais coûte $5 + 1 = 6$, alors que A-B-D compte aussi 2 sauts pour un coût de $1 + 1 = 2$: ici les deux critères coïncident. En revanche, si l'on ajoute une liaison directe coûteuse A-D de coût 3, RIP choisirait ce trajet à **un seul saut**, alors qu'OSPF préférerait toujours A-B-D (coût $2 < 3$).

⚠ ERREUR FRÉQUENTE

Croire qu'un protocole de routage choisit toujours le chemin « le plus court » au sens intuitif. Ce que minimise le protocole dépend entièrement de sa **métrique** : RIP minimise le nombre de sauts (route A-D directe préférée dans l'exemple), OSPF minimise un coût cumulé (route A-B-D préférée si son coût total est plus faible), même si cette dernière traverse davantage de routeurs.

Méthodes

► Méthode directe de travail sur le sujet.

Exercice 1 ◆

En utilisant la fonction tourniquet du cours, déterminer l'ordre d'exécution pour deux processus X (durée totale 4) et Y (durée totale 6), avec une tranche de temps de 3. Combien de tranches de temps sont-elles nécessaires au total pour que les deux processus se terminent ?

Exercice 2 ◆◆

Trois processus se partagent trois ressources : P1 retient R1 et attend R2 ; P2 retient R2 et attend R3 ; P3 retient R3 et attend R1.

1. Représenter cette situation par les deux dictionnaires *attente* et *retenue_par* du cours.
2. Y a-t-il un interblocage ? Justifier en décrivant le cycle d'attente.

Exercice 3 ◆◆

On donne le réseau de coûts suivant :

```
1 reseau = {
2     "A": {"B": 2, "C": 10},
3     "B": {"A": 2, "C": 2},
4     "C": {"A": 10, "B": 2},
5 }
```

1. Quelle route RIP (minimisant le nombre de sauts) choisirait-il de A vers C ? En utilisant `bfs_sauts` du cours, combien de sauts compte-t-elle ?
2. En utilisant `plus_court_cout` du cours, quel est le coût de la route effectivement la moins coûteuse de A vers C ? Ce n'est pas la route directe : pourquoi ?

Exercice 4

En utilisant `chiffrer_xor` du cours, chiffrer le message b"BAC2027" avec la clé b"NSI", puis vérifier qu'appliquer la même fonction une seconde fois sur le résultat, avec la même clé, redonne le message d'origine. Pourquoi ce chiffrement est-il qualifié de **symétrique** ?

Exercice 5

En utilisant la fonction `tourniquet` du cours, déterminer l'ordre d'exécution pour deux processus X (durée totale 4) et Y (durée totale 6), avec une tranche de temps de 3. Combien de tranches de temps sont-elles nécessaires au total pour que les deux processus se terminent ?

Exercice 6

Trois processus se partagent trois ressources : P1 retient R1 et attend R2 ; P2 retient R2 et attend R3 ; P3 retient R3 et attend R1.

1. Représenter cette situation par les deux dictionnaires `attente` et `retenue_par` du cours.
2. Y a-t-il un interblocage ? Justifier en décrivant le cycle d'attente.

Exercice 7

On donne le réseau de coûts suivant :

```
1 reseau = {
2     "A": {"B": 2, "C": 10},
3     "B": {"A": 2, "C": 2},
4     "C": {"A": 10, "B": 2},
5 }
```

1. Quelle route RIP (minimisant le nombre de sauts) choisirait-il de A vers C ? En utilisant `bfs_sauts` du cours, combien de sauts compte-t-elle ?
2. En utilisant `plus_court_cout` du cours, quel est le coût de la route effectivement la moins coûteuse de A vers C ? Ce n'est pas la route directe : pourquoi ?

Exercice 8

En utilisant `chiffrer_xor` du cours, chiffrer le message b"BAC2027" avec la clé b"NSI", puis vérifier qu'appliquer la même fonction une seconde fois sur le résultat, avec la même clé, redonne le message d'origine. Pourquoi ce chiffrement est-il qualifié de **symétrique** ?

Exercice 9

En utilisant la fonction `tourniquet` du cours, déterminer l'ordre d'exécution pour deux processus X (durée totale 4) et Y (durée totale 6), avec une tranche de temps de 3. Combien de tranches de temps sont-elles nécessaires au total pour que les deux processus se terminent ?

Exercice 10

Trois processus se partagent trois ressources : P1 retient R1 et attend R2 ; P2 retient R2 et attend R3 ; P3 retient R3 et attend R1.

1. Représenter cette situation par les deux dictionnaires `attente` et `retenue_par` du cours.
2. Y a-t-il un interblocage ? Justifier en décrivant le cycle d'attente.

Exercice 11 ♦♦

On donne le réseau de coûts suivant :

```
1 reseau = {
2     "A": {"B": 2, "C": 10},
3     "B": {"A": 2, "C": 2},
4     "C": {"A": 10, "B": 2},
5 }
```

1. Quelle route RIP (minimisant le nombre de sauts) choisirait-il de A vers C ? En utilisant `bfs_sauts` du cours, combien de sauts compte-t-elle ?
2. En utilisant `plus_court_cout` du cours, quel est le coût de la route effectivement la moins coûteuse de A vers C ? Ce n'est pas la route directe : pourquoi ?

Exercice 12 ♦

En utilisant `chiffrer_xor` du cours, chiffrer le message b"BAC2027" avec la clé b"NSI", puis vérifier qu'appliquer la même fonction une seconde fois sur le résultat, avec la même clé, redonne le message d'origine. Pourquoi ce chiffrement est-il qualifié de **symétrique** ?

Exercice 13 ♦

En utilisant la fonction `tourniquet` du cours, déterminer l'ordre d'exécution pour deux processus X (durée totale 4) et Y (durée totale 6), avec une tranche de temps de 3. Combien de tranches de temps sont-elles nécessaires au total pour que les deux processus se terminent ?

Exercice 14 ♦♦

Trois processus se partagent trois ressources : P1 retient R1 et attend R2 ; P2 retient R2 et attend R3 ; P3 retient R3 et attend R1.

1. Représenter cette situation par les deux dictionnaires `attente` et `retenue_par` du cours.
2. Y a-t-il un interblocage ? Justifier en décrivant le cycle d'attente.

Exercice 15 ♦♦

On donne le réseau de coûts suivant :

```
1 reseau = {
2     "A": {"B": 2, "C": 10},
3     "B": {"A": 2, "C": 2},
4     "C": {"A": 10, "B": 2},
5 }
```

1. Quelle route RIP (minimisant le nombre de sauts) choisirait-il de A vers C ? En utilisant `bfs_sauts` du cours, combien de sauts compte-t-elle ?
2. En utilisant `plus_court_cout` du cours, quel est le coût de la route effectivement la moins coûteuse de A vers C ? Ce n'est pas la route directe : pourquoi ?

Exercice 16 ♦

En utilisant `chiffrer_xor` du cours, chiffrer le message b"BAC2027" avec la clé b"NSI", puis vérifier qu'appliquer la même fonction une seconde fois sur le résultat, avec la même clé, redonne le message d'origine. Pourquoi ce chiffrement est-il qualifié de **symétrique** ?

Exercice 17

En utilisant la fonction tourniquet du cours, déterminer l'ordre d'exécution pour deux processus X (durée totale 4) et Y (durée totale 6), avec une tranche de temps de 3. Combien de tranches de temps sont-elles nécessaires au total pour que les deux processus se terminent ?

Exercice 18

Trois processus se partagent trois ressources : P1 retient R1 et attend R2 ; P2 retient R2 et attend R3 ; P3 retient R3 et attend R1.

1. Représenter cette situation par les deux dictionnaires `attente` et `retenue_par` du cours.
2. Y a-t-il un interblocage ? Justifier en décrivant le cycle d'attente.

Exercice 19

On donne le réseau de coûts suivant :

```
1 reseau = {
2     "A": {"B": 2, "C": 10},
3     "B": {"A": 2, "C": 2},
4     "C": {"A": 10, "B": 2},
5 }
```

1. Quelle route RIP (minimisant le nombre de sauts) choisirait-il de A vers C ? En utilisant `bfs_sauts` du cours, combien de sauts compte-t-elle ?
2. En utilisant `plus_court_cout` du cours, quel est le coût de la route effectivement la moins coûteuse de A vers C ? Ce n'est pas la route directe : pourquoi ?

Exercice 20

En utilisant `chiffrer_xor` du cours, chiffrer le message b"BAC2027" avec la clé b"NSI", puis vérifier qu'appliquer la même fonction une seconde fois sur le résultat, avec la même clé, redonne le message d'origine. Pourquoi ce chiffrement est-il qualifié de **symétrique** ?

Exercice 21

En utilisant la fonction tourniquet du cours, déterminer l'ordre d'exécution pour deux processus X (durée totale 4) et Y (durée totale 6), avec une tranche de temps de 3. Combien de tranches de temps sont-elles nécessaires au total pour que les deux processus se terminent ?

Exercice 22

Trois processus se partagent trois ressources : P1 retient R1 et attend R2 ; P2 retient R2 et attend R3 ; P3 retient R3 et attend R1.

1. Représenter cette situation par les deux dictionnaires `attente` et `retenue_par` du cours.
2. Y a-t-il un interblocage ? Justifier en décrivant le cycle d'attente.

Exercice 23

On donne le réseau de coûts suivant :

```
1 reseau = {
2     "A": {"B": 2, "C": 10},
3     "B": {"A": 2, "C": 2},
4     "C": {"A": 10, "B": 2},
5 }
```

1. Quelle route RIP (minimisant le nombre de sauts) choisirait-il de A vers C ? En utilisant `bfs_sauts` du cours, combien de sauts compte-t-elle ?
2. En utilisant `plus_court_cout` du cours, quel est le coût de la route effectivement la moins coûteuse de A vers C ? Ce n'est pas la route directe : pourquoi ?

Exercice 24

En utilisant `chiffrer_xor` du cours, chiffrer le message b"BAC2027" avec la clé b"NSI", puis vérifier qu'appliquer la même fonction une seconde fois sur le résultat, avec la même clé, redonne le message d'origine. Pourquoi ce chiffrement est-il qualifié de **symétrique** ?

Exercice 25

En utilisant la fonction `tourniquet` du cours, déterminer l'ordre d'exécution pour deux processus X (durée totale 4) et Y (durée totale 6), avec une tranche de temps de 3. Combien de tranches de temps sont-elles nécessaires au total pour que les deux processus se terminent ?

Exercice 26

Trois processus se partagent trois ressources : P1 retient R1 et attend R2 ; P2 retient R2 et attend R3 ; P3 retient R3 et attend R1.

1. Représenter cette situation par les deux dictionnaires `attente` et `retenue_par` du cours.
2. Y a-t-il un interblocage ? Justifier en décrivant le cycle d'attente.

Exercice 27

On donne le réseau de coûts suivant :

```
1 reseau = {
2     "A": {"B": 2, "C": 10},
3     "B": {"A": 2, "C": 2},
4     "C": {"A": 10, "B": 2},
5 }
```

1. Quelle route RIP (minimisant le nombre de sauts) choisirait-il de A vers C ? En utilisant `bfs_sauts` du cours, combien de sauts compte-t-elle ?
2. En utilisant `plus_court_cout` du cours, quel est le coût de la route effectivement la moins coûteuse de A vers C ? Ce n'est pas la route directe : pourquoi ?

Exercice 28

En utilisant `chiffrer_xor` du cours, chiffrer le message b"BAC2027" avec la clé b"NSI", puis vérifier qu'appliquer la même fonction une seconde fois sur le résultat, avec la même clé, redonne le message d'origine. Pourquoi ce chiffrement est-il qualifié de **symétrique** ?

Exercice 29

En utilisant la fonction `tourniquet` du cours, déterminer l'ordre d'exécution pour deux processus X (durée totale 4) et Y (durée totale 6), avec une tranche de temps de 3. Combien de tranches de temps sont-elles nécessaires au total pour que les deux processus se terminent ?

Exercice 30

Trois processus se partagent trois ressources : P1 retient R1 et attend R2 ; P2 retient R2 et attend R3 ; P3 retient R3 et attend R1.

1. Représenter cette situation par les deux dictionnaires `attente` et `retenue_par` du cours.
2. Y a-t-il un interblocage ? Justifier en décrivant le cycle d'attente.

Exercice 31 ♦♦

On donne le réseau de coûts suivant :

```
1 reseau = {
2     "A": {"B": 2, "C": 10},
3     "B": {"A": 2, "C": 2},
4     "C": {"A": 10, "B": 2},
5 }
```

1. Quelle route RIP (minimisant le nombre de sauts) choisirait-il de A vers C ? En utilisant `bfs_sauts` du cours, combien de sauts compte-t-elle ?
2. En utilisant `plus_court_cout` du cours, quel est le coût de la route effectivement la moins coûteuse de A vers C ? Ce n'est pas la route directe : pourquoi ?

Exercice 32 ♦

En utilisant `chiffrer_xor` du cours, chiffrer le message b"BAC2027" avec la clé b"NSI", puis vérifier qu'appliquer la même fonction une seconde fois sur le résultat, avec la même clé, redonne le message d'origine. Pourquoi ce chiffrement est-il qualifié de **symétrique** ?

Exercice 33 ♦

En utilisant la fonction `tourniquet` du cours, déterminer l'ordre d'exécution pour deux processus X (durée totale 4) et Y (durée totale 6), avec une tranche de temps de 3. Combien de tranches de temps sont-elles nécessaires au total pour que les deux processus se terminent ?

Exercice 34 ♦♦

Trois processus se partagent trois ressources : P1 retient R1 et attend R2 ; P2 retient R2 et attend R3 ; P3 retient R3 et attend R1.

1. Représenter cette situation par les deux dictionnaires `attente` et `retenue_par` du cours.
2. Y a-t-il un interblocage ? Justifier en décrivant le cycle d'attente.

Exercice 35 ♦♦

On donne le réseau de coûts suivant :

```
1 reseau = {
2     "A": {"B": 2, "C": 10},
3     "B": {"A": 2, "C": 2},
4     "C": {"A": 10, "B": 2},
5 }
```

1. Quelle route RIP (minimisant le nombre de sauts) choisirait-il de A vers C ? En utilisant `bfs_sauts` du cours, combien de sauts compte-t-elle ?
2. En utilisant `plus_court_cout` du cours, quel est le coût de la route effectivement la moins coûteuse de A vers C ? Ce n'est pas la route directe : pourquoi ?

Exercice 36 ♦

En utilisant `chiffrer_xor` du cours, chiffrer le message b"BAC2027" avec la clé b"NSI", puis vérifier qu'appliquer la même fonction une seconde fois sur le résultat, avec la même clé, redonne le message d'origine. Pourquoi ce chiffrement est-il qualifié de **symétrique** ?

Exercice 37 ♦

En utilisant la fonction tourniquet du cours, déterminer l'ordre d'exécution pour deux processus X (durée totale 4) et Y (durée totale 6), avec une tranche de temps de 3. Combien de tranches de temps sont-elles nécessaires au total pour que les deux processus se terminent ?

Exercice 38 ♦♦

Trois processus se partagent trois ressources : P1 retient R1 et attend R2 ; P2 retient R2 et attend R3 ; P3 retient R3 et attend R1.

1. Représenter cette situation par les deux dictionnaires `attente` et `retenue_par` du cours.
2. Y a-t-il un interblocage ? Justifier en décrivant le cycle d'attente.

Exercice 39 ♦♦

On donne le réseau de coûts suivant :

```
1 reseau = {
2     "A": {"B": 2, "C": 10},
3     "B": {"A": 2, "C": 2},
4     "C": {"A": 10, "B": 2},
5 }
```

1. Quelle route RIP (minimisant le nombre de sauts) choisirait-il de A vers C ? En utilisant `bfs_sauts` du cours, combien de sauts compte-t-elle ?
2. En utilisant `plus_court_cout` du cours, quel est le coût de la route effectivement la moins coûteuse de A vers C ? Ce n'est pas la route directe : pourquoi ?

Exercice 40 ♦

En utilisant `chiffrer_xor` du cours, chiffrer le message b"BAC2027" avec la clé b"NSI", puis vérifier qu'appliquer la même fonction une seconde fois sur le résultat, avec la même clé, redonne le message d'origine. Pourquoi ce chiffrement est-il qualifié de **symétrique** ?

Exercice 41 ♦

En utilisant la fonction tourniquet du cours, déterminer l'ordre d'exécution pour deux processus X (durée totale 4) et Y (durée totale 6), avec une tranche de temps de 3. Combien de tranches de temps sont-elles nécessaires au total pour que les deux processus se terminent ?

Exercice 42 ♦♦

Trois processus se partagent trois ressources : P1 retient R1 et attend R2 ; P2 retient R2 et attend R3 ; P3 retient R3 et attend R1.

1. Représenter cette situation par les deux dictionnaires `attente` et `retenue_par` du cours.
2. Y a-t-il un interblocage ? Justifier en décrivant le cycle d'attente.

ARCHITECTURES MATÉRIELLES, SYSTÈMES D'EXPLOITATION ET RÉSEAUX — QCM D'AUTO-ÉVALUATION

Pour chaque question, une seule réponse est exacte.

Capacité C1

1. [Q1] Par rapport à des composants séparés, le principal inconvénient d'un système sur puce est :
 - A. la perte de modularité : un composant défaillant ne peut pas être remplacé seul.
 - B. une consommation d'énergie plus élevée.
 - C. un encombrement plus important.
 - D. un coût de fabrication systématiquement supérieur.

Capacité C2

2. [Q2] Créer un processus consiste, pour le système d'exploitation, à :
 - A. compiler le code source du programme.
 - B. lui allouer de la mémoire et l'inscrire parmi les processus à exécuter.
 - C. arrêter tous les autres processus.
 - D. lui attribuer une adresse IP.

Capacité C3

3. [Q3] L'ordonnanceur du système d'exploitation a pour rôle de :
 - A. trier les fichiers sur le disque.
 - B. compiler les programmes avant leur lancement.
 - C. décider quel processus prêt obtient le processeur, et pour combien de temps.
 - D. supprimer les processus terminés de la mémoire, et rien d'autre.

Capacité C4

4. [Q4] Dans le graphe d'attente des processus, un interblocage se manifeste par :
 - A. un cycle.
 - B. un arbre.
 - C. un graphe sans aucune arête.
 - D. un sommet isolé.

Capacité C5

5. [Q5] Le protocole de routage RIP choisit la route qui minimise :
 - A. le coût cumulé des liaisons traversées.
 - B. le nombre de sauts.
 - C. la latence mesurée en temps réel.
 - D. le nombre d'utilisateurs connectés.

Capacité C6

6. [Q6] Le chiffrement asymétrique repose sur :
 - A. une unique clé secrète, partagée par les deux correspondants.
 - B. l'absence de toute clé.
 - C. une paire de clés, l'une publique et l'autre privée.
 - D. une clé différente pour chaque caractère du message.

Capacité C7

7. [Q7] Au cours d'une connexion HTTPS, le chiffrement asymétrique sert principalement à :
 - A. chiffrer la totalité des données échangées pendant toute la session.
 - B. compresser les données avant leur envoi.
 - C. afficher le cadenas dans le navigateur.
 - D. convenir en sécurité d'une clé symétrique de session.

Clé de correction — réservée au professeur

Question	Capacité	Réponse exacte
Q1	C1	A
Q2	C2	B
Q3	C3	C
Q4	C4	A
Q5	C5	B
Q6	C6	C
Q7	C7	D

Diagnostic des réponses erronées

► Q1

- **B** — L'intégration réduit au contraire les distances parcourues par les signaux, donc la consommation. C'est un de ses avantages. *Renvoi : C1, système sur puce.*
- **C** — Regrouper les composants sur une même puce diminue la taille : c'est ce qui rend possibles les appareils mobiles. *Renvoi : C1, système sur puce.*
- **D** — En grande série, l'intégration abaisse le coût unitaire. Le défaut porte sur la réparabilité, non sur le prix. *Renvoi : C1, système sur puce.*

► Q2

- **A** — La compilation précède l'exécution et relève d'un autre outil. Le système lance un programme déjà exécutable. *Renvoi : C2, création d'un processus.*
- **C** — C'est le contraire de la multiprogrammation : de nombreux processus coexistent, et l'ordonnanceur les fait alterner. *Renvoi : C2, création d'un processus.*
- **D** — L'adresse IP identifie une machine sur un réseau, non un processus sur une machine. *Renvoi : C2, création d'un processus.*

► Q3

- **A** — L'élève confond ordonnancement des processus et organisation du système de fichiers, qui sont deux fonctions distinctes. *Renvoi : C3, ordonnancement des processus.*
- **B** — La compilation est antérieure et extérieure au système ; l'ordonnanceur ne manipule que des processus déjà créés. *Renvoi : C3, ordonnancement des processus.*
- **D** — La libération de mémoire suit la fin d'un processus. L'ordonnanceur, lui, décide de l'ATTRIBUTION du processeur aux processus prêts. *Renvoi : C3, ordonnancement des processus.*

► Q4

- **B** — Un arbre est par définition sans cycle : les attentes y remontent vers une racine et finissent donc par se résoudre. *Renvoi : C4, interblocage.*
- **C** — Sans arête, aucun processus n'attend de ressource détenue par un autre. *Renvoi : C4, interblocage.*
- **D** — Un sommet isolé désigne un processus qui n'attend rien : c'est exactement le contraire d'un blocage. *Renvoi : C4, interblocage.*

► Q5

- **A** — Le coût cumulé, qui tient compte du débit des liaisons, est le critère d'OSPF. RIP ignore la qualité des liens. *Renvoi : C5, protocoles de routage.*
- **C** — RIP ne mesure rien en temps réel : sa métrique est fixe et se contente de dénombrer les routeurs traversés. *Renvoi : C5, protocoles de routage.*
- **D** — La charge des machines n'intervient ni dans la métrique de RIP ni dans celle d'OSPF. *Renvoi : C5, protocoles de routage.*

► Q6

- **A** — C'est la définition du chiffrement SYMÉTRIQUE, dont la difficulté est précisément de transmettre cette clé unique. *Renvoi : C6, chiffrement symétrique et asymétrique.*
- **B** — Sans clé, il n'y a plus de secret : n'importe qui pourrait déchiffrer le message. *Renvoi : C6, chiffrement symétrique et asymétrique.*

- **D** — L'élève décrit un chiffrement par flot avec masque jetable, qui reste symétrique et ne correspond pas à la question. *Renvoi : C6, chiffrement symétrique et asymétrique.*

► Q7

- **A** — Le chiffrement asymétrique est bien trop lent pour un flux entier. Il n'intervient qu'à l'établissement de la connexion. *Renvoi : C7, échange de clé et HTTPS.*
- **B** — Chiffrer et compresser sont deux opérations distinctes ; le protocole ne confond pas les deux. *Renvoi : C7, échange de clé et HTTPS.*
- **C** — Le cadenas n'est qu'un indicateur affiché par le navigateur : il rend compte de la sécurité obtenue, il ne la produit pas. *Renvoi : C7, échange de clé et HTTPS.*

Corrigé

Q1. Avantages : compacité, consommation énergétique réduite, coût de fabrication réduit en grande série. Inconvénient : perte de modularité (un composant défaillant oblige à remplacer toute la puce).
Q2. Le système d'exploitation alloue de la mémoire au nouveau processus, y charge le code du programme, et l'inscrit dans la liste des processus à exécuter.

Corrigé

Q1.

```
1 resultat = tourniquet(["P1", "P2", "P3"], 2, {"P1": 2, "P2": 4, "P3": 3})
2 print(resultat) # [('P1', 2), ('P2', 2), ('P3', 2), ('P2', 2), ('P3', 1)]
```

P1 termine en une seule tranche (durée 2), P2 et P3 nécessitent chacun deux passages.

Q2. Non, il n'y a **pas** d'interblocage : P3 retient R3 sans rien attendre, donc la chaîne d'attente P1 → P2 (via R2 puis R1) ne boucle pas – P2 finira par obtenir R1 dès que P1 le libère, sans dépendre de personne d'autre.

Évaluation A — Système sur puce et processus

Durée : 45 minutes. Ordinateur avec interpréteur Python autorisé.

Barème indicatif : Ex. 1 : 8 pts (SoC, création de processus) ; Ex. 2 : 12 pts (ordonnancement, interblocage)

Exercice 1 — Système sur puce et processus

(8 points)

- (4 pts) Citer trois avantages de l'intégration de plusieurs composants sur un système sur puce, et son principal inconvénient.
- (4 pts) Décrire, en une phrase, ce que fait le système d'exploitation lors de la création d'un processus.

Exercice 2 — Ordonnancement et interblocage

(12 points)

- (6 pts) En utilisant tourniquet du cours, déterminer l'ordre d'exécution pour trois processus P1, P2, P3 de durées respectives 2, 4, 3, avec une tranche de temps de 2.
- (6 pts) Trois processus P1, P2, P3 retiennent chacun une ressource R1, R2, R3, et P1 attend R2, P2 attend R1 (pas de troisième attente). Y a-t-il un interblocage ? Justifier avec `a_un_interblocage` du cours.

Corrigé

Q1. RIP choisirait la route directe A-D (un seul saut), quel que soit son coût élevé. `bfs_sauts(reseau, "A", "D")` renvoie 1.

Q2. `plus_court_cout(reseau, "A", "D")` renvoie 3 : la route la moins coûteuse est A-B-C-D (coût $1 + 1 + 1 = 3$), bien moins chère que la route directe (coût 8). Elle **ne coïncide pas** avec la route RIP : RIP privilégie le nombre de sauts, OSPF le coût cumulé.

Corrigé

Q1. Le chiffrement symétrique utilise une **unique clé secrète** partagée pour chiffrer et déchiffrer ; le chiffrement asymétrique utilise une **paire de clés** (publique/privée) : ce qui est chiffré avec la clé publique ne se déchiffre qu'avec la clé privée correspondante.

Q2. Le chiffrement asymétrique est beaucoup plus lent à calculer que le chiffrement symétrique. L'utiliser pour **tout** l'échange (pages web, formulaires, fichiers) ralentirait considérablement la connexion. HTTPS l'utilise donc seulement pour échanger, en toute sécurité, une clé symétrique de session au début de la connexion ; le reste des données est ensuite chiffré avec cette clé symétrique, bien plus rapide.

Évaluation B — Routage et cryptographie

Durée : 45 minutes. Ordinateur avec interpréteur Python autorisé.

Barème indicatif : Ex. 1 : 10 pts (routage) ; Ex. 2 : 10 pts (chiffrement, HTTPS)

Exercice 1 — Routage

(10 points)

```
1 reseau = {
2     "A": {"B": 1, "D": 8},
3     "B": {"A": 1, "C": 1},
4     "C": {"B": 1, "D": 1},
5     "D": {"A": 8, "C": 1},
6 }
```

- (5 pts) En utilisant `bfs_sauts` du cours, quelle route RIP choisirait-il de A vers D, et en combien de sauts ?
- (5 pts) En utilisant `plus_court_cout` du cours, quel est le coût de la route la moins coûteuse de A vers D ? Coïncide-t-elle avec la route RIP ?

Exercice 2 — Chiffrement et HTTPS

(10 points)

- (4 pts) Quelle est la différence essentielle entre chiffrement symétrique et asymétrique ?
- (6 pts) Expliquer pourquoi une connexion HTTPS combine les deux, plutôt que d'utiliser uniquement le chiffrement asymétrique pour tout l'échange.

Exercice 43

Un ordinateur possède un processeur à un seul cœur, et pourtant l'utilisateur peut écouter de la musique tout en tapant du texte dans un traitement de texte, sans interruption perceptible. Ces deux tâches s'exécutent-elles en parallélisme réel ou en concurrence ? Expliquer.

Corrigé

Ces deux tâches s'exécutent en **concurrence**, pas en parallélisme réel : un seul cœur ne peut exécuter qu'une instruction à la fois. L'ordonnanceur alterne très rapidement entre les deux processus (tranches de temps de quelques millisecondes), ce qui donne à l'utilisateur l'**illusion** d'une exécution simultanée, alors qu'en réalité chaque processus n'avance que pendant sa tranche de temps.

Exercice 44

Un ordinateur possède un processeur à un seul cœur, et pourtant l'utilisateur peut écouter de la musique tout en tapant du texte dans un traitement de texte, sans interruption perceptible. Ces deux tâches s'exécutent-elles en parallélisme réel ou en concurrence ? Expliquer.

Corrigé

Ces deux tâches s'exécutent en **concurrence**, pas en parallélisme réel : un seul cœur ne peut exécuter qu'une instruction à la fois. L'ordonnanceur alterne très rapidement entre les deux processus (tranches de temps de quelques millisecondes), ce qui donne à l'utilisateur l'**illusion** d'une exécution simultanée, alors qu'en réalité chaque processus n'avance que pendant sa tranche de temps.

Exercice 45

Un ordinateur possède un processeur à un seul cœur, et pourtant l'utilisateur peut écouter de la musique tout en tapant du texte dans un traitement de texte, sans interruption perceptible. Ces deux tâches s'exécutent-elles en parallélisme réel ou en concurrence ? Expliquer.

Corrigé

Ces deux tâches s'exécutent en **concurrence**, pas en parallélisme réel : un seul cœur ne peut exécuter qu'une instruction à la fois. L'ordonnanceur alterne très rapidement entre les deux processus (tranches de temps de quelques millisecondes), ce qui donne à l'utilisateur l'**illusion** d'une exécution simultanée, alors qu'en réalité chaque processus n'avance que pendant sa tranche de temps.

Exercice 46

Un ordinateur possède un processeur à un seul cœur, et pourtant l'utilisateur peut écouter de la musique tout en tapant du texte dans un traitement de texte, sans interruption perceptible. Ces deux tâches s'exécutent-elles en parallélisme réel ou en concurrence ? Expliquer.

Corrigé

Ces deux tâches s'exécutent en **concurrence**, pas en parallélisme réel : un seul cœur ne peut exécuter qu'une instruction à la fois. L'ordonnanceur alterne très rapidement entre les deux processus (tranches de temps de quelques millisecondes), ce qui donne à l'utilisateur l'**illusion** d'une exécution simultanée, alors qu'en réalité chaque processus n'avance que pendant sa tranche de temps.

Exercice 47

Un ordinateur possède un processeur à un seul cœur, et pourtant l'utilisateur peut écouter de la musique tout en tapant du texte dans un traitement de texte, sans interruption perceptible. Ces deux tâches s'exécutent-elles en parallélisme réel ou en concurrence ? Expliquer.

Corrigé

Ces deux tâches s'exécutent en **concurrence**, pas en parallélisme réel : un seul cœur ne peut exécuter qu'une instruction à la fois. L'ordonnanceur alterne très rapidement entre les deux processus (tranches de temps de quelques millisecondes), ce qui donne à l'utilisateur l'**illusion** d'une exécution simultanée, alors qu'en réalité chaque processus n'avance que pendant sa tranche de temps.

Exercice 48

Un ordinateur possède un processeur à un seul cœur, et pourtant l'utilisateur peut écouter de la musique tout en tapant du texte dans un traitement de texte, sans interruption perceptible. Ces deux tâches s'exécutent-elles en parallélisme réel ou en concurrence ? Expliquer.

Corrigé

Ces deux tâches s'exécutent en **concurrence**, pas en parallélisme réel : un seul cœur ne peut exécuter qu'une instruction à la fois. L'ordonnanceur alterne très rapidement entre les deux processus (tranches de temps de quelques millisecondes), ce qui donne à l'utilisateur l'**illusion** d'une exécution simultanée, alors qu'en réalité chaque processus n'avance que pendant sa tranche de temps.

Exercice 49

Un ordinateur possède un processeur à un seul cœur, et pourtant l'utilisateur peut écouter de la musique tout en tapant du texte dans un traitement de texte, sans interruption perceptible. Ces deux tâches s'exécutent-elles en parallélisme réel ou en concurrence ? Expliquer.

Corrigé

Ces deux tâches s'exécutent en **concurrence**, pas en parallélisme réel : un seul cœur ne peut exécuter qu'une instruction à la fois. L'ordonnanceur alterne très rapidement entre les deux processus (tranches de temps de quelques millisecondes), ce qui donne à l'utilisateur l'**illusion** d'une exécution simultanée, alors qu'en réalité chaque processus n'avance que pendant sa tranche de temps.

Exercice 50

Un ordinateur possède un processeur à un seul cœur, et pourtant l'utilisateur peut écouter de la musique tout en tapant du texte dans un traitement de texte, sans interruption perceptible. Ces deux tâches s'exécutent-elles en parallélisme réel ou en concurrence ? Expliquer.

Corrigé

Ces deux tâches s'exécutent en **concurrence**, pas en parallélisme réel : un seul cœur ne peut exécuter qu'une instruction à la fois. L'ordonnanceur alterne très rapidement entre les deux processus (tranches de temps de quelques millisecondes), ce qui donne à l'utilisateur l'**illusion** d'une exécution simultanée, alors qu'en réalité chaque processus n'avance que pendant sa tranche de temps.

Corrigé

```
1 resultat = tourniquet(["X", "Y"], 3, {"X": 4, "Y": 6})
2 print(resultat) # [('X', 3), ('Y', 3), ('X', 1), ('Y', 3)]
```

X s'exécute 3 puis 1 unité de temps (total 4), Y s'exécute 3 puis 3 unités de temps (total 6). Il faut 4 tranches de temps au total pour que les deux processus se terminent.

Corrigé

Q1.

```
1 attente = {"P1": "R2", "P2": "R3", "P3": "R1"}
2 retenue_par = {"R1": "P1", "R2": "P2", "R3": "P3"}
```

Q2. Oui, il y a interblocage : P1 attend R2 retenue par P2, qui attend R3 retenue par P3, qui attend R1 retenue par P1 – on revient au point de départ. Le cycle d'attente $P1 \rightarrow P2 \rightarrow P3 \rightarrow P1$ signifie qu'aucun des trois processus ne peut jamais obtenir la ressource qu'il attend.

Corrigé

Q1. RIP choisirait la route directe A-C : elle ne compte qu'un seul saut, quel que soit son coût. `bfs_sauts(reseau, "A", "C")` renvoie 1.

Q2. `plus_court_cout(reseau, "A", "C")` renvoie 4 : la route la moins coûteuse est A-B-C (coût $2+2=4$), moins chère que la route directe A-C (coût 10), bien qu'elle traverse un routeur de plus. OSPF

choisirait donc A-B-C, RIP choisirait A-C : les deux protocoles divergent car ils n'optimisent pas le même critère.

Corrigé

```
1 message = b"BAC2027"
2 cle = b"NSI"
3 chiffre = chiffrer_xor(message, cle)
4 dechiffre = chiffrer_xor(chiffre, cle)
5 print(dechiffre == message) # True
```

Ce chiffrement est dit **symétrique** car la **même clé** `cle` sert à la fois à chiffrer et à déchiffrer : il n'y a qu'un seul secret partagé, contrairement au chiffrement asymétrique qui utilise deux clés différentes (publique et privée).

Corrigé

```
1 resultat = tourniquet(["X", "Y"], 3, {"X": 4, "Y": 6})
2 print(resultat) # [('X', 3), ('Y', 3), ('X', 1), ('Y', 3)]
```

X s'exécute 3 puis 1 unité de temps (total 4), Y s'exécute 3 puis 3 unités de temps (total 6). Il faut 4 tranches de temps au total pour que les deux processus se terminent.

Corrigé

Q1.

```
1 attente = {"P1": "R2", "P2": "R3", "P3": "R1"}
2 retenue_par = {"R1": "P1", "R2": "P2", "R3": "P3"}
```

Q2. Oui, il y a un interblocage : P1 attend R2 retenue par P2, qui attend R3 retenue par P3, qui attend R1 retenue par P1 – on revient au point de départ. Le cycle d'attente $P1 \rightarrow P2 \rightarrow P3 \rightarrow P1$ signifie qu'aucun des trois processus ne peut jamais obtenir la ressource qu'il attend.

Corrigé

Q1. RIP choisirait la route directe A-C : elle ne compte qu'un seul saut, quel que soit son coût. `bfs_sauts(reseau, "A", "C")` renvoie 1.

Q2. `plus_court_cout(reseau, "A", "C")` renvoie 4 : la route la moins coûteuse est A-B-C (coût $2+2=4$), moins chère que la route directe A-C (coût 10), bien qu'elle traverse un routeur de plus. OSPF choisirait donc A-B-C, RIP choisirait A-C : les deux protocoles divergent car ils n'optimisent pas le même critère.

Corrigé

```
1 message = b"BAC2027"
2 cle = b"NSI"
3 chiffre = chiffrer_xor(message, cle)
4 dechiffre = chiffrer_xor(chiffre, cle)
5 print(dechiffre == message) # True
```

Ce chiffrement est dit **symétrique** car la **même clé** `cle` sert à la fois à chiffrer et à déchiffrer : il n'y a qu'un seul secret partagé, contrairement au chiffrement asymétrique qui utilise deux clés différentes (publique et privée).

Corrigé

```
1 resultat = tourniquet(["X", "Y"], 3, {"X": 4, "Y": 6})
2 print(resultat) # [('X', 3), ('Y', 3), ('X', 1), ('Y', 3)]
```

X s'exécute 3 puis 1 unité de temps (total 4), Y s'exécute 3 puis 3 unités de temps (total 6). Il faut 4 tranches de temps au total pour que les deux processus se terminent.

Corrigé

Q1.

```
1 attente = {"P1": "R2", "P2": "R3", "P3": "R1"}
2 retenue_par = {"R1": "P1", "R2": "P2", "R3": "P3"}
```

Q2. Oui, il y a interblocage : P1 attend R2 retenue par P2, qui attend R3 retenue par P3, qui attend R1 retenue par P1 – on revient au point de départ. Le cycle d'attente $P1 \rightarrow P2 \rightarrow P3 \rightarrow P1$ signifie qu'aucun des trois processus ne peut jamais obtenir la ressource qu'il attend.

Corrigé

Q1. RIP choisirait la route directe A-C : elle ne compte qu'un seul saut, quel que soit son coût. `bfs_sauts(reseau, "A", "C")` renvoie 1.

Q2. `plus_court_cout(reseau, "A", "C")` renvoie 4 : la route la moins coûteuse est A-B-C (coût $2+2=4$), moins chère que la route directe A-C (coût 10), bien qu'elle traverse un routeur de plus. OSPF choisirait donc A-B-C, RIP choisirait A-C : les deux protocoles divergent car ils n'optimisent pas le même critère.

Corrigé

```
1 message = b"BAC2027"
2 cle = b"NSI"
3 chiffre = chiffrer_xor(message, cle)
4 dechiffre = chiffrer_xor(chiffre, cle)
5 print(dechiffre == message) # True
```

Ce chiffrement est dit **symétrique** car la **même clé** `cle` sert à la fois à chiffrer et à déchiffrer : il n'y a qu'un seul secret partagé, contrairement au chiffrement asymétrique qui utilise deux clés différentes (publique et privée).

Corrigé

```
1 resultat = tourniquet(["X", "Y"], 3, {"X": 4, "Y": 6})
2 print(resultat) # [('X', 3), ('Y', 3), ('X', 1), ('Y', 3)]
```

X s'exécute 3 puis 1 unité de temps (total 4), Y s'exécute 3 puis 3 unités de temps (total 6). Il faut 4 tranches de temps au total pour que les deux processus se terminent.

Corrigé

Q1.

```
1 attente = {"P1": "R2", "P2": "R3", "P3": "R1"}
2 retenue_par = {"R1": "P1", "R2": "P2", "R3": "P3"}
```

Q2. Oui, il y a interblocage : P1 attend R2 retenue par P2, qui attend R3 retenue par P3, qui attend R1 retenue par P1 – on revient au point de départ. Le cycle d'attente $P1 \rightarrow P2 \rightarrow P3 \rightarrow P1$ signifie qu'aucun des trois processus ne peut jamais obtenir la ressource qu'il attend.

Corrigé

Q1. RIP choisirait la route directe A-C : elle ne compte qu'un seul saut, quel que soit son coût. `bfs_sauts(reseau, "A", "C")` renvoie 1.

Q2. `plus_court_cout(reseau, "A", "C")` renvoie 4 : la route la moins coûteuse est A-B-C (coût 2+2 = 4), moins chère que la route directe A-C (coût 10), bien qu'elle traverse un routeur de plus. OSPF choisirait donc A-B-C, RIP choisirait A-C : les deux protocoles divergent car ils n'optimisent pas le même critère.

Corrigé

```
1 message = b"BAC2027"
2 cle = b"NSI"
3 chiffre = chiffrer_xor(message, cle)
4 dechiffre = chiffrer_xor(chiffre, cle)
5 print(dechiffre == message) # True
```

Ce chiffrement est dit **symétrique** car la **même clé** `cle` sert à la fois à chiffrer et à déchiffrer : il n'y a qu'un seul secret partagé, contrairement au chiffrement asymétrique qui utilise deux clés différentes (publique et privée).

Corrigé

```
1 resultat = tourniquet(["X", "Y"], 3, {"X": 4, "Y": 6})
2 print(resultat) # [('X', 3), ('Y', 3), ('X', 1), ('Y', 3)]
```

X s'exécute 3 puis 1 unité de temps (total 4), Y s'exécute 3 puis 3 unités de temps (total 6). Il faut 4 tranches de temps au total pour que les deux processus se terminent.

Corrigé

Q1.

```
1 attente = {"P1": "R2", "P2": "R3", "P3": "R1"}
2 retenue_par = {"R1": "P1", "R2": "P2", "R3": "P3"}
```

Q2. Oui, il y a interblocage : P1 attend R2 retenue par P2, qui attend R3 retenue par P3, qui attend R1 retenue par P1 – on revient au point de départ. Le cycle d'attente $P1 \rightarrow P2 \rightarrow P3 \rightarrow P1$ signifie qu'aucun des trois processus ne peut jamais obtenir la ressource qu'il attend.

Corrigé

Q1. RIP choisirait la route directe A-C : elle ne compte qu'un seul saut, quel que soit son coût. `bfs_sauts(reseau, "A", "C")` renvoie 1.

Q2. `plus_court_cout(reseau, "A", "C")` renvoie 4 : la route la moins coûteuse est A-B-C (coût 2+2 = 4), moins chère que la route directe A-C (coût 10), bien qu'elle traverse un routeur de plus. OSPF choisirait donc A-B-C, RIP choisirait A-C : les deux protocoles divergent car ils n'optimisent pas le même critère.

Corrigé

```
1 message = b"BAC2027"
2 cle = b"NSI"
3 chiffre = chiffrer_xor(message, cle)
4 dechiffre = chiffrer_xor(chiffre, cle)
5 print(dechiffre == message) # True
```

Ce chiffrement est dit **symétrique** car la **même clé** `cle` sert à la fois à chiffrer et à déchiffrer : il n'y a qu'un seul secret partagé, contrairement au chiffrement asymétrique qui utilise deux clés différentes (publique et privée).

Corrigé

```
1 resultat = tourniquet(["X", "Y"], 3, {"X": 4, "Y": 6})
2 print(resultat) # [('X', 3), ('Y', 3), ('X', 1), ('Y', 3)]
```

X s'exécute 3 puis 1 unité de temps (total 4), Y s'exécute 3 puis 3 unités de temps (total 6). Il faut 4 tranches de temps au total pour que les deux processus se terminent.

Corrigé

Q1.

```
1 attente = {"P1": "R2", "P2": "R3", "P3": "R1"}
2 retenue_par = {"R1": "P1", "R2": "P2", "R3": "P3"}
```

Q2. Oui, il y a interblocage : P1 attend R2 retenue par P2, qui attend R3 retenue par P3, qui attend R1 retenue par P1 – on revient au point de départ. Le cycle d'attente $P1 \rightarrow P2 \rightarrow P3 \rightarrow P1$ signifie qu'aucun des trois processus ne peut jamais obtenir la ressource qu'il attend.

Corrigé

Q1. RIP choisirait la route directe A-C : elle ne compte qu'un seul saut, quel que soit son coût. `bfs_sauts(reseau, "A", "C")` renvoie 1.

Q2. `plus_court_cout(reseau, "A", "C")` renvoie 4 : la route la moins coûteuse est A-B-C (coût $2+2=4$), moins chère que la route directe A-C (coût 10), bien qu'elle traverse un routeur de plus. OSPF choisirait donc A-B-C, RIP choisirait A-C : les deux protocoles divergent car ils n'optimisent pas le même critère.

Corrigé

```
1 message = b"BAC2027"
2 cle = b"NSI"
3 chiffrer = chiffrer_xor(message, cle)
4 dechiffrer = chiffrer_xor(chiffrer, cle)
5 print(dechiffrer == message) # True
```

Ce chiffrement est dit **symétrique** car la **même clé** `cle` sert à la fois à chiffrer et à déchiffrer : il n'y a qu'un seul secret partagé, contrairement au chiffrement asymétrique qui utilise deux clés différentes (publique et privée).

Corrigé

```
1 resultat = tourniquet(["X", "Y"], 3, {"X": 4, "Y": 6})
2 print(resultat) # [('X', 3), ('Y', 3), ('X', 1), ('Y', 3)]
```

X s'exécute 3 puis 1 unité de temps (total 4), Y s'exécute 3 puis 3 unités de temps (total 6). Il faut 4 tranches de temps au total pour que les deux processus se terminent.

Corrigé

Q1.

```
1 attente = {"P1": "R2", "P2": "R3", "P3": "R1"}
2 retenue_par = {"R1": "P1", "R2": "P2", "R3": "P3"}
```

Q2. Oui, il y a interblocage : P1 attend R2 retenue par P2, qui attend R3 retenue par P3, qui attend R1 retenue par P1 – on revient au point de départ. Le cycle d'attente P1 → P2 → P3 → P1 signifie qu'aucun des trois processus ne peut jamais obtenir la ressource qu'il attend.

Corrigé

Q1. RIP choisirait la route directe A-C : elle ne compte qu'un seul saut, quel que soit son coût. `bfs_sauts(reseau, "A", "C")` renvoie 1.

Q2. `plus_court_cout(reseau, "A", "C")` renvoie 4 : la route la moins coûteuse est A-B-C (coût 2+2 = 4), moins chère que la route directe A-C (coût 10), bien qu'elle traverse un routeur de plus. OSPF choisirait donc A-B-C, RIP choisirait A-C : les deux protocoles divergent car ils n'optimisent pas le même critère.

Corrigé

```
1 message = b"BAC2027"
2 cle = b"NSI"
3 chiffre = chiffrer_xor(message, cle)
4 dechiffre = chiffrer_xor(chiffre, cle)
5 print(dechiffre == message) # True
```

Ce chiffrement est dit **symétrique** car la **même clé** `cle` sert à la fois à chiffrer et à déchiffrer : il n'y a qu'un seul secret partagé, contrairement au chiffrement asymétrique qui utilise deux clés différentes (publique et privée).

Corrigé

```
1 resultat = tourniquet(["X", "Y"], 3, {"X": 4, "Y": 6})
2 print(resultat) # [('X', 3), ('Y', 3), ('X', 1), ('Y', 3)]
```

X s'exécute 3 puis 1 unité de temps (total 4), Y s'exécute 3 puis 3 unités de temps (total 6). Il faut 4 tranches de temps au total pour que les deux processus se terminent.

Corrigé

Q1.

```
1 attente = {"P1": "R2", "P2": "R3", "P3": "R1"}
2 retenue_par = {"R1": "P1", "R2": "P2", "R3": "P3"}
```

Q2. Oui, il y a interblocage : P1 attend R2 retenue par P2, qui attend R3 retenue par P3, qui attend R1 retenue par P1 – on revient au point de départ. Le cycle d'attente P1 → P2 → P3 → P1 signifie qu'aucun des trois processus ne peut jamais obtenir la ressource qu'il attend.

Corrigé

Q1. RIP choisirait la route directe A-C : elle ne compte qu'un seul saut, quel que soit son coût. `bfs_sauts(reseau, "A", "C")` renvoie 1.

Q2. `plus_court_cout(reseau, "A", "C")` renvoie 4 : la route la moins coûteuse est A-B-C (coût 2+2 = 4), moins chère que la route directe A-C (coût 10), bien qu'elle traverse un routeur de plus. OSPF choisirait donc A-B-C, RIP choisirait A-C : les deux protocoles divergent car ils n'optimisent pas le même critère.

Corrigé

```

1 message = b"BAC2027"
2 cle = b"NSI"
3 chiffre = chiffrer_xor(message, cle)
4 dechiffre = chiffrer_xor(chiffre, cle)
5 print(dechiffre == message) # True

```

Ce chiffrement est dit **symétrique** car la **même clé** `cle` sert à la fois à chiffrer et à déchiffrer : il n'y a qu'un seul secret partagé, contrairement au chiffrement asymétrique qui utilise deux clés différentes (publique et privée).

Corrigé

```

1 resultat = tourniquet(["X", "Y"], 3, {"X": 4, "Y": 6})
2 print(resultat) # [('X', 3), ('Y', 3), ('X', 1), ('Y', 3)]

```

X s'exécute 3 puis 1 unité de temps (total 4), Y s'exécute 3 puis 3 unités de temps (total 6). Il faut 4 tranches de temps au total pour que les deux processus se terminent.

Corrigé

Q1.

```

1 attente = {"P1": "R2", "P2": "R3", "P3": "R1"}
2 retenue_par = {"R1": "P1", "R2": "P2", "R3": "P3"}

```

Q2. Oui, il y a interblocage : P1 attend R2 retenue par P2, qui attend R3 retenue par P3, qui attend R1 retenue par P1 – on revient au point de départ. Le cycle d'attente $P1 \rightarrow P2 \rightarrow P3 \rightarrow P1$ signifie qu'aucun des trois processus ne peut jamais obtenir la ressource qu'il attend.

Corrigé

Q1. RIP choisirait la route directe A-C : elle ne compte qu'un seul saut, quel que soit son coût. `bfs_sauts(reseau, "A", "C")` renvoie 1.

Q2. `plus_court_cout(reseau, "A", "C")` renvoie 4 : la route la moins coûteuse est A-B-C (coût $2+2=4$), moins chère que la route directe A-C (coût 10), bien qu'elle traverse un routeur de plus. OSPF choisirait donc A-B-C, RIP choisirait A-C : les deux protocoles divergent car ils n'optimisent pas le même critère.

Corrigé

```

1 message = b"BAC2027"
2 cle = b"NSI"
3 chiffre = chiffrer_xor(message, cle)
4 dechiffre = chiffrer_xor(chiffre, cle)
5 print(dechiffre == message) # True

```

Ce chiffrement est dit **symétrique** car la **même clé** `cle` sert à la fois à chiffrer et à déchiffrer : il n'y a qu'un seul secret partagé, contrairement au chiffrement asymétrique qui utilise deux clés différentes (publique et privée).

Corrigé

```

1 resultat = tourniquet(["X", "Y"], 3, {"X": 4, "Y": 6})
2 print(resultat) # [('X', 3), ('Y', 3), ('X', 1), ('Y', 3)]

```

X s'exécute 3 puis 1 unité de temps (total 4), Y s'exécute 3 puis 3 unités de temps (total 6). Il faut 4 tranches de temps au total pour que les deux processus se terminent.

Corrigé

Q1.

```
1 attente = {"P1": "R2", "P2": "R3", "P3": "R1"}
2 retenue_par = {"R1": "P1", "R2": "P2", "R3": "P3"}
```

Q2. Oui, il y a interblocage : P1 attend R2 retenue par P2, qui attend R3 retenue par P3, qui attend R1 retenue par P1 – on revient au point de départ. Le cycle d'attente $P1 \rightarrow P2 \rightarrow P3 \rightarrow P1$ signifie qu'aucun des trois processus ne peut jamais obtenir la ressource qu'il attend.

Corrigé

Q1. RIP choisirait la route directe A-C : elle ne compte qu'un seul saut, quel que soit son coût. `bfs_sauts(reseau, "A", "C")` renvoie 1.

Q2. `plus_court_cout(reseau, "A", "C")` renvoie 4 : la route la moins coûteuse est A-B-C (coût $2+2=4$), moins chère que la route directe A-C (coût 10), bien qu'elle traverse un routeur de plus. OSPF choisirait donc A-B-C, RIP choisirait A-C : les deux protocoles divergent car ils n'optimisent pas le même critère.

Corrigé

```
1 message = b"BAC2027"
2 cle = b"NSI"
3 chiffre = chiffrer_xor(message, cle)
4 dechiffre = chiffrer_xor(chiffre, cle)
5 print(dechiffre == message) # True
```

Ce chiffrement est dit **symétrique** car la **même clé** `cle` sert à la fois à chiffrer et à déchiffrer : il n'y a qu'un seul secret partagé, contrairement au chiffrement asymétrique qui utilise deux clés différentes (publique et privée).

Corrigé

```
1 resultat = tourniquet(["X", "Y"], 3, {"X": 4, "Y": 6})
2 print(resultat) # [('X', 3), ('Y', 3), ('X', 1), ('Y', 3)]
```

X s'exécute 3 puis 1 unité de temps (total 4), Y s'exécute 3 puis 3 unités de temps (total 6). Il faut 4 tranches de temps au total pour que les deux processus se terminent.

Corrigé

Q1.

```
1 attente = {"P1": "R2", "P2": "R3", "P3": "R1"}
2 retenue_par = {"R1": "P1", "R2": "P2", "R3": "P3"}
```

Q2. Oui, il y a interblocage : P1 attend R2 retenue par P2, qui attend R3 retenue par P3, qui attend R1 retenue par P1 – on revient au point de départ. Le cycle d'attente $P1 \rightarrow P2 \rightarrow P3 \rightarrow P1$ signifie qu'aucun des trois processus ne peut jamais obtenir la ressource qu'il attend.

3 Bases de données relationnelles

3.1 Le modèle relationnel

DÉFINITION 1

Le **modèle relationnel** organise les données en **relations** (tables). Une relation est constituée de **tuples** (lignes), chacun décrivant une occurrence, et d'**attributs** (colonnes), chacun défini sur un **domaine** (l'ensemble des valeurs possibles, par exemple entier ou texte). Le **schéma** d'une relation liste le nom de la relation et le nom de chacun de ses attributs.

DÉFINITION 2

Une **clé primaire** est un attribut (ou groupe d'attributs) dont la valeur identifie de façon **unique** chaque tuple d'une relation. Une **clé étrangère** est un attribut d'une relation dont la valeur fait référence à la clé primaire d'une autre relation : c'est ainsi que deux relations sont mises en lien.

Exemple. Une bibliothèque gère trois relations liées entre elles :

```
1 Livre(id, titre, auteur, année)
2 Usager(id, nom, prenom)
3 Emprunt(id, id_livre, id_usager, date_emprunt, date_retour)
```

Dans Emprunt, `id_livre` est une clé étrangère vers `Livre.id`, et `id_usager` une clé étrangère vers `Usager.id` : chaque emprunt référence un livre et un usager précis, sans dupliquer leurs informations.

◆ **PROPRIÉTÉ Structure et contenu** La **structure** d'une base de données (son schéma : relations, attributs, clés, contraintes) est stable dans le temps. Le **contenu** (les tuples effectivement stockés) évolue à chaque insertion, modification ou suppression. Modifier le contenu ne change jamais la structure ; modifier la structure (ajouter une colonne, par exemple) laisse le contenu existant intact autant que possible.

⚠ ERREUR FRÉQUENTE

Confondre relation et fichier plat. Stocker toutes les informations d'une bibliothèque dans une seule table `Emprunt(id, titre_livre, auteur, nom_usager, prenom_usager, ...)` **duplique** le titre et l'auteur à chaque emprunt du même livre : c'est une **anomalie de schéma**. Une modification (correction d'une faute dans un titre) doit alors être répercutée sur toutes les lignes concernées, au risque d'incohérences. Séparer `Livre`, `Usager` et `Emprunt` en trois relations liées par des clés étrangères élimine cette duplication.

+ POUR ALLER PLUS LOIN

Ce principe de séparation pour éviter les anomalies porte un nom en théorie des bases de données : la **normalisation** (formes normales 1NF, 2NF, 3NF). Le programme de Terminale ne l'exige pas formellement, mais en comprendre l'intuition (une information ne doit être stockée qu'à un seul endroit) aide à concevoir des schémas robustes.

3.2 Le système de gestion de bases de données

DÉFINITION 1

Un **système de gestion de bases de données** (SGBD) est un logiciel qui prend en charge le stockage, l'interrogation et la modification des données d'une base, en offrant quatre services principaux :

- ▶ la **persistance** : les données restent stockées durablement, indépendamment de l'exécution d'un programme particulier ;
- ▶ la **concurrence** : plusieurs utilisateurs ou programmes peuvent accéder à la base simultanément sans corrompre les données ;
- ▶ l'**efficacité** : les requêtes sur de grands volumes de données restent rapides, notamment grâce à des index ;
- ▶ la **sécurisation** : l'accès aux données est contrôlé (droits d'utilisateurs) et leur intégrité est préservée (contraintes, transactions).

Exemple. La persistance : une base créée dans un fichier reste accessible après la fermeture du programme qui l'a créée.

```
1 import sqlite3
2
3 conn = sqlite3.connect("bibliotheque.db")
4 conn.execute("CREATE TABLE IF NOT EXISTS Livre(id INTEGER PRIMARY KEY, titre TEXT)")
5 conn.execute("INSERT INTO Livre(titre) VALUES ('1984')")
6 conn.commit()
7 conn.close()
8 # Le fichier bibliotheque.db contient désormais la table et sa ligne,
9 # meme apres la fin du programme.
```

◆ **PROPRIÉTÉ Transaction** Une **transaction** regroupe plusieurs opérations en un tout indivisible : soit toutes les opérations réussissent et sont **validées** (COMMIT), soit une échoue et toutes sont **annulées** (ROLLBACK), pour ne jamais laisser la base dans un état intermédiaire incohérent.

⚠ ERREUR FRÉQUENTE

Confondre un SGBD et un simple fichier. Un fichier texte ou CSV stocke des données mais n'offre **aucun** des quatre services ci-dessus : pas de contrôle de concurrence, pas de garantie d'intégrité, pas de langage d'interrogation efficace sur de gros volumes. C'est précisément pour ces raisons qu'on utilise un SGBD dès que les données doivent être partagées, protégées ou interrogées de façon complexe.

3.3 Interroger une base : SELECT, FROM, WHERE

DÉFINITION 1

Le langage **SQL** (*Structured Query Language*) permet d'interroger et de manipuler une base relationnelle. Une requête d'interrogation combine des **clauses**, chacune introduite par un **mot clé** : **SELECT** (quels attributs afficher), **FROM** (dans quelle relation), et optionnellement **WHERE** (quelle condition sur les tuples).

Exemple. Base d'exemple filée sur tout le chapitre (bibliothèque) :

```
1 CREATE TABLE Livre(id INTEGER PRIMARY KEY, titre TEXT, auteur TEXT, année INTEGER);
2 INSERT INTO Livre VALUES
3 (1, 'Le Petit Prince', 'Saint-Exupéry', 1943),
4 (2, '1984', 'Orwell', 1949),
5 (3, 'La Peste', 'Camus', 1947);
```

Afficher uniquement les titres et auteurs de tous les livres :

```
1 SELECT titre, auteur FROM Livre;
```

◆ **PROPRIÉTÉ Filtrer avec WHERE** La clause **WHERE** ne conserve que les tuples vérifiant une **condition**. Les opérateurs de comparaison usuels (=, <, >, <=, >=, <> pour différent) se combinent avec **AND**, **OR**, **NOT** pour des conditions composées ; **LIKE** permet une recherche approchée sur du texte (motif avec le joker %).

Exemple. Livres parus après 1945 :

```
1 SELECT titre FROM Livre WHERE année > 1945;
```

Livres d'Orwell ou de Camus :

```
1 SELECT titre FROM Livre WHERE auteur = 'Orwell' OR auteur = 'Camus';
```

⚠ ERREUR FRÉQUENTE

Oublier les guillemets autour d'une chaîne de caractères. **WHERE** auteur = Orwell est une erreur : SQL interprète Orwell sans guillemets comme un nom de colonne inexistant. Il faut écrire **WHERE** auteur = 'Orwell', avec des apostrophes simples autour du texte.

3.4 Croiser les données : JOIN et ORDER BY

DÉFINITION 1

La clause **JOIN** combine des tuples de deux relations dont les valeurs coïncident sur une colonne commune, en général une clé étrangère et la clé primaire qu'elle référence. La syntaxe **JOIN ... ON ...** précise la condition de rapprochement.

Exemple. Reprenons Livre et une table Emprunt qui référence Livre.id :

```
1 CREATE TABLE Emprunt(id INTEGER PRIMARY KEY, id_livre INTEGER, date_emprunt TEXT);
```

```
2 INSERT INTO Emprunt VALUES (1, 2, '2026-02-01'), (2, 3, '2026-02-03');
3
4 SELECT Emprunt.date_emprunt, Livre.titre
5 FROM Emprunt
6 JOIN Livre ON Emprunt.id_livre = Livre.id;
```

Chaque ligne du résultat associe une date d'emprunt au **titre** du livre correspondant, sans jamais avoir stocké ce titre dans la table Emprunt.

◆ **PROPRIÉTÉ Trier avec ORDER BY** La clause ORDER BY trie les tuples du résultat selon un ou plusieurs attributs, par ordre croissant (ASC, par défaut) ou décroissant (DESC).

Exemple. Livres triés du plus récent au plus ancien :

```
1 SELECT titre, année FROM Livre ORDER BY année DESC;
```

⚠ ERREUR FRÉQUENTE

Oublier que ORDER BY se place après WHERE. L'ordre des clauses est fixe : SELECT ... FROM ... WHERE ... ORDER BY Écrire ORDER BY avant WHERE est une erreur de syntaxe.

3.5 Modifier une base : INSERT, UPDATE, DELETE

DÉFINITION 1

La clause **INSERT INTO** ajoute un ou plusieurs tuples à une relation. On précise la relation, éventuellement la liste des attributs renseignés, puis les valeurs avec VALUES.

Exemple.

```
1 INSERT INTO Livre(titre, auteur, année)
2 VALUES ('Fahrenheit 451', 'Bradbury', 1953);
```

◆ **PROPRIÉTÉ Mettre à jour avec UPDATE** La clause UPDATE ... SET ... WHERE ... modifie la valeur d'un ou plusieurs attributs pour les tuples vérifiant une condition. **Sans clause WHERE, tous les tuples de la relation sont modifiés.**

Exemple. Corriger l'année de parution d'un livre :

```
1 UPDATE Livre SET année = 1954 WHERE titre = 'Fahrenheit 451';
```

DÉFINITION 2

La clause **DELETE FROM ... WHERE ...** supprime les tuples vérifiant une condition. **Sans clause WHERE, tous les tuples de la relation sont supprimés** (la table reste, mais devient vide).

Exemple. Supprimer les livres parus avant 1950 :

```
1 DELETE FROM Livre WHERE année < 1950;
```

ERREUR FRÉQUENTE

Exécuter un **UPDATE** ou un **DELETE** sans **WHERE**. C'est l'erreur la plus dangereuse en SQL : `DELETE FROM Livre;` (sans condition) supprime **tous** les livres, et `UPDATE Livre SET annee = 2000;` écrase l'année de **tous** les livres. Il faut toujours vérifier, avant d'exécuter une modification, quels tuples seraient concernés en testant d'abord la condition avec un **SELECT**.

» **Python À la machine.** Avant tout **UPDATE** ou **DELETE**, exécute d'abord le **SELECT** correspondant avec la même clause **WHERE** pour vérifier quels tuples seront affectés.

3.6 Le modèle relationnel

DÉFINITION 1

Le **modèle relationnel** organise les données en **relations** (tables). Une relation est constituée de **tuples** (lignes), chacun décrivant une occurrence, et d'**attributs** (colonnes), chacun défini sur un **domaine** (l'ensemble des valeurs possibles, par exemple entier ou texte). Le **schéma** d'une relation liste le nom de la relation et le nom de chacun de ses attributs.

DÉFINITION 2

Une **clé primaire** est un attribut (ou groupe d'attributs) dont la valeur identifie de façon **unique** chaque tuple d'une relation. Une **clé étrangère** est un attribut d'une relation dont la valeur fait référence à la clé primaire d'une autre relation : c'est ainsi que deux relations sont mises en lien.

Exemple. Une bibliothèque gère trois relations liées entre elles :

```
1 Livre(id, titre, auteur, année)
2 Usager(id, nom, prenom)
3 Emprunt(id, id_livre, id_usager, date_emprunt, date_retour)
```

Dans `Emprunt`, `id_livre` est une clé étrangère vers `Livre.id`, et `id_usager` une clé étrangère vers `Usager.id` : chaque emprunt référence un livre et un usager précis, sans dupliquer leurs informations.

♦ **PROPRIÉTÉ Structure et contenu** La **structure** d'une base de données (son schéma : relations, attributs, clés, contraintes) est stable dans le temps. Le **contenu** (les tuples effectivement stockés) évolue à chaque insertion, modification ou suppression. Modifier le contenu ne change jamais la structure ; modifier la structure (ajouter une colonne, par exemple) laisse le contenu existant intact autant que possible.

ERREUR FRÉQUENTE

Confondre relation et fichier plat. Stocker toutes les informations d'une bibliothèque dans une seule table `Emprunt(id, titre_livre, auteur, nom_usager, prenom_usager, ...)` **duplique** le titre et l'auteur à chaque emprunt du même livre : c'est une **anomalie de schéma**. Une modification (correction d'une faute dans un titre) doit alors être répercutée sur toutes

les lignes concernées, au risque d'incohérences. Séparer Livre, Usager et Emprunt en trois relations liées par des clés étrangères élimine cette duplication.

+ POUR ALLER PLUS LOIN

Ce principe de séparation pour éviter les anomalies porte un nom en théorie des bases de données : la **normalisation** (formes normales 1NF, 2NF, 3NF). Le programme de Terminale ne l'exige pas formellement, mais en comprendre l'intuition (une information ne doit être stockée qu'à un seul endroit) aide à concevoir des schémas robustes.

3.7 Le système de gestion de bases de données

DÉFINITION 1

Un **système de gestion de bases de données** (SGBD) est un logiciel qui prend en charge le stockage, l'interrogation et la modification des données d'une base, en offrant quatre services principaux :

- ▶ la **persistance** : les données restent stockées durablement, indépendamment de l'exécution d'un programme particulier ;
- ▶ la **concurrence** : plusieurs utilisateurs ou programmes peuvent accéder à la base simultanément sans corrompre les données ;
- ▶ l'**efficacité** : les requêtes sur de grands volumes de données restent rapides, notamment grâce à des index ;
- ▶ la **sécurisation** : l'accès aux données est contrôlé (droits d'utilisateurs) et leur intégrité est préservée (contraintes, transactions).

Exemple. La persistance : une base créée dans un fichier reste accessible après la fermeture du programme qui l'a créée.

```
1 import sqlite3
2
3 conn = sqlite3.connect("bibliotheque.db")
4 conn.execute("CREATE TABLE IF NOT EXISTS Livre(id INTEGER PRIMARY KEY, titre TEXT)")
5 conn.execute("INSERT INTO Livre(titre) VALUES ('1984')")
6 conn.commit()
7 conn.close()
8 # Le fichier bibliotheque.db contient désormais la table et sa ligne,
9 # même après la fin du programme.
```

◆ **PROPRIÉTÉ Transaction** Une **transaction** regroupe plusieurs opérations en un tout indivisible : soit toutes les opérations réussissent et sont **validées** (COMMIT), soit une échoue et toutes sont **annulées** (ROLLBACK), pour ne jamais laisser la base dans un état intermédiaire incohérent.

⚠ ERREUR FRÉQUENTE

Confondre un SGBD et un simple fichier. Un fichier texte ou CSV stocke des données mais n'offre **aucun** des quatre services ci-dessus : pas de contrôle de concurrence, pas de garantie d'intégrité, pas de langage d'interrogation efficace sur de gros volumes. C'est précisément

pour ces raisons qu'on utilise un SGBD dès que les données doivent être partagées, protégées ou interrogées de façon complexe.

3.8 Interroger une base : SELECT, FROM, WHERE

DÉFINITION 1

Le langage **SQL** (*Structured Query Language*) permet d'interroger et de manipuler une base relationnelle. Une requête d'interrogation combine des **clauses**, chacune introduite par un **mot clé** : **SELECT** (quels attributs afficher), **FROM** (dans quelle relation), et optionnellement **WHERE** (quelle condition sur les tuples).

Exemple. Base d'exemple filée sur tout le chapitre (bibliothèque) :

```
1 CREATE TABLE Livre(id INTEGER PRIMARY KEY, titre TEXT, auteur TEXT, année INTEGER);
2 INSERT INTO Livre VALUES
3 (1, 'Le Petit Prince', 'Saint-Exupéry', 1943),
4 (2, '1984', 'Orwell', 1949),
5 (3, 'La Peste', 'Camus', 1947);
```

Afficher uniquement les titres et auteurs de tous les livres :

```
1 SELECT titre, auteur FROM Livre;
```

◆ **PROPRIÉTÉ Filtrer avec WHERE** La clause **WHERE** ne conserve que les tuples vérifiant une **condition**. Les opérateurs de comparaison usuels (=, <, >, <=, >=, <> pour différent) se combinent avec **AND**, **OR**, **NOT** pour des conditions composées ; **LIKE** permet une recherche approchée sur du texte (motif avec le joker %).

Exemple. Livres parus après 1945 :

```
1 SELECT titre FROM Livre WHERE année > 1945;
```

Livres d'Orwell ou de Camus :

```
1 SELECT titre FROM Livre WHERE auteur = 'Orwell' OR auteur = 'Camus';
```

⚠ ERREUR FRÉQUENTE

Oublier les guillemets autour d'une chaîne de caractères. **WHERE** auteur = Orwell est une erreur : SQL interprète Orwell sans guillemets comme un nom de colonne inexistant. Il faut écrire **WHERE** auteur = 'Orwell', avec des apostrophes simples autour du texte.

3.9 Croiser les données : JOIN et ORDER BY

DÉFINITION 1

La clause **JOIN** combine des tuples de deux relations dont les valeurs coïncident sur une colonne commune, en général une clé étrangère et la clé primaire qu'elle référence. La syntaxe `JOIN ... ON ...` précise la condition de rapprochement.

Exemple. Reprenons Livre et une table Emprunt qui référence Livre.id :

```
1 CREATE TABLE Emprunt(id INTEGER PRIMARY KEY, id_livre INTEGER, date_emprunt TEXT);
2 INSERT INTO Emprunt VALUES (1, 2, '2026-02-01'), (2, 3, '2026-02-03');
3
4 SELECT Emprunt.date_emprunt, Livre.titre
5 FROM Emprunt
6 JOIN Livre ON Emprunt.id_livre = Livre.id;
```

Chaque ligne du résultat associe une date d'emprunt au **titre** du livre correspondant, sans jamais avoir stocké ce titre dans la table Emprunt.

◆ **PROPRIÉTÉ Trier avec ORDER BY** La clause **ORDER BY** trie les tuples du résultat selon un ou plusieurs attributs, par ordre croissant (ASC, par défaut) ou décroissant (DESC).

Exemple. Livres triés du plus récent au plus ancien :

```
1 SELECT titre, année FROM Livre ORDER BY année DESC;
```

⚠ ERREUR FRÉQUENTE

Oublier que ORDER BY se place après WHERE. L'ordre des clauses est fixe : `SELECT ... FROM ... WHERE ... ORDER BY ...`. Écrire `ORDER BY` avant `WHERE` est une erreur de syntaxe.

3.10 Modifier une base : INSERT, UPDATE, DELETE

DÉFINITION 1

La clause **INSERT INTO** ajoute un ou plusieurs tuples à une relation. On précise la relation, éventuellement la liste des attributs renseignés, puis les valeurs avec **VALUES**.

Exemple.

```
1 INSERT INTO Livre(titre, auteur, année)
2 VALUES ('Fahrenheit 451', 'Bradbury', 1953);
```

◆ **PROPRIÉTÉ Mettre à jour avec UPDATE** La clause **UPDATE ... SET ... WHERE ...** modifie la valeur d'un ou plusieurs attributs pour les tuples vérifiant une condition. **Sans clause WHERE, tous les tuples de la relation sont modifiés.**

Exemple. Corriger l'année de parution d'un livre :

```
1 UPDATE Livre SET année = 1954 WHERE titre = 'Fahrenheit 451';
```

DÉFINITION 2

La clause **DELETE FROM ... WHERE ...** supprime les tuples vérifiant une condition. **Sans clause WHERE, tous les tuples de la relation sont supprimés** (la table reste, mais devient vide).

Exemple. Supprimer les livres parus avant 1950 :

```
1 DELETE FROM Livre WHERE année < 1950;
```

⚠ ERREUR FRÉQUENTE

Exécuter un UPDATE ou un DELETE sans WHERE. C'est l'erreur la plus dangereuse en SQL : `DELETE FROM Livre;` (sans condition) supprime **tous** les livres, et `UPDATE Livre SET année = 2000;` écrase l'année de **tous** les livres. Il faut toujours vérifier, avant d'exécuter une modification, quels tuples seraient concernés en testant d'abord la condition avec un `SELECT`.

» **Python À la machine.** Avant tout `UPDATE` ou `DELETE`, exécute d'abord le `SELECT` correspondant avec la même clause `WHERE` pour vérifier quels tuples seront affectés.

3.11 Le modèle relationnel

DÉFINITION 1

Le **modèle relationnel** organise les données en **relations** (tables). Une relation est constituée de **tuples** (lignes), chacun décrivant une occurrence, et d'**attributs** (colonnes), chacun défini sur un **domaine** (l'ensemble des valeurs possibles, par exemple entier ou texte). Le **schéma** d'une relation liste le nom de la relation et le nom de chacun de ses attributs.

DÉFINITION 2

Une **clé primaire** est un attribut (ou groupe d'attributs) dont la valeur identifie de façon **unique** chaque tuple d'une relation. Une **clé étrangère** est un attribut d'une relation dont la valeur fait référence à la clé primaire d'une autre relation : c'est ainsi que deux relations sont mises en lien.

Exemple. Une bibliothèque gère trois relations liées entre elles :

```
1 Livre(id, titre, auteur, année)
2 Usager(id, nom, prenom)
3 Emprunt(id, id_livre, id_usager, date_emprunt, date_retour)
```

Dans `Emprunt`, `id_livre` est une clé étrangère vers `Livre.id`, et `id_usager` une clé étrangère vers `Usager.id` : chaque emprunt référence un livre et un usager précis, sans dupliquer leurs informations.

◆ **PROPRIÉTÉ Structure et contenu** La **structure** d'une base de données (son schéma : relations, attributs, clés, contraintes) est stable dans le temps. Le **contenu** (les tuples effectivement stockés) évolue à chaque insertion, modification ou suppression. Modifier le contenu ne change jamais la structure ; modifier la structure (ajouter une colonne, par exemple) laisse le contenu existant intact autant que possible.

⚠ ERREUR FRÉQUENTE

Confondre relation et fichier plat. Stocker toutes les informations d'une bibliothèque dans une seule table `Emprunt(id, titre_livre, auteur, nom_usager, prenom_usager, ...)` **duplique** le titre et l'auteur à chaque emprunt du même livre : c'est une **anomalie de schéma**. Une modification (correction d'une faute dans un titre) doit alors être répercutée sur toutes les lignes concernées, au risque d'incohérences. Séparer `Livre`, `Usager` et `Emprunt` en trois relations liées par des clés étrangères élimine cette duplication.

+ POUR ALLER PLUS LOIN

Ce principe de séparation pour éviter les anomalies porte un nom en théorie des bases de données : la **normalisation** (formes normales 1NF, 2NF, 3NF). Le programme de Terminale ne l'exige pas formellement, mais en comprendre l'intuition (une information ne doit être stockée qu'à un seul endroit) aide à concevoir des schémas robustes.

3.12 Le système de gestion de bases de données

DÉFINITION 1

Un **système de gestion de bases de données** (SGBD) est un logiciel qui prend en charge le stockage, l'interrogation et la modification des données d'une base, en offrant quatre services principaux :

- ▶ la **persistance** : les données restent stockées durablement, indépendamment de l'exécution d'un programme particulier ;
- ▶ la **concurrence** : plusieurs utilisateurs ou programmes peuvent accéder à la base simultanément sans corrompre les données ;
- ▶ l'**efficacité** : les requêtes sur de grands volumes de données restent rapides, notamment grâce à des index ;
- ▶ la **sécurisation** : l'accès aux données est contrôlé (droits d'utilisateurs) et leur intégrité est préservée (contraintes, transactions).

Exemple. La persistance : une base créée dans un fichier reste accessible après la fermeture du programme qui l'a créée.

```
1 import sqlite3
2
3 conn = sqlite3.connect("bibliotheque.db")
4 conn.execute("CREATE TABLE IF NOT EXISTS Livre(id INTEGER PRIMARY KEY, titre TEXT)")
5 conn.execute("INSERT INTO Livre(titre) VALUES ('1984')")
6 conn.commit()
7 conn.close()
8 # Le fichier bibliotheque.db contient désormais la table et sa ligne,
9 # même après la fin du programme.
```

◆ **PROPRIÉTÉ Transaction** Une **transaction** regroupe plusieurs opérations en un tout indivisible : soit toutes les opérations réussissent et sont **validées** (COMMIT), soit une échoue et toutes sont **annulées** (ROLLBACK), pour ne jamais laisser la base dans un état intermédiaire incohérent.

⚠ ERREUR FRÉQUENTE

Confondre un SGBD et un simple fichier. Un fichier texte ou CSV stocke des données mais n'offre **aucun** des quatre services ci-dessus : pas de contrôle de concurrence, pas de garantie d'intégrité, pas de langage d'interrogation efficace sur de gros volumes. C'est précisément pour ces raisons qu'on utilise un SGBD dès que les données doivent être partagées, protégées ou interrogées de façon complexe.

3.13 Interroger une base : SELECT, FROM, WHERE

DÉFINITION 1

Le langage **SQL** (*Structured Query Language*) permet d'interroger et de manipuler une base relationnelle. Une requête d'interrogation combine des **clauses**, chacune introduite par un **mot clé** : SELECT (quels attributs afficher), FROM (dans quelle relation), et optionnellement WHERE (quelle condition sur les tuples).

Exemple. Base d'exemple filée sur tout le chapitre (bibliothèque) :

```
1 CREATE TABLE Livre(id INTEGER PRIMARY KEY, titre TEXT, auteur TEXT, année INTEGER);
2 INSERT INTO Livre VALUES
3 (1, 'Le Petit Prince', 'Saint-Exupéry', 1943),
4 (2, '1984', 'Orwell', 1949),
5 (3, 'La Peste', 'Camus', 1947);
```

Afficher uniquement les titres et auteurs de tous les livres :

```
1 SELECT titre, auteur FROM Livre;
```

◆ **PROPRIÉTÉ Filtrer avec WHERE** La clause WHERE ne conserve que les tuples vérifiant une **condition**. Les opérateurs de comparaison usuels (=, <, >, <=, >=, <> pour différent) se combinent avec AND, OR, NOT pour des conditions composées ; LIKE permet une recherche approchée sur du texte (motif avec le joker %).

Exemple. Livres parus après 1945 :

```
1 SELECT titre FROM Livre WHERE année > 1945;
```

Livres d'Orwell ou de Camus :

```
1 SELECT titre FROM Livre WHERE auteur = 'Orwell' OR auteur = 'Camus';
```

ERREUR FRÉQUENTE

Oublier les guillemets autour d'une chaîne de caractères. `WHERE auteur = Orwell` est une erreur : SQL interprète `Orwell` sans guillemets comme un nom de colonne inexistant. Il faut écrire `WHERE auteur = 'Orwell'`, avec des apostrophes simples autour du texte.

3.14 Croiser les données : JOIN et ORDER BY

DÉFINITION 1

La clause **JOIN** combine des tuples de deux relations dont les valeurs coïncident sur une colonne commune, en général une clé étrangère et la clé primaire qu'elle référence. La syntaxe `JOIN ... ON ...` précise la condition de rapprochement.

Exemple. Reprenons `Livre` et une table `Emprunt` qui référence `Livre.id` :

```
1 CREATE TABLE Emprunt(id INTEGER PRIMARY KEY, id_livre INTEGER, date_emprunt TEXT);
2 INSERT INTO Emprunt VALUES (1, 2, '2026-02-01'), (2, 3, '2026-02-03');
3
4 SELECT Emprunt.date_emprunt, Livre.titre
5 FROM Emprunt
6 JOIN Livre ON Emprunt.id_livre = Livre.id;
```

Chaque ligne du résultat associe une date d'emprunt au **titre** du livre correspondant, sans jamais avoir stocké ce titre dans la table `Emprunt`.

◆ **PROPRIÉTÉ Trier avec ORDER BY** La clause `ORDER BY` trie les tuples du résultat selon un ou plusieurs attributs, par ordre croissant (`ASC`, par défaut) ou décroissant (`DESC`).

Exemple. Livres triés du plus récent au plus ancien :

```
1 SELECT titre, année FROM Livre ORDER BY année DESC;
```

ERREUR FRÉQUENTE

Oublier que **ORDER BY** se place après **WHERE**. L'ordre des clauses est fixe : `SELECT ... FROM ... WHERE ... ORDER BY ...`. Écrire `ORDER BY` avant `WHERE` est une erreur de syntaxe.

3.15 Modifier une base : INSERT, UPDATE, DELETE

DÉFINITION 1

La clause **INSERT INTO** ajoute un ou plusieurs tuples à une relation. On précise la relation, éventuellement la liste des attributs renseignés, puis les valeurs avec `VALUES`.

Exemple.

```
1 INSERT INTO Livre(titre, auteur, année)
2 VALUES ('Fahrenheit 451', 'Bradbury', 1953);
```

◆ **PROPRIÉTÉ Mettre à jour avec UPDATE** La clause **UPDATE ... SET ... WHERE ...** modifie la valeur d'un ou plusieurs attributs pour les tuples vérifiant une condition. **Sans clause WHERE, tous les tuples de la relation sont modifiés.**

Exemple. Corriger l'année de parution d'un livre :

```
1 UPDATE Livre SET année = 1954 WHERE titre = 'Fahrenheit 451';
```

DÉFINITION 2

La clause **DELETE FROM ... WHERE ...** supprime les tuples vérifiant une condition. **Sans clause WHERE, tous les tuples de la relation sont supprimés** (la table reste, mais devient vide).

Exemple. Supprimer les livres parus avant 1950 :

```
1 DELETE FROM Livre WHERE année < 1950;
```

⚠ ERREUR FRÉQUENTE

Exécuter un UPDATE ou un DELETE sans WHERE. C'est l'erreur la plus dangereuse en SQL : `DELETE FROM Livre;` (sans condition) supprime **tous** les livres, et `UPDATE Livre SET année = 2000;` écrase l'année de **tous** les livres. Il faut toujours vérifier, avant d'exécuter une modification, quels tuples seraient concernés en testant d'abord la condition avec un `SELECT`.

» **Python À la machine.** Avant tout `UPDATE` ou `DELETE`, exécute d'abord le `SELECT` correspondant avec la même clause `WHERE` pour vérifier quels tuples seront affectés.

3.16 Le modèle relationnel

DÉFINITION 1

Le **modèle relationnel** organise les données en **relations** (tables). Une relation est constituée de **tuples** (lignes), chacun décrivant une occurrence, et d'**attributs** (colonnes), chacun défini sur un **domaine** (l'ensemble des valeurs possibles, par exemple entier ou texte). Le **schéma** d'une relation liste le nom de la relation et le nom de chacun de ses attributs.

DÉFINITION 2

Une **clé primaire** est un attribut (ou groupe d'attributs) dont la valeur identifie de façon **unique** chaque tuple d'une relation. Une **clé étrangère** est un attribut d'une relation dont la valeur fait référence à la clé primaire d'une autre relation : c'est ainsi que deux relations sont mises en lien.

Exemple. Une bibliothèque gère trois relations liées entre elles :

```
1 Livre(id, titre, auteur, année)
2 Usager(id, nom, prenom)
3 Emprunt(id, id_livre, id_usager, date_emprunt, date_retour)
```

Dans Emprunt, `id_livre` est une clé étrangère vers `Livre.id`, et `id_usager` une clé étrangère vers `Usager.id` : chaque emprunt référence un livre et un usager précis, sans dupliquer leurs informations.

♦ **PROPRIÉTÉ Structure et contenu** La **structure** d'une base de données (son schéma : relations, attributs, clés, contraintes) est stable dans le temps. Le **contenu** (les tuples effectivement stockés) évolue à chaque insertion, modification ou suppression. Modifier le contenu ne change jamais la structure ; modifier la structure (ajouter une colonne, par exemple) laisse le contenu existant intact autant que possible.

⚠ ERREUR FRÉQUENTE

Confondre relation et fichier plat. Stocker toutes les informations d'une bibliothèque dans une seule table `Emprunt(id, titre_livre, auteur, nom_usager, prenom_usager, ...)` **duplique** le titre et l'auteur à chaque emprunt du même livre : c'est une **anomalie de schéma**. Une modification (correction d'une faute dans un titre) doit alors être répercutée sur toutes les lignes concernées, au risque d'incohérences. Séparer `Livre`, `Usager` et `Emprunt` en trois relations liées par des clés étrangères élimine cette duplication.

+ POUR ALLER PLUS LOIN

Ce principe de séparation pour éviter les anomalies porte un nom en théorie des bases de données : la **normalisation** (formes normales 1NF, 2NF, 3NF). Le programme de Terminale ne l'exige pas formellement, mais en comprendre l'intuition (une information ne doit être stockée qu'à un seul endroit) aide à concevoir des schémas robustes.

3.17 Le système de gestion de bases de données

DÉFINITION 1

Un **système de gestion de bases de données** (SGBD) est un logiciel qui prend en charge le stockage, l'interrogation et la modification des données d'une base, en offrant quatre services principaux :

- ▶ la **persistance** : les données restent stockées durablement, indépendamment de l'exécution d'un programme particulier ;
- ▶ la **concurrence** : plusieurs utilisateurs ou programmes peuvent accéder à la base simultanément sans corrompre les données ;
- ▶ l'**efficacité** : les requêtes sur de grands volumes de données restent rapides, notamment grâce à des index ;
- ▶ la **sécurisation** : l'accès aux données est contrôlé (droits d'utilisateurs) et leur intégrité est préservée (contraintes, transactions).

Exemple. La persistance : une base créée dans un fichier reste accessible après la fermeture du programme qui l'a créée.

```
1 import sqlite3
2
3 conn = sqlite3.connect("bibliotheque.db")
4 conn.execute("CREATE TABLE IF NOT EXISTS Livre(id INTEGER PRIMARY KEY, titre TEXT)")
5 conn.execute("INSERT INTO Livre(titre) VALUES ('1984')")
6 conn.commit()
7 conn.close()
```

```
8 # Le fichier bibliotheque.db contient désormais la table et sa ligne,
9 # meme apres la fin du programme.
```

◆ **PROPRIÉTÉ Transaction** Une **transaction** regroupe plusieurs opérations en un tout indivisible : soit toutes les opérations réussissent et sont **validées** (COMMIT), soit une échoue et toutes sont **annulées** (ROLLBACK), pour ne jamais laisser la base dans un état intermédiaire incohérent.

⚠ ERREUR FRÉQUENTE

Confondre un SGBD et un simple fichier. Un fichier texte ou CSV stocke des données mais n'offre **aucun** des quatre services ci-dessus : pas de contrôle de concurrence, pas de garantie d'intégrité, pas de langage d'interrogation efficace sur de gros volumes. C'est précisément pour ces raisons qu'on utilise un SGBD dès que les données doivent être partagées, protégées ou interrogées de façon complexe.

Méthodes

- Méthode directe de travail sur le sujet.

Exercice 1 ◆

Un club de sport stocke tous ses adhérents dans une unique table :

```
1 Inscription(id, nom_adherent, prenom_adherent, nom_sport, jour_seance, nom_coach)
```

1. Un adhérent pratiquant deux sports différents apparaît-il une ou deux fois dans cette table ? Quel problème cela pose-t-il si l'on corrige une faute de frappe dans son nom ?
2. Proposer un découpage de cette table en plusieurs relations (avec leurs attributs) qui élimine ce problème. Préciser au moins une clé étrangère.

Exercice 2 ◆

On dispose de la table suivante :

```
1 CREATE TABLE Film(id INTEGER PRIMARY KEY, titre TEXT, realisateur TEXT, duree INTEGER);
2 INSERT INTO Film VALUES
3 (1, 'Le Voyage de Chihiro', 'Miyazaki', 125),
4 (2, 'Princesse Mononoke', 'Miyazaki', 134),
5 (3, 'Vice-versa', 'Docter', 95);
```

Écrire les requêtes SQL permettant d'afficher :

1. les titres des films réalisés par Miyazaki ;
2. les titres des films dont la durée dépasse 100 minutes.

Exercice 3 ◆◆

On dispose de deux tables :

```
1 Eleve(id, nom)
2 Note(id, id_eleve, matiere, valeur)
```

où `Note.id_eleve` est une clé étrangère vers `Eleve.id`. Écrire une requête SQL affichant, pour chaque note, le nom de l'élève, la matière et la valeur de la note, triées par valeur **décroissante**.

Exercice 4

On dispose de la table :

```
1 CREATE TABLE Stock(id INTEGER PRIMARY KEY, produit TEXT, quantité INTEGER);
2 INSERT INTO Stock VALUES (1, 'Cahier', 40), (2, 'Stylo', 0), (3, 'Règle', 12);
```

Écrire les requêtes SQL permettant, dans l'ordre, de :

1. ajouter un produit 'Gomme' avec une quantité de 25 ;
2. diminuer de 5 la quantité de 'Cahier' (sans réécrire la valeur numérique finale) ;
3. supprimer tous les produits dont la quantité en stock est nulle.

Exercice 5

Un club de sport stocke tous ses adhérents dans une unique table :

```
1 Inscription(id, nom_adherent, prenom_adherent, nom_sport, jour_seance, nom_coach)
```

1. Un adhérent pratiquant deux sports différents apparaît-il une ou deux fois dans cette table ? Quel problème cela pose-t-il si l'on corrige une faute de frappe dans son nom ?
2. Proposer un découpage de cette table en plusieurs relations (avec leurs attributs) qui élimine ce problème. Préciser au moins une clé étrangère.

Exercice 6

On dispose de la table suivante :

```
1 CREATE TABLE Film(id INTEGER PRIMARY KEY, titre TEXT, realisateur TEXT, duree INTEGER);
2 INSERT INTO Film VALUES
3 (1, 'Le Voyage de Chihiro', 'Miyazaki', 125),
4 (2, 'Princesse Mononoke', 'Miyazaki', 134),
5 (3, 'Vice-versa', 'Docter', 95);
```

Écrire les requêtes SQL permettant d'afficher :

1. les titres des films réalisés par Miyazaki ;
2. les titres des films dont la durée dépasse 100 minutes.

Exercice 7

On dispose de deux tables :

```
1 Eleve(id, nom)
2 Note(id, id_eleve, matiere, valeur)
```

où `Note.id_eleve` est une clé étrangère vers `Eleve.id`. Écrire une requête SQL affichant, pour chaque note, le nom de l'élève, la matière et la valeur de la note, triées par valeur **décroissante**.

Exercice 8

On dispose de la table :

```
1 CREATE TABLE Stock(id INTEGER PRIMARY KEY, produit TEXT, quantité INTEGER);
2 INSERT INTO Stock VALUES (1, 'Cahier', 40), (2, 'Stylo', 0), (3, 'Règle', 12);
```

Écrire les requêtes SQL permettant, dans l'ordre, de :

1. ajouter un produit 'Gomme' avec une quantité de 25 ;
2. diminuer de 5 la quantité de 'Cahier' (sans réécrire la valeur numérique finale) ;
3. supprimer tous les produits dont la quantité en stock est nulle.

Exercice 9

Un club de sport stocke tous ses adhérents dans une unique table :

```
1 Inscription(id, nom_adherent, prenom_adherent, nom_sport, jour_seance, nom_coach)
```

1. Un adhérent pratiquant deux sports différents apparaît-il une ou deux fois dans cette table ? Quel problème cela pose-t-il si l'on corrige une faute de frappe dans son nom ?
2. Proposer un découpage de cette table en plusieurs relations (avec leurs attributs) qui élimine ce problème. Préciser au moins une clé étrangère.

Exercice 10

On dispose de la table suivante :

```
1 CREATE TABLE Film(id INTEGER PRIMARY KEY, titre TEXT, realisateur TEXT, duree INTEGER);
2 INSERT INTO Film VALUES
3 (1, 'Le Voyage de Chihiro', 'Miyazaki', 125),
4 (2, 'Princesse Mononoke', 'Miyazaki', 134),
5 (3, 'Vice-versa', 'Docter', 95);
```

Écrire les requêtes SQL permettant d'afficher :

1. les titres des films réalisés par Miyazaki ;
2. les titres des films dont la durée dépasse 100 minutes.

Exercice 11

On dispose de deux tables :

```
1 Eleve(id, nom)
2 Note(id, id_eleve, matiere, valeur)
```

où `Note.id_eleve` est une clé étrangère vers `Eleve.id`. Écrire une requête SQL affichant, pour chaque note, le nom de l'élève, la matière et la valeur de la note, triées par valeur **décroissante**.

Exercice 12

On dispose de la table :

```
1 CREATE TABLE Stock(id INTEGER PRIMARY KEY, produit TEXT, quantité INTEGER);
2 INSERT INTO Stock VALUES (1, 'Cahier', 40), (2, 'Stylo', 0), (3, 'Règle', 12);
```

Écrire les requêtes SQL permettant, dans l'ordre, de :

1. ajouter un produit 'Gomme' avec une quantité de 25 ;
2. diminuer de 5 la quantité de 'Cahier' (sans réécrire la valeur numérique finale) ;
3. supprimer tous les produits dont la quantité en stock est nulle.

Exercice 13

Un club de sport stocke tous ses adhérents dans une unique table :

```
1 Inscription(id, nom_adherent, prenom_adherent, nom_sport, jour_seance, nom_coach)
```

1. Un adhérent pratiquant deux sports différents apparaît-il une ou deux fois dans cette table ? Quel problème cela pose-t-il si l'on corrige une faute de frappe dans son nom ?
2. Proposer un découpage de cette table en plusieurs relations (avec leurs attributs) qui élimine ce problème. Préciser au moins une clé étrangère.

Exercice 14

On dispose de la table suivante :

```
1 CREATE TABLE Film(id INTEGER PRIMARY KEY, titre TEXT, realisateur TEXT, duree INTEGER);
2 INSERT INTO Film VALUES
3 (1, 'Le Voyage de Chihiro', 'Miyazaki', 125),
4 (2, 'Princesse Mononoke', 'Miyazaki', 134),
5 (3, 'Vice-versa', 'Docter', 95);
```

Écrire les requêtes SQL permettant d'afficher :

1. les titres des films réalisés par Miyazaki ;
2. les titres des films dont la durée dépasse 100 minutes.

Exercice 15

On dispose de deux tables :

```
1 Eleve(id, nom)
2 Note(id, id_eleve, matiere, valeur)
```

où Note.id_eleve est une clé étrangère vers Eleve.id. Écrire une requête SQL affichant, pour chaque note, le nom de l'élève, la matière et la valeur de la note, triées par valeur **décroissante**.

Exercice 16

On dispose de la table :

```
1 CREATE TABLE Stock(id INTEGER PRIMARY KEY, produit TEXT, quantité INTEGER);
2 INSERT INTO Stock VALUES (1, 'Cahier', 40), (2, 'Stylo', 0), (3, 'Règle', 12);
```

Écrire les requêtes SQL permettant, dans l'ordre, de :

1. ajouter un produit 'Gomme' avec une quantité de 25 ;
2. diminuer de 5 la quantité de 'Cahier' (sans réécrire la valeur numérique finale) ;
3. supprimer tous les produits dont la quantité en stock est nulle.

Exercice 17

Un club de sport stocke tous ses adhérents dans une unique table :

```
1 Inscription(id, nom_adherent, prenom_adherent, nom_sport, jour_seance, nom_coach)
```

1. Un adhérent pratiquant deux sports différents apparaît-il une ou deux fois dans cette table ? Quel problème cela pose-t-il si l'on corrige une faute de frappe dans son nom ?
2. Proposer un découpage de cette table en plusieurs relations (avec leurs attributs) qui élimine ce problème. Préciser au moins une clé étrangère.

Exercice 18 ◆

On dispose de la table suivante :

```
1 CREATE TABLE Film(id INTEGER PRIMARY KEY, titre TEXT, realisateur TEXT, duree INTEGER);
2 INSERT INTO Film VALUES
3 (1, 'Le Voyage de Chihiro', 'Miyazaki', 125),
4 (2, 'Princesse Mononoke', 'Miyazaki', 134),
5 (3, 'Vice-versa', 'Docter', 95);
```

Écrire les requêtes SQL permettant d'afficher :

1. les titres des films réalisés par Miyazaki ;
2. les titres des films dont la durée dépasse 100 minutes.

Exercice 19 ◆◆

On dispose de deux tables :

```
1 Eleve(id, nom)
2 Note(id, id_eleve, matiere, valeur)
```

où `Note.id_eleve` est une clé étrangère vers `Eleve.id`. Écrire une requête SQL affichant, pour chaque note, le nom de l'élève, la matière et la valeur de la note, triées par valeur **décroissante**.

Exercice 20 ◆◆

On dispose de la table :

```
1 CREATE TABLE Stock(id INTEGER PRIMARY KEY, produit TEXT, quantité INTEGER);
2 INSERT INTO Stock VALUES (1, 'Cahier', 40), (2, 'Stylo', 0), (3, 'Règle', 12);
```

Écrire les requêtes SQL permettant, dans l'ordre, de :

1. ajouter un produit 'Gomme' avec une quantité de 25 ;
2. diminuer de 5 la quantité de 'Cahier' (sans réécrire la valeur numérique finale) ;
3. supprimer tous les produits dont la quantité en stock est nulle.

Exercice 21 ◆

Un club de sport stocke tous ses adhérents dans une unique table :

```
1 Inscription(id, nom_adherent, prenom_adherent, nom_sport, jour_seance, nom_coach)
```

1. Un adhérent pratiquant deux sports différents apparaît-il une ou deux fois dans cette table ? Quel problème cela pose-t-il si l'on corrige une faute de frappe dans son nom ?
2. Proposer un découpage de cette table en plusieurs relations (avec leurs attributs) qui élimine ce problème. Préciser au moins une clé étrangère.

Exercice 22 ◆

On dispose de la table suivante :

```
1 CREATE TABLE Film(id INTEGER PRIMARY KEY, titre TEXT, realisateur TEXT, duree INTEGER);
2 INSERT INTO Film VALUES
3 (1, 'Le Voyage de Chihiro', 'Miyazaki', 125),
4 (2, 'Princesse Mononoke', 'Miyazaki', 134),
5 (3, 'Vice-versa', 'Docter', 95);
```

Écrire les requêtes SQL permettant d'afficher :

1. les titres des films réalisés par Miyazaki ;
2. les titres des films dont la durée dépasse 100 minutes.

Exercice 23 ♦♦

On dispose de deux tables :

```
1 Eleve(id, nom)
2 Note(id, id_eleve, matiere, valeur)
```

où Note.id_eleve est une clé étrangère vers Eleve.id. Écrire une requête SQL affichant, pour chaque note, le nom de l'élève, la matière et la valeur de la note, triées par valeur **décroissante**.

Exercice 24 ♦♦

On dispose de la table :

```
1 CREATE TABLE Stock(id INTEGER PRIMARY KEY, produit TEXT, quantité INTEGER);
2 INSERT INTO Stock VALUES (1, 'Cahier', 40), (2, 'Stylo', 0), (3, 'Règle', 12);
```

Écrire les requêtes SQL permettant, dans l'ordre, de :

1. ajouter un produit 'Gomme' avec une quantité de 25 ;
2. diminuer de 5 la quantité de 'Cahier' (sans réécrire la valeur numérique finale) ;
3. supprimer tous les produits dont la quantité en stock est nulle.

Exercice 25 ♦

Un club de sport stocke tous ses adhérents dans une unique table :

```
1 Inscription(id, nom_adherent, prenom_adherent, nom_sport, jour_seance, nom_coach)
```

1. Un adhérent pratiquant deux sports différents apparaît-il une ou deux fois dans cette table ? Quel problème cela pose-t-il si l'on corrige une faute de frappe dans son nom ?
2. Proposer un découpage de cette table en plusieurs relations (avec leurs attributs) qui élimine ce problème. Préciser au moins une clé étrangère.

Exercice 26 ♦

On dispose de la table suivante :

```
1 CREATE TABLE Film(id INTEGER PRIMARY KEY, titre TEXT, realisateur TEXT, duree INTEGER);
2 INSERT INTO Film VALUES
3 (1, 'Le Voyage de Chihiro', 'Miyazaki', 125),
4 (2, 'Princesse Mononoke', 'Miyazaki', 134),
5 (3, 'Vice-versa', 'Docter', 95);
```

Écrire les requêtes SQL permettant d'afficher :

1. les titres des films réalisés par Miyazaki ;
2. les titres des films dont la durée dépasse 100 minutes.

Exercice 27 ♦♦

On dispose de deux tables :

```

1 Eleve(id, nom)
2 Note(id, id_eleve, matiere, valeur)

```

où `Note.id_eleve` est une clé étrangère vers `Eleve.id`. Écrire une requête SQL affichant, pour chaque note, le nom de l'élève, la matière et la valeur de la note, triées par valeur **décroissante**.

Exercice 28 ♦♦

On dispose de la table :

```

1 CREATE TABLE Stock(id INTEGER PRIMARY KEY, produit TEXT, quantité INTEGER);
2 INSERT INTO Stock VALUES (1, 'Cahier', 40), (2, 'Stylo', 0), (3, 'Règle', 12);

```

Écrire les requêtes SQL permettant, dans l'ordre, de :

1. ajouter un produit 'Gomme' avec une quantité de 25 ;
2. diminuer de 5 la quantité de 'Cahier' (sans réécrire la valeur numérique finale) ;
3. supprimer tous les produits dont la quantité en stock est nulle.

Exercice 29 ♦

Un club de sport stocke tous ses adhérents dans une unique table :

```

1 Inscription(id, nom_adherent, prenom_adherent, nom_sport, jour_seance, nom_coach)

```

1. Un adhérent pratiquant deux sports différents apparaît-il une ou deux fois dans cette table ? Quel problème cela pose-t-il si l'on corrige une faute de frappe dans son nom ?
2. Proposer un découpage de cette table en plusieurs relations (avec leurs attributs) qui élimine ce problème. Préciser au moins une clé étrangère.

Exercice 30 ♦

On dispose de la table suivante :

```

1 CREATE TABLE Film(id INTEGER PRIMARY KEY, titre TEXT, realisateur TEXT, duree INTEGER);
2 INSERT INTO Film VALUES
3 (1, 'Le Voyage de Chihiro', 'Miyazaki', 125),
4 (2, 'Princesse Mononoke', 'Miyazaki', 134),
5 (3, 'Vice-versa', 'Docter', 95);

```

Écrire les requêtes SQL permettant d'afficher :

1. les titres des films réalisés par Miyazaki ;
2. les titres des films dont la durée dépasse 100 minutes.

Exercice 31 ♦♦

On dispose de deux tables :

```

1 Eleve(id, nom)
2 Note(id, id_eleve, matiere, valeur)

```

où `Note.id_eleve` est une clé étrangère vers `Eleve.id`. Écrire une requête SQL affichant, pour chaque note, le nom de l'élève, la matière et la valeur de la note, triées par valeur **décroissante**.

Exercice 32 ♦♦

On dispose de la table :

```
1 CREATE TABLE Stock(id INTEGER PRIMARY KEY, produit TEXT, quantité INTEGER);
2 INSERT INTO Stock VALUES (1, 'Cahier', 40), (2, 'Stylo', 0), (3, 'Règle', 12);
```

Écrire les requêtes SQL permettant, dans l'ordre, de :

1. ajouter un produit 'Gomme' avec une quantité de 25 ;
2. diminuer de 5 la quantité de 'Cahier' (sans réécrire la valeur numérique finale) ;
3. supprimer tous les produits dont la quantité en stock est nulle.

Exercice 33

Un club de sport stocke tous ses adhérents dans une unique table :

```
1 Inscription(id, nom_adherent, prenom_adherent, nom_sport, jour_seance, nom_coach)
```

1. Un adhérent pratiquant deux sports différents apparaît-il une ou deux fois dans cette table ? Quel problème cela pose-t-il si l'on corrige une faute de frappe dans son nom ?
2. Proposer un découpage de cette table en plusieurs relations (avec leurs attributs) qui élimine ce problème. Préciser au moins une clé étrangère.

Exercice 34

On dispose de la table suivante :

```
1 CREATE TABLE Film(id INTEGER PRIMARY KEY, titre TEXT, realisateur TEXT, duree INTEGER);
2 INSERT INTO Film VALUES
3 (1, 'Le Voyage de Chihiro', 'Miyazaki', 125),
4 (2, 'Princesse Mononoke', 'Miyazaki', 134),
5 (3, 'Vice-versa', 'Docter', 95);
```

Écrire les requêtes SQL permettant d'afficher :

1. les titres des films réalisés par Miyazaki ;
2. les titres des films dont la durée dépasse 100 minutes.

Exercice 35

On dispose de deux tables :

```
1 Eleve(id, nom)
2 Note(id, id_eleve, matiere, valeur)
```

où Note.id_eleve est une clé étrangère vers Eleve.id. Écrire une requête SQL affichant, pour chaque note, le nom de l'élève, la matière et la valeur de la note, triées par valeur **décroissante**.

Exercice 36

On dispose de la table :

```
1 CREATE TABLE Stock(id INTEGER PRIMARY KEY, produit TEXT, quantité INTEGER);
2 INSERT INTO Stock VALUES (1, 'Cahier', 40), (2, 'Stylo', 0), (3, 'Règle', 12);
```

Écrire les requêtes SQL permettant, dans l'ordre, de :

1. ajouter un produit 'Gomme' avec une quantité de 25 ;
2. diminuer de 5 la quantité de 'Cahier' (sans réécrire la valeur numérique finale) ;
3. supprimer tous les produits dont la quantité en stock est nulle.

Exercice 37 ◆

Un club de sport stocke tous ses adhérents dans une unique table :

```
1 Inscription(id, nom_adherent, prenom_adherent, nom_sport, jour_seance, nom_coach)
```

1. Un adhérent pratiquant deux sports différents apparaît-il une ou deux fois dans cette table ? Quel problème cela pose-t-il si l'on corrige une faute de frappe dans son nom ?
2. Proposer un découpage de cette table en plusieurs relations (avec leurs attributs) qui élimine ce problème. Préciser au moins une clé étrangère.

Exercice 38 ◆

On dispose de la table suivante :

```
1 CREATE TABLE Film(id INTEGER PRIMARY KEY, titre TEXT, realisateur TEXT, duree INTEGER);
2 INSERT INTO Film VALUES
3 (1, 'Le Voyage de Chihiro', 'Miyazaki', 125),
4 (2, 'Princesse Mononoke', 'Miyazaki', 134),
5 (3, 'Vice-versa', 'Docter', 95);
```

Écrire les requêtes SQL permettant d'afficher :

1. les titres des films réalisés par Miyazaki ;
2. les titres des films dont la durée dépasse 100 minutes.

Exercice 39 ◆◆

On dispose de deux tables :

```
1 Eleve(id, nom)
2 Note(id, id_eleve, matiere, valeur)
```

où `Note.id_eleve` est une clé étrangère vers `Eleve.id`. Écrire une requête SQL affichant, pour chaque note, le nom de l'élève, la matière et la valeur de la note, triées par valeur **décroissante**.

Exercice 40 ◆◆

On dispose de la table :

```
1 CREATE TABLE Stock(id INTEGER PRIMARY KEY, produit TEXT, quantité INTEGER);
2 INSERT INTO Stock VALUES (1, 'Cahier', 40), (2, 'Stylo', 0), (3, 'Règle', 12);
```

Écrire les requêtes SQL permettant, dans l'ordre, de :

1. ajouter un produit 'Gomme' avec une quantité de 25 ;
2. diminuer de 5 la quantité de 'Cahier' (sans réécrire la valeur numérique finale) ;
3. supprimer tous les produits dont la quantité en stock est nulle.

Exercice 41 ◆

Un club de sport stocke tous ses adhérents dans une unique table :

```
1 Inscription(id, nom_adherent, prenom_adherent, nom_sport, jour_seance, nom_coach)
```

1. Un adhérent pratiquant deux sports différents apparaît-il une ou deux fois dans cette table ? Quel problème cela pose-t-il si l'on corrige une faute de frappe dans son nom ?

- Proposer un découpage de cette table en plusieurs relations (avec leurs attributs) qui élimine ce problème. Préciser au moins une clé étrangère.

Exercice 42

On dispose de la table suivante :

```
1 CREATE TABLE Film(id INTEGER PRIMARY KEY, titre TEXT, realisateur TEXT, duree INTEGER);
2 INSERT INTO Film VALUES
3 (1, 'Le Voyage de Chihiro', 'Miyazaki', 125),
4 (2, 'Princesse Mononoke', 'Miyazaki', 134),
5 (3, 'Vice-versa', 'Docter', 95);
```

Écrire les requêtes SQL permettant d'afficher :

- les titres des films réalisés par Miyazaki ;
- les titres des films dont la durée dépasse 100 minutes.

Exercice 43

On dispose de deux tables :

```
1 Eleve(id, nom)
2 Note(id, id_eleve, matiere, valeur)
```

où Note.id_eleve est une clé étrangère vers Eleve.id. Écrire une requête SQL affichant, pour chaque note, le nom de l'élève, la matière et la valeur de la note, triées par valeur **décroissante**.

Exercice 44

On dispose de la table :

```
1 CREATE TABLE Stock(id INTEGER PRIMARY KEY, produit TEXT, quantité INTEGER);
2 INSERT INTO Stock VALUES (1, 'Cahier', 40), (2, 'Stylo', 0), (3, 'Règle', 12);
```

Écrire les requêtes SQL permettant, dans l'ordre, de :

- ajouter un produit 'Gomme' avec une quantité de 25 ;
- diminuer de 5 la quantité de 'Cahier' (sans réécrire la valeur numérique finale) ;
- supprimer tous les produits dont la quantité en stock est nulle.

Exercice 45

Un club de sport stocke tous ses adhérents dans une unique table :

```
1 Inscription(id, nom_adherent, prenom_adherent, nom_sport, jour_seance, nom_coach)
```

- Un adhérent pratiquant deux sports différents apparaît-il une ou deux fois dans cette table ? Quel problème cela pose-t-il si l'on corrige une faute de frappe dans son nom ?
- Proposer un découpage de cette table en plusieurs relations (avec leurs attributs) qui élimine ce problème. Préciser au moins une clé étrangère.

Exercice 46

On dispose de la table suivante :

```
1 CREATE TABLE Film(id INTEGER PRIMARY KEY, titre TEXT, realisateur TEXT, duree INTEGER);
```

```

2 INSERT INTO Film VALUES
3 (1, 'Le Voyage de Chihiro', 'Miyazaki', 125),
4 (2, 'Princesse Mononoke', 'Miyazaki', 134),
5 (3, 'Vice-versa', 'Docter', 95);

```

Écrire les requêtes SQL permettant d'afficher :

1. les titres des films réalisés par Miyazaki ;
2. les titres des films dont la durée dépasse 100 minutes.

Exercice 47 ♦♦

On dispose de deux tables :

```

1 Eleve(id, nom)
2 Note(id, id_eleve, matiere, valeur)

```

où `Note.id_eleve` est une clé étrangère vers `Eleve.id`. Écrire une requête SQL affichant, pour chaque note, le nom de l'élève, la matière et la valeur de la note, triées par valeur **décroissante**.

Exercice 48 ♦♦

On dispose de la table :

```

1 CREATE TABLE Stock(id INTEGER PRIMARY KEY, produit TEXT, quantité INTEGER);
2 INSERT INTO Stock VALUES (1, 'Cahier', 40), (2, 'Stylo', 0), (3, 'Règle', 12);

```

Écrire les requêtes SQL permettant, dans l'ordre, de :

1. ajouter un produit 'Gomme' avec une quantité de 25 ;
2. diminuer de 5 la quantité de 'Cahier' (sans réécrire la valeur numérique finale) ;
3. supprimer tous les produits dont la quantité en stock est nulle.

Exercice 49 ♦

Un club de sport stocke tous ses adhérents dans une unique table :

```

1 Inscription(id, nom_adherent, prenom_adherent, nom_sport, jour_seance, nom_coach)

```

1. Un adhérent pratiquant deux sports différents apparaît-il une ou deux fois dans cette table ? Quel problème cela pose-t-il si l'on corrige une faute de frappe dans son nom ?
2. Proposer un découpage de cette table en plusieurs relations (avec leurs attributs) qui élimine ce problème. Préciser au moins une clé étrangère.

Exercice 50 ♦

On dispose de la table suivante :

```

1 CREATE TABLE Film(id INTEGER PRIMARY KEY, titre TEXT, realisateur TEXT, duree INTEGER);
2 INSERT INTO Film VALUES
3 (1, 'Le Voyage de Chihiro', 'Miyazaki', 125),
4 (2, 'Princesse Mononoke', 'Miyazaki', 134),
5 (3, 'Vice-versa', 'Docter', 95);

```

Écrire les requêtes SQL permettant d'afficher :

1. les titres des films réalisés par Miyazaki ;
2. les titres des films dont la durée dépasse 100 minutes.

BASES DE DONNÉES RELATIONNELLES — QCM D'AUTO-ÉVALUATION

Pour chaque question, une seule réponse est exacte.

Capacité C1

1. [Q1] Dans une relation, un attribut est défini sur :
 - A. un domaine, c'est-à-dire l'ensemble des valeurs qu'il peut prendre.
 - B. un tuple.
 - C. une clé étrangère, et rien d'autre.
 - D. une autre relation.

Capacité C2

2. [Q2] Le schéma d'une base de données décrit :
 - A. les valeurs actuellement enregistrées dans les tables.
 - B. la structure : les relations, leurs attributs, leurs domaines et leurs clés.
 - C. le nombre de lignes de chaque table.
 - D. l'historique des requêtes exécutées.

Capacité C3

3. [Q3] Répéter le titre et l'auteur d'un livre sur chaque ligne d'une table d'emprunts, au lieu de les placer dans une table dédiée, entraîne :
 - A. un gain de temps de calcul systématique.
 - B. une amélioration de la sécurité.
 - C. une duplication des données et un risque d'incohérence lors des mises à jour.
 - D. aucune conséquence particulière.

Capacité C4

4. [Q4] Le service de persistance d'un SGBD garantit que :
 - A. les mots de passe sont chiffrés.
 - B. les requêtes s'exécutent instantanément.
 - C. deux utilisateurs n'accèdent jamais à la base en même temps.
 - D. les données restent conservées durablement, même après l'arrêt du programme.

Capacité C5

5. [Q5] Dans la requête `SELECT nom FROM Client WHERE ville = 'Lyon';`, la clause `FROM` indique :
 - A. la relation dans laquelle les données sont puisées.
 - B. les colonnes à afficher.
 - C. la condition que les lignes doivent vérifier.
 - D. l'ordre d'affichage du résultat.

Capacité C6

6. [Q6] La requête `SELECT nom FROM Client WHERE ville = 'Lyon';` affiche :
 - A. toutes les colonnes des clients lyonnais.
 - B. le seul nom des clients habitant Lyon.
 - C. le nombre de clients lyonnais.
 - D. toutes les villes des clients.

Capacité C7

7. [Q7] Écrire `WHERE ville = Lyon` sans apostrophes, au lieu de `WHERE ville = 'Lyon'` :
 - A. ne change rien au résultat.
 - B. rend la requête plus rapide.
 - C. est une erreur : Lyon est alors compris comme un nom de colonne.
 - D. est obligatoire pour les textes courts.

Capacité C8

8. [Q8] La clause `JOIN B ON A.x = B.y` permet :
 - A. de trier les résultats de la table A.

- B. de renommer la colonne x en y.
- C. de supprimer les doublons de A.
- D. de rapprocher les tuples de A et de B dont les valeurs de x et de y coïncident.

Capacité C9

9. [Q9] Pour afficher les clients du plus jeune au plus âgé, on complète la requête par :
- A. ORDER BY age ASC
 - B. WHERE age
 - C. GROUP BY age
 - D. ORDER BY age DESC

Capacité C10

10. [Q10] Pour ajouter un client nommé Dupont, domicilié à Lyon, dans la table Client(nom, ville), on écrit :
- A. UPDATE Client SET nom = 'Dupont', ville = 'Lyon';
 - B. INSERT INTO Client (nom, ville) VALUES ('Dupont', 'Lyon');
 - C. SELECT 'Dupont', 'Lyon' FROM Client;
 - D. INSERT Client VALUES nom = 'Dupont';

Capacité C11

11. [Q11] Dans une requête UPDATE, omettre la clause WHERE a pour effet :
- A. de faire échouer la requête.
 - B. de n'appliquer la modification qu'à la première ligne.
 - C. de modifier toutes les lignes de la table.
 - D. de créer une nouvelle table.

Capacité C12

12. [Q12] La requête DELETE FROM Emprunt WHERE date_retour IS NOT NULL;;
- A. supprime la table Emprunt tout entière.
 - B. vide la colonne date_retour des lignes concernées.
 - C. supprime toutes les lignes de la table, la condition étant ignorée.
 - D. supprime les lignes dont la date de retour est renseignée.

Clé de correction — réservée au professeur

Question	Capacité	Réponse exacte
Q1	C1	A
Q2	C2	B
Q3	C3	C
Q4	C4	D
Q5	C5	A
Q6	C6	B
Q7	C7	C
Q8	C8	D
Q9	C9	A
Q10	C10	B
Q11	C11	C
Q12	C12	D

Diagnostic des réponses erronées

► Q1

- **B** — L'élève intervertit les deux notions. Le tuple est une LIGNE ; l'attribut est une colonne, définie par le domaine de ses valeurs. *Renvoi : C1, relation, attribut, domaine.*
- **C** — Une clé étrangère est un rôle particulier joué par certains attributs, non la façon dont tout attribut est défini. *Renvoi : C1, relation, attribut, domaine.*
- **D** — Un attribut n'est pas défini par une relation : c'est la relation qui est définie par ses attributs. *Renvoi : C1, relation, attribut, domaine.*

► Q2

- **A** — L'élève décrit le CONTENU. Celui-ci change à chaque insertion, alors que le schéma demeure. *Renvoi : C2, structure et contenu.*
- **C** — L'effectif est une propriété du contenu, variable dans le temps, et non de la structure. *Renvoi : C2, structure et contenu.*
- **D** — Le journal des requêtes relève de l'exploitation du système ; il ne fait pas partie de la description de la base. *Renvoi : C2, structure et contenu.*

► Q3

- **A** — L'élève ne retient que l'économie d'une jointure. La redondance alourdit la base et rend les mises à jour bien plus coûteuses. *Renvoi : C3, anomalies de schéma.*
- **B** — Multiplier les copies d'une même donnée multiplie les endroits où elle peut être altérée : la sécurité en pâtit. *Renvoi : C3, anomalies de schéma.*
- **D** — Corriger un titre mal orthographié exigerait de modifier toutes les lignes concernées, et en oublier une suffit à rendre la base incohérente. *Renvoi : C3, anomalies de schéma.*

► Q4

- **A** — Le chiffrement relève du service de SÉCURISATION, distinct de la conservation durable des données. *Renvoi : C4, services d'un SGBD.*
- **B** — La rapidité relève du service d'EFFICACITÉ, et aucun système ne promet l'instantanéité. *Renvoi : C4, services d'un SGBD.*
- **C** — L'accès simultané est au contraire permis : c'est le service de gestion de la CONCURRENCE qui l'organise sans conflit. *Renvoi : C4, services d'un SGBD.*

► Q5

- **B** — Les colonnes affichées sont énumérées après SELECT. Chaque clause a un rôle distinct. *Renvoi : C5, composants d'une requête.*
- **C** — La condition est portée par WHERE ; FROM ne filtre rien. *Renvoi : C5, composants d'une requête.*
- **D** — L'ordre d'affichage relève de ORDER BY, absent de cette requête. *Renvoi : C5, composants d'une requête.*

► Q6

- **A** — Afficher toutes les colonnes exigerait `SELECT *`. Ici, une seule colonne est demandée. *Renvoi : C6, SELECT et FROM.*
- **C** — Compter demanderait `COUNT`. La requête énumère les noms, elle ne les dénombre pas. *Renvoi : C6, SELECT et FROM.*
- **D** — La ville figure dans la `CONDITION`, non dans la projection : elle n'apparaît donc pas dans le résultat. *Renvoi : C6, SELECT et FROM.*

► **Q7**

- **A** — La différence est essentielle : les apostrophes distinguent une valeur littérale d'un identifiant de colonne. *Renvoi : C7, littéraux et identifiants dans WHERE.*
- **B** — Aucun gain de vitesse n'est en jeu ; la requête est simplement rejetée ou renvoie un résultat faux. *Renvoi : C7, littéraux et identifiants dans WHERE.*
- **D** — La longueur du texte n'entre pas en ligne de compte : toute chaîne littérale est délimitée par des apostrophes. *Renvoi : C7, littéraux et identifiants dans WHERE.*

► **Q8**

- **A** — Le tri relève de `ORDER BY` ; une jointure ne garantit aucun ordre du résultat. *Renvoi : C8, jointure.*
- **B** — Le renommage s'écrit avec `AS`. La condition `ON` compare deux colonnes, elle n'en rebaptise aucune. *Renvoi : C8, jointure.*
- **C** — L'élimination des doublons s'obtient par `DISTINCT`. Une jointure peut au contraire en produire. *Renvoi : C8, jointure.*

► **Q9**

- **B** — `WHERE` filtre les lignes retenues, sans jamais les réordonner. La clause est de surcroît incomplète. *Renvoi : C9, ORDER BY.*
- **C** — `GROUP BY` regroupe les lignes pour les agréger ; il ne définit pas l'ordre d'affichage. *Renvoi : C9, ORDER BY.*
- **D** — `DESC` trie du plus grand au plus petit, donc du plus âgé au plus jeune : c'est l'ordre inverse de celui qui est demandé. *Renvoi : C9, ORDER BY.*

► **Q10**

- **A** — `UPDATE` modifie des lignes existantes. Sans `WHERE`, cette requête écraserait de surcroît TOUTES les lignes de la table. *Renvoi : C10, INSERT.*
- **C** — `SELECT` interroge la base sans jamais la modifier : aucune ligne ne serait ajoutée. *Renvoi : C10, INSERT.*
- **D** — La syntaxe mêle deux formes : `INSERT` attend `INTO` puis une liste de valeurs, jamais des affectations. *Renvoi : C10, INSERT.*

► **Q11**

- **A** — La requête est parfaitement valide : c'est bien ce qui la rend dangereuse, puisque rien ne signale l'oubli. *Renvoi : C11, UPDATE.*
- **B** — Aucune limitation implicite n'existe. Sans condition, la mise à jour porte sur l'intégralité de la relation. *Renvoi : C11, UPDATE.*
- **D** — `UPDATE` agit sur les données d'une table existante ; il ne touche jamais au schéma. *Renvoi : C11, UPDATE.*

► **Q12**

- **A** — Supprimer la table demanderait `DROP TABLE`. `DELETE` retire des lignes et laisse la structure en place. *Renvoi : C12, DELETE.*
- **B** — `DELETE` supprime des LIGNES entières ; vider une seule colonne relèverait d'un `UPDATE`. *Renvoi : C12, DELETE.*
- **C** — La condition est bien prise en compte : seules les lignes qui la vérifient sont supprimées. *Renvoi : C12, DELETE.*

Corrigé

Q1. Une **relation** est une table regroupant des tuples décrits par les mêmes attributs. Un **attribut** est une colonne de la relation, définie sur un domaine. Une **clé primaire** est un attribut (ou groupe d'attributs) dont la valeur identifie de façon unique chaque tuple.

Q2. `Commande.id_client` est une clé étrangère vers `Client.id` : elle permet de savoir quel client a passé chaque commande, sans dupliquer les informations du client dans la table `Commande`.

Corrigé

Q1.

```
1 SELECT nom FROM Produit WHERE categorie = 'Papeterie';
```

Q2.

```
1 SELECT nom FROM Produit WHERE prix < 2.00;
```

Q3.

```
1 INSERT INTO Produit(nom, categorie, prix) VALUES ('Gomme', 'Papeterie', 0.60);
```

Évaluation A — Bases de données

Durée : 45 minutes. Ordinateur avec un SGBD (SQLite ou équivalent) autorisé.

Barème indicatif : Ex. 1 : 8 pts (modèle relationnel); Ex. 2 : 12 pts (SELECT, WHERE, INSERT)

Exercice 1 — Modélisation

(8 points)

- (4 pts) Définir, en une phrase chacun, les termes *relation*, *attribut* et *clé primaire*.
- (4 pts) Donner un exemple de clé étrangère dans le schéma `Commande(id, id_client, date) / Client(id, nom)`, et expliquer à quoi elle sert.

Exercice 2 — SELECT, WHERE, INSERT

(12 points)

On donne la table :

```
1 CREATE TABLE Produit(id INTEGER PRIMARY KEY, nom TEXT, prix REAL, categorie TEXT);
2 INSERT INTO Produit VALUES
3   (1, 'Cahier', 1.50, 'Papeterie'),
4   (2, 'Stylo', 0.80, 'Papeterie'),
5   (3, 'Casque audio', 25.00, 'Electronique');
```

- (4 pts) Écrire la requête affichant le nom des produits de catégorie 'Papeterie'.
- (4 pts) Écrire la requête affichant le nom des produits dont le prix est strictement inférieur à 2.00.
- (4 pts) Écrire la requête ajoutant un produit 'Gomme', catégorie 'Papeterie', prix 0.60.

Corrigé

Q1. Persistance, concurrence, efficacité, sécurisation.

Q2. Un fichier CSV n'offre aucune gestion de la **concurrence** : si deux guichets modifient le fichier en même temps, l'une des deux réservations peut être écrasée ou perdue. Un SGBD sérialise les accès concurrents pour garantir qu'aucune réservation n'est perdue.

Corrigé

Q1.

```

1 SELECT Employe.nom, Service.nom
2 FROM Employe
3 JOIN Service ON Employe.id_service = Service.id
4 ORDER BY Employe.nom ASC;

```

Q2.

```

1 UPDATE Employe SET id_service = 1 WHERE nom = 'Ben Ali';

```

Q3.

```

1 DELETE FROM Employe WHERE id_service = 1;

```

Après la question précédente, 'Ben Ali' et 'Chraibi' sont tous deux dans le service 1 : il ne reste que 'Haddad'. Sans clause WHERE, DELETE FROM Employe; supprimerait **tous** les employés, quel que soit leur service.

Évaluation B — Bases de données

Durée : 45 minutes. Ordinateur avec un SGBD (SQLite ou équivalent) autorisé.

Barème indicatif : Ex. 1 : 6 pts (SGBD); Ex. 2 : 14 pts (JOIN, ORDER BY, UPDATE, DELETE)

Exercice 1 — Services d'un SGBD

(6 points)

- (3 pts) Citer les quatre services rendus par un SGBD relationnel.
- (3 pts) Expliquer pourquoi un simple fichier CSV ne peut pas remplacer un SGBD pour gérer les réservations d'une salle de sport ouverte à plusieurs guichets simultanément.

Exercice 2 — JOIN, ORDER BY, UPDATE, DELETE

(14 points)

On donne :

```

1 CREATE TABLE Employe(id INTEGER PRIMARY KEY, nom TEXT, id_service INTEGER);
2 CREATE TABLE Service(id INTEGER PRIMARY KEY, nom TEXT);
3 INSERT INTO Service VALUES (1, 'Comptabilite'), (2, 'Informatique');
4 INSERT INTO Employe VALUES (1, 'Ben Ali', 2), (2, 'Chraibi', 1), (3, 'Haddad', 2);

```

- (5 pts) Écrire la requête affichant le nom de chaque employé avec le nom de son service, triée par nom d'employé croissant.
- (4 pts) Écrire la requête qui change le service de l'employé 'Ben Ali' vers le service 1.
- (5 pts) Écrire la requête qui supprime tous les employés du service 1 (après la modification de la question précédente). Que se passerait-il si l'on omettait la clause WHERE ?

Exercice 51

Une table Client contient 50 clients. On exécute la requête DELETE FROM Client; (sans clause WHERE). Combien de clients reste-t-il dans la table après cette requête ? La table elle-même existe-t-elle toujours ?

Corrigé

Il ne reste **aucun** client : sans clause WHERE, DELETE supprime **tous** les tuples de la table. La table Client elle-même continue d'exister (son schéma n'est pas touché), mais elle est vide : SELECT COUNT(*) FROM Client; renverrait 0.

Exercice 52

Une table `Client` contient 50 clients. On exécute la requête `DELETE FROM Client;` (sans clause `WHERE`). Combien de clients reste-t-il dans la table après cette requête ? La table elle-même existe-t-elle toujours ?

Corrigé

Il ne reste **aucun** client : sans clause `WHERE`, `DELETE` supprime **tous** les tuples de la table. La table `Client` elle-même continue d'exister (son schéma n'est pas touché), mais elle est vide : `SELECT COUNT(*) FROM Client;` renverrait 0.

Exercice 53

Une table `Client` contient 50 clients. On exécute la requête `DELETE FROM Client;` (sans clause `WHERE`). Combien de clients reste-t-il dans la table après cette requête ? La table elle-même existe-t-elle toujours ?

Corrigé

Il ne reste **aucun** client : sans clause `WHERE`, `DELETE` supprime **tous** les tuples de la table. La table `Client` elle-même continue d'exister (son schéma n'est pas touché), mais elle est vide : `SELECT COUNT(*) FROM Client;` renverrait 0.

Exercice 54

Une table `Client` contient 50 clients. On exécute la requête `DELETE FROM Client;` (sans clause `WHERE`). Combien de clients reste-t-il dans la table après cette requête ? La table elle-même existe-t-elle toujours ?

Corrigé

Il ne reste **aucun** client : sans clause `WHERE`, `DELETE` supprime **tous** les tuples de la table. La table `Client` elle-même continue d'exister (son schéma n'est pas touché), mais elle est vide : `SELECT COUNT(*) FROM Client;` renverrait 0.

Exercice 55

Une table `Client` contient 50 clients. On exécute la requête `DELETE FROM Client;` (sans clause `WHERE`). Combien de clients reste-t-il dans la table après cette requête ? La table elle-même existe-t-elle toujours ?

Corrigé

Il ne reste **aucun** client : sans clause `WHERE`, `DELETE` supprime **tous** les tuples de la table. La table `Client` elle-même continue d'exister (son schéma n'est pas touché), mais elle est vide : `SELECT COUNT(*) FROM Client;` renverrait 0.

Exercice 56

Une table `Client` contient 50 clients. On exécute la requête `DELETE FROM Client;` (sans clause `WHERE`). Combien de clients reste-t-il dans la table après cette requête ? La table elle-même existe-t-elle toujours ?

Corrigé

Il ne reste **aucun** client : sans clause `WHERE`, `DELETE` supprime **tous** les tuples de la table. La table `Client` elle-même continue d'exister (son schéma n'est pas touché), mais elle est vide : `SELECT COUNT(*) FROM Client;` renverrait 0.

Exercice 57 ◆

Une table `Client` contient 50 clients. On exécute la requête `DELETE FROM Client;` (sans clause `WHERE`). Combien de clients reste-t-il dans la table après cette requête ? La table elle-même existe-t-elle toujours ?

Corrigé

Il ne reste **aucun** client : sans clause `WHERE`, `DELETE` supprime **tous** les tuples de la table. La table `Client` elle-même continue d'exister (son schéma n'est pas touché), mais elle est vide : `SELECT COUNT(*) FROM Client;` renverrait 0.

Exercice 58 ◆

Une table `Client` contient 50 clients. On exécute la requête `DELETE FROM Client;` (sans clause `WHERE`). Combien de clients reste-t-il dans la table après cette requête ? La table elle-même existe-t-elle toujours ?

Corrigé

Il ne reste **aucun** client : sans clause `WHERE`, `DELETE` supprime **tous** les tuples de la table. La table `Client` elle-même continue d'exister (son schéma n'est pas touché), mais elle est vide : `SELECT COUNT(*) FROM Client;` renverrait 0.

Exercice 59 ◆

Une table `Client` contient 50 clients. On exécute la requête `DELETE FROM Client;` (sans clause `WHERE`). Combien de clients reste-t-il dans la table après cette requête ? La table elle-même existe-t-elle toujours ?

Corrigé

Il ne reste **aucun** client : sans clause `WHERE`, `DELETE` supprime **tous** les tuples de la table. La table `Client` elle-même continue d'exister (son schéma n'est pas touché), mais elle est vide : `SELECT COUNT(*) FROM Client;` renverrait 0.

Exercice 60 ◆

Une table `Client` contient 50 clients. On exécute la requête `DELETE FROM Client;` (sans clause `WHERE`). Combien de clients reste-t-il dans la table après cette requête ? La table elle-même existe-t-elle toujours ?

Corrigé

Il ne reste **aucun** client : sans clause `WHERE`, `DELETE` supprime **tous** les tuples de la table. La table `Client` elle-même continue d'exister (son schéma n'est pas touché), mais elle est vide : `SELECT COUNT(*) FROM Client;` renverrait 0.

Exercice 61 ◆

Une table `Client` contient 50 clients. On exécute la requête `DELETE FROM Client;` (sans clause `WHERE`). Combien de clients reste-t-il dans la table après cette requête ? La table elle-même existe-t-elle toujours ?

Corrigé

Il ne reste **aucun** client : sans clause `WHERE`, `DELETE` supprime **tous** les tuples de la table. La table `Client` elle-même continue d'exister (son schéma n'est pas touché), mais elle est vide : `SELECT COUNT(*) FROM Client;` renverrait 0.

Exercice 62 ◆

Une table `Client` contient 50 clients. On exécute la requête `DELETE FROM Client;` (sans clause `WHERE`). Combien de clients reste-t-il dans la table après cette requête ? La table elle-même existe-t-elle toujours ?

Corrigé

Il ne reste **aucun** client : sans clause `WHERE`, `DELETE` supprime **tous** les tuples de la table. La table `Client` elle-même continue d'exister (son schéma n'est pas touché), mais elle est vide : `SELECT COUNT(*) FROM Client;` renverrait 0.

Exercice 63 ◆

Une table `Client` contient 50 clients. On exécute la requête `DELETE FROM Client;` (sans clause `WHERE`). Combien de clients reste-t-il dans la table après cette requête ? La table elle-même existe-t-elle toujours ?

Corrigé

Il ne reste **aucun** client : sans clause `WHERE`, `DELETE` supprime **tous** les tuples de la table. La table `Client` elle-même continue d'exister (son schéma n'est pas touché), mais elle est vide : `SELECT COUNT(*) FROM Client;` renverrait 0.

Corrigé

Q1. Un adhérent pratiquant deux sports apparaît **deux fois** dans la table (une ligne par couple adhérent/sport), avec son nom et son prénom **dupliqués**. Si l'on corrige une faute dans son nom sur une seule ligne, les deux lignes deviennent incohérentes entre elles : c'est une anomalie de mise à jour.

Q2. On sépare en trois relations :

```
1 Adherent(id, nom, prenom)
2 Sport(id, nom, coach)
3 Seance(id, id_adherent, id_sport, jour)
```

`Seance.id_adherent` est une clé étrangère vers `Adherent.id`, et `Seance.id_sport` une clé étrangère vers `Sport.id`. Le nom de l'adhérent n'est désormais stocké qu'une seule fois, quel que soit le nombre de sports pratiqués.

Corrigé

Q1.

```
1 SELECT titre FROM Film WHERE realisateur = 'Miyazaki';
```

Q2.

```
1 SELECT titre FROM Film WHERE duree > 100;
```

Corrigé

```
1 SELECT Eleve.nom, Note.matiere, Note.valeur
2 FROM Note
3 JOIN Eleve ON Note.id_eleve = Eleve.id
4 ORDER BY Note.valeur DESC;
```

Le `JOIN` associe chaque note à l'élève correspondant via la clé étrangère `id_eleve`, puis `ORDER BY ... DESC` trie le résultat de la note la plus haute à la plus basse.

Corrigé

```

1 INSERT INTO Stock(produit, quantité) VALUES ('Gomme', 25);
2
3 UPDATE Stock SET quantité = quantité - 5 WHERE produit = 'Cahier';
4
5 DELETE FROM Stock WHERE quantité = 0;

```

Le UPDATE utilise `quantité - 5` pour exprimer la nouvelle valeur **relativement** à l'ancienne, sans avoir besoin de connaître ni réécrire la valeur numérique finale (35). Le DELETE cible précisément `quantité = 0` : seul 'Stylo' est supprimé.

Corrigé

Q1. Un adhérent pratiquant deux sports apparaît **deux fois** dans la table (une ligne par couple adhérent/sport), avec son nom et son prénom **dupliqués**. Si l'on corrige une faute dans son nom sur une seule ligne, les deux lignes deviennent incohérentes entre elles : c'est une anomalie de mise à jour.

Q2. On sépare en trois relations :

```

1 Adherent(id, nom, prenom)
2 Sport(id, nom, coach)
3 Seance(id, id_adherent, id_sport, jour)

```

`Seance.id_adherent` est une clé étrangère vers `Adherent.id`, et `Seance.id_sport` une clé étrangère vers `Sport.id`. Le nom de l'adhérent n'est désormais stocké qu'une seule fois, quel que soit le nombre de sports pratiqués.

Corrigé

Q1.

```

1 SELECT titre FROM Film WHERE realisateur = 'Miyazaki';

```

Q2.

```

1 SELECT titre FROM Film WHERE duree > 100;

```

Corrigé

```

1 SELECT Eleve.nom, Note.matiere, Note.valeur
2 FROM Note
3 JOIN Eleve ON Note.id_eleve = Eleve.id
4 ORDER BY Note.valeur DESC;

```

Le JOIN associe chaque note à l'élève correspondant via la clé étrangère `id_eleve`, puis `ORDER BY ... DESC` trie le résultat de la note la plus haute à la plus basse.

Corrigé

```

1 INSERT INTO Stock(produit, quantité) VALUES ('Gomme', 25);
2
3 UPDATE Stock SET quantité = quantité - 5 WHERE produit = 'Cahier';
4
5 DELETE FROM Stock WHERE quantité = 0;

```

Le UPDATE utilise `quantite - 5` pour exprimer la nouvelle valeur **relativement** à l'ancienne, sans avoir besoin de connaître ni réécrire la valeur numérique finale (35). Le DELETE cible précisément `quantite = 0` : seul 'Stylo' est supprimé.

Corrigé

Q1. Un adhérent pratiquant deux sports apparaît **deux fois** dans la table (une ligne par couple adhérent/sport), avec son nom et son prénom **dupliqués**. Si l'on corrige une faute dans son nom sur une seule ligne, les deux lignes deviennent incohérentes entre elles : c'est une anomalie de mise à jour.

Q2. On sépare en trois relations :

```
1 Adherent(id, nom, prenom)
2 Sport(id, nom, coach)
3 Seance(id, id_adherent, id_sport, jour)
```

`Seance.id_adherent` est une clé étrangère vers `Adherent.id`, et `Seance.id_sport` une clé étrangère vers `Sport.id`. Le nom de l'adhérent n'est désormais stocké qu'une seule fois, quel que soit le nombre de sports pratiqués.

Corrigé

Q1.

```
1 SELECT titre FROM Film WHERE realisateur = 'Miyazaki';
```

Q2.

```
1 SELECT titre FROM Film WHERE duree > 100;
```

Corrigé

```
1 SELECT Eleve.nom, Note.matiere, Note.valeur
2 FROM Note
3 JOIN Eleve ON Note.id_eleve = Eleve.id
4 ORDER BY Note.valeur DESC;
```

Le JOIN associe chaque note à l'élève correspondant via la clé étrangère `id_eleve`, puis `ORDER BY ... DESC` trie le résultat de la note la plus haute à la plus basse.

Corrigé

```
1 INSERT INTO Stock(produit, quantité) VALUES ('Gomme', 25);
2
3 UPDATE Stock SET quantité = quantité - 5 WHERE produit = 'Cahier';
4
5 DELETE FROM Stock WHERE quantité = 0;
```

Le UPDATE utilise `quantite - 5` pour exprimer la nouvelle valeur **relativement** à l'ancienne, sans avoir besoin de connaître ni réécrire la valeur numérique finale (35). Le DELETE cible précisément `quantite = 0` : seul 'Stylo' est supprimé.

Corrigé

Q1. Un adhérent pratiquant deux sports apparaît **deux fois** dans la table (une ligne par couple adhérent/sport), avec son nom et son prénom **dupliqués**. Si l'on corrige une faute dans son nom sur une seule ligne, les deux lignes deviennent incohérentes entre elles : c'est une anomalie de mise à jour.

Q2. On sépare en trois relations :

```

1 Adherent(id, nom, prenom)
2 Sport(id, nom, coach)
3 Seance(id, id_adherent, id_sport, jour)

```

Seance.id_adherent est une clé étrangère vers Adherent.id, et Seance.id_sport une clé étrangère vers Sport.id. Le nom de l'adhérent n'est désormais stocké qu'une seule fois, quel que soit le nombre de sports pratiqués.

Corrigé

Q1.

```

1 SELECT titre FROM Film WHERE realisateur = 'Miyazaki';

```

Q2.

```

1 SELECT titre FROM Film WHERE duree > 100;

```

Corrigé

```

1 SELECT Eleve.nom, Note.matiere, Note.valeur
2 FROM Note
3 JOIN Eleve ON Note.id_eleve = Eleve.id
4 ORDER BY Note.valeur DESC;

```

Le JOIN associe chaque note à l'élève correspondant via la clé étrangère id_eleve, puis ORDER BY ... DESC trie le résultat de la note la plus haute à la plus basse.

Corrigé

```

1 INSERT INTO Stock(produit, quantité) VALUES ('Gomme', 25);
2
3 UPDATE Stock SET quantité = quantité - 5 WHERE produit = 'Cahier';
4
5 DELETE FROM Stock WHERE quantité = 0;

```

Le UPDATE utilise quantité - 5 pour exprimer la nouvelle valeur **relativement** à l'ancienne, sans avoir besoin de connaître ni réécrire la valeur numérique finale (35). Le DELETE cible précisément quantité = 0 : seul 'Stylo' est supprimé.

Corrigé

Q1. Un adhérent pratiquant deux sports apparaît **deux fois** dans la table (une ligne par couple adhérent/sport), avec son nom et son prénom **dupliqués**. Si l'on corrige une faute dans son nom sur une seule ligne, les deux lignes deviennent incohérentes entre elles : c'est une anomalie de mise à jour.

Q2. On sépare en trois relations :

```

1 Adherent(id, nom, prenom)
2 Sport(id, nom, coach)
3 Seance(id, id_adherent, id_sport, jour)

```

Seance.id_adherent est une clé étrangère vers Adherent.id, et Seance.id_sport une clé étrangère vers Sport.id. Le nom de l'adhérent n'est désormais stocké qu'une seule fois, quel que soit le nombre de sports pratiqués.

Corrigé

Q1.

```
1 SELECT titre FROM Film WHERE realisateur = 'Miyazaki';
```

Q2.

```
1 SELECT titre FROM Film WHERE duree > 100;
```

Corrigé

```
1 SELECT Eleve.nom, Note.matiere, Note.valeur
2 FROM Note
3 JOIN Eleve ON Note.id_eleve = Eleve.id
4 ORDER BY Note.valeur DESC;
```

Le JOIN associe chaque note à l'élève correspondant via la clé étrangère `id_eleve`, puis `ORDER BY ... DESC` trie le résultat de la note la plus haute à la plus basse.

Corrigé

```
1 INSERT INTO Stock(produit, quantité) VALUES ('Gomme', 25);
2
3 UPDATE Stock SET quantité = quantité - 5 WHERE produit = 'Cahier';
4
5 DELETE FROM Stock WHERE quantité = 0;
```

Le UPDATE utilise `quantité - 5` pour exprimer la nouvelle valeur **relativement** à l'ancienne, sans avoir besoin de connaître ni réécrire la valeur numérique finale (35). Le DELETE cible précisément `quantité = 0` : seul 'Stylo' est supprimé.

Corrigé

Q1. Un adhérent pratiquant deux sports apparaît **deux fois** dans la table (une ligne par couple adhérent/sport), avec son nom et son prénom **dupliqués**. Si l'on corrige une faute dans son nom sur une seule ligne, les deux lignes deviennent incohérentes entre elles : c'est une anomalie de mise à jour.

Q2. On sépare en trois relations :

```
1 Adherent(id, nom, prenom)
2 Sport(id, nom, coach)
3 Seance(id, id_adherent, id_sport, jour)
```

`Seance.id_adherent` est une clé étrangère vers `Adherent.id`, et `Seance.id_sport` une clé étrangère vers `Sport.id`. Le nom de l'adhérent n'est désormais stocké qu'une seule fois, quel que soit le nombre de sports pratiqués.

Corrigé

Q1.

```
1 SELECT titre FROM Film WHERE realisateur = 'Miyazaki';
```

Q2.

```
1 SELECT titre FROM Film WHERE duree > 100;
```

Corrigé

```

1 SELECT Eleve.nom, Note.matiere, Note.valeur
2 FROM Note
3 JOIN Eleve ON Note.id_eleve = Eleve.id
4 ORDER BY Note.valeur DESC;

```

Le JOIN associe chaque note à l'élève correspondant via la clé étrangère `id_eleve`, puis ORDER BY ... DESC trie le résultat de la note la plus haute à la plus basse.

Corrigé

```

1 INSERT INTO Stock(produit, quantité) VALUES ('Gomme', 25);
2
3 UPDATE Stock SET quantité = quantité - 5 WHERE produit = 'Cahier';
4
5 DELETE FROM Stock WHERE quantité = 0;

```

Le UPDATE utilise `quantité - 5` pour exprimer la nouvelle valeur **relativement** à l'ancienne, sans avoir besoin de connaître ni réécrire la valeur numérique finale (35). Le DELETE cible précisément `quantité = 0` : seul 'Stylo' est supprimé.

Corrigé

Q1. Un adhérent pratiquant deux sports apparaît **deux fois** dans la table (une ligne par couple adhérent/sport), avec son nom et son prénom **dupliqués**. Si l'on corrige une faute dans son nom sur une seule ligne, les deux lignes deviennent incohérentes entre elles : c'est une anomalie de mise à jour.

Q2. On sépare en trois relations :

```

1 Adherent(id, nom, prenom)
2 Sport(id, nom, coach)
3 Seance(id, id_adherent, id_sport, jour)

```

`Seance.id_adherent` est une clé étrangère vers `Adherent.id`, et `Seance.id_sport` une clé étrangère vers `Sport.id`. Le nom de l'adhérent n'est désormais stocké qu'une seule fois, quel que soit le nombre de sports pratiqués.

Corrigé

Q1.

```

1 SELECT titre FROM Film WHERE realisateur = 'Miyazaki';

```

Q2.

```

1 SELECT titre FROM Film WHERE duree > 100;

```

Corrigé

```

1 SELECT Eleve.nom, Note.matiere, Note.valeur
2 FROM Note
3 JOIN Eleve ON Note.id_eleve = Eleve.id
4 ORDER BY Note.valeur DESC;

```

Le JOIN associe chaque note à l'élève correspondant via la clé étrangère `id_eleve`, puis ORDER BY ... DESC trie le résultat de la note la plus haute à la plus basse.

Corrigé

```
1 INSERT INTO Stock(produit, quantité) VALUES ('Gomme', 25);
2
3 UPDATE Stock SET quantité = quantité - 5 WHERE produit = 'Cahier';
4
5 DELETE FROM Stock WHERE quantité = 0;
```

Le UPDATE utilise `quantité - 5` pour exprimer la nouvelle valeur **relativement** à l'ancienne, sans avoir besoin de connaître ni réécrire la valeur numérique finale (35). Le DELETE cible précisément `quantité = 0` : seul 'Stylo' est supprimé.

Corrigé

Q1. Un adhérent pratiquant deux sports apparaît **deux fois** dans la table (une ligne par couple adhérent/sport), avec son nom et son prénom **dupliqués**. Si l'on corrige une faute dans son nom sur une seule ligne, les deux lignes deviennent incohérentes entre elles : c'est une anomalie de mise à jour.

Q2. On sépare en trois relations :

```
1 Adherent(id, nom, prenom)
2 Sport(id, nom, coach)
3 Seance(id, id_adherent, id_sport, jour)
```

`Seance.id_adherent` est une clé étrangère vers `Adherent.id`, et `Seance.id_sport` une clé étrangère vers `Sport.id`. Le nom de l'adhérent n'est désormais stocké qu'une seule fois, quel que soit le nombre de sports pratiqués.

Corrigé

Q1.

```
1 SELECT titre FROM Film WHERE realisateur = 'Miyazaki';
```

Q2.

```
1 SELECT titre FROM Film WHERE duree > 100;
```

Corrigé

```
1 SELECT Eleve.nom, Note.matiere, Note.valeur
2 FROM Note
3 JOIN Eleve ON Note.id_eleve = Eleve.id
4 ORDER BY Note.valeur DESC;
```

Le JOIN associe chaque note à l'élève correspondant via la clé étrangère `id_eleve`, puis `ORDER BY ... DESC` trie le résultat de la note la plus haute à la plus basse.

Corrigé

```
1 INSERT INTO Stock(produit, quantité) VALUES ('Gomme', 25);
2
3 UPDATE Stock SET quantité = quantité - 5 WHERE produit = 'Cahier';
4
5 DELETE FROM Stock WHERE quantité = 0;
```

Le UPDATE utilise `quantité - 5` pour exprimer la nouvelle valeur **relativement** à l'ancienne, sans avoir besoin de connaître ni réécrire la valeur numérique finale (35). Le DELETE cible précisément `quantité = 0` : seul 'Stylo' est supprimé.

Corrigé

Q1. Un adhérent pratiquant deux sports apparaît **deux fois** dans la table (une ligne par couple adhérent/sport), avec son nom et son prénom **dupliqués**. Si l'on corrige une faute dans son nom sur une seule ligne, les deux lignes deviennent incohérentes entre elles : c'est une anomalie de mise à jour.

Q2. On sépare en trois relations :

```
1 Adherent(id, nom, prenom)
2 Sport(id, nom, coach)
3 Seance(id, id_adherent, id_sport, jour)
```

Seance.id_adherent est une clé étrangère vers Adherent.id, et Seance.id_sport une clé étrangère vers Sport.id. Le nom de l'adhérent n'est désormais stocké qu'une seule fois, quel que soit le nombre de sports pratiqués.

Corrigé

Q1.

```
1 SELECT titre FROM Film WHERE realisateur = 'Miyazaki';
```

Q2.

```
1 SELECT titre FROM Film WHERE duree > 100;
```

Corrigé

```
1 SELECT Eleve.nom, Note.matiere, Note.valeur
2 FROM Note
3 JOIN Eleve ON Note.id_eleve = Eleve.id
4 ORDER BY Note.valeur DESC;
```

Le JOIN associe chaque note à l'élève correspondant via la clé étrangère id_eleve, puis ORDER BY ... DESC trie le résultat de la note la plus haute à la plus basse.

Corrigé

```
1 INSERT INTO Stock(produit, quantité) VALUES ('Gomme', 25);
2
3 UPDATE Stock SET quantité = quantité - 5 WHERE produit = 'Cahier';
4
5 DELETE FROM Stock WHERE quantité = 0;
```

Le UPDATE utilise quantité - 5 pour exprimer la nouvelle valeur **relativement** à l'ancienne, sans avoir besoin de connaître ni réécrire la valeur numérique finale (35). Le DELETE cible précisément quantité = 0 : seul 'Stylo' est supprimé.

Corrigé

Q1. Un adhérent pratiquant deux sports apparaît **deux fois** dans la table (une ligne par couple adhérent/sport), avec son nom et son prénom **dupliqués**. Si l'on corrige une faute dans son nom sur une seule ligne, les deux lignes deviennent incohérentes entre elles : c'est une anomalie de mise à jour.

Q2. On sépare en trois relations :

```
1 Adherent(id, nom, prenom)
2 Sport(id, nom, coach)
3 Seance(id, id_adherent, id_sport, jour)
```

Seance.id_adherent est une clé étrangère vers Adherent.id, et Seance.id_sport une clé étrangère vers Sport.id. Le nom de l'adhérent n'est désormais stocké qu'une seule fois, quel que soit le nombre de sports pratiqués.

Corrigé

Q1.

```
1 SELECT titre FROM Film WHERE realisateur = 'Miyazaki';
```

Q2.

```
1 SELECT titre FROM Film WHERE duree > 100;
```

Corrigé

```
1 SELECT Eleve.nom, Note.matiere, Note.valeur
2 FROM Note
3 JOIN Eleve ON Note.id_eleve = Eleve.id
4 ORDER BY Note.valeur DESC;
```

Le JOIN associe chaque note à l'élève correspondant via la clé étrangère id_eleve, puis ORDER BY ... DESC trie le résultat de la note la plus haute à la plus basse.

Corrigé

```
1 INSERT INTO Stock(produit, quantité) VALUES ('Gomme', 25);
2
3 UPDATE Stock SET quantité = quantité - 5 WHERE produit = 'Cahier';
4
5 DELETE FROM Stock WHERE quantité = 0;
```

Le UPDATE utilise quantité - 5 pour exprimer la nouvelle valeur **relativement** à l'ancienne, sans avoir besoin de connaître ni réécrire la valeur numérique finale (35). Le DELETE cible précisément quantité = 0 : seul 'Stylo' est supprimé.

Corrigé

Q1. Un adhérent pratiquant deux sports apparaît **deux fois** dans la table (une ligne par couple adhérent/sport), avec son nom et son prénom **dupliqués**. Si l'on corrige une faute dans son nom sur une seule ligne, les deux lignes deviennent incohérentes entre elles : c'est une anomalie de mise à jour.

Q2. On sépare en trois relations :

```
1 Adherent(id, nom, prenom)
2 Sport(id, nom, coach)
3 Seance(id, id_adherent, id_sport, jour)
```

Seance.id_adherent est une clé étrangère vers Adherent.id, et Seance.id_sport une clé étrangère vers Sport.id. Le nom de l'adhérent n'est désormais stocké qu'une seule fois, quel que soit le nombre de sports pratiqués.

Corrigé

Q1.

```
1 SELECT titre FROM Film WHERE realisateur = 'Miyazaki';
```

Q2.

```
1 SELECT titre FROM Film WHERE duree > 100;
```

Corrigé

```
1 SELECT Eleve.nom, Note.matiere, Note.valeur
2 FROM Note
3 JOIN Eleve ON Note.id_eleve = Eleve.id
4 ORDER BY Note.valeur DESC;
```

Le JOIN associe chaque note à l'élève correspondant via la clé étrangère `id_eleve`, puis ORDER BY ... DESC trie le résultat de la note la plus haute à la plus basse.

Corrigé

```
1 INSERT INTO Stock(produit, quantité) VALUES ('Gomme', 25);
2
3 UPDATE Stock SET quantité = quantité - 5 WHERE produit = 'Cahier';
4
5 DELETE FROM Stock WHERE quantité = 0;
```

Le UPDATE utilise `quantité - 5` pour exprimer la nouvelle valeur **relativement** à l'ancienne, sans avoir besoin de connaître ni réécrire la valeur numérique finale (35). Le DELETE cible précisément `quantité = 0` : seul 'Stylo' est supprimé.

Corrigé

Q1. Un adhérent pratiquant deux sports apparaît **deux fois** dans la table (une ligne par couple adhérent/sport), avec son nom et son prénom **dupliqués**. Si l'on corrige une faute dans son nom sur une seule ligne, les deux lignes deviennent incohérentes entre elles : c'est une anomalie de mise à jour.

Q2. On sépare en trois relations :

```
1 Adherent(id, nom, prenom)
2 Sport(id, nom, coach)
3 Seance(id, id_adherent, id_sport, jour)
```

`Seance.id_adherent` est une clé étrangère vers `Adherent.id`, et `Seance.id_sport` une clé étrangère vers `Sport.id`. Le nom de l'adhérent n'est désormais stocké qu'une seule fois, quel que soit le nombre de sports pratiqués.

Corrigé

Q1.

```
1 SELECT titre FROM Film WHERE realisateur = 'Miyazaki';
```

Q2.

```
1 SELECT titre FROM Film WHERE duree > 100;
```

Corrigé

```
1 SELECT Eleve.nom, Note.matiere, Note.valeur
2 FROM Note
3 JOIN Eleve ON Note.id_eleve = Eleve.id
4 ORDER BY Note.valeur DESC;
```

Le JOIN associe chaque note à l'élève correspondant via la clé étrangère `id_eleve`, puis `ORDER BY ... DESC` trie le résultat de la note la plus haute à la plus basse.

Corrigé

```
1 INSERT INTO Stock(produit, quantité) VALUES ('Gomme', 25);
2
3 UPDATE Stock SET quantité = quantité - 5 WHERE produit = 'Cahier';
4
5 DELETE FROM Stock WHERE quantité = 0;
```

Le UPDATE utilise `quantité - 5` pour exprimer la nouvelle valeur **relativement** à l'ancienne, sans avoir besoin de connaître ni réécrire la valeur numérique finale (35). Le DELETE cible précisément `quantité = 0` : seul 'Stylo' est supprimé.

Corrigé

Q1. Un adhérent pratiquant deux sports apparaît **deux fois** dans la table (une ligne par couple adhérent/sport), avec son nom et son prénom **dupliqués**. Si l'on corrige une faute dans son nom sur une seule ligne, les deux lignes deviennent incohérentes entre elles : c'est une anomalie de mise à jour.

Q2. On sépare en trois relations :

```
1 Adherent(id, nom, prenom)
2 Sport(id, nom, coach)
3 Seance(id, id_adherent, id_sport, jour)
```

`Seance.id_adherent` est une clé étrangère vers `Adherent.id`, et `Seance.id_sport` une clé étrangère vers `Sport.id`. Le nom de l'adhérent n'est désormais stocké qu'une seule fois, quel que soit le nombre de sports pratiqués.

Corrigé

Q1.

```
1 SELECT titre FROM Film WHERE realisateur = 'Miyazaki';
```

Q2.

```
1 SELECT titre FROM Film WHERE duree > 100;
```

4 Histoire de l'informatique

4.1 Événements clés de l'histoire de l'informatique

◆ PROPRIÉTÉ Repères 1936–1948 : la machine universelle

- ▶ **1936** : Alan Turing formalise le concept de **machine universelle** (machine de Turing), capable en théorie d'exécuter tout algorithme calculable. C'est la première fois que les notions de **machine**, **algorithme**, **langage** et **information** sont pensées comme un tout cohérent.
- ▶ **1945** : John von Neumann décrit l'**architecture** qui porte son nom : programme et données stockés dans une même mémoire (voir chapitre Architectures matérielles).
- ▶ **1948** : construction des premiers ordinateurs électroniques à programme enregistré (par exemple le *Manchester Baby*).

◆ PROPRIÉTÉ De 1948 à aujourd'hui : croissance et diversification

- ▶ La puissance de calcul des ordinateurs a évolué de manière **exponentielle** (loi empirique de Moore, 1965 : doublement approximatif du nombre de transistors par circuit tous les deux ans).
- ▶ Les ordinateurs se sont **diversifiés** en taille, forme et usage : ordinateurs personnels, serveurs, fermes de calcul, téléphones, tablettes, montres connectées, objets connectés.
- ▶ **1969** : mise en service du réseau ARPANET, ancêtre d'**Internet**, qui relie aujourd'hui ordinateurs et objets connectés à l'échelle mondiale.

EXEMPLE Situer sur une frise chronologique : machine de Turing (1936), premiers ordinateurs à programme enregistré (1948), ARPANET (1969), micro-ordinateurs personnels (fin des années 1970), World Wide Web (1989-1991), smartphones (années 2000).

⚠ ERREUR FRÉQUENTE

Confondre Internet et le World Wide Web. Internet (réseau de réseaux, depuis 1969/ARPANET) est l'infrastructure de communication ; le Web (inventé par Tim Berners-Lee en 1989-1991) est un **service** qui fonctionne au-dessus d'Internet, au même titre que le courrier électronique. Le Web n'est qu'une des utilisations d'Internet.

4.2 Évolution des rôles du logiciel et du matériel

◆ PROPRIÉTÉ Des débuts au logiciel dominant

- ▶ Dans les tout premiers ordinateurs (fin des années 1940), le **matériel** était prédominant : programmer consistait souvent à reconfigurer physiquement des câblages ou à saisir du code machine directement.

- ▶ L'apparition des **langages de programmation** de haut niveau (Fortran 1957, puis de nombreux autres) et des **compilateurs** déplace progressivement la complexité vers le **logiciel** : le matériel devient plus générique et reconfigurable par programmation.
- ▶ Les **systèmes d'exploitation** (années 1960-1970) organisent le partage des ressources matérielles entre plusieurs programmes, ajoutant une couche logicielle essentielle entre l'utilisateur et la machine.

◆ **PROPRIÉTÉ Aujourd'hui : le logiciel omniprésent** Un même matériel (smartphone, ordinateur) exécute une multitude de logiciels différents grâce à des couches d'abstraction (système d'exploitation, machine virtuelle, navigateur). Le **coût de développement logiciel** dépasse souvent aujourd'hui le coût du matériel dans de nombreux projets numériques.

EXEMPLE Comparer un four à micro-ondes des années 1980 (circuits électroniques dédiés, comportement figé) à un four connecté moderne (matériel générique + logiciel embarqué mis à jour à distance) : le second illustre le basculement du rôle de spécialisation vers le logiciel.

⚠ ERREUR FRÉQUENTE

Croire que le matériel est devenu secondaire. Le matériel reste indispensable (sans lui, aucun logiciel ne peut s'exécuter) : c'est son **rôle** qui a évolué, passant de porteur de la logique spécifique à plateforme générique exécutant une logique portée par le logiciel.

4.3 Événements clés de l'histoire de l'informatique

◆ PROPRIÉTÉ Repères 1936–1948 : la machine universelle

- ▶ **1936** : Alan Turing formalise le concept de **machine universelle** (machine de Turing), capable en théorie d'exécuter tout algorithme calculable. C'est la première fois que les notions de **machine**, **algorithme**, **langage** et **information** sont pensées comme un tout cohérent.
- ▶ **1945** : John von Neumann décrit l'**architecture** qui porte son nom : programme et données stockés dans une même mémoire (voir chapitre Architectures matérielles).
- ▶ **1948** : construction des premiers ordinateurs électroniques à programme enregistré (par exemple le *Manchester Baby*).

◆ PROPRIÉTÉ De 1948 à aujourd'hui : croissance et diversification

- ▶ La puissance de calcul des ordinateurs a évolué de manière **exponentielle** (loi empirique de Moore, 1965 : doublement approximatif du nombre de transistors par circuit tous les deux ans).
- ▶ Les ordinateurs se sont **diversifiés** en taille, forme et usage : ordinateurs personnels, serveurs, fermes de calcul, téléphones, tablettes, montres connectées, objets connectés.
- ▶ **1969** : mise en service du réseau ARPANET, ancêtre d'**Internet**, qui relie aujourd'hui ordinateurs et objets connectés à l'échelle mondiale.

EXEMPLE Situer sur une frise chronologique : machine de Turing (1936), premiers ordinateurs à programme enregistré (1948), ARPANET (1969), micro-ordinateurs personnels (fin des années 1970), World Wide Web (1989-1991), smartphones (années 2000).

ERREUR FRÉQUENTE

Confondre Internet et le World Wide Web. Internet (reseau de reseaux, depuis 1969/ARPANET) est l'infrastructure de communication ; le Web (inventé par Tim Berners-Lee en 1989-1991) est un **service** qui fonctionne au-dessus d'Internet, au même titre que le courrier électronique. Le Web n'est qu'une des utilisations d'Internet.

4.4 Évolution des rôles du logiciel et du matériel

◆ PROPRIÉTÉ Des débuts au logiciel dominant

- ▶ Dans les tout premiers ordinateurs (fin des années 1940), le **matériel** était prédominant : programmer consistait souvent à reconfigurer physiquement des câblages ou à saisir du code machine directement.
- ▶ L'apparition des **langages de programmation** de haut niveau (Fortran 1957, puis de nombreux autres) et des **compilateurs** déplace progressivement la complexité vers le **logiciel** : le matériel devient plus générique et reconfigurable par programmation.
- ▶ Les **systèmes d'exploitation** (années 1960-1970) organisent le partage des ressources matérielles entre plusieurs programmes, ajoutant une couche logicielle essentielle entre l'utilisateur et la machine.

◆ **PROPRIÉTÉ Aujourd'hui : le logiciel omniprésent** Un même matériel (smartphone, ordinateur) exécute une multitude de logiciels différents grâce à des couches d'abstraction (système d'exploitation, machine virtuelle, navigateur). Le **coût de développement logiciel** dépasse souvent aujourd'hui le coût du matériel dans de nombreux projets numériques.

EXEMPLE Comparer un four à micro-ondes des années 1980 (circuits électroniques dédiés, comportement figé) à un four connecté moderne (matériel générique + logiciel embarqué mis à jour à distance) : le second illustre le basculement du rôle de spécialisation vers le logiciel.

ERREUR FRÉQUENTE

Croire que le matériel est devenu secondaire. Le matériel reste indispensable (sans lui, aucun logiciel ne peut s'exécuter) : c'est son **rôle** qui a évolué, passant de porteur de la logique spécifique à plateforme générique exécutant une logique portée par le logiciel.

Méthodes

- ▶ Méthode directe de travail sur le sujet.

Exercice 1 ◆

Remettre dans l'ordre chronologique les événements suivants, en précisant l'année de chacun :

- ▶ Mise en service du réseau ARPANET.
- ▶ Formalisation de la machine universelle par Alan Turing.
- ▶ Construction des premiers ordinateurs à programme enregistré.

Exercice 2 ◆◆

Un smartphone moderne peut exécuter des milliers d'applications différentes sans que son matériel change.

1. Expliquer, en une phrase, pourquoi cela illustre l'évolution du rôle du matériel vers plus de généralité.
2. Citer deux couches logicielles (parmi système d'exploitation, machine virtuelle, navigateur, compilateur) qui permettent cette flexibilité, et expliquer brièvement le rôle de chacune.

Exercice 3 ◆

Remettre dans l'ordre chronologique les événements suivants, en précisant l'année de chacun :

- Mise en service du réseau ARPANET.
- Formalisation de la machine universelle par Alan Turing.
- Construction des premiers ordinateurs à programme enregistré.

Exercice 4 ◆◆

Un smartphone moderne peut exécuter des milliers d'applications différentes sans que son matériel change.

1. Expliquer, en une phrase, pourquoi cela illustre l'évolution du rôle du matériel vers plus de généralité.
2. Citer deux couches logicielles (parmi système d'exploitation, machine virtuelle, navigateur, compilateur) qui permettent cette flexibilité, et expliquer brièvement le rôle de chacune.

Exercice 5 ◆

Remettre dans l'ordre chronologique les événements suivants, en précisant l'année de chacun :

- Mise en service du réseau ARPANET.
- Formalisation de la machine universelle par Alan Turing.
- Construction des premiers ordinateurs à programme enregistré.

Exercice 6 ◆◆

Un smartphone moderne peut exécuter des milliers d'applications différentes sans que son matériel change.

1. Expliquer, en une phrase, pourquoi cela illustre l'évolution du rôle du matériel vers plus de généralité.
2. Citer deux couches logicielles (parmi système d'exploitation, machine virtuelle, navigateur, compilateur) qui permettent cette flexibilité, et expliquer brièvement le rôle de chacune.

Exercice 7 ◆

Remettre dans l'ordre chronologique les événements suivants, en précisant l'année de chacun :

- Mise en service du réseau ARPANET.
- Formalisation de la machine universelle par Alan Turing.
- Construction des premiers ordinateurs à programme enregistré.

Exercice 8 ◆◆

Un smartphone moderne peut exécuter des milliers d'applications différentes sans que son matériel change.

1. Expliquer, en une phrase, pourquoi cela illustre l'évolution du rôle du matériel vers plus de généralité.
2. Citer deux couches logicielles (parmi système d'exploitation, machine virtuelle, navigateur, compilateur) qui permettent cette flexibilité, et expliquer brièvement le rôle de chacune.

Exercice 9 ♦

Remettre dans l'ordre chronologique les événements suivants, en précisant l'année de chacun :

- Mise en service du réseau ARPANET.
- Formalisation de la machine universelle par Alan Turing.
- Construction des premiers ordinateurs à programme enregistré.

Exercice 10 ♦♦

Un smartphone moderne peut exécuter des milliers d'applications différentes sans que son matériel change.

1. Expliquer, en une phrase, pourquoi cela illustre l'évolution du rôle du matériel vers plus de généralité.
2. Citer deux couches logicielles (parmi système d'exploitation, machine virtuelle, navigateur, compilateur) qui permettent cette flexibilité, et expliquer brièvement le rôle de chacune.

Exercice 11 ♦

Remettre dans l'ordre chronologique les événements suivants, en précisant l'année de chacun :

- Mise en service du réseau ARPANET.
- Formalisation de la machine universelle par Alan Turing.
- Construction des premiers ordinateurs à programme enregistré.

Exercice 12 ♦♦

Un smartphone moderne peut exécuter des milliers d'applications différentes sans que son matériel change.

1. Expliquer, en une phrase, pourquoi cela illustre l'évolution du rôle du matériel vers plus de généralité.
2. Citer deux couches logicielles (parmi système d'exploitation, machine virtuelle, navigateur, compilateur) qui permettent cette flexibilité, et expliquer brièvement le rôle de chacune.

Exercice 13 ♦

Remettre dans l'ordre chronologique les événements suivants, en précisant l'année de chacun :

- Mise en service du réseau ARPANET.
- Formalisation de la machine universelle par Alan Turing.
- Construction des premiers ordinateurs à programme enregistré.

Exercice 14 ♦♦

Un smartphone moderne peut exécuter des milliers d'applications différentes sans que son matériel change.

1. Expliquer, en une phrase, pourquoi cela illustre l'évolution du rôle du matériel vers plus de généralité.
2. Citer deux couches logicielles (parmi système d'exploitation, machine virtuelle, navigateur, compilateur) qui permettent cette flexibilité, et expliquer brièvement le rôle de chacune.

Exercice 15 ♦

Remettre dans l'ordre chronologique les événements suivants, en précisant l'année de chacun :

- ▶ Mise en service du réseau ARPANET.
- ▶ Formalisation de la machine universelle par Alan Turing.
- ▶ Construction des premiers ordinateurs à programme enregistré.

Exercice 16 ♦♦

Un smartphone moderne peut exécuter des milliers d'applications différentes sans que son matériel change.

1. Expliquer, en une phrase, pourquoi cela illustre l'évolution du rôle du matériel vers plus de génère.
2. Citer deux couches logicielles (parmi système d'exploitation, machine virtuelle, navigateur, compilateur) qui permettent cette flexibilité, et expliquer brièvement le rôle de chacune.

Exercice 17 ♦

Remettre dans l'ordre chronologique les événements suivants, en précisant l'année de chacun :

- ▶ Mise en service du réseau ARPANET.
- ▶ Formalisation de la machine universelle par Alan Turing.
- ▶ Construction des premiers ordinateurs à programme enregistré.

Exercice 18 ♦♦

Un smartphone moderne peut exécuter des milliers d'applications différentes sans que son matériel change.

1. Expliquer, en une phrase, pourquoi cela illustre l'évolution du rôle du matériel vers plus de génère.
2. Citer deux couches logicielles (parmi système d'exploitation, machine virtuelle, navigateur, compilateur) qui permettent cette flexibilité, et expliquer brièvement le rôle de chacune.

Exercice 19 ♦

Remettre dans l'ordre chronologique les événements suivants, en précisant l'année de chacun :

- ▶ Mise en service du réseau ARPANET.
- ▶ Formalisation de la machine universelle par Alan Turing.
- ▶ Construction des premiers ordinateurs à programme enregistré.

Exercice 20 ♦♦

Un smartphone moderne peut exécuter des milliers d'applications différentes sans que son matériel change.

1. Expliquer, en une phrase, pourquoi cela illustre l'évolution du rôle du matériel vers plus de génère.
2. Citer deux couches logicielles (parmi système d'exploitation, machine virtuelle, navigateur, compilateur) qui permettent cette flexibilité, et expliquer brièvement le rôle de chacune.

Exercice 21 ♦

Remettre dans l'ordre chronologique les événements suivants, en précisant l'année de chacun :

- ▶ Mise en service du réseau ARPANET.

- ▶ Formalisation de la machine universelle par Alan Turing.
- ▶ Construction des premiers ordinateurs à programme enregistré.

Exercice 22 ♦♦

Un smartphone moderne peut exécuter des milliers d'applications différentes sans que son matériel change.

1. Expliquer, en une phrase, pourquoi cela illustre l'évolution du rôle du matériel vers plus de générique.
2. Citer deux couches logicielles (parmi système d'exploitation, machine virtuelle, navigateur, compilateur) qui permettent cette flexibilité, et expliquer brièvement le rôle de chacune.

Exercice 23 ♦

Remettre dans l'ordre chronologique les événements suivants, en précisant l'année de chacun :

- ▶ Mise en service du réseau ARPANET.
- ▶ Formalisation de la machine universelle par Alan Turing.
- ▶ Construction des premiers ordinateurs à programme enregistré.

Exercice 24 ♦♦

Un smartphone moderne peut exécuter des milliers d'applications différentes sans que son matériel change.

1. Expliquer, en une phrase, pourquoi cela illustre l'évolution du rôle du matériel vers plus de générique.
2. Citer deux couches logicielles (parmi système d'exploitation, machine virtuelle, navigateur, compilateur) qui permettent cette flexibilité, et expliquer brièvement le rôle de chacune.

HISTOIRE DE L'INFORMATIQUE — QCM D'AUTO-ÉVALUATION

Pour chaque question, une seule réponse est exacte.

Capacité C1

1. [Q1] Alan Turing formalise le concept de machine universelle en :
 - A. 1936.
 - B. 1948.
 - C. 1969.
 - D. 1991.
2. [Q2] ARPANET, ancêtre d'Internet, est mis en service en :
 - A. 1948.
 - B. 1969.
 - C. 1989.
 - D. 2000.

Capacité C2

3. [Q3] Depuis les débuts de l'informatique, le rôle du matériel a évolué vers :
 - A. une spécialisation croissante, chaque machine ne traitant qu'une seule tâche.
 - B. une disparition progressive au profit du seul logiciel.
 - C. davantage de générique, la logique particulière étant portée par le logiciel.
 - D. aucune évolution notable.
4. [Q4] Internet et le World Wide Web sont dans le rapport suivant :

- A. ce sont deux noms de la même chose.
- B. ce sont deux réseaux physiques distincts et indépendants.
- C. Internet est un service qui fonctionne au-dessus du Web.
- D. le Web est un service qui fonctionne au-dessus du réseau Internet.

Clé de correction — réservée au professeur

Question	Capacité	Réponse exacte
Q1	C1	A
Q2	C1	B
Q3	C2	C
Q4	C2	D

Diagnostic des réponses erronées

► Q1

- **B** — 1948 est l'époque des premières machines à programme enregistré. Le modèle théorique les précède de plus de dix ans. *Renvoi : C1, repères chronologiques.*
- **C** — 1969 est la mise en service d'ARPANET, un jalon des réseaux et non du modèle de calcul. *Renvoi : C1, repères chronologiques.*
- **D** — 1991 est la naissance de Python et l'ouverture du Web au public : bien après les fondements théoriques. *Renvoi : C1, repères chronologiques.*

► Q2

- **A** — 1948 précède l'existence même des ordinateurs interconnectés ; les premières machines à programme enregistré datent de cette période. *Renvoi : C1, repères chronologiques.*
- **C** — 1989 est l'année où Tim Berners-Lee propose le Web, soit vingt ans APRÈS le réseau qui le porte. *Renvoi : C1, Internet et le Web.*
- **D** — L'élève date le réseau de sa diffusion grand public, largement postérieure à sa création. *Renvoi : C1, repères chronologiques.*

► Q3

- **A** — C'est l'inverse du mouvement historique : les premières machines étaient câblées pour une tâche, et le programme enregistré les a rendues polyvalentes. *Renvoi : C2, rôles relatifs du logiciel et du matériel.*
- **B** — Tout logiciel s'exécute sur du matériel. Sa part relative dans la valeur a crû, mais le support n'a pas disparu. *Renvoi : C2, rôles relatifs du logiciel et du matériel.*
- **D** — Le déplacement de la logique du câblage vers le logiciel est l'une des évolutions majeures de la discipline. *Renvoi : C2, rôles relatifs du logiciel et du matériel.*

► Q4

- **A** — Le courriel et bien d'autres services empruntent Internet sans passer par le Web : les deux ne se confondent donc pas. *Renvoi : C2, Internet et le Web.*
- **B** — Le Web n'est pas un réseau physique : c'est un ensemble de pages liées, échangées grâce au protocole HTTP. *Renvoi : C2, Internet et le Web.*
- **C** — L'élève intervertit les deux niveaux. L'infrastructure est Internet ; le Web est l'un des services qu'elle transporte. *Renvoi : C2, Internet et le Web.*

Corrigé

Q1. Machine universelle de Turing : 1936. Premiers ordinateurs à programme enregistré : 1948. ARPANET : 1969.

Q2. En 1936, les notions de machine, algorithme, langage et information sont pour la première fois pensées comme un tout cohérent (concept de machine universelle, capable d'exécuter tout algorithme calculable).

Q3. Exemple accepté : diversification en téléphones, tablettes, montres connectées, objets connectés, fermes de calcul (toute réponse cohérente avec le cours est valable).

Corrigé

Q1. Aux débuts de l'informatique, le matériel portait directement la logique de calcul (câblages spécifiques). Aujourd'hui, un même matériel générique (smartphone, ordinateur) exécute des logiques très variées portées par le logiciel : le rôle du matériel a évolué vers plus de généralité.

Q2. Le système d'exploitation gère le partage des ressources matérielles (mémoire, processeur, périphériques) entre plusieurs programmes, ajoutant une couche logicielle entre l'utilisateur/les applications et le matériel, ce qui permet à ce dernier de rester générique.

Évaluation A — Histoire de l'informatique

Durée : 30 minutes. Documents interdits.

Barème indicatif : Ex. 1 : 10 pts (repères chronologiques) ; Ex. 2 : 10 pts (évolution logiciel/matériel)

Exercice 1 — Repères chronologiques (10 points)

1. (4 pts) Associer chacun des événements suivants à son année : machine universelle de Turing ; premiers ordinateurs à programme enregistré ; mise en service d'ARPANET.
2. (3 pts) Expliquer en une phrase pourquoi 1936 est considérée comme une date charnière dans l'histoire de l'informatique.
3. (3 pts) Citer un exemple de diversification des formes d'ordinateurs depuis les années 1980.

Exercice 2 — Évolution logiciel/matériel (10 points)

1. (5 pts) Expliquer, avec un exemple, comment le rôle du matériel a évolué depuis les débuts de l'informatique.
2. (5 pts) Expliquer le rôle d'un système d'exploitation dans cette évolution.

Corrigé

Q1. 1936, par Alan Turing.

Q2. 1948.

Q3. Exemple accepté : invention du World Wide Web par Tim Berners-Lee (1989-1991), qui popularise l'usage grand public d'Internet.

Corrigé

Q1. Dans les années 1950, le matériel portait directement la logique de calcul (cablages, code machine) : peu de flexibilité, chaque machine était quasi dédiée à une tâche. Aujourd'hui, le matériel est générique (processeurs polyvalents) et c'est le logiciel installé qui détermine entièrement le comportement de la machine, permettant une flexibilité quasi illimitée.

Q2. Exemple accepté : une console de jeux ou un ordinateur personnel peut exécuter des milliers de jeux ou d'applications très différents sans modification matérielle, uniquement par installation de logiciels différents.

Évaluation B — Histoire de l'informatique

Durée : 30 minutes. Documents interdits.

Barème indicatif : Ex. 1 : 10 pts (repères chronologiques) ; Ex. 2 : 10 pts (évolution logiciel/matériel)

Exercice 1 — Repères chronologiques (10 points)

1. (4 pts) Quelle année marque la formalisation du concept de machine universelle, et par qui ?
2. (3 pts) En quelle année les premiers ordinateurs à programme enregistré sont-ils construits ?

3. (3 pts) Citer un événement marquant du développement d'Internet après 1969 (par exemple le World Wide Web).

Exercice 2 — Évolution logiciel/matériel

(10 points)

- (5 pts) Expliquer la différence entre l'informatique des années 1950 (matériel dominant) et l'informatique actuelle (logiciel dominant).
- (5 pts) Donner un exemple concret (autre que celui du cours) illustrant cette évolution.

4.5 Événements clés de l'histoire de l'informatique

◆ PROPRIÉTÉ Repères 1936–1948 : la machine universelle

- **1936** : Alan Turing formalise le concept de **machine universelle** (machine de Turing), capable en théorie d'exécuter tout algorithme calculable. C'est la première fois que les notions de **machine**, **algorithme**, **langage** et **information** sont pensées comme un tout cohérent.
- **1945** : John von Neumann décrit l'**architecture** qui porte son nom : programme et données stockés dans une même mémoire (voir chapitre Architectures matérielles).
- **1948** : construction des premiers ordinateurs électroniques à programme enregistré (par exemple le *Manchester Baby*).

◆ PROPRIÉTÉ De 1948 à aujourd'hui : croissance et diversification

- La puissance de calcul des ordinateurs a évolué de manière **exponentielle** (loi empirique de Moore, 1965 : doublement approximatif du nombre de transistors par circuit tous les deux ans).
- Les ordinateurs se sont **diversifiés** en taille, forme et usage : ordinateurs personnels, serveurs, fermes de calcul, téléphones, tablettes, montres connectées, objets connectés.
- **1969** : mise en service du réseau ARPANET, ancêtre d'**Internet**, qui relie aujourd'hui ordinateurs et objets connectés à l'échelle mondiale.

EXEMPLE Situer sur une frise chronologique : machine de Turing (1936), premiers ordinateurs à programme enregistré (1948), ARPANET (1969), micro-ordinateurs personnels (fin des années 1970), World Wide Web (1989-1991), smartphones (années 2000).

⚠ ERREUR FRÉQUENTE

Confondre Internet et le World Wide Web. Internet (réseau de réseaux, depuis 1969/ARPANET) est l'infrastructure de communication ; le Web (inventé par Tim Berners-Lee en 1989-1991) est un **service** qui fonctionne au-dessus d'Internet, au même titre que le courrier électronique. Le Web n'est qu'une des utilisations d'Internet.

4.6 Évolution des rôles du logiciel et du matériel

♦ PROPRIÉTÉ Des débuts au logiciel dominant

- ▶ Dans les tout premiers ordinateurs (fin des années 1940), le **matériel** était prédominant : programmer consistait souvent à reconfigurer physiquement des câblages ou à saisir du code machine directement.
- ▶ L'apparition des **langages de programmation** de haut niveau (Fortran 1957, puis de nombreux autres) et des **compilateurs** déplace progressivement la complexité vers le **logiciel** : le matériel devient plus générique et reconfigurable par programmation.
- ▶ Les **systèmes d'exploitation** (années 1960-1970) organisent le partage des ressources matérielles entre plusieurs programmes, ajoutant une couche logicielle essentielle entre l'utilisateur et la machine.

♦ **PROPRIÉTÉ Aujourd'hui : le logiciel omniprésent** Un même matériel (smartphone, ordinateur) exécute une multitude de logiciels différents grâce à des couches d'abstraction (système d'exploitation, machine virtuelle, navigateur). Le **coût de développement logiciel** dépasse souvent aujourd'hui le coût du matériel dans de nombreux projets numériques.

EXEMPLE Comparer un four à micro-ondes des années 1980 (circuits électroniques dédiés, comportement figé) à un four connecté moderne (matériel générique + logiciel embarqué mis à jour à distance) : le second illustre le basculement du rôle de spécialisation vers le logiciel.

⚠ ERREUR FRÉQUENTE

Croire que le matériel est devenu secondaire. Le matériel reste indispensable (sans lui, aucun logiciel ne peut s'exécuter) : c'est son **rôle** qui a évolué, passant de porteur de la logique spécifique à plateforme générique exécutant une logique portée par le logiciel.

Corrigé

1. **1936** : Alan Turing formalise le concept de machine universelle.
2. **1948** : construction des premiers ordinateurs à programme enregistré.
3. **1969** : mise en service du réseau ARPANET.

Corrigé

Q1. Le matériel (processeur, mémoire, écran) reste identique quelle que soit l'application exécutée : c'est le logiciel installé qui détermine le comportement du smartphone, illustrant un matériel devenu générique et polyvalent.

Q2. Le **système d'exploitation** gère l'accès aux ressources matérielles (mémoire, processeur, périphériques) et fournit une interface commune aux applications. Le **navigateur** permet d'exécuter des applications web (écrites en HTML/CSS/JavaScript) directement, sans installation dédiée, ajoutant une couche d'abstraction supplémentaire au-dessus du système d'exploitation.

Corrigé

1. **1936** : Alan Turing formalise le concept de machine universelle.
2. **1948** : construction des premiers ordinateurs à programme enregistré.
3. **1969** : mise en service du réseau ARPANET.

Corrigé

Q1. Le matériel (processeur, mémoire, écran) reste identique quelle que soit l'application exécutée : c'est le logiciel installé qui détermine le comportement du smartphone, illustrant un matériel devenu générique et polyvalent.

Q2. Le **système d'exploitation** gère l'accès aux ressources matérielles (mémoire, processeur, périphériques) et fournit une interface commune aux applications. Le **navigateur** permet d'exécuter des applications web (écrites en HTML/CSS/JavaScript) directement, sans installation dédiée, ajoutant une couche d'abstraction supplémentaire au-dessus du système d'exploitation.

Corrigé

1. **1936** : Alan Turing formalise le concept de machine universelle.
2. **1948** : construction des premiers ordinateurs à programme enregistré.
3. **1969** : mise en service du réseau ARPANET.

Corrigé

Q1. Le matériel (processeur, mémoire, écran) reste identique quelle que soit l'application exécutée : c'est le logiciel installé qui détermine le comportement du smartphone, illustrant un matériel devenu générique et polyvalent.

Q2. Le **système d'exploitation** gère l'accès aux ressources matérielles (mémoire, processeur, périphériques) et fournit une interface commune aux applications. Le **navigateur** permet d'exécuter des applications web (écrites en HTML/CSS/JavaScript) directement, sans installation dédiée, ajoutant une couche d'abstraction supplémentaire au-dessus du système d'exploitation.

Corrigé

1. **1936** : Alan Turing formalise le concept de machine universelle.
2. **1948** : construction des premiers ordinateurs à programme enregistré.
3. **1969** : mise en service du réseau ARPANET.

Corrigé

Q1. Le matériel (processeur, mémoire, écran) reste identique quelle que soit l'application exécutée : c'est le logiciel installé qui détermine le comportement du smartphone, illustrant un matériel devenu générique et polyvalent.

Q2. Le **système d'exploitation** gère l'accès aux ressources matérielles (mémoire, processeur, périphériques) et fournit une interface commune aux applications. Le **navigateur** permet d'exécuter des applications web (écrites en HTML/CSS/JavaScript) directement, sans installation dédiée, ajoutant une couche d'abstraction supplémentaire au-dessus du système d'exploitation.

Corrigé

1. **1936** : Alan Turing formalise le concept de machine universelle.
2. **1948** : construction des premiers ordinateurs à programme enregistré.
3. **1969** : mise en service du réseau ARPANET.

Corrigé

Q1. Le matériel (processeur, mémoire, écran) reste identique quelle que soit l'application exécutée : c'est le logiciel installé qui détermine le comportement du smartphone, illustrant un matériel devenu générique et polyvalent.

Q2. Le **système d'exploitation** gere l'accès aux ressources matérielles (memoire, processeur, peripheriques) et fournit une interface commune aux applications. Le **navigateur** permet d'exécuter des applications web (écrites en HTML/CSS/JavaScript) directement, sans installation dédiée, ajoutant une couche d'abstraction supplémentaire au-dessus du système d'exploitation.

Corrigé

1. **1936** : Alan Turing formalise le concept de machine universelle.
2. **1948** : construction des premiers ordinateurs à programme enregistré.
3. **1969** : mise en service du réseau ARPANET.

Corrigé

Q1. Le matériel (processeur, memoire, ecran) reste identique quelle que soit l'application exécutée : c'est le logiciel installé qui détermine le comportement du smartphone, illustrant un matériel devenu générique et polyvalent.

Q2. Le **système d'exploitation** gere l'accès aux ressources matérielles (memoire, processeur, peripheriques) et fournit une interface commune aux applications. Le **navigateur** permet d'exécuter des applications web (écrites en HTML/CSS/JavaScript) directement, sans installation dédiée, ajoutant une couche d'abstraction supplémentaire au-dessus du système d'exploitation.

Corrigé

1. **1936** : Alan Turing formalise le concept de machine universelle.
2. **1948** : construction des premiers ordinateurs à programme enregistré.
3. **1969** : mise en service du réseau ARPANET.

Corrigé

Q1. Le matériel (processeur, memoire, ecran) reste identique quelle que soit l'application exécutée : c'est le logiciel installé qui détermine le comportement du smartphone, illustrant un matériel devenu générique et polyvalent.

Q2. Le **système d'exploitation** gere l'accès aux ressources matérielles (memoire, processeur, peripheriques) et fournit une interface commune aux applications. Le **navigateur** permet d'exécuter des applications web (écrites en HTML/CSS/JavaScript) directement, sans installation dédiée, ajoutant une couche d'abstraction supplémentaire au-dessus du système d'exploitation.

Corrigé

1. **1936** : Alan Turing formalise le concept de machine universelle.
2. **1948** : construction des premiers ordinateurs à programme enregistré.
3. **1969** : mise en service du réseau ARPANET.

Corrigé

Q1. Le matériel (processeur, memoire, ecran) reste identique quelle que soit l'application exécutée : c'est le logiciel installé qui détermine le comportement du smartphone, illustrant un matériel devenu générique et polyvalent.

Q2. Le **système d'exploitation** gere l'accès aux ressources matérielles (memoire, processeur, peripheriques) et fournit une interface commune aux applications. Le **navigateur** permet d'exécuter des applications web (écrites en HTML/CSS/JavaScript) directement, sans installation dédiée, ajoutant une couche d'abstraction supplémentaire au-dessus du système d'exploitation.

Corrigé

1. 1936 : Alan Turing formalise le concept de machine universelle.
2. 1948 : construction des premiers ordinateurs a programme enregistre.
3. 1969 : mise en service du reseau ARPANET.

Corrigé

Q1. Le matériel (processeur, memoire, ecran) reste identique quelle que soit l'application executee : c'est le logiciel installe qui déterminé le comportement du smartphone, illustrant un matériel devenu generique et polyvalent.

Q2. Le **système d'exploitation** gere l'accès aux ressources materielles (memoire, processeur, peripheriques) et fournit une interface commune aux applications. Le **navigateur** permet d'exécuter des applications web (ecrites en HTML/CSS/JavaScript) directement, sans installation dediee, ajoutant une couche d'abstraction supplementaire au-dessus du système d'exploitation.

Corrigé

1. 1936 : Alan Turing formalise le concept de machine universelle.
2. 1948 : construction des premiers ordinateurs a programme enregistre.
3. 1969 : mise en service du reseau ARPANET.

Corrigé

Q1. Le matériel (processeur, memoire, ecran) reste identique quelle que soit l'application executee : c'est le logiciel installe qui déterminé le comportement du smartphone, illustrant un matériel devenu generique et polyvalent.

Q2. Le **système d'exploitation** gere l'accès aux ressources materielles (memoire, processeur, peripheriques) et fournit une interface commune aux applications. Le **navigateur** permet d'exécuter des applications web (ecrites en HTML/CSS/JavaScript) directement, sans installation dediee, ajoutant une couche d'abstraction supplementaire au-dessus du système d'exploitation.

Corrigé

1. 1936 : Alan Turing formalise le concept de machine universelle.
2. 1948 : construction des premiers ordinateurs a programme enregistre.
3. 1969 : mise en service du reseau ARPANET.

Corrigé

Q1. Le matériel (processeur, memoire, ecran) reste identique quelle que soit l'application executee : c'est le logiciel installe qui déterminé le comportement du smartphone, illustrant un matériel devenu generique et polyvalent.

Q2. Le **système d'exploitation** gere l'accès aux ressources materielles (memoire, processeur, peripheriques) et fournit une interface commune aux applications. Le **navigateur** permet d'exécuter des applications web (ecrites en HTML/CSS/JavaScript) directement, sans installation dediee, ajoutant une couche d'abstraction supplementaire au-dessus du système d'exploitation.

Corrigé

1. 1936 : Alan Turing formalise le concept de machine universelle.

2. 1948 : construction des premiers ordinateurs a programme enregistré.
3. 1969 : mise en service du reseau ARPANET.

Corrigé

Q1. Le matériel (processeur, memoire, ecran) reste identique quelle que soit l'application executee : c'est le logiciel installe qui déterminé le comportement du smartphone, illustrant un matériel devenu generique et polyvalent.

Q2. Le **système d'exploitation** gere l'accès aux ressources materielles (memoire, processeur, peripheriques) et fournit une interface commune aux applications. Le **navigateur** permet d'executer des applications web (ecrites en HTML/CSS/JavaScript) directement, sans installation dediee, ajoutant une couche d'abstraction supplementaire au-dessus du système d'exploitation.

5 Langages, récursivité et paradigmes de programmation

5.1 Programme, donnée et calculabilité

DÉFINITION 1

Un programme est lui-même une suite de caractères (son code source) ou d'octets (son code compilé) : c'est donc une **donnée** comme une autre, qui peut être stockée, copiée, transmise sur un réseau, ou même **lue et exécutée par un autre programme** (un interpréteur ou un compilateur).

Exemple. Un programme peut être représenté comme une simple liste d'instructions (une donnée), puis exécuté par un **interpréteur** qui la parcourt :

```
1 def interpreter(programme, accumulateur=0):
2     for instruction, valeur in programme:
3         if instruction == "ADD":
4             accumulateur += valeur
5         elif instruction == "MUL":
6             accumulateur *= valeur
7     return accumulateur
```

◆ **PROPRIÉTÉ Calculabilité indépendante du langage** Ce qu'un programme peut calculer ne dépend pas du **langage** dans lequel il est écrit : un algorithme calculable en Python l'est tout autant en C, en OCaml ou dans n'importe quel autre langage de programmation à usage général. Un langage peut rendre un calcul plus simple ou plus rapide à écrire, mais il ne rend jamais calculable ce qui ne l'était pas dans un autre langage.

DÉFINITION 2

Le **problème de l'arrêt** consiste à se demander s'il existe un programme capable de décider, pour **n'importe quel** programme P et **n'importe quelle** entrée e , si l'exécution de P sur e s'arrête ou boucle indéfiniment. Ce problème est **indécidable** : aucun programme ne peut le résoudre dans tous les cas.

◆ **PROPRIÉTÉ Argument intuitif d'indécidabilité** Supposons qu'existe une fonction $s_arrete(P, e)$ qui renvoie toujours correctement True si le programme P s'arrête sur l'entrée e , et False sinon. Construisons alors un programme paradoxe qui, exécuté sur lui-même comme entrée, fait le **contraire** de ce que prédit s_arrete :

- ▶ si $s_arrete(paradoxe, paradoxe)$ vaut True (prédit que paradoxe s'arrête), alors paradoxe boucle volontairement à l'infini ;
- ▶ si $s_arrete(paradoxe, paradoxe)$ vaut False (prédit que paradoxe boucle), alors paradoxe s'arrête immédiatement.

Dans les deux cas, `s_arrete` se trompe sur paradoxe appliqué à lui-même : une telle fonction `s_arrete` ne peut donc pas exister pour **tous** les cas.

⚠ ERREUR FRÉQUENTE

Croire que l'indécidabilité du problème de l'arrêt signifie qu'on ne peut jamais savoir si un programme donné s'arrête. Ce n'est pas ce que dit le résultat : on peut très bien prouver, au cas par cas, que tel ou tel programme particulier s'arrête (ou boucle). Ce qui est impossible, c'est d'écrire un **unique** programme qui répondrait correctement pour **tous** les programmes et toutes les entrées possibles, sans exception.

5.2 Écrire et analyser un programme récursif

DÉFINITION 1

Un programme **récursif** est une fonction qui s'appelle elle-même, directement ou indirectement. Il comporte toujours au moins un **cas de base** (qui s'arrête sans appel récursif) et un **cas général** (qui se ramène à une instance plus simple du même problème).

Exemple. La factorielle, un exemple classique dans un domaine mathématique :

```
1 def factorielle(n):
2     if n == 0:
3         return 1
4     return n * factorielle(n - 1)
```

Exemple. Compter les occurrences d'un caractère dans une chaîne, dans un domaine différent (traitement de texte) :

```
1 def compter(chaine, caractere):
2     if chaine == "":
3         return 0
4     premier = 1 if chaine[0] == caractere else 0
5     return premier + compter(chaine[1:], caractere)
```

DÉFINITION 2

Analyser un programme récursif consiste à comprendre son **arbre d'appels** (quels appels déclenchent quels autres appels), à vérifier sa **terminaison** (le cas de base est-il toujours atteint ?), et à évaluer le nombre d'appels effectués.

Exemple. La version naïve du calcul de Fibonacci effectue un très grand nombre d'appels redondants : on peut le mesurer en instrumentant le code avec un compteur.

```
1 compteur_appels = 0
2
3 def fib_naif(n):
4     global compteur_appels
5     compteur_appels += 1
6     if n <= 1:
7         return n
```

```
8 return fib_naif(n - 1) + fib_naif(n - 2)
```

◆ **PROPRIÉTÉ Lecture de l'arbre d'appels** L'appel `fib_naif(4)` déclenche `fib_naif(3)` et `fib_naif(2)` ; `fib_naif(3)` déclenche à son tour `fib_naif(2)` et `fib_naif(1)`. On observe que `fib_naif(2)` est appelé **deux fois indépendamment**, avec le même résultat recalculé à chaque fois : c'est cette redondance, visible sur l'arbre d'appels, qui explique la croissance très rapide du nombre d'appels (voir le chapitre *Algorithmique* pour la solution par mémoïsation).

⚠ ERREUR FRÉQUENTE

Confondre « la fonction s'exécute plusieurs fois » et « la fonction boucle indéfiniment ». Un grand nombre d'appels récursifs (comme pour `fib_naif`) n'est pas un signe de non-terminaison : chaque branche de l'arbre d'appels atteint bien le cas de base ($n \leq 1$), la fonction termine toujours, mais avec un coût qui peut devenir très élevé.

5.3 API, bibliothèques et modules

DÉFINITION 1

Une **bibliothèque** (ou *librairie*) est un ensemble de fonctions déjà écrites et testées, réutilisables dans d'autres programmes. Son **API** (*Application Programming Interface*) est l'ensemble des fonctions, classes et paramètres qu'elle expose à l'utilisateur, **sans qu'il ait besoin de connaître leur implémentation interne**.

Exemple. La bibliothèque standard `statistics` fournit des fonctions déjà écrites et testées :

```
1 import statistics
2
3 notes = [12, 15, 8, 17, 14]
4 moyenne = statistics.mean(notes)
5 mediane = statistics.median(notes)
6 ecart_type = statistics.stdev(notes)
```

◆ **PROPRIÉTÉ Exploiter la documentation** Avant d'utiliser une fonction d'une bibliothèque, on consulte sa **documentation** : le nom et l'ordre des paramètres, leur type attendu, la valeur renvoyée, et des exemples d'utilisation. En Python, `help(fonction)` affiche la **docstring** (chaîne de documentation) associée à une fonction.

DÉFINITION 2

Un **module** est un fichier Python (`.py`) regroupant des fonctions liées par un thème commun, réutilisable dans d'autres programmes via `import`. Chaque fonction d'un module bien conçu est **documentée** par une docstring décrivant son rôle, ses paramètres et sa valeur de retour.

Exemple. Un module `geometrie.py` regroupant des fonctions sur les figures planes, chacune documentée :

```
1 def aire_rectangle(largeur, hauteur):
```

```

2     """Renvoie l'aire d'un rectangle.
3
4     Parametres : largeur, hauteur (nombres positifs).
5     Renvoie : l'aire (largeur * hauteur).
6     """
7     return largeur * hauteur
8
9 def perimetre_rectangle(largeur, hauteur):
10     """Renvoie le perimetre d'un rectangle."""
11     return 2 * (largeur + hauteur)

```

⚠ ERREUR FRÉQUENTE

Documenter le « comment » au lieu du « quoi ». Une bonne docstring décrit **ce que fait** la fonction, ses paramètres attendus et sa valeur de retour – pas le détail de son implémentation interne (qui peut changer sans que la documentation ait besoin d’être mise à jour, tant que le comportement observable reste le même).

5.4 Paradigmes de programmation

DÉFINITION 1

Un **paradigme de programmation** est une façon générale de structurer un programme et de raisonner sur son fonctionnement. Un même langage, et même un même programme, peut mobiliser **plusieurs paradigmes**.

Exemple. Calculer la somme des carrés des nombres pairs d’une liste, dans trois styles différents.

Style impératif (une suite d’instructions qui modifient un état) :

```

1 def somme_carres_pairs_imperatif(liste):
2     total = 0
3     for x in liste:
4         if x % 2 == 0:
5             total += x ** 2
6     return total

```

Style fonctionnel (composition de fonctions, sans modifier d’état) :

```

1 def somme_carres_pairs_fonctionnel(liste):
2     return sum(x ** 2 for x in liste if x % 2 == 0)

```

Style objet (les données et les traitements sont regroupés dans une classe) :

```

1 class ListeNombres:
2     def __init__(self, valeurs):
3         self.valeurs = valeurs
4
5     def somme_carres_pairs(self):
6         return sum(x ** 2 for x in self.valeurs if x % 2 == 0)

```

◆ PROPRIÉTÉ Choisir un paradigme selon le contexte

- ▶ Le style **impératif** est naturel pour décrire une suite d'étapes avec un état qui évolue (simulation, algorithme pas à pas).
- ▶ Le style **fonctionnel** favorise la lisibilité et limite les effets de bord pour des transformations de données (filtrer, transformer, agréger).
- ▶ Le style **objet** est adapté quand on manipule des **entités** avec un état propre et des comportements associés (une classe Compte, Pile, Noeud...).

Ce choix n'est presque jamais exclusif : un même projet combine typiquement les trois styles selon les besoins de chaque partie.

⚠ ERREUR FRÉQUENTE

Croire qu'un langage « impose » un seul paradigme. Python permet d'écrire du code impératif, fonctionnel (fonctions map, filter, expressions génératrices) et objet (classes) au sein d'un même programme, voire d'une même fonction. Le paradigme est un choix de conception, pas une contrainte du langage.

5.5 Mise au point : causes typiques de bugs

DÉFINITION 1

La **mise au point** (*débogage*) consiste à localiser et corriger les causes d'un comportement incorrect d'un programme. Certaines causes de bugs reviennent très fréquemment : erreurs liées aux **nombre flottants**, aux **effets de bord**, aux **inégalités**, et au **nommage**.

Exemple. Les nombres flottants ne représentent pas exactement toutes les valeurs décimales :

```
1 resultat = 0.1 + 0.2
2 print(resultat == 0.3)           # False, a priori surprenant
3 print(abs(resultat - 0.3) < 1e-9) # True : comparaison a une tolerance pres
```

Exemple. Un **effet de bord** inattendu par aliasing : deux noms qui désignent le même objet mutable.

```
1 def vider(liste):
2     liste.clear()
3
4 a = [1, 2, 3]
5 b = a           # b designe le MEME objet liste que a, pas une copie
6 vider(b)
```

⚠ ERREUR FRÉQUENTE

Croire que `b = a` crée une copie de la liste `a`. En Python, cette instruction fait seulement pointer `b` vers le **même objet** que `a` : modifier `b` (par une méthode qui modifie en place, comme `clear`, `append`, `sort`) modifie donc aussi `a`. Pour obtenir une copie indépendante, il faut écrire explicitement `b = a.copy()` ou `b = list(a)`.

Exemple. Une erreur d'**inégalité stricte/large** (*off-by-one*) très fréquente dans une boucle :

```
1 def indices_valides(liste, indice):
2     # Version correcte : indice doit verifier 0 <= indice < len(liste)
```

```
3 | return 0 <= indice < len(liste)
```

Exemple. Une erreur de **nommage** : masquer (*shadow*) une fonction native de Python.

```
1 | def somme_avec_bug(valeurs):
2 |     sum = 0           # masque la fonction native sum() !
3 |     for v in valeurs:
4 |         sum += v
5 |     return sum        # fonctionne ici, mais sum() natif est devenu inaccessible
```

⚠ ERREUR FRÉQUENTE

Nommer une variable comme une fonction native (`sum`, `list`, `type`, `id`...). Python l'autorise sans erreur, mais la fonction native devient **inaccessible sous ce nom** dans le reste du bloc de code : si l'on a besoin d'appeler `sum(...)` plus loin dans la même fonction, l'appel échouera avec une erreur de type « `int object is not callable` » puisque `sum` désigne maintenant un entier.

5.6 Programme, donnée et calculabilité

DÉFINITION 1

Un programme est lui-même une suite de caractères (son code source) ou d'octets (son code compilé) : c'est donc une **donnée** comme une autre, qui peut être stockée, copiée, transmise sur un réseau, ou même **lue et exécutée par un autre programme** (un interpréteur ou un compilateur).

Exemple. Un programme peut être représenté comme une simple liste d'instructions (une donnée), puis exécuté par un **interpréteur** qui la parcourt :

```
1 | def interpreter(programme, accumulateur=0):
2 |     for instruction, valeur in programme:
3 |         if instruction == "ADD":
4 |             accumulateur += valeur
5 |         elif instruction == "MUL":
6 |             accumulateur *= valeur
7 |     return accumulateur
```

♦ **PROPRIÉTÉ Calculabilité indépendante du langage** Ce qu'un programme peut calculer ne dépend pas du **langage** dans lequel il est écrit : un algorithme calculable en Python l'est tout autant en C, en OCaml ou dans n'importe quel autre langage de programmation à usage général. Un langage peut rendre un calcul plus simple ou plus rapide à écrire, mais il ne rend jamais calculable ce qui ne l'était pas dans un autre langage.

DÉFINITION 2

Le **problème de l'arrêt** consiste à se demander s'il existe un programme capable de décider, pour **n'importe quel** programme P et **n'importe quelle** entrée e , si l'exécution de P sur e s'arrête ou boucle indéfiniment. Ce problème est **indécidable** : aucun programme ne peut le résoudre dans tous les cas.

◆ **PROPRIÉTÉ Argument intuitif d'indécidabilité** Supposons qu'existe une fonction `s_arrete(P, e)` qui renvoie toujours correctement `True` si le programme `P` s'arrête sur l'entrée `e`, et `False` sinon. Construisons alors un programme paradoxe qui, exécuté sur lui-même comme entrée, fait **le contraire** de ce que prédit `s_arrete` :

- ▶ si `s_arrete(paradoxe, paradoxe)` vaut `True` (prédit que paradoxe s'arrête), alors paradoxe boucle volontairement à l'infini ;
- ▶ si `s_arrete(paradoxe, paradoxe)` vaut `False` (prédit que paradoxe boucle), alors paradoxe s'arrête immédiatement.

Dans les deux cas, `s_arrete` se trompe sur paradoxe appliqué à lui-même : une telle fonction `s_arrete` ne peut donc pas exister pour **tous** les cas.

⚠ ERREUR FRÉQUENTE

Croire que l'indécidabilité du problème de l'arrêt signifie qu'on ne peut jamais savoir si un programme donné s'arrête. Ce n'est pas ce que dit le résultat : on peut très bien prouver, au cas par cas, que tel ou tel programme particulier s'arrête (ou boucle). Ce qui est impossible, c'est d'écrire un **unique** programme qui répondrait correctement pour **tous** les programmes et toutes les entrées possibles, sans exception.

5.7 Écrire et analyser un programme récursif

DÉFINITION 1

Un programme **récursif** est une fonction qui s'appelle elle-même, directement ou indirectement. Il comporte toujours au moins un **cas de base** (qui s'arrête sans appel récursif) et un **cas général** (qui se ramène à une instance plus simple du même problème).

Exemple. La factorielle, un exemple classique dans un domaine mathématique :

```
1 def factorielle(n):
2     if n == 0:
3         return 1
4     return n * factorielle(n - 1)
```

Exemple. Compter les occurrences d'un caractère dans une chaîne, dans un domaine différent (traitement de texte) :

```
1 def compter(chaine, caractere):
2     if chaine == "":
3         return 0
4     premier = 1 if chaine[0] == caractere else 0
5     return premier + compter(chaine[1:], caractere)
```

DÉFINITION 2

Analyser un programme récursif consiste à comprendre son **arbre d'appels** (quels appels déclenchent quels autres appels), à vérifier sa **terminaison** (le cas de base est-il toujours atteint ?), et à évaluer le nombre d'appels effectués.

Exemple. La version naïve du calcul de Fibonacci effectue un très grand nombre d'appels redondants : on peut le mesurer en instrumentant le code avec un compteur.

```
1 compteur_appels = 0
2
3 def fib_naif(n):
4     global compteur_appels
5     compteur_appels += 1
6     if n <= 1:
7         return n
8     return fib_naif(n - 1) + fib_naif(n - 2)
```

◆ **PROPRIÉTÉ Lecture de l'arbre d'appels** L'appel `fib_naif(4)` déclenche `fib_naif(3)` et `fib_naif(2)` ; `fib_naif(3)` déclenche à son tour `fib_naif(2)` et `fib_naif(1)`. On observe que `fib_naif(2)` est appelé **deux fois indépendamment**, avec le même résultat recalculé à chaque fois : c'est cette redondance, visible sur l'arbre d'appels, qui explique la croissance très rapide du nombre d'appels (voir le chapitre *Algorithmique* pour la solution par mémoïsation).

⚠ ERREUR FRÉQUENTE

Confondre « la fonction s'exécute plusieurs fois » et « la fonction boucle indéfiniment ». Un grand nombre d'appels récursifs (comme pour `fib_naif`) n'est pas un signe de non-terminaison : chaque branche de l'arbre d'appels atteint bien le cas de base ($n \leq 1$), la fonction termine toujours, mais avec un coût qui peut devenir très élevé.

5.8 API, bibliothèques et modules

DÉFINITION 1

Une **bibliothèque** (ou *librairie*) est un ensemble de fonctions déjà écrites et testées, réutilisables dans d'autres programmes. Son **API** (*Application Programming Interface*) est l'ensemble des fonctions, classes et paramètres qu'elle expose à l'utilisateur, **sans qu'il ait besoin de connaître leur implémentation interne**.

Exemple. La bibliothèque standard `statistics` fournit des fonctions déjà écrites et testées :

```
1 import statistics
2
3 notes = [12, 15, 8, 17, 14]
4 moyenne = statistics.mean(notes)
5 mediane = statistics.median(notes)
6 ecart_type = statistics.stdev(notes)
```

◆ **PROPRIÉTÉ Exploiter la documentation** Avant d'utiliser une fonction d'une bibliothèque, on consulte sa **documentation** : le nom et l'ordre des paramètres, leur type attendu, la valeur renvoyée, et des exemples d'utilisation. En Python, `help(fonction)` affiche la **docstring** (chaîne de documentation) associée à une fonction.

DÉFINITION 2

Un **module** est un fichier Python (.py) regroupant des fonctions liées par un thème commun, réutilisable dans d'autres programmes via import. Chaque fonction d'un module bien conçu est **documentée** par une docstring décrivant son rôle, ses paramètres et sa valeur de retour.

Exemple. Un module `geometrie.py` regroupant des fonctions sur les figures planes, chacune documentée :

```
1 def aire_rectangle(largeur, hauteur):
2     """Renvoie l'aire d'un rectangle.
3
4     Parametres : largeur, hauteur (nombres positifs).
5     Renvoie : l'aire (largeur * hauteur).
6     """
7     return largeur * hauteur
8
9 def perimetre_rectangle(largeur, hauteur):
10    """Renvoie le perimetre d'un rectangle."""
11    return 2 * (largeur + hauteur)
```

⚠ ERREUR FRÉQUENTE

Documenter le « comment » au lieu du « quoi ». Une bonne docstring décrit **ce que fait** la fonction, ses paramètres attendus et sa valeur de retour – pas le détail de son implémentation interne (qui peut changer sans que la documentation ait besoin d'être mise à jour, tant que le comportement observable reste le même).

5.9 Paradigmes de programmation

DÉFINITION 1

Un **paradigme de programmation** est une façon générale de structurer un programme et de raisonner sur son fonctionnement. Un même langage, et même un même programme, peut mobiliser **plusieurs paradigmes**.

Exemple. Calculer la somme des carrés des nombres pairs d'une liste, dans trois styles différents.

Style impératif (une suite d'instructions qui modifient un état) :

```
1 def somme_carres_pairs_imperatif(liste):
2     total = 0
3     for x in liste:
4         if x % 2 == 0:
5             total += x ** 2
6     return total
```

Style fonctionnel (composition de fonctions, sans modifier d'état) :

```
1 def somme_carres_pairs_fonctionnel(liste):
2     return sum(x ** 2 for x in liste if x % 2 == 0)
```

Style objet (les données et les traitements sont regroupés dans une classe) :

```

1 class ListeNombres:
2     def __init__(self, valeurs):
3         self.valeurs = valeurs
4
5     def somme_carres_pairs(self):
6         return sum(x ** 2 for x in self.valeurs if x % 2 == 0)

```

◆ PROPRIÉTÉ Choisir un paradigme selon le contexte

- ▶ Le style **impératif** est naturel pour décrire une suite d'étapes avec un état qui évolue (simulation, algorithme pas à pas).
- ▶ Le style **fonctionnel** favorise la lisibilité et limite les effets de bord pour des transformations de données (filtrer, transformer, agréger).
- ▶ Le style **objet** est adapté quand on manipule des **entités** avec un état propre et des comportements associés (une classe Compte, Pile, Noeud...).

Ce choix n'est presque jamais exclusif : un même projet combine typiquement les trois styles selon les besoins de chaque partie.

⚠ ERREUR FRÉQUENTE

Croire qu'un langage « impose » un seul paradigme. Python permet d'écrire du code impératif, fonctionnel (fonctions map, filter, expressions génératrices) et objet (classes) au sein d'un **même** programme, voire d'une même fonction. Le paradigme est un choix de conception, pas une contrainte du langage.

5.10 Mise au point : causes typiques de bugs

DÉFINITION 1

La **mise au point** (*débogage*) consiste à localiser et corriger les causes d'un comportement incorrect d'un programme. Certaines causes de bugs reviennent très fréquemment : erreurs liées aux **nombre flottants**, aux **effets de bord**, aux **inégalités**, et au **nommage**.

Exemple. Les nombres flottants ne représentent pas exactement toutes les valeurs décimales :

```

1 resultat = 0.1 + 0.2
2 print(resultat == 0.3)           # False, a priori surprenant
3 print(abs(resultat - 0.3) < 1e-9) # True : comparaison a une tolerance pres

```

Exemple. Un **effet de bord** inattendu par aliasing : deux noms qui désignent le même objet mutable.

```

1 def vider(liste):
2     liste.clear()
3
4 a = [1, 2, 3]
5 b = a           # b designe le MEME objet liste que a, pas une copie
6 vider(b)

```

⚠ ERREUR FRÉQUENTE

Croire que `b = a` crée une copie de la liste `a`. En Python, cette instruction fait seulement pointer `b` vers le **même objet** que `a` : modifier `b` (par une méthode qui modifie en place, comme `clear`, `append`, `sort`) modifie donc aussi `a`. Pour obtenir une copie indépendante, il faut écrire explicitement `b = a.copy()` ou `b = list(a)`.

Exemple. Une erreur d'**inégalité stricte/large** (*off-by-one*) très fréquente dans une boucle :

```
1 def indices_valides(liste, indice):
2     # Version correcte : indice doit vérifier 0 <= indice < len(liste)
3     return 0 <= indice < len(liste)
```

Exemple. Une erreur de **nommage** : masquer (*shadow*) une fonction native de Python.

```
1 def somme_avec_bug(valeurs):
2     sum = 0           # masque la fonction native sum() !
3     for v in valeurs:
4         sum += v
5     return sum        # fonctionne ici, mais sum() natif est devenu inaccessible
```

⚠ ERREUR FRÉQUENTE

Nommer une variable comme une fonction native (`sum`, `list`, `type`, `id`...). Python l'autorise sans erreur, mais la fonction native devient **inaccessible sous ce nom** dans le reste du bloc de code : si l'on a besoin d'appeler `sum(...)` plus loin dans la même fonction, l'appel échouera avec une erreur de type « `int object is not callable` » puisque `sum` désigne maintenant un entier.

5.11 Programme, donnée et calculabilité

DÉFINITION 1

Un programme est lui-même une suite de caractères (son code source) ou d'octets (son code compilé) : c'est donc une **donnée** comme une autre, qui peut être stockée, copiée, transmise sur un réseau, ou même **lue et exécutée par un autre programme** (un interpréteur ou un compilateur).

Exemple. Un programme peut être représenté comme une simple liste d'instructions (une donnée), puis exécuté par un **interpréteur** qui la parcourt :

```
1 def interpreter(programme, accumulateur=0):
2     for instruction, valeur in programme:
3         if instruction == "ADD":
4             accumulateur += valeur
5         elif instruction == "MUL":
6             accumulateur *= valeur
7     return accumulateur
```

◆ **PROPRIÉTÉ Calculabilité indépendante du langage** Ce qu'un programme peut calculer ne dépend pas du **langage** dans lequel il est écrit : un algorithme calculable en Python l'est tout

autant en C, en OCaml ou dans n'importe quel autre langage de programmation à usage général. Un langage peut rendre un calcul plus simple ou plus rapide à écrire, mais il ne rend jamais calculable ce qui ne l'était pas dans un autre langage.

DÉFINITION 2

Le **problème de l'arrêt** consiste à se demander s'il existe un programme capable de décider, pour **n'importe quel** programme P et **n'importe quelle** entrée e , si l'exécution de P sur e s'arrête ou boucle indéfiniment. Ce problème est **indécidable** : aucun programme ne peut le résoudre dans tous les cas.

◆ **PROPRIÉTÉ Argument intuitif d'indécidabilité** Supposons qu'existe une fonction $s_arrete(P, e)$ qui renvoie toujours correctement True si le programme P s'arrête sur l'entrée e , et False sinon. Construisons alors un programme paradoxe qui, exécuté sur lui-même comme entrée, fait le **contraire** de ce que prédit s_arrete :

- ▶ si $s_arrete(paradoxe, paradoxe)$ vaut True (prédit que paradoxe s'arrête), alors paradoxe boucle volontairement à l'infini ;
- ▶ si $s_arrete(paradoxe, paradoxe)$ vaut False (prédit que paradoxe boucle), alors paradoxe s'arrête immédiatement.

Dans les deux cas, s_arrete se trompe sur paradoxe appliqué à lui-même : une telle fonction s_arrete ne peut donc pas exister pour **tous** les cas.

⚠ ERREUR FRÉQUENTE

Croire que l'indécidabilité du problème de l'arrêt signifie qu'on ne peut jamais savoir si un programme donné s'arrête. Ce n'est pas ce que dit le résultat : on peut très bien prouver, au cas par cas, que tel ou tel programme particulier s'arrête (ou boucle). Ce qui est impossible, c'est d'écrire un **unique** programme qui répondrait correctement pour **tous** les programmes et toutes les entrées possibles, sans exception.

5.12 Écrire et analyser un programme récursif

DÉFINITION 1

Un programme **récursif** est une fonction qui s'appelle elle-même, directement ou indirectement. Il comporte toujours au moins un **cas de base** (qui s'arrête sans appel récursif) et un **cas général** (qui se ramène à une instance plus simple du même problème).

Exemple. La factorielle, un exemple classique dans un domaine mathématique :

```
1 def factorielle(n):
2     if n == 0:
3         return 1
4     return n * factorielle(n - 1)
```

Exemple. Compter les occurrences d'un caractère dans une chaîne, dans un domaine différent (traitement de texte) :

```
1 def compter(chaine, caractere):
```

```

2  if chaine == "":
3      return 0
4  premier = 1 if chaine[0] == caractere else 0
5  return premier + compter(chaine[1:], caractere)

```

DÉFINITION 2

Analyser un programme récursif consiste à comprendre son **arbre d'appels** (quels appels déclenchent quels autres appels), à vérifier sa **terminaison** (le cas de base est-il toujours atteint ?), et à évaluer le nombre d'appels effectués.

Exemple. La version naïve du calcul de Fibonacci effectue un très grand nombre d'appels redondants : on peut le mesurer en instrumentant le code avec un compteur.

```

1  compteur_appels = 0
2
3  def fib_naif(n):
4      global compteur_appels
5      compteur_appels += 1
6      if n <= 1:
7          return n
8      return fib_naif(n - 1) + fib_naif(n - 2)

```

◆ **PROPRIÉTÉ Lecture de l'arbre d'appels** L'appel `fib_naif(4)` déclenche `fib_naif(3)` et `fib_naif(2)` ; `fib_naif(3)` déclenche à son tour `fib_naif(2)` et `fib_naif(1)`. On observe que `fib_naif(2)` est appelé **deux fois indépendamment**, avec le même résultat recalculé à chaque fois : c'est cette redondance, visible sur l'arbre d'appels, qui explique la croissance très rapide du nombre d'appels (voir le chapitre *Algorithmique* pour la solution par mémoïsation).

⚠ ERREUR FRÉQUENTE

Confondre « la fonction s'exécute plusieurs fois » et « la fonction boucle indéfiniment ». Un grand nombre d'appels récursifs (comme pour `fib_naif`) n'est pas un signe de non-terminaison : chaque branche de l'arbre d'appels atteint bien le cas de base ($n \leq 1$), la fonction termine toujours, mais avec un coût qui peut devenir très élevé.

5.13 API, bibliothèques et modules

DÉFINITION 1

Une **bibliothèque** (ou *librairie*) est un ensemble de fonctions déjà écrites et testées, réutilisables dans d'autres programmes. Son **API** (*Application Programming Interface*) est l'ensemble des fonctions, classes et paramètres qu'elle expose à l'utilisateur, **sans qu'il ait besoin de connaître leur implémentation interne**.

Exemple. La bibliothèque standard `statistics` fournit des fonctions déjà écrites et testées :

```

1  import statistics
2
3  notes = [12, 15, 8, 17, 14]
4  moyenne = statistics.mean(notes)

```

```
5 mediane = statistics.median(notes)
6 ecart_type = statistics.stdev(notes)
```

◆ **PROPRIÉTÉ Exploiter la documentation** Avant d'utiliser une fonction d'une bibliothèque, on consulte sa **documentation** : le nom et l'ordre des paramètres, leur type attendu, la valeur renvoyée, et des exemples d'utilisation. En Python, `help(fonction)` affiche la **docstring** (chaîne de documentation) associée à une fonction.

DÉFINITION 2

Un **module** est un fichier Python (.py) regroupant des fonctions liées par un thème commun, réutilisable dans d'autres programmes via `import`. Chaque fonction d'un module bien conçu est **documentée** par une docstring décrivant son rôle, ses paramètres et sa valeur de retour.

Exemple. Un module `geometrie.py` regroupant des fonctions sur les figures planes, chacune documentée :

```
1 def aire_rectangle(largeur, hauteur):
2     """Renvoie l'aire d'un rectangle.
3
4     Parametres : largeur, hauteur (nombres positifs).
5     Renvoie : l'aire (largeur * hauteur).
6     """
7     return largeur * hauteur
8
9 def perimetre_rectangle(largeur, hauteur):
10     """Renvoie le perimetre d'un rectangle."""
11     return 2 * (largeur + hauteur)
```

⚠ ERREUR FRÉQUENTE

Documenter le « comment » au lieu du « quoi ». Une bonne docstring décrit **ce que fait** la fonction, ses paramètres attendus et sa valeur de retour – pas le détail de son implémentation interne (qui peut changer sans que la documentation ait besoin d'être mise à jour, tant que le comportement observable reste le même).

5.14 Paradigmes de programmation

DÉFINITION 1

Un **paradigme de programmation** est une façon générale de structurer un programme et de raisonner sur son fonctionnement. Un même langage, et même un même programme, peut mobiliser **plusieurs paradigmes**.

Exemple. Calculer la somme des carrés des nombres pairs d'une liste, dans trois styles différents.

Style impératif (une suite d'instructions qui modifient un état) :

```
1 def somme_carres_paires_imperatif(liste):
2     total = 0
```

```

3     for x in liste:
4         if x % 2 == 0:
5             total += x ** 2
6     return total

```

Style fonctionnel (composition de fonctions, sans modifier d'état) :

```

1 def somme_carres_pairs_fonctionnel(liste):
2     return sum(x ** 2 for x in liste if x % 2 == 0)

```

Style objet (les données et les traitements sont regroupés dans une classe) :

```

1 class ListeNombres:
2     def __init__(self, valeurs):
3         self.valeurs = valeurs
4
5     def somme_carres_pairs(self):
6         return sum(x ** 2 for x in self.valeurs if x % 2 == 0)

```

◆ PROPRIÉTÉ Choisir un paradigme selon le contexte

- ▶ Le style **impératif** est naturel pour décrire une suite d'étapes avec un état qui évolue (simulation, algorithme pas à pas).
- ▶ Le style **fonctionnel** favorise la lisibilité et limite les effets de bord pour des transformations de données (filtrer, transformer, agréger).
- ▶ Le style **objet** est adapté quand on manipule des **entités** avec un état propre et des comportements associés (une classe Compte, Pile, Noeud...).

Ce choix n'est presque jamais exclusif : un même projet combine typiquement les trois styles selon les besoins de chaque partie.

⚠ ERREUR FRÉQUENTE

Croire qu'un langage « impose » un seul paradigme. Python permet d'écrire du code impératif, fonctionnel (fonctions map, filter, expressions génératrices) et objet (classes) au sein d'un même programme, voire d'une même fonction. Le paradigme est un choix de conception, pas une contrainte du langage.

5.15 Mise au point : causes typiques de bugs

DÉFINITION 1

La **mise au point** (*débogage*) consiste à localiser et corriger les causes d'un comportement incorrect d'un programme. Certaines causes de bugs reviennent très fréquemment : erreurs liées aux **nombre flottants**, aux **effets de bord**, aux **inégalités**, et au **nommage**.

Exemple. Les nombres flottants ne représentent pas exactement toutes les valeurs décimales :

```

1 resultat = 0.1 + 0.2
2 print(resultat == 0.3)           # False, a priori surprenant
3 print(abs(resultat - 0.3) < 1e-9) # True : comparaison a une tolerance pres

```

Exemple. Un **effet de bord** inattendu par aliasing : deux noms qui désignent le même objet mutable.

```

1 def vider(liste):
2     liste.clear()
3
4 a = [1, 2, 3]
5 b = a          # b designe le MEME objet liste que a, pas une copie
6 vider(b)

```

⚠ ERREUR FRÉQUENTE

Croire que `b = a` crée une copie de la liste `a`. En Python, cette instruction fait seulement pointer `b` vers le **même objet** que `a` : modifier `b` (par une méthode qui modifie en place, comme `clear`, `append`, `sort`) modifie donc aussi `a`. Pour obtenir une copie indépendante, il faut écrire explicitement `b = a.copy()` ou `b = list(a)`.

Exemple. Une erreur d'**inégalité stricte/large** (*off-by-one*) très fréquente dans une boucle :

```

1 def indices_valides(liste, indice):
2     # Version correcte : indice doit verifier 0 <= indice < len(liste)
3     return 0 <= indice < len(liste)

```

Exemple. Une erreur de **nommage** : masquer (*shadow*) une fonction native de Python.

```

1 def somme_avec_bug(valeurs):
2     sum = 0          # masque la fonction native sum() !
3     for v in valeurs:
4         sum += v
5     return sum       # fonctionne ici, mais sum() natif est devenu inaccessible

```

⚠ ERREUR FRÉQUENTE

Nommer une variable comme une fonction native (`sum`, `list`, `type`, `id`...). Python l'autorise sans erreur, mais la fonction native devient **inaccessible sous ce nom** dans le reste du bloc de code : si l'on a besoin d'appeler `sum(...)` plus loin dans la même fonction, l'appel échouera avec une erreur de type « `int object is not callable` » puisque `sum` désigne maintenant un entier.

5.16 Programme, donnée et calculabilité

DÉFINITION 1

Un programme est lui-même une suite de caractères (son code source) ou d'octets (son code compilé) : c'est donc une **donnée** comme une autre, qui peut être stockée, copiée, transmise sur un réseau, ou même **lue et exécutée par un autre programme** (un interpréteur ou un compilateur).

Exemple. Un programme peut être représenté comme une simple liste d'instructions (une donnée), puis exécuté par un **interpréteur** qui la parcourt :

```

1 def interpreter(programme, accumulateur=0):
2     for instruction, valeur in programme:
3         if instruction == "ADD":
4             accumulateur += valeur
5         elif instruction == "MUL":

```

```

6     accumulateur *= valeur
7     return accumulateur

```

◆ **PROPRIÉTÉ Calculabilité indépendante du langage** Ce qu'un programme peut calculer ne dépend pas du **langage** dans lequel il est écrit : un algorithme calculable en Python l'est tout autant en C, en OCaml ou dans n'importe quel autre langage de programmation à usage général. Un langage peut rendre un calcul plus simple ou plus rapide à écrire, mais il ne rend jamais calculable ce qui ne l'était pas dans un autre langage.

DÉFINITION 2

Le **problème de l'arrêt** consiste à se demander s'il existe un programme capable de décider, pour **n'importe quel** programme P et **n'importe quelle** entrée e , si l'exécution de P sur e s'arrête ou boucle indéfiniment. Ce problème est **indécidable** : aucun programme ne peut le résoudre dans tous les cas.

◆ **PROPRIÉTÉ Argument intuitif d'indécidabilité** Supposons qu'existe une fonction $s_arrete(P, e)$ qui renvoie toujours correctement True si le programme P s'arrête sur l'entrée e , et False sinon. Construisons alors un programme paradoxe qui, exécuté sur lui-même comme entrée, fait **le contraire** de ce que prédit s_arrete :

- ▶ si $s_arrete(paradoxe, paradoxe)$ vaut True (prédit que paradoxe s'arrête), alors paradoxe boucle volontairement à l'infini ;
- ▶ si $s_arrete(paradoxe, paradoxe)$ vaut False (prédit que paradoxe boucle), alors paradoxe s'arrête immédiatement.

Dans les deux cas, s_arrete se trompe sur paradoxe appliqué à lui-même : une telle fonction s_arrete ne peut donc pas exister pour **tous** les cas.

⚠ ERREUR FRÉQUENTE

Croire que l'indécidabilité du problème de l'arrêt signifie qu'on ne peut jamais savoir si un programme donné s'arrête. Ce n'est pas ce que dit le résultat : on peut très bien prouver, au cas par cas, que tel ou tel programme particulier s'arrête (ou boucle). Ce qui est impossible, c'est d'écrire un **unique** programme qui répondrait correctement pour **tous** les programmes et toutes les entrées possibles, sans exception.

Méthodes

- ▶ Méthode directe de travail sur le sujet.

Exercice 1 ◆

Écrire une fonction récursive `somme_chiffres(n)` qui renvoie la somme des chiffres d'un entier positif n (par exemple, $1234 \rightarrow 1 + 2 + 3 + 4 = 10$). Quel est le cas de base ? Combien d'appels récurifs sont nécessaires pour `somme_chiffres(1234)` ?

Exercice 2 ◆

En consultant la documentation du module standard `math` (par exemple avec `help(math.hypot)` et `help(math.gcd)`) :

1. Utiliser `math.hypot(x, y)` pour calculer l'hypoténuse d'un triangle rectangle de côtés 3 et 4.
2. Utiliser `math.gcd(a, b)` pour calculer le plus grand diviseur commun de 48 et 18.
3. La documentation précise que `math.sqrt(x)` attend un nombre. Que se passe-t-il si l'on appelle `math.sqrt("4")` (une chaîne de caractères) au lieu de `math.sqrt(4)` ? Vérifier en lisant le message d'erreur obtenu.

Exercice 3

La fonction suivante, écrite en style **impératif**, sélectionne et met en majuscules les mots d'une liste dont la longueur atteint un minimum donné :

```
1 def mots_longs_imperatif(mots, longueur_min):
2     resultat = []
3     for mot in mots:
4         if len(mot) >= longueur_min:
5             resultat.append(mot.upper())
6     return resultat
```

1. Réécrire cette fonction en style **fonctionnel**, sous le nom `mots_longs_fonctionnel`, en une seule expression (liste en compréhension).
2. Vérifier que les deux versions renvoient le même résultat sur `["chat", "ordinateur", "cle", "programmation"]` avec `longueur_min=6`.

Exercice 4

Le code suivant contient un bug lié à un **effet de bord** inattendu :

```
1 def ajouter_note(bulletin, note):
2     bulletin.append(note)
3     return bulletin
4
5 bulletin_A = []
6 bulletin_B = ajouter_note(bulletin_A, 15)
7 bulletin_C = ajouter_note(bulletin_A, 12)
```

1. Que contiennent `bulletin_A`, `bulletin_B` et `bulletin_C` après ces trois lignes ? Le résultat est-il celui attendu si l'on voulait deux bulletins **indépendants** (un avec la note 15, un autre avec la note 12) ?
2. Corriger `ajouter_note` pour qu'elle renvoie un **nouveau** bulletin sans modifier celui passé en argument.

Exercice 5

Écrire une fonction récursive `somme_chiffres(n)` qui renvoie la somme des chiffres d'un entier positif `n` (par exemple, $1234 \rightarrow 1 + 2 + 3 + 4 = 10$). Quel est le cas de base ? Combien d'appels récurifs sont nécessaires pour `somme_chiffres(1234)` ?

Exercice 6

En consultant la documentation du module standard `math` (par exemple avec `help(math.hypot)` et `help(math.gcd)`) :

1. Utiliser `math.hypot(x, y)` pour calculer l'hypoténuse d'un triangle rectangle de côtés 3 et 4.
2. Utiliser `math.gcd(a, b)` pour calculer le plus grand diviseur commun de 48 et 18.
3. La documentation précise que `math.sqrt(x)` attend un nombre. Que se passe-t-il si l'on appelle `math.sqrt("4")` (une chaîne de caractères) au lieu de `math.sqrt(4)` ? Vérifier en lisant le message d'erreur obtenu.

Exercice 7 ♦♦

La fonction suivante, écrite en style **impératif**, sélectionne et met en majuscules les mots d'une liste dont la longueur atteint un minimum donné :

```
1 def mots_longs_imperatif(mots, longueur_min):
2     resultat = []
3     for mot in mots:
4         if len(mot) >= longueur_min:
5             resultat.append(mot.upper())
6     return resultat
```

1. Réécrire cette fonction en style **fonctionnel**, sous le nom `mots_longs_fonctionnel`, en une seule expression (liste en compréhension).
2. Vérifier que les deux versions renvoient le même résultat sur `["chat", "ordinateur", "cle", "programmation"]` avec `longueur_min=6`.

Exercice 8 ♦

Le code suivant contient un bug lié à un **effet de bord** inattendu :

```
1 def ajouter_note(bulletin, note):
2     bulletin.append(note)
3     return bulletin
4
5 bulletin_A = []
6 bulletin_B = ajouter_note(bulletin_A, 15)
7 bulletin_C = ajouter_note(bulletin_A, 12)
```

1. Que contiennent `bulletin_A`, `bulletin_B` et `bulletin_C` après ces trois lignes ? Le résultat est-il celui attendu si l'on voulait deux bulletins **indépendants** (un avec la note 15, un autre avec la note 12) ?
2. Corriger `ajouter_note` pour qu'elle renvoie un **nouveau** bulletin sans modifier celui passé en argument.

Exercice 9 ♦

Écrire une fonction récursive `somme_chiffres(n)` qui renvoie la somme des chiffres d'un entier positif `n` (par exemple, $1234 \rightarrow 1 + 2 + 3 + 4 = 10$). Quel est le cas de base ? Combien d'appels récursifs sont nécessaires pour `somme_chiffres(1234)` ?

Exercice 10 ♦

En consultant la documentation du module standard `math` (par exemple avec `help(math.hypot)` et `help(math.gcd)`) :

1. Utiliser `math.hypot(x, y)` pour calculer l'hypoténuse d'un triangle rectangle de côtés 3 et 4.
2. Utiliser `math.gcd(a, b)` pour calculer le plus grand diviseur commun de 48 et 18.
3. La documentation précise que `math.sqrt(x)` attend un nombre. Que se passe-t-il si l'on appelle `math.sqrt("4")` (une chaîne de caractères) au lieu de `math.sqrt(4)` ? Vérifier en lisant le message d'erreur obtenu.

Exercice 11 ♦♦

La fonction suivante, écrite en style **impératif**, sélectionne et met en majuscules les mots d'une liste dont la longueur atteint un minimum donné :

```
1 def mots_longes_imperatif(mots, longueur_min):
2     resultat = []
3     for mot in mots:
4         if len(mot) >= longueur_min:
5             resultat.append(mot.upper())
6     return resultat
```

1. Réécrire cette fonction en style **fonctionnel**, sous le nom `mots_longes_fonctionnel`, en une seule expression (liste en compréhension).
2. Vérifier que les deux versions renvoient le même résultat sur `["chat", "ordinateur", "cle", "programmation"]` avec `longueur_min=6`.

Exercice 12

Le code suivant contient un bug lié à un **effet de bord** inattendu :

```
1 def ajouter_note(bulletin, note):
2     bulletin.append(note)
3     return bulletin
4
5 bulletin_A = []
6 bulletin_B = ajouter_note(bulletin_A, 15)
7 bulletin_C = ajouter_note(bulletin_A, 12)
```

1. Que contiennent `bulletin_A`, `bulletin_B` et `bulletin_C` après ces trois lignes ? Le résultat est-il celui attendu si l'on voulait deux bulletins **indépendants** (un avec la note 15, un autre avec la note 12) ?
2. Corriger `ajouter_note` pour qu'elle renvoie un **nouveau** bulletin sans modifier celui passé en argument.

Exercice 13

Écrire une fonction récursive `somme_chiffres(n)` qui renvoie la somme des chiffres d'un entier positif `n` (par exemple, $1234 \rightarrow 1 + 2 + 3 + 4 = 10$). Quel est le cas de base ? Combien d'appels récurifs sont nécessaires pour `somme_chiffres(1234)` ?

Exercice 14

En consultant la documentation du module standard `math` (par exemple avec `help(math.hypot)` et `help(math.gcd)`) :

1. Utiliser `math.hypot(x, y)` pour calculer l'hypoténuse d'un triangle rectangle de côtés 3 et 4.
2. Utiliser `math.gcd(a, b)` pour calculer le plus grand diviseur commun de 48 et 18.
3. La documentation précise que `math.sqrt(x)` attend un nombre. Que se passe-t-il si l'on appelle `math.sqrt("4")` (une chaîne de caractères) au lieu de `math.sqrt(4)` ? Vérifier en lisant le message d'erreur obtenu.

Exercice 15

La fonction suivante, écrite en style **impératif**, sélectionne et met en majuscules les mots d'une liste dont la longueur atteint un minimum donné :

```
1 def mots_longes_imperatif(mots, longueur_min):
2     resultat = []
3     for mot in mots:
4         if len(mot) >= longueur_min:
5             resultat.append(mot.upper())
6     return resultat
```

1. Réécrire cette fonction en style **fonctionnel**, sous le nom `mots_longs_fonctionnel`, en une seule expression (liste en compréhension).
2. Vérifier que les deux versions renvoient le même résultat sur `["chat", "ordinateur", "cle", "programmation"]` avec `longueur_min=6`.

Exercice 16

Le code suivant contient un bug lié à un **effet de bord** inattendu :

```
1 def ajouter_note(bulletin, note):
2     bulletin.append(note)
3     return bulletin
4
5 bulletin_A = []
6 bulletin_B = ajouter_note(bulletin_A, 15)
7 bulletin_C = ajouter_note(bulletin_A, 12)
```

1. Que contiennent `bulletin_A`, `bulletin_B` et `bulletin_C` après ces trois lignes ? Le résultat est-il celui attendu si l'on voulait deux bulletins **indépendants** (un avec la note 15, un autre avec la note 12) ?
2. Corriger `ajouter_note` pour qu'elle renvoie un **nouveau** bulletin sans modifier celui passé en argument.

Exercice 17

Écrire une fonction récursive `somme_chiffres(n)` qui renvoie la somme des chiffres d'un entier positif `n` (par exemple, $1234 \rightarrow 1 + 2 + 3 + 4 = 10$). Quel est le cas de base ? Combien d'appels récursifs sont nécessaires pour `somme_chiffres(1234)` ?

Exercice 18

En consultant la documentation du module standard `math` (par exemple avec `help(math.hypot)` et `help(math.gcd)`) :

1. Utiliser `math.hypot(x, y)` pour calculer l'hypoténuse d'un triangle rectangle de côtés 3 et 4.
2. Utiliser `math.gcd(a, b)` pour calculer le plus grand diviseur commun de 48 et 18.
3. La documentation précise que `math.sqrt(x)` attend un nombre. Que se passe-t-il si l'on appelle `math.sqrt("4")` (une chaîne de caractères) au lieu de `math.sqrt(4)` ? Vérifier en lisant le message d'erreur obtenu.

Exercice 19

La fonction suivante, écrite en style **impératif**, sélectionne et met en majuscules les mots d'une liste dont la longueur atteint un minimum donné :

```
1 def mots_longs_imperatif(mots, longueur_min):
2     resultat = []
3     for mot in mots:
4         if len(mot) >= longueur_min:
5             resultat.append(mot.upper())
6     return resultat
```

1. Réécrire cette fonction en style **fonctionnel**, sous le nom `mots_longs_fonctionnel`, en une seule expression (liste en compréhension).
2. Vérifier que les deux versions renvoient le même résultat sur `["chat", "ordinateur", "cle", "programmation"]` avec `longueur_min=6`.

Exercice 20

Le code suivant contient un bug lié à un **effet de bord** inattendu :

```
1 def ajouter_note(bulletin, note):
2     bulletin.append(note)
3     return bulletin
4
5 bulletin_A = []
6 bulletin_B = ajouter_note(bulletin_A, 15)
7 bulletin_C = ajouter_note(bulletin_A, 12)
```

1. Que contiennent `bulletin_A`, `bulletin_B` et `bulletin_C` après ces trois lignes ? Le résultat est-il celui attendu si l'on voulait deux bulletins **indépendants** (un avec la note 15, un autre avec la note 12) ?
2. Corriger `ajouter_note` pour qu'elle renvoie un **nouveau** bulletin sans modifier celui passé en argument.

Exercice 21

Écrire une fonction récursive `somme_chiffres(n)` qui renvoie la somme des chiffres d'un entier positif `n` (par exemple, $1234 \rightarrow 1 + 2 + 3 + 4 = 10$). Quel est le cas de base ? Combien d'appels récurifs sont nécessaires pour `somme_chiffres(1234)` ?

Exercice 22

En consultant la documentation du module standard `math` (par exemple avec `help(math.hypot)` et `help(math.gcd)`) :

1. Utiliser `math.hypot(x, y)` pour calculer l'hypoténuse d'un triangle rectangle de côtés 3 et 4.
2. Utiliser `math.gcd(a, b)` pour calculer le plus grand diviseur commun de 48 et 18.
3. La documentation précise que `math.sqrt(x)` attend un nombre. Que se passe-t-il si l'on appelle `math.sqrt("4")` (une chaîne de caractères) au lieu de `math.sqrt(4)` ? Vérifier en lisant le message d'erreur obtenu.

Exercice 23

La fonction suivante, écrite en style **impératif**, sélectionne et met en majuscules les mots d'une liste dont la longueur atteint un minimum donné :

```
1 def mots_longs_imperatif(mots, longueur_min):
2     resultat = []
3     for mot in mots:
4         if len(mot) >= longueur_min:
5             resultat.append(mot.upper())
6     return resultat
```

1. Réécrire cette fonction en style **fonctionnel**, sous le nom `mots_longs_fonctionnel`, en une seule expression (liste en compréhension).
2. Vérifier que les deux versions renvoient le même résultat sur `["chat", "ordinateur", "cle", "programmation"]` avec `longueur_min=6`.

Exercice 24

Le code suivant contient un bug lié à un **effet de bord** inattendu :

```
1 def ajouter_note(bulletin, note):
2     bulletin.append(note)
```

```

3     return bulletin
4
5     bulletin_A = []
6     bulletin_B = ajouter_note(bulletin_A, 15)
7     bulletin_C = ajouter_note(bulletin_A, 12)

```

1. Que contiennent `bulletin_A`, `bulletin_B` et `bulletin_C` après ces trois lignes ? Le résultat est-il celui attendu si l'on voulait deux bulletins **indépendants** (un avec la note 15, un autre avec la note 12) ?
2. Corriger `ajouter_note` pour qu'elle renvoie un **nouveau** bulletin sans modifier celui passé en argument.

Exercice 25

Écrire une fonction récursive `somme_chiffres(n)` qui renvoie la somme des chiffres d'un entier positif n (par exemple, $1234 \rightarrow 1 + 2 + 3 + 4 = 10$). Quel est le cas de base ? Combien d'appels récur­sifs sont nécessaires pour `somme_chiffres(1234)` ?

Exercice 26

En consultant la documentation du module standard `math` (par exemple avec `help(math.hypot)` et `help(math.gcd)`) :

1. Utiliser `math.hypot(x, y)` pour calculer l'hypoténuse d'un triangle rectangle de côtés 3 et 4.
2. Utiliser `math.gcd(a, b)` pour calculer le plus grand diviseur commun de 48 et 18.
3. La documentation précise que `math.sqrt(x)` attend un nombre. Que se passe-t-il si l'on appelle `math.sqrt("4")` (une chaîne de caractères) au lieu de `math.sqrt(4)` ? Vérifier en lisant le message d'erreur obtenu.

Exercice 27

La fonction suivante, écrite en style **impératif**, sélectionne et met en majuscules les mots d'une liste dont la longueur atteint un minimum donné :

```

1 def mots_longs_imperatif(mots, longueur_min):
2     resultat = []
3     for mot in mots:
4         if len(mot) >= longueur_min:
5             resultat.append(mot.upper())
6     return resultat

```

1. Réécrire cette fonction en style **fonctionnel**, sous le nom `mots_longs_fonctionnel`, en une seule expression (liste en compréhension).
2. Vérifier que les deux versions renvoient le même résultat sur `["chat", "ordinateur", "cle", "programmation"]` avec `longueur_min=6`.

Exercice 28

Le code suivant contient un bug lié à un **effet de bord** inattendu :

```

1 def ajouter_note(bulletin, note):
2     bulletin.append(note)
3     return bulletin
4
5     bulletin_A = []
6     bulletin_B = ajouter_note(bulletin_A, 15)
7     bulletin_C = ajouter_note(bulletin_A, 12)

```

1. Que contiennent `bulletin_A`, `bulletin_B` et `bulletin_C` après ces trois lignes ? Le résultat est-il celui attendu si l'on voulait deux bulletins **indépendants** (un avec la note 15, un autre avec la note 12) ?
2. Corriger `ajouter_note` pour qu'elle renvoie un **nouveau** bulletin sans modifier celui passé en argument.

Exercice 29

Écrire une fonction récursive `somme_chiffres(n)` qui renvoie la somme des chiffres d'un entier positif `n` (par exemple, $1234 \rightarrow 1 + 2 + 3 + 4 = 10$). Quel est le cas de base ? Combien d'appels récurifs sont nécessaires pour `somme_chiffres(1234)` ?

Exercice 30

En consultant la documentation du module standard `math` (par exemple avec `help(math.hypot)` et `help(math.gcd)`) :

1. Utiliser `math.hypot(x, y)` pour calculer l'hypoténuse d'un triangle rectangle de côtés 3 et 4.
2. Utiliser `math.gcd(a, b)` pour calculer le plus grand diviseur commun de 48 et 18.
3. La documentation précise que `math.sqrt(x)` attend un nombre. Que se passe-t-il si l'on appelle `math.sqrt("4")` (une chaîne de caractères) au lieu de `math.sqrt(4)` ? Vérifier en lisant le message d'erreur obtenu.

Exercice 31

La fonction suivante, écrite en style **impératif**, sélectionne et met en majuscules les mots d'une liste dont la longueur atteint un minimum donné :

```
1 def mots_longs_imperatif(mots, longueur_min):
2     resultat = []
3     for mot in mots:
4         if len(mot) >= longueur_min:
5             resultat.append(mot.upper())
6     return resultat
```

1. Réécrire cette fonction en style **fonctionnel**, sous le nom `mots_longs_fonctionnel`, en une seule expression (liste en compréhension).
2. Vérifier que les deux versions renvoient le même résultat sur `["chat", "ordinateur", "cle", "programmation"]` avec `longueur_min=6`.

Exercice 32

Le code suivant contient un bug lié à un **effet de bord** inattendu :

```
1 def ajouter_note(bulletin, note):
2     bulletin.append(note)
3     return bulletin
4
5 bulletin_A = []
6 bulletin_B = ajouter_note(bulletin_A, 15)
7 bulletin_C = ajouter_note(bulletin_A, 12)
```

1. Que contiennent `bulletin_A`, `bulletin_B` et `bulletin_C` après ces trois lignes ? Le résultat est-il celui attendu si l'on voulait deux bulletins **indépendants** (un avec la note 15, un autre avec la note 12) ?
2. Corriger `ajouter_note` pour qu'elle renvoie un **nouveau** bulletin sans modifier celui passé en argument.

Exercice 33 ◆

Écrire une fonction récursive `somme_chiffres(n)` qui renvoie la somme des chiffres d'un entier positif n (par exemple, $1234 \rightarrow 1 + 2 + 3 + 4 = 10$). Quel est le cas de base ? Combien d'appels récursifs sont nécessaires pour `somme_chiffres(1234)` ?

Exercice 34 ◆

En consultant la documentation du module standard `math` (par exemple avec `help(math.hypot)` et `help(math.gcd)`) :

1. Utiliser `math.hypot(x, y)` pour calculer l'hypoténuse d'un triangle rectangle de côtés 3 et 4.
2. Utiliser `math.gcd(a, b)` pour calculer le plus grand diviseur commun de 48 et 18.
3. La documentation précise que `math.sqrt(x)` attend un nombre. Que se passe-t-il si l'on appelle `math.sqrt("4")` (une chaîne de caractères) au lieu de `math.sqrt(4)` ? Vérifier en lisant le message d'erreur obtenu.

Exercice 35 ◆◆

La fonction suivante, écrite en style **impératif**, sélectionne et met en majuscules les mots d'une liste dont la longueur atteint un minimum donné :

```
1 def mots_longs_imperatif(mots, longueur_min):
2     resultat = []
3     for mot in mots:
4         if len(mot) >= longueur_min:
5             resultat.append(mot.upper())
6     return resultat
```

1. Réécrire cette fonction en style **fonctionnel**, sous le nom `mots_longs_fonctionnel`, en une seule expression (liste en compréhension).
2. Vérifier que les deux versions renvoient le même résultat sur `["chat", "ordinateur", "cle", "programmation"]` avec `longueur_min=6`.

Exercice 36 ◆

Le code suivant contient un bug lié à un **effet de bord** inattendu :

```
1 def ajouter_note(bulletin, note):
2     bulletin.append(note)
3     return bulletin
4
5 bulletin_A = []
6 bulletin_B = ajouter_note(bulletin_A, 15)
7 bulletin_C = ajouter_note(bulletin_A, 12)
```

1. Que contiennent `bulletin_A`, `bulletin_B` et `bulletin_C` après ces trois lignes ? Le résultat est-il celui attendu si l'on voulait deux bulletins **indépendants** (un avec la note 15, un autre avec la note 12) ?
2. Corriger `ajouter_note` pour qu'elle renvoie un **nouveau** bulletin sans modifier celui passé en argument.

Exercice 37 ◆

Écrire une fonction récursive `somme_chiffres(n)` qui renvoie la somme des chiffres d'un entier positif n (par exemple, $1234 \rightarrow 1 + 2 + 3 + 4 = 10$). Quel est le cas de base ? Combien d'appels récursifs sont nécessaires pour `somme_chiffres(1234)` ?

Exercice 38

En consultant la documentation du module standard `math` (par exemple avec `help(math.hypot)` et `help(math.gcd)`) :

1. Utiliser `math.hypot(x, y)` pour calculer l'hypoténuse d'un triangle rectangle de côtés 3 et 4.
2. Utiliser `math.gcd(a, b)` pour calculer le plus grand diviseur commun de 48 et 18.
3. La documentation précise que `math.sqrt(x)` attend un nombre. Que se passe-t-il si l'on appelle `math.sqrt("4")` (une chaîne de caractères) au lieu de `math.sqrt(4)` ? Vérifier en lisant le message d'erreur obtenu.

Exercice 39

La fonction suivante, écrite en style **impératif**, sélectionne et met en majuscules les mots d'une liste dont la longueur atteint un minimum donné :

```
1 def mots_longs_imperatif(mots, longueur_min):
2     resultat = []
3     for mot in mots:
4         if len(mot) >= longueur_min:
5             resultat.append(mot.upper())
6     return resultat
```

1. Réécrire cette fonction en style **fonctionnel**, sous le nom `mots_longs_fonctionnel`, en une seule expression (liste en compréhension).
2. Vérifier que les deux versions renvoient le même résultat sur `["chat", "ordinateur", "cle", "programmation"]` avec `longueur_min=6`.

Exercice 40

Le code suivant contient un bug lié à un **effet de bord** inattendu :

```
1 def ajouter_note(bulletin, note):
2     bulletin.append(note)
3     return bulletin
4
5 bulletin_A = []
6 bulletin_B = ajouter_note(bulletin_A, 15)
7 bulletin_C = ajouter_note(bulletin_A, 12)
```

1. Que contiennent `bulletin_A`, `bulletin_B` et `bulletin_C` après ces trois lignes ? Le résultat est-il celui attendu si l'on voulait deux bulletins **indépendants** (un avec la note 15, un autre avec la note 12) ?
2. Corriger `ajouter_note` pour qu'elle renvoie un **nouveau** bulletin sans modifier celui passé en argument.

Exercice 41

Écrire une fonction récursive `somme_chiffres(n)` qui renvoie la somme des chiffres d'un entier positif `n` (par exemple, $1234 \rightarrow 1 + 2 + 3 + 4 = 10$). Quel est le cas de base ? Combien d'appels récurifs sont nécessaires pour `somme_chiffres(1234)` ?

Exercice 42

En consultant la documentation du module standard `math` (par exemple avec `help(math.hypot)` et `help(math.gcd)`) :

1. Utiliser `math.hypot(x, y)` pour calculer l'hypoténuse d'un triangle rectangle de côtés 3 et 4.

- Utiliser `math.gcd(a, b)` pour calculer le plus grand diviseur commun de 48 et 18.
- La documentation précise que `math.sqrt(x)` attend un nombre. Que se passe-t-il si l'on appelle `math.sqrt("4")` (une chaîne de caractères) au lieu de `math.sqrt(4)` ? Vérifier en lisant le message d'erreur obtenu.

Exercice 43

La fonction suivante, écrite en style **impératif**, sélectionne et met en majuscules les mots d'une liste dont la longueur atteint un minimum donné :

```
1 def mots_longs_imperatif(mots, longueur_min):
2     resultat = []
3     for mot in mots:
4         if len(mot) >= longueur_min:
5             resultat.append(mot.upper())
6     return resultat
```

- Réécrire cette fonction en style **fonctionnel**, sous le nom `mots_longs_fonctionnel`, en une seule expression (liste en compréhension).
- Vérifier que les deux versions renvoient le même résultat sur `["chat", "ordinateur", "cle", "programmation"]` avec `longueur_min=6`.

Exercice 44

Le code suivant contient un bug lié à un **effet de bord** inattendu :

```
1 def ajouter_note(bulletin, note):
2     bulletin.append(note)
3     return bulletin
4
5 bulletin_A = []
6 bulletin_B = ajouter_note(bulletin_A, 15)
7 bulletin_C = ajouter_note(bulletin_A, 12)
```

- Que contiennent `bulletin_A`, `bulletin_B` et `bulletin_C` après ces trois lignes ? Le résultat est-il celui attendu si l'on voulait deux bulletins **indépendants** (un avec la note 15, un autre avec la note 12) ?
- Corriger `ajouter_note` pour qu'elle renvoie un **nouveau** bulletin sans modifier celui passé en argument.

Exercice 45

Écrire une fonction récursive `somme_chiffres(n)` qui renvoie la somme des chiffres d'un entier positif `n` (par exemple, $1234 \rightarrow 1 + 2 + 3 + 4 = 10$). Quel est le cas de base ? Combien d'appels récurifs sont nécessaires pour `somme_chiffres(1234)` ?

Exercice 46

En consultant la documentation du module standard `math` (par exemple avec `help(math.hypot)` et `help(math.gcd)`) :

- Utiliser `math.hypot(x, y)` pour calculer l'hypoténuse d'un triangle rectangle de côtés 3 et 4.
- Utiliser `math.gcd(a, b)` pour calculer le plus grand diviseur commun de 48 et 18.
- La documentation précise que `math.sqrt(x)` attend un nombre. Que se passe-t-il si l'on appelle `math.sqrt("4")` (une chaîne de caractères) au lieu de `math.sqrt(4)` ? Vérifier en lisant le message d'erreur obtenu.

Exercice 47 ♦♦

La fonction suivante, écrite en style **impératif**, sélectionne et met en majuscules les mots d'une liste dont la longueur atteint un minimum donné :

```
1 def mots_longes_imperatif(mots, longueur_min):
2     resultat = []
3     for mot in mots:
4         if len(mot) >= longueur_min:
5             resultat.append(mot.upper())
6     return resultat
```

1. Réécrire cette fonction en style **fonctionnel**, sous le nom `mots_longes_fonctionnel`, en une seule expression (liste en compréhension).
2. Vérifier que les deux versions renvoient le même résultat sur `["chat", "ordinateur", "cle", "programmation"]` avec `longueur_min=6`.

Exercice 48 ♦

Le code suivant contient un bug lié à un **effet de bord** inattendu :

```
1 def ajouter_note(bulletin, note):
2     bulletin.append(note)
3     return bulletin
4
5 bulletin_A = []
6 bulletin_B = ajouter_note(bulletin_A, 15)
7 bulletin_C = ajouter_note(bulletin_A, 12)
```

1. Que contiennent `bulletin_A`, `bulletin_B` et `bulletin_C` après ces trois lignes ? Le résultat est-il celui attendu si l'on voulait deux bulletins **indépendants** (un avec la note 15, un autre avec la note 12) ?
2. Corriger `ajouter_note` pour qu'elle renvoie un **nouveau** bulletin sans modifier celui passé en argument.

Exercice 49 ♦

Écrire une fonction récursive `somme_chiffres(n)` qui renvoie la somme des chiffres d'un entier positif `n` (par exemple, $1234 \rightarrow 1 + 2 + 3 + 4 = 10$). Quel est le cas de base ? Combien d'appels récurifs sont nécessaires pour `somme_chiffres(1234)` ?

Exercice 50 ♦

En consultant la documentation du module standard `math` (par exemple avec `help(math.hypot)` et `help(math.gcd)`) :

1. Utiliser `math.hypot(x, y)` pour calculer l'hypoténuse d'un triangle rectangle de côtés 3 et 4.
2. Utiliser `math.gcd(a, b)` pour calculer le plus grand diviseur commun de 48 et 18.
3. La documentation précise que `math.sqrt(x)` attend un nombre. Que se passe-t-il si l'on appelle `math.sqrt("4")` (une chaîne de caractères) au lieu de `math.sqrt(4)` ? Vérifier en lisant le message d'erreur obtenu.

LANGAGES, RÉCURSIVITÉ ET PARADIGMES — QCM D'AUTO-ÉVALUATION

Pour chaque question, une seule réponse est exacte.

Capacité C1

1. [Q1] Affirmer qu'un programme est aussi une donnée signifie que :
- A. un programme peut être lu, transformé ou exécuté par un autre programme.
 - B. un programme n'a aucun effet lorsqu'on l'exécute.
 - C. les programmes et les données occupent la même quantité de mémoire.
 - D. un programme ne peut être écrit que par une machine.

Capacité C2

2. [Q2] Dire que la calculabilité ne dépend pas du langage employé signifie que :
- A. tous les langages s'exécutent à la même vitesse.
 - B. ce qui est calculable dans un langage l'est dans tout autre langage universel.
 - C. tous les langages ont la même syntaxe.
 - D. un algorithme ne peut s'écrire que dans un seul langage.

Capacité C3

3. [Q3] Dire que le problème de l'arrêt est indécidable signifie que :
- A. aucun programme ne s'arrête jamais.
 - B. on ne peut jamais prouver qu'un programme donné s'arrête.
 - C. aucun programme unique ne peut décider, pour tout programme et toute entrée, si l'exécution s'arrête.
 - D. cette difficulté ne concerne que Python.

Capacité C4

4. [Q4] Dans une fonction récursive, le cas de base sert à :
- A. accélérer le calcul.
 - B. déclarer les variables locales.
 - C. vérifier le type des arguments reçus.
 - D. arrêter la récursion en fournissant une valeur sans nouvel appel.

Capacité C5

5. [Q5] Un grand nombre d'appels dans l'arbre d'appels d'une fonction récursive indique que :
- A. la fonction termine, mais peut coûter cher en temps.
 - B. le code comporte nécessairement une erreur.
 - C. la fonction ne termine jamais.
 - D. le cas de base est mal écrit.

Capacité C6

6. [Q6] L'API d'une bibliothèque désigne :
- A. son code source complet.
 - B. l'ensemble des fonctions et des paramètres qu'elle met à disposition.
 - C. le nom de son auteur.
 - D. sa vitesse d'exécution.

Capacité C7

7. [Q7] La documentation d'une fonction indique qu'elle « renvoie None si la clé est absente ». Le programme appelant doit :
- A. modifier la bibliothèque pour qu'elle lève une exception.
 - B. ignorer cette mention, qui ne concerne que les développeurs de la bibliothèque.
 - C. traiter explicitement ce cas avant d'utiliser la valeur renvoyée.
 - D. supposer que la clé est toujours présente.

Capacité C8

8. [Q8] Dans un module Python destiné à être réutilisé, le bloc `if __name__ == "__main__":` sert à :
- A. déclarer les fonctions exportées par le module.
 - B. documenter le module.
 - C. importer automatiquement toutes les dépendances.
 - D. n'exécuter le code de démonstration ou de test que si le fichier est lancé directement.

Capacité C9

9. [Q9] Un même programme Python peut mobiliser :
- A. plusieurs paradigmes à la fois : impératif, fonctionnel et objet.
 - B. le seul paradigme objet.
 - C. un seul paradigme, imposé par le langage.
 - D. aucun paradigme identifiable.

Capacité C10

10. [Q10] Pour appliquer une même transformation à chaque élément d'une liste, sans modifier la liste de départ, le style le mieux adapté est :
- A. impératif, en modifiant la liste sur place case par case.
 - B. fonctionnel, par une compréhension ou une application de fonction produisant une nouvelle liste.
 - C. objet, en créant une classe par élément.
 - D. aucun : cette transformation est impossible.

Capacité C11

11. [Q11] Comparer deux nombres flottants avec l'opérateur == est une source de bugs parce que :
- A. Python interdit cette comparaison.
 - B. l'opérateur ne fonctionne qu'entre entiers.
 - C. les flottants ne représentent pas exactement toutes les valeurs décimales.
 - D. tous les flottants sont considérés comme égaux entre eux.
12. [Q12] Écrire `b = a`, où `a` est une liste :
- A. vide la liste `a`.
 - B. crée une copie indépendante de `a`.
 - C. provoque une erreur.
 - D. fait désigner à `b` le même objet que `a`.

Clé de correction — réservée au professeur

Question	Capacité	Réponse exacte
Q1	C1	A
Q2	C2	B
Q3	C3	C
Q4	C4	D
Q5	C5	A
Q6	C6	B
Q7	C7	C
Q8	C8	D
Q9	C9	A
Q10	C10	B
Q11	C11	C
Q12	C11	D

Diagnostic des réponses erronées

► Q1

- **B** — Être une donnée ne l'empêche pas d'être exécutable : c'est justement de porter les deux statuts qui fait sa force. *Renvoi : C1, programme et donnée.*
- **C** — L'élève rapproche deux notions sans rapport. Il s'agit de la NATURE du programme, non de son encombrement. *Renvoi : C1, programme et donnée.*
- **D** — Que des programmes puissent en produire d'autres, comme le fait un compilateur, ne retire rien à l'écriture humaine. *Renvoi : C1, programme et donnée.*

► Q2

- **A** — L'élève confond calculabilité et performance. Le résultat porte sur ce qui est possible, jamais sur le temps mis à l'obtenir. *Renvoi : C2, calculabilité et langages.*
- **C** — Les syntaxes diffèrent nettement ; c'est le POUVOIR d'expression qui coïncide, non l'écriture. *Renvoi : C2, calculabilité et langages.*
- **D** — C'est exactement l'inverse : un même algorithme s'écrit dans n'importe quel langage universel. *Renvoi : C2, calculabilité et langages.*

► Q3

- **A** — Beaucoup de programmes s'arrêtent, et on le constate. Ce qui est impossible est de trancher pour TOUS les cas par un procédé unique. *Renvoi : C3, indécidabilité du problème de l'arrêt.*
- **B** — L'élève passe du général au particulier. Pour un programme donné, une preuve est souvent accessible ; c'est la méthode universelle qui n'existe pas. *Renvoi : C3, indécidabilité du problème de l'arrêt.*
- **D** — Le résultat est indépendant du langage : il vaut pour tout modèle de calcul universel. *Renvoi : C3, indécidabilité du problème de l'arrêt.*

► Q4

- **A** — Le cas de base ne vise pas la rapidité mais la TERMINAISON : sans lui, les appels se poursuivent jusqu'au débordement de la pile. *Renvoi : C4, écrire un programme récursif.*
- **B** — La déclaration de variables n'a aucun rapport avec la condition d'arrêt des appels. *Renvoi : C4, écrire un programme récursif.*
- **C** — La vérification des arguments est une précaution indépendante, qui n'interrompt pas la récursion. *Renvoi : C4, écrire un programme récursif.*

► Q5

- **B** — La récursion naïve du calcul de Fibonacci est correcte, tout en engendrant un nombre exponentiel d'appels. *Renvoi : C5, analyse d'un programme récursif.*
- **C** — Un arbre d'appels FINI, même vaste, décrit précisément une exécution qui se termine. *Renvoi : C5, analyse d'un programme récursif.*
- **D** — Un cas de base fautif produirait une récursion sans fin, donc une erreur d'exécution, et non simplement un grand arbre. *Renvoi : C5, analyse d'un programme récursif.*

► Q6

- A — L'élève confond l'interface et sa réalisation. L'API est ce que la bibliothèque promet ; le code source est la façon dont elle tient sa promesse. *Renvoi : C6, utiliser une API.*
- C — La paternité relève des métadonnées du paquet, sans rapport avec les points d'entrée disponibles. *Renvoi : C6, utiliser une API.*
- D — La performance est une propriété mesurée, et non une description de ce qui est offert. *Renvoi : C6, utiliser une API.*

► Q7

- A — Modifier une dépendance pour éviter d'en lire le contrat est disproportionné, et rend toute mise à jour impossible. *Renvoi : C7, exploiter une documentation.*
- B — Cette mention s'adresse au contraire à l'appelant : elle décrit ce qu'il recevra, et donc ce qu'il doit prévoir. *Renvoi : C7, exploiter une documentation.*
- D — La documentation avertit précisément du contraire. L'ignorer conduira à manipuler None comme s'il s'agissait d'une valeur. *Renvoi : C7, exploiter une documentation.*

► Q8

- A — Les fonctions sont exportées du seul fait d'être définies ; ce bloc n'en déclare aucune. *Renvoi : C8, créer et documenter un module.*
- B — La documentation est portée par la docstring placée en tête du module, non par une condition d'exécution. *Renvoi : C8, créer et documenter un module.*
- C — Les imports s'écrivent explicitement en tête de fichier ; aucun mécanisme automatique n'existe. *Renvoi : C8, créer et documenter un module.*

► Q9

- B — Une simple boucle for qui modifie une variable relève de l'impératif, sans le moindre objet défini par le programmeur. *Renvoi : C9, paradigmes de programmation.*
- C — Python est un langage multiparadigme : il n'en impose aucun et les laisse coexister. *Renvoi : C9, paradigmes de programmation.*
- D — Tout programme relève d'au moins un style d'organisation ; l'absence de paradigme n'a pas de sens. *Renvoi : C9, paradigmes de programmation.*

► Q10

- A — Modifier sur place contredit l'exigence énoncée : la liste initiale doit rester intacte. *Renvoi : C10, choisir un paradigme adapté.*
- C — Introduire une classe par élément alourdit considérablement le programme sans rien apporter à une simple transformation. *Renvoi : C10, choisir un paradigme adapté.*
- D — L'opération est au contraire courante ; c'est même l'exemple type du style fonctionnel. *Renvoi : C10, choisir un paradigme adapté.*

► Q11

- A — La comparaison est autorisée et ne lève aucune erreur : c'est bien ce qui la rend trompeuse. *Renvoi : C11, causes typiques de bugs.*
- B — == s'applique à tous les types ; le problème vient de la représentation des flottants, non de l'opérateur. *Renvoi : C11, causes typiques de bugs.*
- D — Deux flottants distincts sont bien reconnus comme différents. La difficulté est que deux calculs mathématiquement égaux peuvent donner deux valeurs distinctes. *Renvoi : C11, causes typiques de bugs.*

► Q12

- A — Le contenu de a n'est en rien affecté : seul un nouveau nom est introduit. *Renvoi : C11, alias et objets mutables.*
- B — L'affectation ne copie jamais un objet mutable : elle ne fait que lui donner un second nom, et toute modification par l'un se voit par l'autre. *Renvoi : C11, alias et objets mutables.*
- C — L'opération est parfaitement licite ; elle est même très courante, ce qui explique que le piège passe inaperçu. *Renvoi : C11, alias et objets mutables.*

Corrigé

Q1. Le code source d'un programme est une suite de caractères, stockable et manipulable comme n'importe quelle donnée : par exemple, un interpréteur lit le code d'un autre programme comme une donnée en entrée, pour l'exécuter instruction par instruction.

Q2. Si un programme `s_arrete(P, e)` pouvait toujours décider correctement si `P` s'arrête sur `e`, on pourrait construire un programme paradoxe qui fait exactement le contraire de ce que `s_arrete` (paradoxe, paradoxe) prédit pour lui-même : `s_arrete` se tromperait alors nécessairement sur ce cas précis, ce qui contredit l'hypothèse qu'il décide correctement dans tous les cas. Un tel programme ne peut donc pas exister.

Corrigé

Q1.

```
1 def puissance(base, exposant):
2     if exposant == 0:
3         return 1
4     return base * puissance(base, exposant - 1)
```

Q2. `puissance(2, 4)` appelle `puissance(2, 3)`, qui appelle `puissance(2, 2)`, qui appelle `puissance(2, 1)`, qui appelle `puissance(2, 0)` : ce dernier atteint le cas de base (`exposant == 0`) et renvoie 1 directement. Les résultats remontent ensuite en cascade : $1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow 16$.

Évaluation A — Calculabilité et récursivité

Durée : 45 minutes. Ordinateur avec interpréteur Python autorisé.

Barème indicatif : Ex. 1 : 8 pts (programme comme donnée, problème de l'arrêt) ; Ex. 2 : 12 pts (récursivité)

Exercice 1 — Programme comme donnée et calculabilité

(8 points)

- (4 pts) Expliquer, avec un exemple, en quoi un programme est aussi une donnée.
- (4 pts) Résumer en quelques phrases l'argument montrant que le problème de l'arrêt est indécidable (sans formalisme, l'idée du raisonnement par l'absurde suffit).

Exercice 2 — Récursivité

(12 points)

- (6 pts) Écrire une fonction récursive `puissance(base, exposant)` qui calcule $\text{base}^{\text{exposant}}$ pour un exposant entier positif ou nul (sans utiliser l'opérateur `**`).
- (6 pts) Décrire l'arbre d'appels de `puissance(2, 4)` : quels appels successifs sont déclenchés, et quel est le cas de base atteint ?

Corrigé

```
1 def est_premier(n):
2     """Indique si l'entier n (n > 1) est premier.
3
4     Parametre : n, entier strictement supérieur à 1.
5     Renvoie : True si n est premier, False sinon.
6     """
7     for d in range(2, n):
8         if n % d == 0:
9             return False
10    return True
```

```

11
12 print(est_premier(7)) # True
13 print(est_premier(8)) # False

```

Corrigé

Q1.

```

1 def carres_positifs_fonctionnel(valeurs):
2     return [v ** 2 for v in valeurs if v > 0]

```

Q2. La fonction native masquée est `sum`. Le code fonctionne malgré tout ici car la variable locale `sum` n'est utilisée que comme accumulateur, sans jamais appeler la fonction native `sum(...)` dans le reste de la fonction. Le risque apparaîtrait si l'on avait besoin d'appeler `sum(...)` plus loin dans le même bloc : l'appel échouerait car `sum` désignerait alors un entier, plus une fonction.

Q3. `resultat == 0.1` renvoie `False` : à cause des arrondis internes de la représentation binaire des flottants, $0.3 - 0.2$ ne vaut pas exactement 0.1 (l'écart est de l'ordre de 10^{-17}). Comparer deux flottants avec `==` est risqué pour cette raison ; il faut comparer leur écart à une tolérance près : `abs(resultat - 0.1) < 1e-9`.

Évaluation B — Modules, paradigmes et débogage

Durée : 45 minutes. Ordinateur avec interpréteur Python autorisé.

Barème indicatif : Ex. 1 : 8 pts (module documenté) ; Ex. 2 : 12 pts (paradigmes, débogage)

Exercice 1 — Créer un module documenté

(8 points)

- (5 pts) Écrire une fonction `est_premier(n)`, destinée à un module `arithmetique.py`, qui renvoie `True` si l'entier n (supérieur à 1) est premier, `False` sinon. Ajouter une docstring décrivant son rôle, son paramètre et sa valeur de retour.
- (3 pts) Tester la fonction sur $n = 7$ (premier) et $n = 8$ (non premier).

Exercice 2 — Paradigmes et débogage

(12 points)

- (4 pts) Réécrire en style fonctionnel (une liste en compréhension) la fonction impérative suivante :

```

1 def carres_positifs_imperatif(valeurs):
2     resultat = []
3     for v in valeurs:
4         if v > 0:
5             resultat.append(v ** 2)
6     return resultat

```

- (4 pts) Le code suivant contient un bug de nommage :

```

1 def calculer_moyenne(notes):
2     sum = 0
3     for n in notes:
4         sum += n
5     return sum / len(notes)

```

Quelle fonction native est masquée ? Ce code fonctionne-t-il malgré tout pour calculer une moyenne simple ? Justifier.

- (4 pts) Le code suivant compare deux résultats de calculs flottants avec `==`. Que renvoie-t-il, pourquoi est-ce risqué de comparer ainsi, et comment corriger ?

```
1 resultat = 0.3 - 0.2
2 print(resultat == 0.1)
```

Exercice 51

Une fonction censée vérifier qu'un indice est valide pour une liste de longueur n est écrite ainsi :

```
1 def indice_valide_bug(indice, n):
2     return 0 <= indice <= n
```

Pour une liste de longueur $n = 5$ (indices valides : 0 à 4), que renvoie `indice_valide_bug(5, 5)` ? Est-ce correct ? Corriger la fonction.

Corrigé

`indice_valide_bug(5, 5)` renvoie `True`, ce qui est **incorrect** : pour une liste de longueur 5, les indices valides sont 0, 1, 2, 3, 4 (indice 5 est hors limites, il provoquerait un `IndexError`). L'erreur vient de l'inégalité **large** `<= n` au lieu de **stricte** `< n` :

```
1 def indice_valide_corrige(indice, n):
2     return 0 <= indice < n
```

Exercice 52

Une fonction censée vérifier qu'un indice est valide pour une liste de longueur n est écrite ainsi :

```
1 def indice_valide_bug(indice, n):
2     return 0 <= indice <= n
```

Pour une liste de longueur $n = 5$ (indices valides : 0 à 4), que renvoie `indice_valide_bug(5, 5)` ? Est-ce correct ? Corriger la fonction.

Corrigé

`indice_valide_bug(5, 5)` renvoie `True`, ce qui est **incorrect** : pour une liste de longueur 5, les indices valides sont 0, 1, 2, 3, 4 (indice 5 est hors limites, il provoquerait un `IndexError`). L'erreur vient de l'inégalité **large** `<= n` au lieu de **stricte** `< n` :

```
1 def indice_valide_corrige(indice, n):
2     return 0 <= indice < n
```

Exercice 53

Une fonction censée vérifier qu'un indice est valide pour une liste de longueur n est écrite ainsi :

```
1 def indice_valide_bug(indice, n):
2     return 0 <= indice <= n
```

Pour une liste de longueur $n = 5$ (indices valides : 0 à 4), que renvoie `indice_valide_bug(5, 5)` ? Est-ce correct ? Corriger la fonction.

Corrigé

`indice_valide_bug(5, 5)` renvoie `True`, ce qui est **incorrect** : pour une liste de longueur 5, les indices valides sont 0, 1, 2, 3, 4 (indice 5 est hors limites, il provoquerait un `IndexError`). L'erreur vient de l'inégalité **large** `<= n` au lieu de **stricte** `< n` :

```
1 def indice_valide_corrige(indice, n):
2     return 0 <= indice < n
```

Exercice 54 ◆

Une fonction censée vérifier qu'un indice est valide pour une liste de longueur n est écrite ainsi :

```
1 def indice_valide_bug(indice, n):
2     return 0 <= indice <= n
```

Pour une liste de longueur $n = 5$ (indices valides : 0 à 4), que renvoie `indice_valide_bug(5, 5)` ? Est-ce correct ? Corriger la fonction.

Corrigé

`indice_valide_bug(5, 5)` renvoie `True`, ce qui est **incorrect** : pour une liste de longueur 5, les indices valides sont 0, 1, 2, 3, 4 (indice 5 est hors limites, il provoquerait un `IndexError`). L'erreur vient de l'inégalité **large** `<= n` au lieu de **stricte** `< n` :

```
1 def indice_valide_corrige(indice, n):
2     return 0 <= indice < n
```

Exercice 55 ◆

Une fonction censée vérifier qu'un indice est valide pour une liste de longueur n est écrite ainsi :

```
1 def indice_valide_bug(indice, n):
2     return 0 <= indice <= n
```

Pour une liste de longueur $n = 5$ (indices valides : 0 à 4), que renvoie `indice_valide_bug(5, 5)` ? Est-ce correct ? Corriger la fonction.

Corrigé

`indice_valide_bug(5, 5)` renvoie `True`, ce qui est **incorrect** : pour une liste de longueur 5, les indices valides sont 0, 1, 2, 3, 4 (indice 5 est hors limites, il provoquerait un `IndexError`). L'erreur vient de l'inégalité **large** `<= n` au lieu de **stricte** `< n` :

```
1 def indice_valide_corrige(indice, n):
2     return 0 <= indice < n
```

Exercice 56 ◆

Une fonction censée vérifier qu'un indice est valide pour une liste de longueur n est écrite ainsi :

```
1 def indice_valide_bug(indice, n):
2     return 0 <= indice <= n
```

Pour une liste de longueur $n = 5$ (indices valides : 0 à 4), que renvoie `indice_valide_bug(5, 5)` ? Est-ce correct ? Corriger la fonction.

Corrigé

`indice_valide_bug(5, 5)` renvoie `True`, ce qui est **incorrect** : pour une liste de longueur 5, les indices valides sont 0, 1, 2, 3, 4 (indice 5 est hors limites, il provoquerait un `IndexError`). L'erreur vient de l'inégalité **large** `<= n` au lieu de **stricte** `< n` :

```
1 def indice_valide_corrige(indice, n):
2     return 0 <= indice < n
```

Exercice 57

Une fonction censée vérifier qu'un indice est valide pour une liste de longueur n est écrite ainsi :

```
1 def indice_valide_bug(indice, n):
2     return 0 <= indice <= n
```

Pour une liste de longueur $n = 5$ (indices valides : 0 à 4), que renvoie `indice_valide_bug(5, 5)` ? Est-ce correct ? Corriger la fonction.

Corrigé

`indice_valide_bug(5, 5)` renvoie `True`, ce qui est **incorrect** : pour une liste de longueur 5, les indices valides sont 0, 1, 2, 3, 4 (indice 5 est hors limites, il provoquerait un `IndexError`). L'erreur vient de l'inégalité **large** `<= n` au lieu de **stricte** `< n` :

```
1 def indice_valide_corrige(indice, n):
2     return 0 <= indice < n
```

Exercice 58

Une fonction censée vérifier qu'un indice est valide pour une liste de longueur n est écrite ainsi :

```
1 def indice_valide_bug(indice, n):
2     return 0 <= indice <= n
```

Pour une liste de longueur $n = 5$ (indices valides : 0 à 4), que renvoie `indice_valide_bug(5, 5)` ? Est-ce correct ? Corriger la fonction.

Corrigé

`indice_valide_bug(5, 5)` renvoie `True`, ce qui est **incorrect** : pour une liste de longueur 5, les indices valides sont 0, 1, 2, 3, 4 (indice 5 est hors limites, il provoquerait un `IndexError`). L'erreur vient de l'inégalité **large** `<= n` au lieu de **stricte** `< n` :

```
1 def indice_valide_corrige(indice, n):
2     return 0 <= indice < n
```

Exercice 59

Une fonction censée vérifier qu'un indice est valide pour une liste de longueur n est écrite ainsi :

```
1 def indice_valide_bug(indice, n):
2     return 0 <= indice <= n
```

Pour une liste de longueur $n = 5$ (indices valides : 0 à 4), que renvoie `indice_valide_bug(5, 5)` ? Est-ce correct ? Corriger la fonction.

Corrigé

`indice_valide_bug(5, 5)` renvoie `True`, ce qui est **incorrect** : pour une liste de longueur 5, les indices valides sont 0, 1, 2, 3, 4 (indice 5 est hors limites, il provoquerait un `IndexError`). L'erreur vient de l'inégalité **large** `<= n` au lieu de **stricte** `< n` :

```
1 def indice_valide_corrige(indice, n):
2     return 0 <= indice < n
```

Exercice 60 ◆

Une fonction censée vérifier qu'un indice est valide pour une liste de longueur n est écrite ainsi :

```
1 def indice_valide_bug(indice, n):
2     return 0 <= indice <= n
```

Pour une liste de longueur $n = 5$ (indices valides : 0 à 4), que renvoie `indice_valide_bug(5, 5)` ? Est-ce correct ? Corriger la fonction.

Corrigé

`indice_valide_bug(5, 5)` renvoie `True`, ce qui est **incorrect** : pour une liste de longueur 5, les indices valides sont 0, 1, 2, 3, 4 (indice 5 est hors limites, il provoquerait un `IndexError`). L'erreur vient de l'inégalité **large** `<= n` au lieu de **stricte** `< n` :

```
1 def indice_valide_corrige(indice, n):
2     return 0 <= indice < n
```

Exercice 61 ◆

Une fonction censée vérifier qu'un indice est valide pour une liste de longueur n est écrite ainsi :

```
1 def indice_valide_bug(indice, n):
2     return 0 <= indice <= n
```

Pour une liste de longueur $n = 5$ (indices valides : 0 à 4), que renvoie `indice_valide_bug(5, 5)` ? Est-ce correct ? Corriger la fonction.

Corrigé

`indice_valide_bug(5, 5)` renvoie `True`, ce qui est **incorrect** : pour une liste de longueur 5, les indices valides sont 0, 1, 2, 3, 4 (indice 5 est hors limites, il provoquerait un `IndexError`). L'erreur vient de l'inégalité **large** `<= n` au lieu de **stricte** `< n` :

```
1 def indice_valide_corrige(indice, n):
2     return 0 <= indice < n
```

Exercice 62 ◆

Une fonction censée vérifier qu'un indice est valide pour une liste de longueur n est écrite ainsi :

```
1 def indice_valide_bug(indice, n):
2     return 0 <= indice <= n
```

Pour une liste de longueur $n = 5$ (indices valides : 0 à 4), que renvoie `indice_valide_bug(5, 5)` ? Est-ce correct ? Corriger la fonction.

Corrigé

`indice_valide_bug(5, 5)` renvoie `True`, ce qui est **incorrect** : pour une liste de longueur 5, les indices valides sont 0, 1, 2, 3, 4 (indice 5 est hors limites, il provoquerait un `IndexError`). L'erreur vient de l'inégalité **large** `<= n` au lieu de **stricte** `< n` :

```

1 def indice_valide_corrige(indice, n):
2     return 0 <= indice < n

```

Corrigé

```

1 def somme_chiffres(n):
2     if n < 10:
3         return n
4     return n % 10 + somme_chiffres(n // 10)

```

Le cas de base est $n < 10$ (un seul chiffre) : la fonction renvoie n directement. Le cas général sépare le dernier chiffre ($n \% 10$) du reste du nombre ($n // 10$), et additionne récursivement. Pour `somme_chiffres(1234)`, l'arbre d'appels est `somme_chiffres(1234) → somme_chiffres(123) → somme_chiffres(12) → somme_chiffres(1)` : 4 appels au total (le dernier atteint le cas de base).

Corrigé

```

1 import math
2
3 print(math.hypot(3, 4)) # 5.0
4 print(math.gcd(48, 18)) # 6
5
6 try:
7     math.sqrt("4")
8 except TypeError as erreur:
9     print(erreur) # "must be real number, not str"

```

`math.sqrt` attend un nombre (entier ou flottant), pas une chaîne de caractères : appeler `math.sqrt("4")` lève une `TypeError`, même si "4" « ressemble » à un nombre. Il faudrait d'abord convertir explicitement avec `math.sqrt(int("4"))` ou `math.sqrt(float("4"))`.

Corrigé

```

1 def mots_longfs_fonctionnel(mots, longueur_min):
2     return [mot.upper() for mot in mots if len(mot) >= longueur_min]

1 mots = ["chat", "ordinateur", "cle", "programmation"]
2 print(mots_longfs_imperatif(mots, 6) == mots_longfs_fonctionnel(mots, 6)) # True
3 print(mots_longfs_fonctionnel(mots, 6)) # ['ORDINATEUR', 'PROGRAMMATION']

```

Les deux versions produisent le même résultat : la version fonctionnelle exprime en une seule expression ce que la version impérative construit pas à pas avec une variable d'accumulation (résultat) modifiée par des instructions successives.

Corrigé

Q1. `bulletin_A`, `bulletin_B` et `bulletin_C` désignent tous les **trois le même objet liste**, qui contient finalement `[15, 12]`. Ce n'est **pas** le résultat attendu : `append` modifie la liste **en place** (effet de bord), donc chaque appel modifie le même `bulletin_A` au lieu de créer un bulletin séparé pour chaque élève.

Q2.

```

1 def ajouter_note_corrige(bulletin, note):
2     return bulletin + [note]

```

L'opérateur `+` sur des listes crée une **nouvelle** liste (concaténation), sans modifier `bulletin` : `bulletin_A2` reste `[]`, `bulletin_B2` vaut `[15]` et `bulletin_C2` vaut `[12]`, chacun indépendant des autres.

Corrigé

```

1 def somme_chiffres(n):
2     if n < 10:
3         return n
4     return n % 10 + somme_chiffres(n // 10)

```

Le cas de base est $n < 10$ (un seul chiffre) : la fonction renvoie n directement. Le cas général sépare le dernier chiffre ($n \% 10$) du reste du nombre ($n // 10$), et additionne récursivement. Pour `somme_chiffres(1234)`, l'arbre d'appels est `somme_chiffres(1234) → somme_chiffres(123) → somme_chiffres(12) → somme_chiffres(1)` : 4 appels au total (le dernier atteint le cas de base).

Corrigé

```

1 import math
2
3 print(math.hypot(3, 4)) # 5.0
4 print(math.gcd(48, 18)) # 6
5
6 try:
7     math.sqrt("4")
8 except TypeError as erreur:
9     print(erreur) # "must be real number, not str"

```

`math.sqrt` attend un nombre (entier ou flottant), pas une chaîne de caractères : appeler `math.sqrt("4")` lève une `TypeError`, même si "4" « ressemble » à un nombre. Il faudrait d'abord convertir explicitement avec `math.sqrt(int("4"))` ou `math.sqrt(float("4"))`.

Corrigé

```

1 def mots_longfs_fonctionnel(mots, longueur_min):
2     return [mot.upper() for mot in mots if len(mot) >= longueur_min]

```

```

1 mots = ["chat", "ordinateur", "cle", "programmation"]
2 print(mots_longfs_imperatif(mots, 6) == mots_longfs_fonctionnel(mots, 6)) # True
3 print(mots_longfs_fonctionnel(mots, 6)) # ['ORDINATEUR', 'PROGRAMMATION']

```

Les deux versions produisent le même résultat : la version fonctionnelle exprime en une seule expression ce que la version impérative construit pas à pas avec une variable d'accumulation (résultat) modifiée par des instructions successives.

Corrigé

Q1. `bulletin_A`, `bulletin_B` et `bulletin_C` désignent tous les **trois le même objet liste**, qui contient finalement `[15, 12]`. Ce n'est **pas** le résultat attendu : `append` modifie la liste **en place** (effet de bord), donc chaque appel modifie le même `bulletin_A` au lieu de créer un bulletin séparé pour chaque élève.

Q2.

```

1 def ajouter_note_corrige(bulletin, note):
2     return bulletin + [note]

```

L'opérateur `+` sur des listes crée une **nouvelle** liste (concaténation), sans modifier `bulletin` : `bulletin_A2` reste `[]`, `bulletin_B2` vaut `[15]` et `bulletin_C2` vaut `[12]`, chacun indépendant des autres.

Corrigé

```

1 def somme_chiffres(n):
2     if n < 10:
3         return n
4     return n % 10 + somme_chiffres(n // 10)

```

Le cas de base est $n < 10$ (un seul chiffre) : la fonction renvoie n directement. Le cas général sépare le dernier chiffre ($n \% 10$) du reste du nombre ($n // 10$), et additionne récursivement. Pour `somme_chiffres(1234)`, l'arbre d'appels est `somme_chiffres(1234) → somme_chiffres(123) → somme_chiffres(12) → somme_chiffres(1)` : 4 appels au total (le dernier atteint le cas de base).

Corrigé

```

1 import math
2
3 print(math.hypot(3, 4)) # 5.0
4 print(math.gcd(48, 18)) # 6
5
6 try:
7     math.sqrt("4")
8 except TypeError as erreur:
9     print(erreur) # "must be real number, not str"

```

`math.sqrt` attend un nombre (entier ou flottant), pas une chaîne de caractères : appeler `math.sqrt("4")` lève une `TypeError`, même si "4" « ressemble » à un nombre. Il faudrait d'abord convertir explicitement avec `math.sqrt(int("4"))` ou `math.sqrt(float("4"))`.

Corrigé

```

1 def mots_longfs_fonctionnel(mots, longueur_min):
2     return [mot.upper() for mot in mots if len(mot) >= longueur_min]
3
4 mots = ["chat", "ordinateur", "cle", "programmation"]
5 print(mots_longfs_imperatif(mots, 6) == mots_longfs_fonctionnel(mots, 6)) # True
6 print(mots_longfs_fonctionnel(mots, 6)) # ['ORDINATEUR', 'PROGRAMMATION']

```

Les deux versions produisent le même résultat : la version fonctionnelle exprime en une seule expression ce que la version impérative construit pas à pas avec une variable d'accumulation (résultat) modifiée par des instructions successives.

Corrigé

Q1. `bulletin_A`, `bulletin_B` et `bulletin_C` désignent tous les **trois le même objet liste**, qui contient finalement `[15, 12]`. Ce n'est **pas** le résultat attendu : `append` modifie la liste **en place** (effet de bord), donc chaque appel modifie le même `bulletin_A` au lieu de créer un bulletin séparé pour chaque élève.

Q2.

```

1 def ajouter_note_corrige(bulletin, note):
2     return bulletin + [note]

```

L'opérateur `+` sur des listes crée une **nouvelle** liste (concaténation), sans modifier `bulletin` : `bulletin_A2` reste `[]`, `bulletin_B2` vaut `[15]` et `bulletin_C2` vaut `[12]`, chacun indépendant des autres.

Corrigé

```

1 def somme_chiffres(n):
2     if n < 10:

```

```
3     return n
4     return n % 10 + somme_chiffres(n // 10)
```

Le cas de base est $n < 10$ (un seul chiffre) : la fonction renvoie n directement. Le cas général sépare le dernier chiffre ($n \% 10$) du reste du nombre ($n // 10$), et additionne récursivement. Pour `somme_chiffres(1234)`, l'arbre d'appels est `somme_chiffres(1234) → somme_chiffres(123) → somme_chiffres(12) → somme_chiffres(1)` : 4 appels au total (le dernier atteint le cas de base).

Corrigé

```
1 import math
2
3 print(math.hypot(3, 4)) # 5.0
4 print(math.gcd(48, 18)) # 6
5
6 try:
7     math.sqrt("4")
8 except TypeError as erreur:
9     print(erreur) # "must be real number, not str"
```

`math.sqrt` attend un nombre (entier ou flottant), pas une chaîne de caractères : appeler `math.sqrt("4")` lève une `TypeError`, même si "4" « ressemble » à un nombre. Il faudrait d'abord convertir explicitement avec `math.sqrt(int("4"))` ou `math.sqrt(float("4"))`.

Corrigé

```
1 def mots_longfs_fonctionnel(mots, longueur_min):
2     return [mot.upper() for mot in mots if len(mot) >= longueur_min]
3
4 mots = ["chat", "ordinateur", "cle", "programmation"]
5 print(mots_longfs_imperatif(mots, 6) == mots_longfs_fonctionnel(mots, 6)) # True
6 print(mots_longfs_fonctionnel(mots, 6)) # ['ORDINATEUR', 'PROGRAMMATION']
```

Les deux versions produisent le même résultat : la version fonctionnelle exprime en une seule expression ce que la version impérative construit pas à pas avec une variable d'accumulation (resultat) modifiée par des instructions successives.

Corrigé

Q1. `bulletin_A`, `bulletin_B` et `bulletin_C` désignent tous les **trois le même objet liste**, qui contient finalement `[15, 12]`. Ce n'est **pas** le résultat attendu : `append` modifie la liste **en place** (effet de bord), donc chaque appel modifie le même `bulletin_A` au lieu de créer un bulletin séparé pour chaque élève.

Q2.

```
1 def ajouter_note_corrige(bulletin, note):
2     return bulletin + [note]
```

L'opérateur `+` sur des listes crée une **nouvelle** liste (concaténation), sans modifier `bulletin` : `bulletin_A2` reste `[]`, `bulletin_B2` vaut `[15]` et `bulletin_C2` vaut `[12]`, chacun indépendant des autres.

Corrigé

```
1 def somme_chiffres(n):
2     if n < 10:
3         return n
4     return n % 10 + somme_chiffres(n // 10)
```

Le cas de base est $n < 10$ (un seul chiffre) : la fonction renvoie n directement. Le cas général sépare le dernier chiffre ($n \% 10$) du reste du nombre ($n // 10$), et additionne récursivement. Pour `somme_chiffres(1234)`, l'arbre d'appels est `somme_chiffres(1234) → somme_chiffres(123) → somme_chiffres(12) → somme_chiffres(1)` : 4 appels au total (le dernier atteint le cas de base).

Corrigé

```
1 import math
2
3 print(math.hypot(3, 4)) # 5.0
4 print(math.gcd(48, 18)) # 6
5
6 try:
7     math.sqrt("4")
8 except TypeError as erreur:
9     print(erreur) # "must be real number, not str"
```

`math.sqrt` attend un nombre (entier ou flottant), pas une chaîne de caractères : appeler `math.sqrt("4")` lève une `TypeError`, même si "4" « ressemble » à un nombre. Il faudrait d'abord convertir explicitement avec `math.sqrt(int("4"))` ou `math.sqrt(float("4"))`.

Corrigé

```
1 def mots_longfs_fonctionnel(mots, longueur_min):
2     return [mot.upper() for mot in mots if len(mot) >= longueur_min]
3
4 mots = ["chat", "ordinateur", "cle", "programmation"]
5 print(mots_longfs_imperatif(mots, 6) == mots_longfs_fonctionnel(mots, 6)) # True
6 print(mots_longfs_fonctionnel(mots, 6)) # ['ORDINATEUR', 'PROGRAMMATION']
```

Les deux versions produisent le même résultat : la version fonctionnelle exprime en une seule expression ce que la version impérative construit pas à pas avec une variable d'accumulation (résultat) modifiée par des instructions successives.

Corrigé

Q1. `bulletin_A`, `bulletin_B` et `bulletin_C` désignent tous les **trois le même objet liste**, qui contient finalement `[15, 12]`. Ce n'est **pas** le résultat attendu : `append` modifie la liste **en place** (effet de bord), donc chaque appel modifie le même `bulletin_A` au lieu de créer un bulletin séparé pour chaque élève.

Q2.

```
1 def ajouter_note_corrige(bulletin, note):
2     return bulletin + [note]
```

L'opérateur `+` sur des listes crée une **nouvelle** liste (concaténation), sans modifier `bulletin` : `bulletin_A2` reste `[]`, `bulletin_B2` vaut `[15]` et `bulletin_C2` vaut `[12]`, chacun indépendant des autres.

Corrigé

```
1 def somme_chiffres(n):
2     if n < 10:
3         return n
4     return n % 10 + somme_chiffres(n // 10)
```

Le cas de base est $n < 10$ (un seul chiffre) : la fonction renvoie n directement. Le cas général sépare le dernier chiffre ($n \% 10$) du reste du nombre ($n // 10$), et additionne récursivement. Pour `somme_`

chiffres(1234), l'arbre d'appels est somme_chiffres(1234) → somme_chiffres(123) → somme_chiffres(12) → somme_chiffres(1) : 4 appels au total (le dernier atteint le cas de base).

Corrigé

```
1 import math
2
3 print(math.hypot(3, 4)) # 5.0
4 print(math.gcd(48, 18)) # 6
5
6 try:
7     math.sqrt("4")
8 except TypeError as erreur:
9     print(erreur) # "must be real number, not str"
```

math.sqrt attend un nombre (entier ou flottant), pas une chaîne de caractères : appeler math.sqrt("4") lève une TypeError, même si "4" « ressemble » à un nombre. Il faudrait d'abord convertir explicitement avec math.sqrt(int("4")) ou math.sqrt(float("4")).

Corrigé

```
1 def mots_long_fonctionnel(mots, longueur_min):
2     return [mot.upper() for mot in mots if len(mot) >= longueur_min]
3
4 mots = ["chat", "ordinateur", "cle", "programmation"]
5 print(mots_long_imperatif(mots, 6) == mots_long_fonctionnel(mots, 6)) # True
6 print(mots_long_fonctionnel(mots, 6)) # ['ORDINATEUR', 'PROGRAMMATION']
```

Les deux versions produisent le même résultat : la version fonctionnelle exprime en une seule expression ce que la version impérative construit pas à pas avec une variable d'accumulation (resultat) modifiée par des instructions successives.

Corrigé

Q1. bulletin_A, bulletin_B et bulletin_C désignent tous les **trois le même objet liste**, qui contient finalement [15, 12]. Ce n'est **pas** le résultat attendu : append modifie la liste **en place** (effet de bord), donc chaque appel modifie le même bulletin_A au lieu de créer un bulletin séparé pour chaque élève.

Q2.

```
1 def ajouter_note_corrige(bulletin, note):
2     return bulletin + [note]
```

L'opérateur + sur des listes crée une **nouvelle** liste (concaténation), sans modifier bulletin : bulletin_A2 reste [], bulletin_B2 vaut [15] et bulletin_C2 vaut [12], chacun indépendant des autres.

Corrigé

```
1 def somme_chiffres(n):
2     if n < 10:
3         return n
4     return n % 10 + somme_chiffres(n // 10)
```

Le cas de base est $n < 10$ (un seul chiffre) : la fonction renvoie n directement. Le cas général sépare le dernier chiffre ($n \% 10$) du reste du nombre ($n // 10$), et additionne récursivement. Pour somme_chiffres(1234), l'arbre d'appels est somme_chiffres(1234) → somme_chiffres(123) → somme_chiffres(12) → somme_chiffres(1) : 4 appels au total (le dernier atteint le cas de base).

Corrigé

```

1 import math
2
3 print(math.hypot(3, 4)) # 5.0
4 print(math.gcd(48, 18)) # 6
5
6 try:
7     math.sqrt("4")
8 except TypeError as erreur:
9     print(erreur) # "must be real number, not str"

```

`math.sqrt` attend un nombre (entier ou flottant), pas une chaîne de caractères : appeler `math.sqrt("4")` lève une `TypeError`, même si "4" « ressemble » à un nombre. Il faudrait d'abord convertir explicitement avec `math.sqrt(int("4"))` ou `math.sqrt(float("4"))`.

Corrigé

```

1 def mots_longfs_fonctionnel(mots, longueur_min):
2     return [mot.upper() for mot in mots if len(mot) >= longueur_min]

```

```

1 mots = ["chat", "ordinateur", "cle", "programmation"]
2 print(mots_longfs_imperatif(mots, 6) == mots_longfs_fonctionnel(mots, 6)) # True
3 print(mots_longfs_fonctionnel(mots, 6)) # ['ORDINATEUR', 'PROGRAMMATION']

```

Les deux versions produisent le même résultat : la version fonctionnelle exprime en une seule expression ce que la version impérative construit pas à pas avec une variable d'accumulation (résultat) modifiée par des instructions successives.

Corrigé

Q1. `bulletin_A`, `bulletin_B` et `bulletin_C` désignent tous les **trois le même objet liste**, qui contient finalement `[15, 12]`. Ce n'est **pas** le résultat attendu : `append` modifie la liste **en place** (effet de bord), donc chaque appel modifie le même `bulletin_A` au lieu de créer un bulletin séparé pour chaque élève.

Q2.

```

1 def ajouter_note_corrige(bulletin, note):
2     return bulletin + [note]

```

L'opérateur `+` sur des listes crée une **nouvelle** liste (concaténation), sans modifier `bulletin` : `bulletin_A2` reste `[]`, `bulletin_B2` vaut `[15]` et `bulletin_C2` vaut `[12]`, chacun indépendant des autres.

Corrigé

```

1 def somme_chiffres(n):
2     if n < 10:
3         return n
4     return n % 10 + somme_chiffres(n // 10)

```

Le cas de base est `n < 10` (un seul chiffre) : la fonction renvoie `n` directement. Le cas général sépare le dernier chiffre (`n % 10`) du reste du nombre (`n // 10`), et additionne récursivement. Pour `somme_chiffres(1234)`, l'arbre d'appels est `somme_chiffres(1234) → somme_chiffres(123) → somme_chiffres(12) → somme_chiffres(1)` : 4 appels au total (le dernier atteint le cas de base).

Corrigé

```

1 import math
2
3 print(math.hypot(3, 4)) # 5.0
4 print(math.gcd(48, 18)) # 6
5
6 try:
7     math.sqrt("4")
8 except TypeError as erreur:
9     print(erreur) # "must be real number, not str"

```

`math.sqrt` attend un nombre (entier ou flottant), pas une chaîne de caractères : appeler `math.sqrt("4")` lève une `TypeError`, même si "4" « ressemble » à un nombre. Il faudrait d'abord convertir explicitement avec `math.sqrt(int("4"))` ou `math.sqrt(float("4"))`.

Corrigé

```

1 def mots_longfs_fonctionnel(mots, longueur_min):
2     return [mot.upper() for mot in mots if len(mot) >= longueur_min]

1 mots = ["chat", "ordinateur", "cle", "programmation"]
2 print(mots_longfs_imperatif(mots, 6) == mots_longfs_fonctionnel(mots, 6)) # True
3 print(mots_longfs_fonctionnel(mots, 6)) # ['ORDINATEUR', 'PROGRAMMATION']

```

Les deux versions produisent le même résultat : la version fonctionnelle exprime en une seule expression ce que la version impérative construit pas à pas avec une variable d'accumulation (resultat) modifiée par des instructions successives.

Corrigé

Q1. `bulletin_A`, `bulletin_B` et `bulletin_C` désignent tous les **trois le même objet liste**, qui contient finalement `[15, 12]`. Ce n'est **pas** le résultat attendu : `append` modifie la liste **en place** (effet de bord), donc chaque appel modifie le même `bulletin_A` au lieu de créer un bulletin séparé pour chaque élève.

Q2.

```

1 def ajouter_note_corrige(bulletin, note):
2     return bulletin + [note]

```

L'opérateur `+` sur des listes crée une **nouvelle** liste (concaténation), sans modifier `bulletin` : `bulletin_A2` reste `[]`, `bulletin_B2` vaut `[15]` et `bulletin_C2` vaut `[12]`, chacun indépendant des autres.

Corrigé

```

1 def somme_chiffres(n):
2     if n < 10:
3         return n
4     return n % 10 + somme_chiffres(n // 10)

```

Le cas de base est $n < 10$ (un seul chiffre) : la fonction renvoie n directement. Le cas général sépare le dernier chiffre ($n \% 10$) du reste du nombre ($n // 10$), et additionne récursivement. Pour `somme_chiffres(1234)`, l'arbre d'appels est `somme_chiffres(1234) → somme_chiffres(123) → somme_chiffres(12) → somme_chiffres(1)` : 4 appels au total (le dernier atteint le cas de base).

Corrigé

```

1 import math
2

```

```

3 print(math.hypot(3, 4)) # 5.0
4 print(math.gcd(48, 18)) # 6
5
6 try:
7     math.sqrt("4")
8 except TypeError as erreur:
9     print(erreur) # "must be real number, not str"

```

`math.sqrt` attend un nombre (entier ou flottant), pas une chaîne de caractères : appeler `math.sqrt("4")` lève une `TypeError`, même si "4" « ressemble » à un nombre. Il faudrait d'abord convertir explicitement avec `math.sqrt(int("4"))` ou `math.sqrt(float("4"))`.

Corrigé

```

1 def mots_longfs_fonctionnel(mots, longueur_min):
2     return [mot.upper() for mot in mots if len(mot) >= longueur_min]

1 mots = ["chat", "ordinateur", "cle", "programmation"]
2 print(mots_longfs_imperatif(mots, 6) == mots_longfs_fonctionnel(mots, 6)) # True
3 print(mots_longfs_fonctionnel(mots, 6)) # ['ORDINATEUR', 'PROGRAMMATION']

```

Les deux versions produisent le même résultat : la version fonctionnelle exprime en une seule expression ce que la version impérative construit pas à pas avec une variable d'accumulation (résultat) modifiée par des instructions successives.

Corrigé

Q1. `bulletin_A`, `bulletin_B` et `bulletin_C` désignent tous les **trois le même objet liste**, qui contient finalement `[15, 12]`. Ce n'est **pas** le résultat attendu : `append` modifie la liste **en place** (effet de bord), donc chaque appel modifie le même `bulletin_A` au lieu de créer un bulletin séparé pour chaque élève.

Q2.

```

1 def ajouter_note_corrige(bulletin, note):
2     return bulletin + [note]

```

L'opérateur `+` sur des listes crée une **nouvelle** liste (concaténation), sans modifier `bulletin` : `bulletin_A2` reste `[]`, `bulletin_B2` vaut `[15]` et `bulletin_C2` vaut `[12]`, chacun indépendant des autres.

Corrigé

```

1 def somme_chiffres(n):
2     if n < 10:
3         return n
4     return n % 10 + somme_chiffres(n // 10)

```

Le cas de base est `n < 10` (un seul chiffre) : la fonction renvoie `n` directement. Le cas général sépare le dernier chiffre (`n % 10`) du reste du nombre (`n // 10`), et additionne récursivement. Pour `somme_chiffres(1234)`, l'arbre d'appels est `somme_chiffres(1234) → somme_chiffres(123) → somme_chiffres(12) → somme_chiffres(1)` : 4 appels au total (le dernier atteint le cas de base).

Corrigé

```

1 import math
2
3 print(math.hypot(3, 4)) # 5.0
4 print(math.gcd(48, 18)) # 6

```

```

5
6 try:
7     math.sqrt("4")
8 except TypeError as erreur:
9     print(erreur) # "must be real number, not str"

```

`math.sqrt` attend un nombre (entier ou flottant), pas une chaîne de caractères : appeler `math.sqrt("4")` lève une `TypeError`, même si "4" « ressemble » à un nombre. Il faudrait d'abord convertir explicitement avec `math.sqrt(int("4"))` ou `math.sqrt(float("4"))`.

Corrigé

```

1 def mots_longfs_fonctionnel(mots, longueur_min):
2     return [mot.upper() for mot in mots if len(mot) >= longueur_min]

1 mots = ["chat", "ordinateur", "cle", "programmation"]
2 print(mots_longfs_imperatif(mots, 6) == mots_longfs_fonctionnel(mots, 6)) # True
3 print(mots_longfs_fonctionnel(mots, 6)) # ['ORDINATEUR', 'PROGRAMMATION']

```

Les deux versions produisent le même résultat : la version fonctionnelle exprime en une seule expression ce que la version impérative construit pas à pas avec une variable d'accumulation (resultat) modifiée par des instructions successives.

Corrigé

Q1. `bulletin_A`, `bulletin_B` et `bulletin_C` désignent tous les **trois le même objet liste**, qui contient finalement `[15, 12]`. Ce n'est **pas** le résultat attendu : `append` modifie la liste **en place** (effet de bord), donc chaque appel modifie le même `bulletin_A` au lieu de créer un bulletin séparé pour chaque élève.

Q2.

```

1 def ajouter_note_corrige(bulletin, note):
2     return bulletin + [note]

```

L'opérateur `+` sur des listes crée une **nouvelle** liste (concaténation), sans modifier `bulletin` : `bulletin_A2` reste `[]`, `bulletin_B2` vaut `[15]` et `bulletin_C2` vaut `[12]`, chacun indépendant des autres.

Corrigé

```

1 def somme_chiffres(n):
2     if n < 10:
3         return n
4     return n % 10 + somme_chiffres(n // 10)

```

Le cas de base est `n < 10` (un seul chiffre) : la fonction renvoie `n` directement. Le cas général sépare le dernier chiffre (`n % 10`) du reste du nombre (`n // 10`), et additionne récursivement. Pour `somme_chiffres(1234)`, l'arbre d'appels est `somme_chiffres(1234) → somme_chiffres(123) → somme_chiffres(12) → somme_chiffres(1)` : 4 appels au total (le dernier atteint le cas de base).

Corrigé

```

1 import math
2
3 print(math.hypot(3, 4)) # 5.0
4 print(math.gcd(48, 18)) # 6
5
6 try:

```

```

7  math.sqrt("4")
8  except TypeError as erreur:
9      print(erreur) # "must be real number, not str"

```

`math.sqrt` attend un nombre (entier ou flottant), pas une chaîne de caractères : appeler `math.sqrt("4")` lève une `TypeError`, même si "4" « ressemble » à un nombre. Il faudrait d'abord convertir explicitement avec `math.sqrt(int("4"))` ou `math.sqrt(float("4"))`.

Corrigé

```

1  def mots_longs_fonctionnel(mots, longueur_min):
2      return [mot.upper() for mot in mots if len(mot) >= longueur_min]

```

```

1  mots = ["chat", "ordinateur", "cle", "programmation"]
2  print(mots_longs_imperatif(mots, 6) == mots_longs_fonctionnel(mots, 6)) # True
3  print(mots_longs_fonctionnel(mots, 6)) # ['ORDINATEUR', 'PROGRAMMATION']

```

Les deux versions produisent le même résultat : la version fonctionnelle exprime en une seule expression ce que la version impérative construit pas à pas avec une variable d'accumulation (résultat) modifiée par des instructions successives.

Corrigé

Q1. `bulletin_A`, `bulletin_B` et `bulletin_C` désignent tous les **trois le même objet liste**, qui contient finalement `[15, 12]`. Ce n'est **pas** le résultat attendu : `append` modifie la liste **en place** (effet de bord), donc chaque appel modifie le même `bulletin_A` au lieu de créer un bulletin séparé pour chaque élève.

Q2.

```

1  def ajouter_note_corrige(bulletin, note):
2      return bulletin + [note]

```

L'opérateur `+` sur des listes crée une **nouvelle** liste (concaténation), sans modifier `bulletin` : `bulletin_A2` reste `[]`, `bulletin_B2` vaut `[15]` et `bulletin_C2` vaut `[12]`, chacun indépendant des autres.

Corrigé

```

1  def somme_chiffres(n):
2      if n < 10:
3          return n
4      return n % 10 + somme_chiffres(n // 10)

```

Le cas de base est $n < 10$ (un seul chiffre) : la fonction renvoie n directement. Le cas général sépare le dernier chiffre ($n \% 10$) du reste du nombre ($n // 10$), et additionne récursivement. Pour `somme_chiffres(1234)`, l'arbre d'appels est `somme_chiffres(1234) → somme_chiffres(123) → somme_chiffres(12) → somme_chiffres(1)` : 4 appels au total (le dernier atteint le cas de base).

Corrigé

```

1  import math
2
3  print(math.hypot(3, 4)) # 5.0
4  print(math.gcd(48, 18)) # 6
5
6  try:
7      math.sqrt("4")
8  except TypeError as erreur:
9      print(erreur) # "must be real number, not str"

```

`math.sqrt` attend un nombre (entier ou flottant), pas une chaîne de caractères : appeler `math.sqrt("4")` lève une `TypeError`, même si "4" « ressemble » à un nombre. Il faudrait d'abord convertir explicitement avec `math.sqrt(int("4"))` ou `math.sqrt(float("4"))`.

Corrigé

```
1 def mots_long_fonctionnel(mots, longueur_min):
2     return [mot.upper() for mot in mots if len(mot) >= longueur_min]

1 mots = ["chat", "ordinateur", "cle", "programmation"]
2 print(mots_long_imperatif(mots, 6) == mots_long_fonctionnel(mots, 6)) # True
3 print(mots_long_fonctionnel(mots, 6)) # ['ORDINATEUR', 'PROGRAMMATION']
```

Les deux versions produisent le même résultat : la version fonctionnelle exprime en une seule expression ce que la version impérative construit pas à pas avec une variable d'accumulation (resultat) modifiée par des instructions successives.

Corrigé

Q1. `bulletin_A`, `bulletin_B` et `bulletin_C` désignent tous les **trois le même objet liste**, qui contient finalement `[15, 12]`. Ce n'est **pas** le résultat attendu : `append` modifie la liste **en place** (effet de bord), donc chaque appel modifie le même `bulletin_A` au lieu de créer un bulletin séparé pour chaque élève.

Q2.

```
1 def ajouter_note_corrige(bulletin, note):
2     return bulletin + [note]
```

L'opérateur `+` sur des listes crée une **nouvelle** liste (concaténation), sans modifier `bulletin` : `bulletin_A2` reste `[]`, `bulletin_B2` vaut `[15]` et `bulletin_C2` vaut `[12]`, chacun indépendant des autres.

Corrigé

```
1 def somme_chiffres(n):
2     if n < 10:
3         return n
4     return n % 10 + somme_chiffres(n // 10)
```

Le cas de base est `n < 10` (un seul chiffre) : la fonction renvoie `n` directement. Le cas général sépare le dernier chiffre (`n % 10`) du reste du nombre (`n // 10`), et additionne récursivement. Pour `somme_chiffres(1234)`, l'arbre d'appels est `somme_chiffres(1234) → somme_chiffres(123) → somme_chiffres(12) → somme_chiffres(1)` : 4 appels au total (le dernier atteint le cas de base).

Corrigé

```
1 import math
2
3 print(math.hypot(3, 4)) # 5.0
4 print(math.gcd(48, 18)) # 6
5
6 try:
7     math.sqrt("4")
8 except TypeError as erreur:
9     print(erreur) # "must be real number, not str"
```

`math.sqrt` attend un nombre (entier ou flottant), pas une chaîne de caractères : appeler `math.sqrt("4")` lève une `TypeError`, même si "4" « ressemble » à un nombre. Il faudrait d'abord convertir explicitement avec `math.sqrt(int("4"))` ou `math.sqrt(float("4"))`.

6 Structures de données

6.1 Interface et implémentation d'une structure de données

DÉFINITION 1

L'**interface** d'une structure de données est l'ensemble des **opérations** qu'elle propose (ce que l'on peut faire), **sans préciser comment** elles sont réalisées. Une **implémentation** est une façon concrète de coder ces opérations (avec quelles données internes, quel algorithme).

EXEMPLE Interface d'une pile (Stack) : empiler(valeur), depiler(), est_vide().

Implémentation 1 — avec une liste Python (fin de liste = sommet) :

```
1 class PileListe:
2     def __init__(self):
3         self._donnees = []
4
5     def empiler(self, valeur):
6         self._donnees.append(valeur)
7
8     def depiler(self):
9         return self._donnees.pop()
10
11    def est_vide(self):
12        return len(self._donnees) == 0
```

Implémentation 2 — avec une liste chaînée (debut = sommet) :

```
1 class Maillon:
2     def __init__(self, valeur, suivant=None):
3         self.valeur = valeur
4         self.suivant = suivant
5
6 class PileChaînee:
7     def __init__(self):
8         self._sommet = None
9
10    def empiler(self, valeur):
11        self._sommet = Maillon(valeur, self._sommet)
12
13    def depiler(self):
14        v = self._sommet.valeur
15        self._sommet = self._sommet.suivant
16        return v
17
18    def est_vide(self):
19        return self._sommet is None
```

EXEMPLE Ces deux implementations offrent **exactement la même interface** (mêmes trois méthodes, même comportement observable), mais des structures de données internes totalement différentes (tableau dynamique contre liste chaînée).

⚠ ERREUR FRÉQUENTE

Confondre interface et implementation lors d'un test. Un test doit porter sur le comportement observable via l'interface (par exemple : après avoir empiler 1, 2, 3, dépiler doit renvoyer 3), pas sur les détails internes (comme la structure exacte de `_donnees`). Cela permet au même jeu de tests de valider plusieurs implementations.

6.2 Définir et utiliser une classe en Python

DÉFINITION 1

Une **classe** est un modèle qui décrit la structure (**attributs**, l'état) et le comportement (**méthodes**) des objets qui en sont des **instances**. En Python, une classe se définit avec le mot-cle `class`, et le constructeur `__init__` initialise les attributs d'une nouvelle instance.

Exemple.

```
1 class Point:
2     def __init__(self, x, y):
3         self.x = x
4         self.y = y
5
6     def distance_origine(self):
7         return (self.x ** 2 + self.y ** 2) ** 0.5
8
9     def translater(self, dx, dy):
10        self.x = self.x + dx
11        self.y = self.y + dy
```

Utilisation :

```
1 p = Point(3, 4)
2 print(p.x, p.y)           # acces aux attributs
3 print(p.distance_origine()) # appel de methode : 5.0
4 p.translater(1, 1)
5 print(p.x, p.y)           # 4 5
```

- ◆ **PROPRIÉTÉ Vocabulaire**
 - **Attribut** : donnée associée à un objet (état), accédée par objet.attribut.
 - **Méthode** : fonction associée à une classe, appelée par objet.methode(...). Le premier paramètre `self` désigne l'objet lui-même (fourni automatiquement lors de l'appel).
 - **Instance** : un objet particulier créé à partir d'une classe (par exemple `p` est une instance de `Point`).

ERREUR FRÉQUENTE

Oublier self dans la définition d'une méthode. Toute méthode d'instance doit avoir self comme premier paramètre (même si on ne l'appelle jamais explicitement lors de l'appel objet.methode()) : Python le fournit automatiquement.

6.3 Piles, files, et choix de structure adaptée

DÉFINITION 1

Une **pile** (Stack) fonctionne selon le principe **LIFO** (*Last In, First Out*) : le dernier élément ajouté est le premier retire. Interface : empiler, depiler, est_vide.

Une **file** (Queue) fonctionne selon le principe **FIFO** (*First In, First Out*) : le premier élément ajouté est le premier retire. Interface : enfiler, defiler, est_vide.

Exemple. Implementation d'une file avec une liste Python (collections.deque est préférable en pratique pour l'efficacité, mais une liste suffit pour illustrer le principe) :

```

1 class File:
2     def __init__(self):
3         self._donnees = []
4
5     def enfiler(self, valeur):
6         self._donnees.append(valeur)
7
8     def defiler(self):
9         return self._donnees.pop(0)
10
11    def est_vide(self):
12        return len(self._donnees) == 0

```

Ici, "A" est le premier entre et le premier sorti : comportement FIFO, à l'opposé du comportement LIFO d'une pile.

◆ **PROPRIÉTÉ Choisir une structure adaptée** Le choix dépend des **opérations dominantes** de la situation à modéliser :

- ▶ **Pile** : annulation d'actions (undo), évaluation d'expressions, parcours en profondeur.
- ▶ **File** : gestion de tâches dans l'ordre d'arrivée, parcours en largeur, files d'attente.
- ▶ **Liste** : accès par position (indice), parcours séquentiel.
- ▶ **Dictionnaire** : accès direct par cle (recherche rapide en moyenne, indépendante de la taille), sans notion d'ordre de position.

ERREUR FRÉQUENTE

Confondre .pop() et .pop(0). Pour une liste Python, .pop() retire et renvoie le **dernier** élément (comportement LIFO, utile pour une pile), tandis que .pop(0) retire et renvoie le **premier** élément (comportement FIFO, utile pour une file, mais coûteux car il faut décaler tous les éléments restants).

6.4 Arbres binaires

DÉFINITION 1

Un **arbre binaire** est soit **vide**, soit constitué d'une **racine** (une valeur) et de deux **sous-arbres** (gauche et droit), eux-mêmes des arbres binaires. Une **feuille** est un noeud sans sous-arbre (les deux sous-arbres sont vides).

- ◆ **PROPRIÉTÉ Mesures usuelles**
- La **taille** d'un arbre est son nombre total de noeuds.
 - La **hauteur** d'un arbre est la longueur du plus long chemin de la racine à une feuille (hauteur 0 pour un arbre réduit à une seule feuille, hauteur -1 ou non définie pour un arbre vide selon la convention retenue).
 - Le nombre de **feuilles** est le nombre de noeuds sans sous-arbre.

Exemple. Implementation d'un arbre binaire et calcul de ses mesures :

```

1 class Arbre:
2     def __init__(self, valeur, gauche=None, droit=None):
3         self.valeur = valeur
4         self.gauche = gauche
5         self.droit = droit
6
7     def taille(a):
8         if a is None:
9             return 0
10        return 1 + taille(a.gauche) + taille(a.droit)
11
12    def hauteur(a):
13        if a is None:
14            return -1
15        return 1 + max(hauteur(a.gauche), hauteur(a.droit))
16
17    def nb_feuilles(a):
18        if a is None:
19            return 0
20        if a.gauche is None and a.droit is None:
21            return 1
22        return nb_feuilles(a.gauche) + nb_feuilles(a.droit)

```

Pour l'arbre $(1, (2, (4, \emptyset, \emptyset), \emptyset), (3, \emptyset, \emptyset))$ (racine 1, fils gauche 2 ayant lui-même un fils gauche 4, fils droit 3) :

taille = 4, hauteur = 2 (chemin $1 \rightarrow 2 \rightarrow 4$), nombre de feuilles = 2 (les noeuds 4 et 3).

⚠ ERREUR FRÉQUENTE

Oublier le cas de base dans une fonction recursive sur un arbre. Toute fonction recursive sur un arbre binaire doit traiter explicitement le cas `a is None` (arbre vide) en premier, sous peine d'erreur (`AttributeError`) lorsque la recursion atteint un sous-arbre vide.

6.5 Graphes : modélisation et représentations

DÉFINITION 1

Un **graphe** est constitué de **sommets** (ou noeuds) reliés par des **arêtes**. Un graphe est **orienté** si ses arêtes ont un sens (representees par des fleches), **non orienté** sinon.

◆ **PROPRIÉTÉ Représentation par matrice d'adjacence** Pour un graphe a n sommets numerotes de 0 a $n - 1$, la **matrice d'adjacence** est un tableau $n \times n$ ou la case (i, j) vaut 1 (ou True) s'il existe une arête de i vers j , 0 sinon.

Exemple. Graphe orienté a 3 sommets : $0 \rightarrow 1, 0 \rightarrow 2, 1 \rightarrow 2$.

```
1 matrice = [
2     [0, 1, 1],
3     [0, 0, 1],
4     [0, 0, 0],
5 ]
```

◆ **PROPRIÉTÉ Représentation par listes de successeurs** Pour chaque sommet, on stocke la **liste des sommets accessibles directement** (ses successeurs), par exemple sous forme de dictionnaire.

Exemple. Le même graphe ($0 \rightarrow 1, 0 \rightarrow 2, 1 \rightarrow 2$) représente par listes de successeurs :

```
1 successeurs = {
2     0: [1, 2],
3     1: [2],
4     2: [],
5 }
```

Conversion. Pour passer de la matrice aux listes de successeurs, on parcourt chaque ligne i et on retient les indices j ou $\text{matrice}[i][j] == 1$:

```
1 def matrice_vers_listes(matrice):
2     n = len(matrice)
3     return {i: [j for j in range(n) if matrice[i][j] == 1] for i in range(n)}
```

◆ **PROPRIÉTÉ Comparaison** La **matrice d'adjacence** permet de tester en temps constant l'existence d'une arête (i, j) , mais occupe une memoire en n^2 même si le graphe est peu dense (**creux**). Les **listes de successeurs** occupent une memoire proportionnelle au nombre d'arêtes reellement presentes, plus adaptees aux graphes creux, mais tester l'existence d'une arête précise peut necessiter de parcourir une liste.

⚠ ERREUR FRÉQUENTE

Oublier qu'un sommet sans successeur doit quand même apparaitre. Dans la représentation par listes de successeurs, un sommet isole ou puits (sans arête sortante) doit être present dans

le dictionnaire avec une liste **vide**, pas absent du dictionnaire : sinon, un parcours du graphe risque d'ignorer ce sommet.

6.6 Interface et implémentation d'une structure de données

DÉFINITION 1

L'**interface** d'une structure de données est l'ensemble des **opérations** qu'elle propose (ce que l'on peut faire), **sans préciser comment** elles sont réalisées. Une **implémentation** est une façon concrète de coder ces opérations (avec quelles données internes, quel algorithme).

EXEMPLE Interface d'une pile (Stack) : empiler(valeur), depiler(), est_vide().

Implémentation 1 — avec une liste Python (fin de liste = sommet) :

```

1 class PileListe:
2     def __init__(self):
3         self._donnees = []
4
5     def empiler(self, valeur):
6         self._donnees.append(valeur)
7
8     def depiler(self):
9         return self._donnees.pop()
10
11    def est_vide(self):
12        return len(self._donnees) == 0

```

Implémentation 2 — avec une liste chaînée (debut = sommet) :

```

1 class Maillon:
2     def __init__(self, valeur, suivant=None):
3         self.valeur = valeur
4         self.suivant = suivant
5
6 class PileChaînee:
7     def __init__(self):
8         self._sommets = None
9
10    def empiler(self, valeur):
11        self._sommets = Maillon(valeur, self._sommets)
12
13    def depiler(self):
14        v = self._sommets.valeur
15        self._sommets = self._sommets.suivant
16        return v
17
18    def est_vide(self):
19        return self._sommets is None

```

EXEMPLE Ces deux implémentations offrent **exactement la même interface** (mêmes trois méthodes, même comportement observable), mais des structures de données internes totalement différentes (tableau dynamique contre liste chaînée).

ERREUR FRÉQUENTE

Confondre interface et implémentation lors d'un test. Un test doit porter sur le comportement observable via l'interface (par exemple : après avoir empiler 1, 2, 3, dépiler doit renvoyer 3), pas sur les détails internes (comme la structure exacte de `_donnees`). Cela permet au même jeu de tests de valider plusieurs implémentations.

6.7 Définir et utiliser une classe en Python

DÉFINITION 1

Une **classe** est un modèle qui décrit la structure (**attributs**, l'état) et le comportement (**méthodes**) des objets qui en sont des **instances**. En Python, une classe se définit avec le mot-cle `class`, et le constructeur `__init__` initialise les attributs d'une nouvelle instance.

Exemple.

```
1 class Point:
2     def __init__(self, x, y):
3         self.x = x
4         self.y = y
5
6     def distance_origine(self):
7         return (self.x ** 2 + self.y ** 2) ** 0.5
8
9     def translater(self, dx, dy):
10        self.x = self.x + dx
11        self.y = self.y + dy
```

Utilisation :

```
1 p = Point(3, 4)
2 print(p.x, p.y)           # acces aux attributs
3 print(p.distance_origine()) # appel de methode : 5.0
4 p.translater(1, 1)
5 print(p.x, p.y)           # 4 5
```

◆ **PROPRIÉTÉ Vocabulaire** • **Attribut** : donnée associée à un objet (état), accédée par objet.attribut.

• **Méthode** : fonction associée à une classe, appelée par objet.methode(...). Le premier paramètre `self` désigne l'objet lui-même (fourni automatiquement lors de l'appel).

• **Instance** : un objet particulier créé à partir d'une classe (par exemple `p` est une instance de `Point`).

ERREUR FRÉQUENTE

Oublier `self` dans la définition d'une méthode. Toute méthode d'instance doit avoir `self` comme premier paramètre (même si on ne l'appelle jamais explicitement lors de l'appel `objet.methode()`) : Python le fournit automatiquement.

6.8 Piles, files, et choix de structure adaptée

DÉFINITION 1

Une **pile** (Stack) fonctionne selon le principe **LIFO** (*Last In, First Out*) : le dernier élément ajouté est le premier retire. Interface : empiler, depiler, est_vide.

Une **file** (Queue) fonctionne selon le principe **FIFO** (*First In, First Out*) : le premier élément ajouté est le premier retire. Interface : enfiler, defiler, est_vide.

Exemple. Implementation d'une file avec une liste Python (`collections.deque` est préférable en pratique pour l'efficacité, mais une liste suffit pour illustrer le principe) :

```
1 class File:
2     def __init__(self):
3         self._donnees = []
4
5     def enfiler(self, valeur):
6         self._donnees.append(valeur)
7
8     def defiler(self):
9         return self._donnees.pop(0)
10
11    def est_vide(self):
12        return len(self._donnees) == 0
```

Ici, "A" est le premier entre et le premier sorti : comportement FIFO, à l'opposé du comportement LIFO d'une pile.

◆ **PROPRIÉTÉ Choisir une structure adaptée** Le choix dépend des **opérations dominantes** de la situation à modéliser :

- ▶ **Pile** : annulation d'actions (undo), évaluation d'expressions, parcours en profondeur.
- ▶ **File** : gestion de tâches dans l'ordre d'arrivée, parcours en largeur, files d'attente.
- ▶ **Liste** : accès par position (indice), parcours séquentiel.
- ▶ **Dictionnaire** : accès direct par cle (recherche rapide en moyenne, indépendante de la taille), sans notion d'ordre de position.

⚠ ERREUR FRÉQUENTE

Confondre `.pop()` et `.pop(0)`. Pour une liste Python, `.pop()` retire et renvoie le **dernier** élément (comportement LIFO, utile pour une pile), tandis que `.pop(0)` retire et renvoie le **premier** élément (comportement FIFO, utile pour une file, mais coûteux car il faut décaler tous les éléments restants).

6.9 Arbres binaires

DÉFINITION 1

Un **arbre binaire** est soit **vide**, soit constitué d'une **racine** (une valeur) et de deux **sous-arbres** (gauche et droit), eux-mêmes des arbres binaires. Une **feuille** est un nœud sans sous-arbre (les deux sous-arbres sont vides).

- ◆ **PROPRIÉTÉ Mesures usuelles** • La **taille** d'un arbre est son nombre total de noeuds.
- La **hauteur** d'un arbre est la longueur du plus long chemin de la racine à une feuille (hauteur 0 pour un arbre réduit à une seule feuille, hauteur -1 ou non définie pour un arbre vide selon la convention retenue).
- Le nombre de **feuilles** est le nombre de noeuds sans sous-arbre.

Exemple. Implementation d'un arbre binaire et calcul de ses mesures :

```

1 class Arbre:
2     def __init__(self, valeur, gauche=None, droit=None):
3         self.valeur = valeur
4         self.gauche = gauche
5         self.droit = droit
6
7     def taille(a):
8         if a is None:
9             return 0
10        return 1 + taille(a.gauche) + taille(a.droit)
11
12    def hauteur(a):
13        if a is None:
14            return -1
15        return 1 + max(hauteur(a.gauche), hauteur(a.droit))
16
17    def nb_feuilles(a):
18        if a is None:
19            return 0
20        if a.gauche is None and a.droit is None:
21            return 1
22        return nb_feuilles(a.gauche) + nb_feuilles(a.droit)

```

Pour l'arbre $(1, (2, (4, \emptyset, \emptyset), \emptyset), (3, \emptyset, \emptyset))$ (racine 1, fils gauche 2 ayant lui-même un fils gauche 4, fils droit 3) :

taille = 4, hauteur = 2 (chemin $1 \rightarrow 2 \rightarrow 4$), nombre de feuilles = 2 (les noeuds 4 et 3).

⚠ ERREUR FRÉQUENTE

Oublier le cas de base dans une fonction recursive sur un arbre. Toute fonction recursive sur un arbre binaire doit traiter explicitement le cas $a \text{ is None}$ (arbre vide) en premier, sous peine d'erreur (AttributeError) lorsque la recursion atteint un sous-arbre vide.

6.10 Graphes : modélisation et représentations

DÉFINITION 1

Un **graphe** est constitué de **sommets** (ou noeuds) reliés par des **arêtes**. Un graphe est **orienté** si ses arêtes ont un sens (representées par des fleches), **non orienté** sinon.

- ◆ **PROPRIÉTÉ Représentation par matrice d'adjacence** Pour un graphe à n sommets numérotés de 0 à $n - 1$, la **matrice d'adjacence** est un tableau $n \times n$ où la case (i, j) vaut 1 (ou True) s'il existe une arête de i vers j , 0 sinon.

Exemple. Graphe orienté à 3 sommets : $0 \rightarrow 1, 0 \rightarrow 2, 1 \rightarrow 2$.

```
1 matrice = [
2     [0, 1, 1],
3     [0, 0, 1],
4     [0, 0, 0],
5 ]
```

◆ **PROPRIÉTÉ Représentation par listes de successeurs** Pour chaque sommet, on stocke la **liste des sommets accessibles directement** (ses successeurs), par exemple sous forme de dictionnaire.

Exemple. Le même graphe ($0 \rightarrow 1, 0 \rightarrow 2, 1 \rightarrow 2$) représenté par listes de successeurs :

```
1 successeurs = {
2     0: [1, 2],
3     1: [2],
4     2: [],
5 }
```

Conversion. Pour passer de la matrice aux listes de successeurs, on parcourt chaque ligne i et on retient les indices j où $\text{matrice}[i][j] == 1$:

```
1 def matrice_vers_listes(matrice):
2     n = len(matrice)
3     return {i: [j for j in range(n) if matrice[i][j] == 1] for i in range(n)}
```

◆ **PROPRIÉTÉ Comparaison** La **matrice d'adjacence** permet de tester en temps constant l'existence d'une arête (i, j) , mais occupe une mémoire en n^2 même si le graphe est peu dense (**creux**). Les **listes de successeurs** occupent une mémoire proportionnelle au nombre d'arêtes réellement présentes, plus adaptées aux graphes creux, mais tester l'existence d'une arête précise peut nécessiter de parcourir une liste.

⚠ ERREUR FRÉQUENTE

Oublier qu'un sommet sans successeur doit quand même apparaître. Dans la représentation par listes de successeurs, un sommet isolé ou puits (sans arête sortante) doit être présent dans le dictionnaire avec une liste **vide**, pas absent du dictionnaire : sinon, un parcours du graphe risque d'ignorer ce sommet.

6.11 Interface et implémentation d'une structure de données

DÉFINITION 1

L'**interface** d'une structure de données est l'ensemble des **opérations** qu'elle propose (ce que l'on peut faire), **sans préciser comment** elles sont réalisées. Une **implémentation** est une façon concrète de coder ces opérations (avec quelles données internes, quel algorithme).

EXEMPLE Interface d'une pile (Stack) : `empiler(valeur)`, `depiler()`, `est_vide()`.

Implementation 1 — avec une liste Python (fin de liste = sommet) :

```

1 class PileListe:
2     def __init__(self):
3         self._donnees = []
4
5     def empiler(self, valeur):
6         self._donnees.append(valeur)
7
8     def depiler(self):
9         return self._donnees.pop()
10
11    def est_vide(self):
12        return len(self._donnees) == 0

```

Implementation 2 — avec une liste chaînée (debut = sommet) :

```

1 class Maillon:
2     def __init__(self, valeur, suivant=None):
3         self.valeur = valeur
4         self.suivant = suivant
5
6 class PileChaînee:
7     def __init__(self):
8         self._sommet = None
9
10    def empiler(self, valeur):
11        self._sommet = Maillon(valeur, self._sommet)
12
13    def depiler(self):
14        v = self._sommet.valeur
15        self._sommet = self._sommet.suivant
16        return v
17
18    def est_vide(self):
19        return self._sommet is None

```

EXEMPLE Ces deux implementations offrent **exactement la même interface** (mêmes trois méthodes, même comportement observable), mais des structures de données internes totalement différentes (tableau dynamique contre liste chaînée).

⚠ ERREUR FRÉQUENTE

Confondre interface et implémentation lors d'un test. Un test doit porter sur le comportement observable via l'interface (par exemple : après avoir empiler 1, 2, 3, dépiler doit renvoyer 3), pas sur les détails internes (comme la structure exacte de `_donnees`). Cela permet au même jeu de tests de valider plusieurs implémentations.

6.12 Définir et utiliser une classe en Python

DÉFINITION 1

Une **classe** est un modèle qui décrit la structure (**attributs**, l'état) et le comportement (**methodes**) des objets qui en sont des **instances**. En Python, une classe se définit avec le mot-cle `class`, et le constructeur `__init__` initialise les attributs d'une nouvelle instance.

Exemple.

```
1 class Point:
2     def __init__(self, x, y):
3         self.x = x
4         self.y = y
5
6     def distance_origine(self):
7         return (self.x ** 2 + self.y ** 2) ** 0.5
8
9     def translater(self, dx, dy):
10        self.x = self.x + dx
11        self.y = self.y + dy
```

Utilisation :

```
1 p = Point(3, 4)
2 print(p.x, p.y)           # acces aux attributs
3 print(p.distance_origine()) # appel de methode : 5.0
4 p.translater(1, 1)
5 print(p.x, p.y)           # 4 5
```

- ◆ **PROPRIÉTÉ Vocabulaire**
 - **Attribut** : donnée associée à un objet (état), accédée par objet.attribut.
 - **Méthode** : fonction associée à une classe, appelée par objet.methode(...). Le premier paramètre `self` désigne l'objet lui-même (fourni automatiquement lors de l'appel).
 - **Instance** : un objet particulier créé à partir d'une classe (par exemple `p` est une instance de `Point`).

⚠ ERREUR FRÉQUENTE

Oublier `self` dans la définition d'une méthode. Toute méthode d'instance doit avoir `self` comme premier paramètre (même si on ne l'appelle jamais explicitement lors de l'appel `objet.methode()`) : Python le fournit automatiquement.

6.13 Piles, files, et choix de structure adaptée

DÉFINITION 1

Une **pile** (Stack) fonctionne selon le principe **LIFO** (*Last In, First Out*) : le dernier élément ajouté est le premier retiré. Interface : empiler, dépiler, `est_vide`.

Une **file** (Queue) fonctionne selon le principe **FIFO** (*First In, First Out*) : le premier élément ajouté est le premier retiré. Interface : enfiler, défiler, `est_vide`.

Exemple. Implementation d'une file avec une liste Python (`collections.deque` est préférable en pratique pour l'efficacité, mais une liste suffit pour illustrer le principe) :

```

1 class File:
2     def __init__(self):
3         self._donnees = []
4
5     def enfiler(self, valeur):
6         self._donnees.append(valeur)
7
8     def defiler(self):
9         return self._donnees.pop(0)
10
11    def est_vide(self):
12        return len(self._donnees) == 0

```

Ici, "A" est le premier entre et le premier sorti : comportement FIFO, à l'opposé du comportement LIFO d'une pile.

◆ **PROPRIÉTÉ Choisir une structure adaptée** Le choix dépend des **opérations dominantes** de la situation à modéliser :

- ▶ **Pile** : annulation d'actions (undo), évaluation d'expressions, parcours en profondeur.
- ▶ **File** : gestion de tâches dans l'ordre d'arrivée, parcours en largeur, files d'attente.
- ▶ **Liste** : accès par position (indice), parcours séquentiel.
- ▶ **Dictionnaire** : accès direct par clé (recherche rapide en moyenne, indépendante de la taille), sans notion d'ordre de position.

⚠ ERREUR FRÉQUENTE

Confondre `.pop()` et `.pop(0)`. Pour une liste Python, `.pop()` retire et renvoie le **dernier** élément (comportement LIFO, utile pour une pile), tandis que `.pop(0)` retire et renvoie le **premier** élément (comportement FIFO, utile pour une file, mais coûteux car il faut décaler tous les éléments restants).

6.14 Arbres binaires

DÉFINITION 1

Un **arbre binaire** est soit **vide**, soit constitué d'une **racine** (une valeur) et de deux **sous-arbres** (gauche et droit), eux-mêmes des arbres binaires. Une **feuille** est un nœud sans sous-arbre (les deux sous-arbres sont vides).

- ◆ **PROPRIÉTÉ Mesures usuelles**
- La **taille** d'un arbre est son nombre total de nœuds.
 - La **hauteur** d'un arbre est la longueur du plus long chemin de la racine à une feuille (hauteur 0 pour un arbre réduit à une seule feuille, hauteur -1 ou non définie pour un arbre vide selon la convention retenue).
 - Le nombre de **feuilles** est le nombre de nœuds sans sous-arbre.

Exemple. Implementation d'un arbre binaire et calcul de ses mesures :

```

1 class Arbre:
2     def __init__(self, valeur, gauche=None, droit=None):
3         self.valeur = valeur
4         self.gauche = gauche
5         self.droit = droit
6
7     def taille(a):
8         if a is None:
9             return 0
10        return 1 + taille(a.gauche) + taille(a.droit)
11
12    def hauteur(a):
13        if a is None:
14            return -1
15        return 1 + max(hauteur(a.gauche), hauteur(a.droit))
16
17    def nb_feuilles(a):
18        if a is None:
19            return 0
20        if a.gauche is None and a.droit is None:
21            return 1
22        return nb_feuilles(a.gauche) + nb_feuilles(a.droit)

```

Pour l'arbre $(1, (2, (4, \emptyset, \emptyset), \emptyset), (3, \emptyset, \emptyset))$ (racine 1, fils gauche 2 ayant lui-même un fils gauche 4, fils droit 3) :

taille = 4, hauteur = 2 (chemin $1 \rightarrow 2 \rightarrow 4$), nombre de feuilles = 2 (les noeuds 4 et 3).

⚠ ERREUR FRÉQUENTE

Oublier le cas de base dans une fonction recursive sur un arbre. Toute fonction recursive sur un arbre binaire doit traiter explicitement le cas `a is None` (arbre vide) en premier, sous peine d'erreur (`AttributeError`) lorsque la recursion atteint un sous-arbre vide.

6.15 Graphes : modélisation et représentations

DÉFINITION 1

Un **graphe** est constitué de **sommets** (ou noeuds) reliés par des **arêtes**. Un graphe est **orienté** si ses arêtes ont un sens (representees par des fleches), **non orienté** sinon.

◆ **PROPRIÉTÉ Représentation par matrice d'adjacence** Pour un graphe a n sommets numerotes de 0 a $n - 1$, la **matrice d'adjacence** est un tableau $n \times n$ ou la case (i, j) vaut 1 (ou True) s'il existe une arête de i vers j , 0 sinon.

Exemple. Graphe orienté a 3 sommets : $0 \rightarrow 1, 0 \rightarrow 2, 1 \rightarrow 2$.

```

1 matrice = [
2     [0, 1, 1],
3     [0, 0, 1],
4     [0, 0, 0],
5 ]

```

◆ **PROPRIÉTÉ Représentation par listes de successeurs** Pour chaque sommet, on stocke la **liste des sommets accessibles directement** (ses successeurs), par exemple sous forme de dictionnaire.

Exemple. Le même graphe ($0 \rightarrow 1, 0 \rightarrow 2, 1 \rightarrow 2$) représente par listes de successeurs :

```
1 successeurs = {
2     0: [1, 2],
3     1: [2],
4     2: [],
5 }
```

Conversion. Pour passer de la matrice aux listes de successeurs, on parcourt chaque ligne i et on retient les indices j où $\text{matrice}[i][j] == 1$:

```
1 def matrice_vers_listes(matrice):
2     n = len(matrice)
3     return {i: [j for j in range(n) if matrice[i][j] == 1] for i in range(n)}
```

◆ **PROPRIÉTÉ Comparaison** La **matrice d'adjacence** permet de tester en temps constant l'existence d'une arête (i, j) , mais occupe une mémoire en n^2 même si le graphe est peu dense (**creux**). Les **listes de successeurs** occupent une mémoire proportionnelle au nombre d'arêtes réellement présentes, plus adaptées aux graphes creux, mais tester l'existence d'une arête précise peut nécessiter de parcourir une liste.

⚠ ERREUR FRÉQUENTE

Oublier qu'un sommet sans successeur doit quand même apparaître. Dans la représentation par listes de successeurs, un sommet isolé ou puits (sans arête sortante) doit être présent dans le dictionnaire avec une liste **vide**, pas absent du dictionnaire : sinon, un parcours du graphe risque d'ignorer ce sommet.

6.16 Interface et implémentation d'une structure de données

DÉFINITION 1

L'**interface** d'une structure de données est l'ensemble des **opérations** qu'elle propose (ce que l'on peut faire), **sans préciser comment** elles sont réalisées. Une **implémentation** est une façon concrète de coder ces opérations (avec quelles données internes, quel algorithme).

EXEMPLE Interface d'une pile (Stack) : `empiler(valeur)`, `depiler()`, `est_vide()`.

Implémentation 1 — avec une liste Python (fin de liste = sommet) :

```
1 class PileListe:
2     def __init__(self):
3         self._donnees = []
4
5     def empiler(self, valeur):
6         self._donnees.append(valeur)
7
```

```

8     def depiler(self):
9         return self._donnees.pop()
10
11    def est_vide(self):
12        return len(self._donnees) == 0

```

Implementation 2 — avec une liste chaînée (debut = sommet) :

```

1 class Maillon:
2     def __init__(self, valeur, suivant=None):
3         self.valeur = valeur
4         self.suivant = suivant
5
6 class PileChaine:
7     def __init__(self):
8         self._sommet = None
9
10    def empiler(self, valeur):
11        self._sommet = Maillon(valeur, self._sommet)
12
13    def depiler(self):
14        v = self._sommet.valeur
15        self._sommet = self._sommet.suivant
16        return v
17
18    def est_vide(self):
19        return self._sommet is None

```

EXEMPLE Ces deux implementations offrent **exactement la même interface** (mêmes trois méthodes, même comportement observable), mais des structures de données internes totalement différentes (tableau dynamique contre liste chaînée).

⚠ ERREUR FRÉQUENTE

Confondre interface et implementation lors d'un test. Un test doit porter sur le comportement observable via l'interface (par exemple : après avoir empiler 1, 2, 3, depiler doit renvoyer 3), pas sur les détails internes (comme la structure exacte de `_donnees`). Cela permet au même jeu de tests de valider plusieurs implementations.

6.17 Définir et utiliser une classe en Python

DÉFINITION 1

Une **classe** est un modèle qui décrit la structure (**attributs**, l'état) et le comportement (**méthodes**) des objets qui en sont des **instances**. En Python, une classe se définit avec le mot-cle `class`, et le constructeur `__init__` initialise les attributs d'une nouvelle instance.

Exemple.

```

1 class Point:
2     def __init__(self, x, y):
3         self.x = x
4         self.y = y
5
6     def distance_origine(self):

```

```

7         return (self.x ** 2 + self.y ** 2) ** 0.5
8
9     def translater(self, dx, dy):
10         self.x = self.x + dx
11         self.y = self.y + dy

```

Utilisation :

```

1 p = Point(3, 4)
2 print(p.x, p.y)           # acces aux attributs
3 print(p.distance_origine()) # appel de methode : 5.0
4 p.translater(1, 1)
5 print(p.x, p.y)           # 4 5

```

- ◆ **PROPRIÉTÉ Vocabulaire**
- **Attribut** : donnée associée à un objet (état), accédée par objet.attribut.
 - **Méthode** : fonction associée à une classe, appelée par objet.methode(...). Le premier paramètre `self` désigne l'objet lui-même (fourni automatiquement lors de l'appel).
 - **Instance** : un objet particulier créé à partir d'une classe (par exemple `p` est une instance de `Point`).

⚠ ERREUR FRÉQUENTE

Oublier `self` dans la définition d'une méthode. Toute méthode d'instance doit avoir `self` comme premier paramètre (même si on ne l'appelle jamais explicitement lors de l'appel `objet.methode()`) : Python le fournit automatiquement.

6.18 Piles, files, et choix de structure adaptée

DÉFINITION 1

Une **pile** (Stack) fonctionne selon le principe **LIFO** (*Last In, First Out*) : le dernier élément ajouté est le premier retiré. Interface : empiler, dépiler, `est_vide`.

Une **file** (Queue) fonctionne selon le principe **FIFO** (*First In, First Out*) : le premier élément ajouté est le premier retiré. Interface : enfiler, defiler, `est_vide`.

Exemple. Implémentation d'une file avec une liste Python (`collections.deque` est préférable en pratique pour l'efficacité, mais une liste suffit pour illustrer le principe) :

```

1 class File:
2     def __init__(self):
3         self._donnees = []
4
5     def enfiler(self, valeur):
6         self._donnees.append(valeur)
7
8     def defiler(self):
9         return self._donnees.pop(0)
10
11    def est_vide(self):
12        return len(self._donnees) == 0

```

Ici, "A" est le premier entre et le premier sorti : comportement FIFO, à l'opposé du comportement LIFO d'une pile.

◆ **PROPRIÉTÉ Choisir une structure adaptée** Le choix dépend des **opérations dominantes** de la situation à modéliser :

- ▶ **Pile** : annulation d'actions (undo), évaluation d'expressions, parcours en profondeur.
- ▶ **File** : gestion de tâches dans l'ordre d'arrivée, parcours en largeur, files d'attente.
- ▶ **Liste** : accès par position (indice), parcours séquentiel.
- ▶ **Dictionnaire** : accès direct par cle (recherche rapide en moyenne, indépendante de la taille), sans notion d'ordre de position.

⚠ ERREUR FRÉQUENTE

Confondre .pop() et .pop(0). Pour une liste Python, .pop() retire et renvoie le **dernier** élément (comportement LIFO, utile pour une pile), tandis que .pop(0) retire et renvoie le **premier** élément (comportement FIFO, utile pour une file, mais coûteux car il faut décaler tous les éléments restants).

6.19 Arbres binaires

DÉFINITION 1

Un **arbre binaire** est soit **vide**, soit constitué d'une **racine** (une valeur) et de deux **sous-arbres** (gauche et droit), eux-mêmes des arbres binaires. Une **feuille** est un nœud sans sous-arbre (les deux sous-arbres sont vides).

- ◆ **PROPRIÉTÉ Mesures usuelles**
- La **taille** d'un arbre est son nombre total de nœuds.
 - La **hauteur** d'un arbre est la longueur du plus long chemin de la racine à une feuille (hauteur 0 pour un arbre réduit à une seule feuille, hauteur -1 ou non définie pour un arbre vide selon la convention retenue).
 - Le nombre de **feuilles** est le nombre de nœuds sans sous-arbre.

Exemple. Implementation d'un arbre binaire et calcul de ses mesures :

```

1 class Arbre:
2     def __init__(self, valeur, gauche=None, droit=None):
3         self.valeur = valeur
4         self.gauche = gauche
5         self.droit = droit
6
7     def taille(a):
8         if a is None:
9             return 0
10        return 1 + taille(a.gauche) + taille(a.droit)
11
12    def hauteur(a):
13        if a is None:
14            return -1
15        return 1 + max(hauteur(a.gauche), hauteur(a.droit))
16

```

```

17 def nb_feuilles(a):
18     if a is None:
19         return 0
20     if a.gauche is None and a.droit is None:
21         return 1
22     return nb_feuilles(a.gauche) + nb_feuilles(a.droit)

```

Pour l'arbre $(1, (2, (4, \emptyset, \emptyset), \emptyset), (3, \emptyset, \emptyset))$ (racine 1, fils gauche 2 ayant lui-même un fils gauche 4, fils droit 3) :

taille = 4, hauteur = 2 (chemin $1 \rightarrow 2 \rightarrow 4$), nombre de feuilles = 2 (les noeuds 4 et 3).

⚠ ERREUR FRÉQUENTE

Oublier le cas de base dans une fonction récursive sur un arbre. Toute fonction récursive sur un arbre binaire doit traiter explicitement le cas `a is None` (arbre vide) en premier, sous peine d'erreur (`AttributeError`) lorsque la récursion atteint un sous-arbre vide.

Méthodes

► Méthode directe de travail sur le sujet.

Exercice 1 ♦♦

On veut vérifier si une expression comportant des parenthèses `()`, crochets `[]` et accolades `{}` est **bien parenthesée** (chaque symbole ouvrant a un symbole fermant correspondant, dans le bon ordre).

1. Expliquer pourquoi une **pile** est une structure adaptée à ce problème (justifier par les opérations dominantes).
2. Écrire une fonction `est_equilibre(expr)` qui utilise une pile (interface `empiler`, `depiler`, `est_vide`) pour résoudre ce problème.
3. Tester votre fonction sur `"([{}])"` (bien parenthesée) et `"([)]"` (mal parenthesée).

Exercice 2 ♦

Écrire une classe `Compte` représentant un compte bancaire simplifié, avec :

- un constructeur prenant un titulaire et un solde initial (par défaut 0) ;
- une méthode `deposer(montant)` qui augmente le solde ;
- une méthode `retirer(montant)` qui diminue le solde, et lève une erreur (`ValueError`) si le montant demandé dépasse le solde disponible.

Tester votre classe avec un compte initial de 100, un dépôt de 50, puis un retrait de 30.

Exercice 3 ♦♦

On construit un arbre binaire "en peigne" : la racine vaut 1, son fils gauche vaut 2 et a lui-même un fils gauche 3, qui a un fils gauche 4, qui a un fils gauche 5 (aucun fils droit nulle part).

1. En utilisant la classe `Arbre` du cours et les fonctions `taille` et `hauteur`, construire cet arbre en Python.
2. Sans exécuter de code, prédire la taille et la hauteur de cet arbre. Justifier.
3. Vérifier votre prédiction en appelant `taille` et `hauteur` sur l'arbre construit.

Exercice 4 ♦♦

On donne le graphe orienté à 4 sommets $(0, 1, 2, 3)$ représenté par la matrice d'adjacence :

```
1 matrice = [
2     [0, 1, 0, 1],
3     [0, 0, 1, 0],
4     [0, 0, 0, 0],
5     [0, 0, 1, 0],
6 ]
```

1. Lister toutes les arêtes de ce graphe (sous la forme $i \rightarrow j$).
2. Le sommet 2 a-t-il des successeurs ? Interpréter.
3. En utilisant la fonction `matrice_vers_listes` du cours, donner la représentation par listes de successeurs de ce graphe.

Exercice 5

On veut vérifier si une expression comportant des parenthèses `()`, crochets `[]` et accolades `{}` est **bien parenthesée** (chaque symbole ouvrant a un symbole fermant correspondant, dans le bon ordre).

1. Expliquer pourquoi une **pile** est une structure adaptée à ce problème (justifier par les opérations dominantes).
2. Écrire une fonction `est_equilibre(expr)` qui utilise une pile (interface `empiler`, `depiler`, `est_vide`) pour résoudre ce problème.
3. Tester votre fonction sur `"([{}])"` (bien parenthesée) et `"([)]"` (mal parenthesée).

Exercice 6

Écrire une classe `Compte` représentant un compte bancaire simplifié, avec :

- ▶ un constructeur prenant un titulaire et un solde initial (par défaut 0) ;
- ▶ une méthode `deposer(montant)` qui augmente le solde ;
- ▶ une méthode `retirer(montant)` qui diminue le solde, et lève une erreur (`ValueError`) si le montant demande dépasse le solde disponible.

Tester votre classe avec un compte initial de 100, un dépôt de 50, puis un retrait de 30.

Exercice 7

On construit un arbre binaire "en peigne" : la racine vaut 1, son fils gauche vaut 2 et a lui-même un fils gauche 3, qui a un fils gauche 4, qui a un fils gauche 5 (aucun fils droit nulle part).

1. En utilisant la classe `Arbre` du cours et les fonctions `taille` et `hauteur`, construire cet arbre en Python.
2. Sans exécuter de code, prédire la taille et la hauteur de cet arbre. Justifier.
3. Vérifier votre prédiction en appelant `taille` et `hauteur` sur l'arbre construit.

Exercice 8

On donne le graphe orienté à 4 sommets (0, 1, 2, 3) représenté par la matrice d'adjacence :

```
1 matrice = [
2     [0, 1, 0, 1],
3     [0, 0, 1, 0],
4     [0, 0, 0, 0],
5     [0, 0, 1, 0],
6 ]
```

1. Lister toutes les arêtes de ce graphe (sous la forme $i \rightarrow j$).
2. Le sommet 2 a-t-il des successeurs ? Interpréter.

- En utilisant la fonction `matrice_vers_listes` du cours, donner la représentation par listes de successeurs de ce graphe.

Exercice 9

On veut vérifier si une expression comportant des parenthèses `()`, crochets `[]` et accolades `{}` est **bien parenthesee** (chaque symbole ouvrant a un symbole fermant correspondant, dans le bon ordre).

- Expliquer pourquoi une **pile** est une structure adaptée à ce problème (justifier par les opérations dominantes).
- Écrire une fonction `est_equilibre(expr)` qui utilise une pile (interface `empiler`, `depiler`, `est_vide`) pour résoudre ce problème.
- Tester votre fonction sur `"([{}])"` (bien parenthesee) et `"([)]"` (mal parenthesee).

Exercice 10

Écrire une classe `Compte` représentant un compte bancaire simplifié, avec :

- un constructeur prenant un titulaire et un solde initial (par défaut 0) ;
- une méthode `deposer(montant)` qui augmente le solde ;
- une méthode `retirer(montant)` qui diminue le solde, et lève une erreur (`ValueError`) si le montant demande dépasse le solde disponible.

Tester votre classe avec un compte initial de 100, un dépôt de 50, puis un retrait de 30.

Exercice 11

On construit un arbre binaire "en peigne" : la racine vaut 1, son fils gauche vaut 2 et a lui-même un fils gauche 3, qui a un fils gauche 4, qui a un fils gauche 5 (aucun fils droit nulle part).

- En utilisant la classe `Arbre` du cours et les fonctions `taille` et `hauteur`, construire cet arbre en Python.
- Sans exécuter de code, prédire la taille et la hauteur de cet arbre. Justifier.
- Vérifier votre prédiction en appelant `taille` et `hauteur` sur l'arbre construit.

Exercice 12

On donne le graphe orienté à 4 sommets $(0, 1, 2, 3)$ représenté par la matrice d'adjacence :

```
1 matrice = [
2     [0, 1, 0, 1],
3     [0, 0, 1, 0],
4     [0, 0, 0, 0],
5     [0, 0, 1, 0],
6 ]
```

- Lister toutes les arêtes de ce graphe (sous la forme $i \rightarrow j$).
- Le sommet 2 a-t-il des successeurs ? Interpréter.
- En utilisant la fonction `matrice_vers_listes` du cours, donner la représentation par listes de successeurs de ce graphe.

Exercice 13

On veut vérifier si une expression comportant des parenthèses `()`, crochets `[]` et accolades `{}` est **bien parenthesee** (chaque symbole ouvrant a un symbole fermant correspondant, dans le bon ordre).

- Expliquer pourquoi une **pile** est une structure adaptée à ce problème (justifier par les opérations dominantes).

- Écrire une fonction `est_equilibre(expr)` qui utilise une pile (interface `empiler`, `depiler`, `est_vide`) pour résoudre ce problème.
- Tester votre fonction sur `"([{}])"` (bien parenthésée) et `"([)]"` (mal parenthésée).

Exercice 14

Écrire une classe `Compte` représentant un compte bancaire simplifié, avec :

- un constructeur prenant un titulaire et un solde initial (par défaut 0) ;
- une méthode `deposer(montant)` qui augmente le solde ;
- une méthode `retirer(montant)` qui diminue le solde, et lève une erreur (`ValueError`) si le montant demandé dépasse le solde disponible.

Tester votre classe avec un compte initial de 100, un dépôt de 50, puis un retrait de 30.

Exercice 15

On construit un arbre binaire "en peigne" : la racine vaut 1, son fils gauche vaut 2 et a lui-même un fils gauche 3, qui a un fils gauche 4, qui a un fils gauche 5 (aucun fils droit nulle part).

- En utilisant la classe `Arbre` du cours et les fonctions `taille` et `hauteur`, construire cet arbre en Python.
- Sans exécuter de code, prédire la taille et la hauteur de cet arbre. Justifier.
- Vérifier votre prédiction en appelant `taille` et `hauteur` sur l'arbre construit.

Exercice 16

On donne le graphe orienté à 4 sommets (0, 1, 2, 3) représenté par la matrice d'adjacence :

```
1 matrice = [
2     [0, 1, 0, 1],
3     [0, 0, 1, 0],
4     [0, 0, 0, 0],
5     [0, 0, 1, 0],
6 ]
```

- Lister toutes les arêtes de ce graphe (sous la forme $i \rightarrow j$).
- Le sommet 2 a-t-il des successeurs ? Interpréter.
- En utilisant la fonction `matrice_vers_listes` du cours, donner la représentation par listes de successeurs de ce graphe.

Exercice 17

On veut vérifier si une expression comportant des parenthèses `()`, crochets `[]` et accolades `{}` est **bien parenthésée** (chaque symbole ouvrant a un symbole fermant correspondant, dans le bon ordre).

- Expliquer pourquoi une **pile** est une structure adaptée à ce problème (justifier par les opérations dominantes).
- Écrire une fonction `est_equilibre(expr)` qui utilise une pile (interface `empiler`, `depiler`, `est_vide`) pour résoudre ce problème.
- Tester votre fonction sur `"([{}])"` (bien parenthésée) et `"([)]"` (mal parenthésée).

Exercice 18

Écrire une classe `Compte` représentant un compte bancaire simplifié, avec :

- un constructeur prenant un titulaire et un solde initial (par défaut 0) ;

- ▶ une méthode `deposer(montant)` qui augmente le solde ;
- ▶ une méthode `retirer(montant)` qui diminue le solde, et lève une erreur (`ValueError`) si le montant demande dépasse le solde disponible.

Tester votre classe avec un compte initial de 100, un dépôt de 50, puis un retrait de 30.

Exercice 19 ♦♦

On construit un arbre binaire "en peigne" : la racine vaut 1, son fils gauche vaut 2 et a lui-même un fils gauche 3, qui a un fils gauche 4, qui a un fils gauche 5 (aucun fils droit nulle part).

1. En utilisant la classe `Arbre` du cours et les fonctions `taille` et `hauteur`, construire cet arbre en Python.
2. Sans exécuter de code, prédire la taille et la hauteur de cet arbre. Justifier.
3. Vérifier votre prédiction en appelant `taille` et `hauteur` sur l'arbre construit.

Exercice 20 ♦♦

On donne le graphe orienté à 4 sommets (0,1,2,3) représenté par la matrice d'adjacence :

```
1 matrice = [
2     [0, 1, 0, 1],
3     [0, 0, 1, 0],
4     [0, 0, 0, 0],
5     [0, 0, 1, 0],
6 ]
```

1. Lister toutes les arêtes de ce graphe (sous la forme $i \rightarrow j$).
2. Le sommet 2 a-t-il des successeurs ? Interpréter.
3. En utilisant la fonction `matrice_vers_listes` du cours, donner la représentation par listes de successeurs de ce graphe.

Exercice 21 ♦♦

On veut vérifier si une expression comportant des parenthèses `()`, crochets `[]` et accolades `{}` est **bien parenthesée** (chaque symbole ouvrant a un symbole fermant correspondant, dans le bon ordre).

1. Expliquer pourquoi une **pile** est une structure adaptée à ce problème (justifier par les opérations dominantes).
2. Écrire une fonction `est_equilibre(expr)` qui utilise une pile (interface `empiler`, `depiler`, `est_vide`) pour résoudre ce problème.
3. Tester votre fonction sur `"([{}])"` (bien parenthesée) et `"([)]"` (mal parenthesée).

Exercice 22 ♦

Écrire une classe `Compte` représentant un compte bancaire simplifié, avec :

- ▶ un constructeur prenant un titulaire et un solde initial (par défaut 0) ;
- ▶ une méthode `deposer(montant)` qui augmente le solde ;
- ▶ une méthode `retirer(montant)` qui diminue le solde, et lève une erreur (`ValueError`) si le montant demande dépasse le solde disponible.

Tester votre classe avec un compte initial de 100, un dépôt de 50, puis un retrait de 30.

Exercice 23 ♦♦

On construit un arbre binaire "en peigne" : la racine vaut 1, son fils gauche vaut 2 et a lui-même un fils gauche 3, qui a un fils gauche 4, qui a un fils gauche 5 (aucun fils droit nulle part).

1. En utilisant la classe `Arbre` du cours et les fonctions `taille` et `hauteur`, construire cet arbre en Python.
2. Sans executer de code, predire la taille et la hauteur de cet arbre. Justifier.
3. Vérifier votre prediction en appelant `taille` et `hauteur` sur l'arbre construit.

Exercice 24

On donne le graphe orienté à 4 sommets (0, 1, 2, 3) représenté par la matrice d'adjacence :

```
1 matrice = [
2     [0, 1, 0, 1],
3     [0, 0, 1, 0],
4     [0, 0, 0, 0],
5     [0, 0, 1, 0],
6 ]
```

1. Lister toutes les arêtes de ce graphe (sous la forme $i \rightarrow j$).
2. Le sommet 2 a-t-il des successeurs ? Interpréter.
3. En utilisant la fonction `matrice_vers_listes` du cours, donner la représentation par listes de successeurs de ce graphe.

Exercice 25

On veut vérifier si une expression comportant des parenthèses `()`, crochets `[]` et accolades `{}` est **bien parenthesée** (chaque symbole ouvrant a un symbole fermant correspondant, dans le bon ordre).

1. Expliquer pourquoi une **pile** est une structure adaptée à ce problème (justifier par les opérations dominantes).
2. Écrire une fonction `est_equilibre(expr)` qui utilise une pile (interface `empiler`, `depiler`, `est_vide`) pour résoudre ce problème.
3. Tester votre fonction sur `"([{}])"` (bien parenthesée) et `"([)]"` (mal parenthesée).

Exercice 26

Écrire une classe `Compte` représentant un compte bancaire simplifié, avec :

- ▶ un constructeur prenant un titulaire et un solde initial (par défaut 0) ;
- ▶ une méthode `deposer(montant)` qui augmente le solde ;
- ▶ une méthode `retirer(montant)` qui diminue le solde, et lève une erreur (`ValueError`) si le montant demandé dépasse le solde disponible.

Tester votre classe avec un compte initial de 100, un dépôt de 50, puis un retrait de 30.

Exercice 27

On construit un arbre binaire "en peigne" : la racine vaut 1, son fils gauche vaut 2 et a lui-même un fils gauche 3, qui a un fils gauche 4, qui a un fils gauche 5 (aucun fils droit nulle part).

1. En utilisant la classe `Arbre` du cours et les fonctions `taille` et `hauteur`, construire cet arbre en Python.
2. Sans executer de code, predire la taille et la hauteur de cet arbre. Justifier.
3. Vérifier votre prediction en appelant `taille` et `hauteur` sur l'arbre construit.

Exercice 28

On donne le graphe orienté à 4 sommets (0, 1, 2, 3) représenté par la matrice d'adjacence :

```

1 matrice = [
2     [0, 1, 0, 1],
3     [0, 0, 1, 0],
4     [0, 0, 0, 0],
5     [0, 0, 1, 0],
6 ]

```

1. Lister toutes les arêtes de ce graphe (sous la forme $i \rightarrow j$).
2. Le sommet 2 a-t-il des successeurs ? Interpréter.
3. En utilisant la fonction `matrice_vers_listes` du cours, donner la représentation par listes de successeurs de ce graphe.

Exercice 29

On veut vérifier si une expression comportant des parenthèses `()`, crochets `[]` et accolades `{}` est **bien parenthesée** (chaque symbole ouvrant a un symbole fermant correspondant, dans le bon ordre).

1. Expliquer pourquoi une **pile** est une structure adaptée à ce problème (justifier par les opérations dominantes).
2. Écrire une fonction `est_equilibre(expr)` qui utilise une pile (interface `empiler`, `depiler`, `est_vide`) pour résoudre ce problème.
3. Tester votre fonction sur `"([{}])"` (bien parenthesée) et `"([)]"` (mal parenthesée).

Exercice 30

Écrire une classe `Compte` représentant un compte bancaire simplifié, avec :

- ▶ un constructeur prenant un titulaire et un solde initial (par défaut 0) ;
- ▶ une méthode `deposer(montant)` qui augmente le solde ;
- ▶ une méthode `retirer(montant)` qui diminue le solde, et lève une erreur (`ValueError`) si le montant demandé dépasse le solde disponible.

Tester votre classe avec un compte initial de 100, un dépôt de 50, puis un retrait de 30.

Exercice 31

On construit un arbre binaire "en peigne" : la racine vaut 1, son fils gauche vaut 2 et a lui-même un fils gauche 3, qui a un fils gauche 4, qui a un fils gauche 5 (aucun fils droit nulle part).

1. En utilisant la classe `Arbre` du cours et les fonctions `taille` et `hauteur`, construire cet arbre en Python.
2. Sans exécuter de code, prédire la taille et la hauteur de cet arbre. Justifier.
3. Vérifier votre prédiction en appelant `taille` et `hauteur` sur l'arbre construit.

Exercice 32

On donne le graphe orienté à 4 sommets $(0, 1, 2, 3)$ représenté par la matrice d'adjacence :

```

1 matrice = [
2     [0, 1, 0, 1],
3     [0, 0, 1, 0],
4     [0, 0, 0, 0],
5     [0, 0, 1, 0],
6 ]

```

1. Lister toutes les arêtes de ce graphe (sous la forme $i \rightarrow j$).
2. Le sommet 2 a-t-il des successeurs ? Interpréter.

- En utilisant la fonction `matrice_vers_listes` du cours, donner la représentation par listes de successeurs de ce graphe.

Exercice 33

On veut vérifier si une expression comportant des parenthèses `()`, crochets `[]` et accolades `{}` est **bien parenthesée** (chaque symbole ouvrant a un symbole fermant correspondant, dans le bon ordre).

- Expliquer pourquoi une **pile** est une structure adaptée à ce problème (justifier par les opérations dominantes).
- Écrire une fonction `est_equilibre(expr)` qui utilise une pile (interface `empiler`, `depiler`, `est_vide`) pour résoudre ce problème.
- Tester votre fonction sur `"([{}])"` (bien parenthesée) et `"([)]"` (mal parenthesée).

Exercice 34

Écrire une classe `Compte` représentant un compte bancaire simplifié, avec :

- un constructeur prenant un titulaire et un solde initial (par défaut 0) ;
- une méthode `deposer(montant)` qui augmente le solde ;
- une méthode `retirer(montant)` qui diminue le solde, et lève une erreur (`ValueError`) si le montant demande dépasse le solde disponible.

Tester votre classe avec un compte initial de 100, un dépôt de 50, puis un retrait de 30.

Exercice 35

On construit un arbre binaire "en peigne" : la racine vaut 1, son fils gauche vaut 2 et a lui-même un fils gauche 3, qui a un fils gauche 4, qui a un fils gauche 5 (aucun fils droit nulle part).

- En utilisant la classe `Arbre` du cours et les fonctions `taille` et `hauteur`, construire cet arbre en Python.
- Sans exécuter de code, prédire la taille et la hauteur de cet arbre. Justifier.
- Vérifier votre prédiction en appelant `taille` et `hauteur` sur l'arbre construit.

Exercice 36

On donne le graphe orienté à 4 sommets $(0, 1, 2, 3)$ représenté par la matrice d'adjacence :

```
1 matrice = [
2     [0, 1, 0, 1],
3     [0, 0, 1, 0],
4     [0, 0, 0, 0],
5     [0, 0, 1, 0],
6 ]
```

- Lister toutes les arêtes de ce graphe (sous la forme $i \rightarrow j$).
- Le sommet 2 a-t-il des successeurs ? Interpréter.
- En utilisant la fonction `matrice_vers_listes` du cours, donner la représentation par listes de successeurs de ce graphe.

Exercice 37

On veut vérifier si une expression comportant des parenthèses `()`, crochets `[]` et accolades `{}` est **bien parenthesée** (chaque symbole ouvrant a un symbole fermant correspondant, dans le bon ordre).

- Expliquer pourquoi une **pile** est une structure adaptée à ce problème (justifier par les opérations dominantes).

- Écrire une fonction `est_equilibre(expr)` qui utilise une pile (interface `empiler`, `depiler`, `est_vide`) pour résoudre ce problème.
- Tester votre fonction sur `"([{}])"` (bien parenthésée) et `"([)]"` (mal parenthésée).

Exercice 38

Écrire une classe `Compte` représentant un compte bancaire simplifié, avec :

- un constructeur prenant un titulaire et un solde initial (par défaut 0) ;
- une méthode `deposer(montant)` qui augmente le solde ;
- une méthode `retirer(montant)` qui diminue le solde, et lève une erreur (`ValueError`) si le montant demandé dépasse le solde disponible.

Tester votre classe avec un compte initial de 100, un dépôt de 50, puis un retrait de 30.

Exercice 39

On construit un arbre binaire "en peigne" : la racine vaut 1, son fils gauche vaut 2 et a lui-même un fils gauche 3, qui a un fils gauche 4, qui a un fils gauche 5 (aucun fils droit nulle part).

- En utilisant la classe `Arbre` du cours et les fonctions `taille` et `hauteur`, construire cet arbre en Python.
- Sans exécuter de code, prédire la taille et la hauteur de cet arbre. Justifier.
- Vérifier votre prédiction en appelant `taille` et `hauteur` sur l'arbre construit.

Exercice 40

On donne le graphe orienté à 4 sommets (0, 1, 2, 3) représenté par la matrice d'adjacence :

```

1 matrice = [
2     [0, 1, 0, 1],
3     [0, 0, 1, 0],
4     [0, 0, 0, 0],
5     [0, 0, 1, 0],
6 ]

```

- Lister toutes les arêtes de ce graphe (sous la forme $i \rightarrow j$).
- Le sommet 2 a-t-il des successeurs ? Interpréter.
- En utilisant la fonction `matrice_vers_listes` du cours, donner la représentation par listes de successeurs de ce graphe.

Exercice 41

On veut vérifier si une expression comportant des parenthèses `()`, crochets `[]` et accolades `{}` est **bien parenthésée** (chaque symbole ouvrant a un symbole fermant correspondant, dans le bon ordre).

- Expliquer pourquoi une **pile** est une structure adaptée à ce problème (justifier par les opérations dominantes).
- Écrire une fonction `est_equilibre(expr)` qui utilise une pile (interface `empiler`, `depiler`, `est_vide`) pour résoudre ce problème.
- Tester votre fonction sur `"([{}])"` (bien parenthésée) et `"([)]"` (mal parenthésée).

Exercice 42

Écrire une classe `Compte` représentant un compte bancaire simplifié, avec :

- un constructeur prenant un titulaire et un solde initial (par défaut 0) ;

- ▶ une méthode `deposer(montant)` qui augmente le solde ;
- ▶ une méthode `retirer(montant)` qui diminue le solde, et leve une erreur (`ValueError`) si le montant demande depasse le solde disponible.

Tester votre classe avec un compte initial de 100, un depot de 50, puis un retrait de 30.

Exercice 43 ♦♦

On construit un arbre binaire "en peigne" : la racine vaut 1, son fils gauche vaut 2 et a lui-même un fils gauche 3, qui a un fils gauche 4, qui a un fils gauche 5 (aucun fils droit nulle part).

1. En utilisant la classe `Arbre` du cours et les fonctions `taille` et `hauteur`, construire cet arbre en Python.
2. Sans executer de code, predire la taille et la hauteur de cet arbre. Justifier.
3. Vérifier votre prediction en appelant `taille` et `hauteur` sur l'arbre construit.

Exercice 44 ♦♦

On donne le graphe oriente a 4 sommets (0,1,2,3) représente par la matrice d'adjacence :

```
1 matrice = [
2     [0, 1, 0, 1],
3     [0, 0, 1, 0],
4     [0, 0, 0, 0],
5     [0, 0, 1, 0],
6 ]
```

1. Lister toutes les arêtes de ce graphe (sous la forme $i \rightarrow j$).
2. Le sommet 2 a-t-il des successeurs ? Interpréter.
3. En utilisant la fonction `matrice_vers_listes` du cours, donner la représentation par listes de successeurs de ce graphe.

Exercice 45 ♦♦

On veut vérifier si une expression comportant des parenthesés (`()`), crochets [`[]`] et accolades `{}` est **bien parenthesee** (chaque symbole ouvrant a un symbole fermant correspondant, dans le bon ordre).

1. Expliquer pourquoi une **pile** est une structure adaptee a ce probleme (justifier par les opérations dominantes).
2. Écrire une fonction `est_equilibre(expr)` qui utilise une pile (interface `empiler`, `depiler`, `est_vide`) pour résoudre ce probleme.
3. Tester votre fonction sur `"([{}])"` (bien parenthesee) et `"([)]"` (mal parenthesee).

Exercice 46 ♦

Écrire une classe `Compte` representant un compte bancaire simplifie, avec :

- ▶ un constructeur prenant un titulaire et un solde initial (par défaut 0) ;
- ▶ une méthode `deposer(montant)` qui augmente le solde ;
- ▶ une méthode `retirer(montant)` qui diminue le solde, et leve une erreur (`ValueError`) si le montant demande depasse le solde disponible.

Tester votre classe avec un compte initial de 100, un depot de 50, puis un retrait de 30.

Exercice 47 ♦♦

On construit un arbre binaire "en peigne" : la racine vaut 1, son fils gauche vaut 2 et a lui-même un fils gauche 3, qui a un fils gauche 4, qui a un fils gauche 5 (aucun fils droit nulle part).

1. En utilisant la classe `Arbre` du cours et les fonctions `taille` et `hauteur`, construire cet arbre en Python.
2. Sans exécuter de code, prédire la taille et la hauteur de cet arbre. Justifier.
3. Vérifier votre prédiction en appelant `taille` et `hauteur` sur l'arbre construit.

Exercice 48

On donne le graphe orienté à 4 sommets $(0, 1, 2, 3)$ représenté par la matrice d'adjacence :

```
1 matrice = [
2     [0, 1, 0, 1],
3     [0, 0, 1, 0],
4     [0, 0, 0, 0],
5     [0, 0, 1, 0],
6 ]
```

1. Lister toutes les arêtes de ce graphe (sous la forme $i \rightarrow j$).
2. Le sommet 2 a-t-il des successeurs ? Interpréter.
3. En utilisant la fonction `matrice_vers_listes` du cours, donner la représentation par listes de successeurs de ce graphe.

Exercice 49

On veut vérifier si une expression comportant des parenthèses `()`, crochets `[]` et accolades `{}` est **bien parenthesée** (chaque symbole ouvrant a un symbole fermant correspondant, dans le bon ordre).

1. Expliquer pourquoi une **pile** est une structure adaptée à ce problème (justifier par les opérations dominantes).
2. Écrire une fonction `est_equilibre(expr)` qui utilise une pile (interface `empiler`, `depiler`, `est_vide`) pour résoudre ce problème.
3. Tester votre fonction sur `"({})"` (bien parenthesée) et `"([)]"` (mal parenthesée).

Exercice 50

Écrire une classe `Compte` représentant un compte bancaire simplifié, avec :

- un constructeur prenant un titulaire et un solde initial (par défaut 0) ;
- une méthode `deposer(montant)` qui augmente le solde ;
- une méthode `retirer(montant)` qui diminue le solde, et lève une erreur (`ValueError`) si le montant demandé dépasse le solde disponible.

Tester votre classe avec un compte initial de 100, un dépôt de 50, puis un retrait de 30.

STRUCTURES DE DONNÉES — QCM D'AUTO-ÉVALUATION

Pour chaque question, une seule réponse est exacte.

Capacité C1

1. [Q1] L'interface d'une structure de données décrit :
 - A. les opérations disponibles, sans préciser comment elles sont réalisées.
 - B. la façon dont les données sont rangées en mémoire.
 - C. le nombre d'octets occupés.
 - D. le langage de programmation employé.

Capacité C2

2. [Q2] Passer d'une implémentation d'une pile par tableau à une implémentation par liste chaînée, sans changer son interface :
- A. oblige à réécrire tous les programmes qui utilisent la pile.
 - B. laisse inchangés les programmes qui n'utilisent que `empiler` et `depiler`.
 - C. change le principe LIFO en principe FIFO.
 - D. est impossible : l'interface dépend de l'implémentation.

Capacité C3

3. [Q3] Une file peut être implémentée par une liste Python où l'on retire en tête avec `pop(0)`, ou par deux piles. Ces deux implémentations :
- A. respectent le même contrat, mais n'ont pas le même coût.
 - B. sont interdites : une file n'admet qu'une implémentation.
 - C. produisent des ordres de sortie différents.
 - D. ont exactement les mêmes performances.

Capacité C4

4. [Q4] Dans la définition d'une classe Python, la méthode chargée d'initialiser un nouvel objet se nomme :
- A. `init`
 - B. `__init__`
 - C. `create`
 - D. `new`

Capacité C5

5. [Q5] Dans une méthode de classe Python, le paramètre `self` désigne :
- A. le premier attribut déclaré.
 - B. la classe elle-même.
 - C. l'objet sur lequel la méthode est appelée.
 - D. rien de particulier : il est facultatif.

Capacité C6

6. [Q6] On empile successivement 1, puis 2, puis 3 dans une pile vide, puis on dépile une fois. La valeur obtenue est :
- A. 1
 - B. la plus petite des trois.
 - C. 2
 - D. 3

Capacité C7

7. [Q7] Pour mémoriser les pages visitées afin de revenir en arrière dans un navigateur, la structure adaptée est :
- A. une pile.
 - B. un arbre binaire de recherche.
 - C. une file.
 - D. une matrice d'adjacence.

Capacité C8

8. [Q8] Rechercher une valeur associée à une clé est en moyenne plus rapide dans un dictionnaire que dans une liste, parce que :
- A. un dictionnaire est toujours plus petit qu'une liste.
 - B. l'accès passe par une table de hachage, sans dépendre du nombre d'éléments.
 - C. l'affirmation est fausse : les deux coûtent autant.
 - D. les dictionnaires sont triés automatiquement.

Capacité C9

9. [Q9] Parmi ces situations, laquelle appelle le plus naturellement une structure arborescente ?
- A. La liste des élèves d'une classe, par ordre alphabétique.
 - B. La file d'attente des travaux d'impression.
 - C. L'arborescence des dossiers et fichiers d'un disque.

D. Le relevé des températures d'une journée, heure par heure.

Capacité C10

10. [Q10] La taille d'un arbre binaire est :

- A. la longueur du plus long chemin de la racine à une feuille.
- B. le nombre de ses niveaux moins un.
- C. son nombre de feuilles.
- D. son nombre de nœuds.

Capacité C11

11. [Q11] Pour modéliser un réseau social où l'amitié est réciproque, on emploie :

- A. un graphe non orienté, les personnes étant les sommets et les amitiés les arêtes.
- B. une pile de profils.
- C. un arbre binaire de recherche.
- D. un graphe orienté, car l'amitié a toujours un sens.

Capacité C12

12. [Q12] Dans la matrice d'adjacence d'un graphe à n sommets, la case (i, j) vaut 1 lorsque :

- A. les sommets i et j portent le même numéro.
- B. il existe une arête reliant i à j .
- C. le sommet i est une feuille.
- D. la somme $i + j$ vaut n .

Capacité C13

13. [Q13] Dans une implémentation par listes de successeurs, l'espace occupé par un graphe à n sommets et m arêtes est de l'ordre de :

- A. n^2 , quel que soit m .
- B. m seulement, sans dépendre de n .
- C. $n + m$.
- D. $n \times m$.

Capacité C14

14. [Q14] Pour convertir une matrice d'adjacence en listes de successeurs, il faut :

- A. conserver seulement la diagonale de la matrice.
- B. trier les sommets par nombre de voisins.
- C. recopier chaque ligne de la matrice telle quelle, zéros compris.
- D. pour chaque sommet, retenir les indices des colonnes dont la case vaut 1.

Clé de correction — réservée au professeur

Question	Capacité	Réponse exacte
Q1	C1	A
Q2	C2	B
Q3	C3	A
Q4	C4	B
Q5	C5	C
Q6	C6	D
Q7	C7	A
Q8	C8	B
Q9	C9	C
Q10	C10	D
Q11	C11	A
Q12	C12	B
Q13	C13	C
Q14	C14	D

Diagnostic des réponses erronées

► Q1

- **B** — L'élève décrit l'IMPLEMENTATION. L'interface énonce le contrat ; le rangement en mémoire est un choix de réalisation qu'elle laisse libre. *Renvoi : C1, spécification par l'interface.*
- **C** — L'encombrement dépend de l'implémentation retenue, et varie donc d'une réalisation à l'autre pour une même interface. *Renvoi : C1, spécification par l'interface.*
- **D** — Une même interface, par exemple celle d'une pile, se retrouve dans tous les langages : elle en est indépendante. *Renvoi : C1, spécification par l'interface.*

► Q2

- **A** — C'est précisément ce que la séparation évite. Tant que les opérations gardent leur nom et leur comportement, le code appelant ne voit rien. *Renvoi : C2, interface et implémentation.*
- **C** — L'ordre de service fait partie du contrat, donc de l'interface. Changer d'implémentation ne peut pas le modifier. *Renvoi : C2, interface et implémentation.*
- **D** — L'élève inverse la dépendance : c'est l'implémentation qui doit respecter l'interface, et non l'inverse. *Renvoi : C2, interface et implémentation.*

► Q3

- **B** — C'est l'inverse : pouvoir réaliser une même structure de plusieurs façons est justement l'intérêt de la séparation entre interface et implémentation. *Renvoi : C3, implémentations multiples.*
- **C** — Si l'ordre différait, l'une des deux ne serait pas une file. Toutes deux servent les éléments dans leur ordre d'arrivée. *Renvoi : C3, implémentations multiples.*
- **D** — `pop(0)` décale tous les éléments restants, à un coût linéaire, alors que la version à deux piles amortit ce coût. *Renvoi : C3, coût d'une implémentation.*

► Q4

- **A** — Sans les doubles soulignements, la méthode est ordinaire : Python ne l'appelle pas automatiquement à la construction. *Renvoi : C4, définition d'une classe.*
- **C** — Aucune convention Python ne reconnaît ce nom ; la méthode ne serait jamais appelée d'elle-même. *Renvoi : C4, définition d'une classe.*
- **D** — L'élève transpose le mot-clé d'autres langages. En Python, l'instanciation s'écrit sans `new`. *Renvoi : C4, définition d'une classe.*

► Q5

- **A** — `self` donne accès à TOUS les attributs, par exemple `self.taille` ; il n'en désigne aucun en particulier. *Renvoi : C5, attributs et méthodes.*
- **B** — La classe est le moule ; `self` désigne l'exemplaire. Deux objets d'une même classe reçoivent des `self` différents. *Renvoi : C5, attributs et méthodes.*
- **D** — Il est au contraire indispensable : l'omettre prive la méthode de tout accès à l'objet et provoque une erreur à l'appel. *Renvoi : C5, attributs et méthodes.*

► Q6

- A — L'élève applique le principe FIFO, celui de la FILE. Une pile sert en dernier entré, premier sorti : c'est 3 qui ressort. *Renvoi : C6, LIFO et FIFO.*
- B — Une pile ne compare jamais les valeurs ; elle ne connaît que l'ordre d'arrivée. *Renvoi : C6, LIFO et FIFO.*
- C — Aucun élément intermédiaire n'est accessible : seul le sommet de la pile peut être retiré. *Renvoi : C6, LIFO et FIFO.*

► Q7

- B — L'arbre de recherche organise les données par valeur. Ici, c'est l'ordre de visite qui compte, non un ordre sur les adresses. *Renvoi : C7, choix d'une structure.*
- C — Une file rendrait d'abord la page la PLUS ANCIENNE, alors que le retour arrière doit rendre la plus récente. *Renvoi : C7, choix d'une structure.*
- D — La matrice d'adjacence décrit des liens entre sommets ; elle ne mémorise aucune chronologie. *Renvoi : C7, choix d'une structure.*

► Q8

- A — La taille n'entre pas en jeu : l'écart tient à la méthode d'accès, et il grandit justement avec le nombre d'éléments. *Renvoi : C8, recherche par clé et par parcours.*
- C — La recherche dans une liste parcourt les éléments un à un, à un coût linéaire, tandis que l'accès par clé est en temps quasi constant. *Renvoi : C8, recherche par clé et par parcours.*
- D — Aucun tri n'a lieu, et il ne servirait pas ici : le hachage calcule directement l'emplacement de la clé. *Renvoi : C8, recherche par clé et par parcours.*

► Q9

- A — Une liste ordonnée est linéaire. Un arbre ne s'impose que si l'on veut de surcroît accélérer les recherches. *Renvoi : C9, situations arborescentes.*
- B — Une file d'attente est purement linéaire : chaque travail a un seul suivant, sans ramification. *Renvoi : C9, situations arborescentes.*
- D — Une suite de mesures indexée par le temps est un tableau ; aucune hiérarchie n'y apparaît. *Renvoi : C9, situations arborescentes.*

► Q10

- A — L'élève définit la HAUTEUR. Les deux mesures diffèrent : un arbre dégénéré à n nœuds a pour taille n et pour hauteur $n - 1$. *Renvoi : C10, taille, hauteur et feuilles.*
- B — C'est encore la hauteur, exprimée par le nombre de niveaux. La taille dénombre les nœuds, pas les niveaux. *Renvoi : C10, taille, hauteur et feuilles.*
- C — Les feuilles ne forment qu'une partie des nœuds ; les nœuds internes comptent aussi dans la taille. *Renvoi : C10, taille, hauteur et feuilles.*

► Q11

- B — Une pile ordonne dans le temps ; elle ne représente aucune relation entre deux éléments. *Renvoi : C11, modélisation par un graphe.*
- C — Un arbre interdit les cycles et impose une racine. Or trois personnes amies deux à deux forment un cycle, situation banale ici. *Renvoi : C11, modélisation par un graphe.*
- D — L'orientation ne s'impose que si la relation est à sens unique, comme l'abonnement. Une amitié réciproque se représente par une arête sans direction. *Renvoi : C11, graphes orientés et non orientés.*

► Q12

- A — La diagonale ne vaut 1 que si le graphe comporte une boucle sur ce sommet, ce qui est un cas particulier et non la règle. *Renvoi : C12, matrice d'adjacence.*
- C — La notion de feuille appartient au vocabulaire des arbres ; la matrice code des arêtes, non des rôles de sommets. *Renvoi : C12, matrice d'adjacence.*
- D — L'élève invente une relation arithmétique entre indices. Le contenu de la matrice décrit le graphe, non la numérotation. *Renvoi : C12, matrice d'adjacence.*

► Q13

- **A** — C'est le coût de la MATRICE d'adjacence, qui réserve une case par couple de sommets, même absent. Les listes de successeurs ne stockent que les arêtes présentes. *Renvoi : C13, listes de successeurs.*
- **B** — Il faut de toute façon une entrée par sommet, y compris pour un sommet isolé sans aucune arête. *Renvoi : C13, listes de successeurs.*
- **D** — Aucune structure ne duplique les arêtes pour chaque sommet ; chaque arête n'est enregistrée qu'une ou deux fois. *Renvoi : C13, listes de successeurs.*

► Q14

- **A** — La diagonale ne renseigne que sur d'éventuelles boucles ; toutes les autres arêtes seraient perdues. *Renvoi : C14, conversion entre représentations.*
- **B** — Le tri modifierait l'ordre sans rien apporter : la conversion doit préserver exactement les mêmes arêtes. *Renvoi : C14, conversion entre représentations.*
- **C** — Conserver les zéros reviendrait à garder la matrice sous un autre nom, et à perdre le gain de place recherché. *Renvoi : C14, conversion entre représentations.*

Corrigé

```

1 class Rectangle:
2     def __init__(self, largeur, hauteur):
3         self.largeur = largeur
4         self.hauteur = hauteur
5
6     def aire(self):
7         return self.largeur * self.hauteur
8
9     def perimetre(self):
10        return 2 * (self.largeur + self.hauteur)

```

Corrigé

Q1. Interface : empiler(v), depiler(), est_vide(). Principe LIFO : le dernier élément empilé est le premier dépiler.

Q2. Après avoir empilé 5, 3, 8, 1, la pile est [5, 3, 8, 1] (du bas vers le sommet). Dépiler trois fois retire 1, puis 8, puis 3 (LIFO) : il reste [5].

Évaluation A — Structures de données

Durée : 45 minutes. Ordinateur avec interpréteur Python autorisé.

Barème indicatif : Ex. 1 : 10 pts (programmer une classe) ; Ex. 2 : 10 pts (pile, LIFO)

Exercice 1 — Classe Rectangle

(10 points)

1. (4 pts) Écrire une classe Rectangle avec un constructeur prenant largeur et hauteur.
2. (3 pts) Ajouter une méthode aire() qui renvoie l'aire du rectangle.
3. (3 pts) Ajouter une méthode perimetre() qui renvoie le perimètre du rectangle.

Exercice 2 — Pile

(10 points)

1. (4 pts) Rappeler l'interface d'une pile (trois méthodes) et le principe LIFO.
2. (6 pts) On empile successivement 5, 3, 8, 1, puis on dépile trois fois. Donner, en le justifiant, le contenu final de la pile.

Corrigé

Q1. Un graphe modélisé naturellement des éléments (stations) en relation (lignes directes) : les stations sont les sommets, les lignes directes les arêtes.

Q2. Si les lignes fonctionnent dans les deux sens, le graphe doit être **non orienté** (une arête entre deux stations représente une liaison utilisable dans les deux sens, sans distinction de direction).

Corrigé

Q1. $0 \rightarrow 1, 0 \rightarrow 2, 2 \rightarrow 0$.

Q2. La liste associée au sommet 1 est vide : 1 n'a aucun successeur, c'est un puits du graphe (aucune arête sortante).

Q3.

```
1 matrice = [
2     [0, 1, 1],
3     [0, 0, 0],
4     [1, 0, 0],
5 ]
```

Évaluation B — Structures de données

Durée : 45 minutes. Ordinateur avec interpréteur Python autorisé.

Barème indicatif : Ex. 1 : 8 pts (modélisation) ; Ex. 2 : 12 pts (graphes, conversion)

Exercice 1 — Modélisation

(8 points)

Un réseau de transport relie des stations par des lignes directes.

- (4 pts) Justifier qu'un graphe est une structure adaptée pour modéliser ce réseau.
- (4 pts) Ce graphe doit-il être orienté ou non orienté si les lignes fonctionnent dans les deux sens ? Justifier.

Exercice 2 — Graphes, conversion

(12 points)

On donne un graphe orienté à 3 sommets (0, 1, 2) par ses listes de successeurs :

```
1 successeurs = {0: [1, 2], 1: [], 2: [0]}
```

- (4 pts) Lister toutes les arêtes de ce graphe.
- (4 pts) Le sommet 1 a-t-il des successeurs ? Qu'est-ce que cela signifie ?
- (4 pts) Donner la matrice d'adjacence correspondant à ce graphe.

Exercice 51 ♦

On empile successivement les valeurs 10, 20, 30 dans une pile, puis on depile deux fois. Quelle est la valeur au sommet de la pile après ces opérations ?

Corrigé

Après avoir empilé 10, 20, 30, la pile est (du bas vers le sommet) [10, 20, 30]. Depiler une fois retire 30 (LIFO), depiler une deuxième fois retire 20. Il reste 10 au sommet.

Exercice 52

On empile successivement les valeurs 10, 20, 30 dans une pile, puis on depile deux fois. Quelle est la valeur au sommet de la pile après ces opérations ?

Corrigé

Après avoir empile 10, 20, 30, la pile est (du bas vers le sommet) [10, 20, 30]. Depiler une fois retire 30 (LIFO), depiler une deuxième fois retire 20. Il reste 10 au sommet.

Exercice 53

On empile successivement les valeurs 10, 20, 30 dans une pile, puis on depile deux fois. Quelle est la valeur au sommet de la pile après ces opérations ?

Corrigé

Après avoir empile 10, 20, 30, la pile est (du bas vers le sommet) [10, 20, 30]. Depiler une fois retire 30 (LIFO), depiler une deuxième fois retire 20. Il reste 10 au sommet.

Exercice 54

On empile successivement les valeurs 10, 20, 30 dans une pile, puis on depile deux fois. Quelle est la valeur au sommet de la pile après ces opérations ?

Corrigé

Après avoir empile 10, 20, 30, la pile est (du bas vers le sommet) [10, 20, 30]. Depiler une fois retire 30 (LIFO), depiler une deuxième fois retire 20. Il reste 10 au sommet.

Exercice 55

On empile successivement les valeurs 10, 20, 30 dans une pile, puis on depile deux fois. Quelle est la valeur au sommet de la pile après ces opérations ?

Corrigé

Après avoir empile 10, 20, 30, la pile est (du bas vers le sommet) [10, 20, 30]. Depiler une fois retire 30 (LIFO), depiler une deuxième fois retire 20. Il reste 10 au sommet.

Exercice 56

On empile successivement les valeurs 10, 20, 30 dans une pile, puis on depile deux fois. Quelle est la valeur au sommet de la pile après ces opérations ?

Corrigé

Après avoir empile 10, 20, 30, la pile est (du bas vers le sommet) [10, 20, 30]. Depiler une fois retire 30 (LIFO), depiler une deuxième fois retire 20. Il reste 10 au sommet.

Exercice 57

On empile successivement les valeurs 10, 20, 30 dans une pile, puis on depile deux fois. Quelle est la valeur au sommet de la pile après ces opérations ?

Corrigé

Après avoir empile 10, 20, 30, la pile est (du bas vers le sommet) [10, 20, 30]. Depiler une fois retire 30 (LIFO), depiler une deuxième fois retire 20. Il reste 10 au sommet.

Exercice 58 ◆

On empile successivement les valeurs 10, 20, 30 dans une pile, puis on depile deux fois. Quelle est la valeur au sommet de la pile après ces opérations ?

Corrigé

Après avoir empilé 10, 20, 30, la pile est (du bas vers le sommet) [10, 20, 30]. Depiler une fois retire 30 (LIFO), depiler une deuxième fois retire 20. Il reste 10 au sommet.

Exercice 59 ◆

On empile successivement les valeurs 10, 20, 30 dans une pile, puis on depile deux fois. Quelle est la valeur au sommet de la pile après ces opérations ?

Corrigé

Après avoir empilé 10, 20, 30, la pile est (du bas vers le sommet) [10, 20, 30]. Depiler une fois retire 30 (LIFO), depiler une deuxième fois retire 20. Il reste 10 au sommet.

Exercice 60 ◆

On empile successivement les valeurs 10, 20, 30 dans une pile, puis on depile deux fois. Quelle est la valeur au sommet de la pile après ces opérations ?

Corrigé

Après avoir empilé 10, 20, 30, la pile est (du bas vers le sommet) [10, 20, 30]. Depiler une fois retire 30 (LIFO), depiler une deuxième fois retire 20. Il reste 10 au sommet.

Exercice 61 ◆

On empile successivement les valeurs 10, 20, 30 dans une pile, puis on depile deux fois. Quelle est la valeur au sommet de la pile après ces opérations ?

Corrigé

Après avoir empilé 10, 20, 30, la pile est (du bas vers le sommet) [10, 20, 30]. Depiler une fois retire 30 (LIFO), depiler une deuxième fois retire 20. Il reste 10 au sommet.

Exercice 62 ◆

On empile successivement les valeurs 10, 20, 30 dans une pile, puis on depile deux fois. Quelle est la valeur au sommet de la pile après ces opérations ?

Corrigé

Après avoir empilé 10, 20, 30, la pile est (du bas vers le sommet) [10, 20, 30]. Depiler une fois retire 30 (LIFO), depiler une deuxième fois retire 20. Il reste 10 au sommet.

Exercice 63 ◆

On empile successivement les valeurs 10, 20, 30 dans une pile, puis on depile deux fois. Quelle est la valeur au sommet de la pile après ces opérations ?

Corrigé

Après avoir empilé 10, 20, 30, la pile est (du bas vers le sommet) [10, 20, 30]. Depiler une fois retire 30 (LIFO), depiler une deuxième fois retire 20. Il reste 10 au sommet.

Exercice 64 ◆

On empile successivement les valeurs 10, 20, 30 dans une pile, puis on depile deux fois. Quelle est la valeur au sommet de la pile après ces opérations ?

Corrigé

Après avoir empilé 10, 20, 30, la pile est (du bas vers le sommet) [10, 20, 30]. Depiler une fois retire 30 (LIFO), depiler une deuxième fois retire 20. Il reste 10 au sommet.

Exercice 65 ◆

On empile successivement les valeurs 10, 20, 30 dans une pile, puis on depile deux fois. Quelle est la valeur au sommet de la pile après ces opérations ?

Corrigé

Après avoir empilé 10, 20, 30, la pile est (du bas vers le sommet) [10, 20, 30]. Depiler une fois retire 30 (LIFO), depiler une deuxième fois retire 20. Il reste 10 au sommet.

Corrigé

Q1. Une pile est adaptée car on doit toujours refermer le symbole ouvert le **plus récemment** en premier (comportement LIFO) : chaque symbole ouvrant est empilé, et chaque symbole fermant doit correspondre au sommet de la pile.

Q2.

```

1 class Pile:
2     def __init__(self):
3         self._d = []
4
5     def empiler(self, v):
6         self._d.append(v)
7
8     def depiler(self):
9         return self._d.pop()
10
11    def est_vide(self):
12        return len(self._d) == 0
13
14
15    def est_equilibre(expr):
16        p = Pile()
17        paires = {'(': ')', '[': ']', '{': '}'
18        for c in expr:
19            if c in '([{':
20                p.empiler(c)
21            elif c in ')]}':
22                if p.est_vide() or p.depiler() != paires[c]:
23                    return False
24        return p.est_vide()

```

Q3. `est_equilibre("([{}])")` renvoie True ; `est_equilibre("([)])")` renvoie False (le `)` rencontre alors que le sommet de la pile est `[`, qui ne correspond pas).

Corrigé

```

1 class Compte:

```

```

2  def __init__(self, titulaire, solde=0):
3      self.titulaire = titulaire
4      self.solde = solde
5
6  def deposer(self, montant):
7      self.solde = self.solde + montant
8
9  def retirer(self, montant):
10     if montant > self.solde:
11         raise ValueError("solde insuffisant")
12     self.solde = self.solde - montant

```

Test : `c = Compte("Alice", 100)` ; après `c.deposer(50)`, `c.solde` vaut 150 ; après `c.retirer(30)`, `c.solde` vaut 120.

Corrigé

Q1.

```
1 arbre = Arbre(1, Arbre(2, Arbre(3, Arbre(4, Arbre(5)))))
```

Q2. L'arbre a 5 noeuds (1,2,3,4,5), donc **taille** = 5. Le plus long chemin racine-feuille passe par tous les noeuds (chaîne de fils gauches), soit 4 arêtes parcourues : **hauteur** = 4.

Q3. `taille(arbre)` renvoie 5, `hauteur(arbre)` renvoie 4 : la prédiction est confirmée.

Corrigé

Q1. En lisant les 1 de la matrice : $0 \rightarrow 1$, $0 \rightarrow 3$, $1 \rightarrow 2$, $3 \rightarrow 2$.

Q2. La ligne 2 de la matrice ne contient que des 0 : le sommet 2 n'a **aucun successeur** (c'est un puits du graphe).

Q3.

```
1 successeurs = {0: [1, 3], 1: [2], 2: [], 3: [2]}
```

Le sommet 2, bien que sans successeur, apparaît dans le dictionnaire avec une liste vide (conformément à la remarque du cours).

Corrigé

Q1. Une pile est adaptée car on doit toujours refermer le symbole ouvert le **plus récemment** en premier (comportement LIFO) : chaque symbole ouvrant est empilé, et chaque symbole fermant doit correspondre au sommet de la pile.

Q2.

```

1 class Pile:
2     def __init__(self):
3         self._d = []
4
5     def empiler(self, v):
6         self._d.append(v)
7
8     def depiler(self):
9         return self._d.pop()
10
11    def est_vide(self):
12        return len(self._d) == 0
13
14

```

```

15 def est_equilibre(expr):
16     p = Pile()
17     paires = {'(': ')', '[': ']', '{': '}'
18     for c in expr:
19         if c in '([{':
20             p.empiler(c)
21         elif c in ')]}':
22             if p.est_vide() or p.depiler() != paires[c]:
23                 return False
24     return p.est_vide()

```

Q3. `est_equilibre("({})")` renvoie True ; `est_equilibre("([)")` renvoie False (le `)` rencontre alors que le sommet de la pile est `[`, qui ne correspond pas).

Corrigé

```

1 class Compte:
2     def __init__(self, titulaire, solde=0):
3         self.titulaire = titulaire
4         self.solde = solde
5
6     def deposer(self, montant):
7         self.solde = self.solde + montant
8
9     def retirer(self, montant):
10        if montant > self.solde:
11            raise ValueError("solde insuffisant")
12        self.solde = self.solde - montant

```

Test : `c = Compte("Alice", 100)` ; après `c.deposer(50)`, `c.solde` vaut 150 ; après `c.retirer(30)`, `c.solde` vaut 120.

Corrigé

Q1.

```

1 arbre = Arbre(1, Arbre(2, Arbre(3, Arbre(4, Arbre(5)))))

```

Q2. L'arbre a 5 noeuds (1,2,3,4,5), donc **taille** = 5. Le plus long chemin racine-feuille passe par tous les noeuds (chaîne de fils gauches), soit 4 arêtes parcourues : **hauteur** = 4.

Q3. `taille(arbre)` renvoie 5, `hauteur(arbre)` renvoie 4 : la prediction est confirmée.

Corrigé

Q1. En lisant les 1 de la matrice : $0 \rightarrow 1, 0 \rightarrow 3, 1 \rightarrow 2, 3 \rightarrow 2$.

Q2. La ligne 2 de la matrice ne contient que des 0 : le sommet 2 n'a **aucun successeur** (c'est un puits du graphe).

Q3.

```

1 successeurs = {0: [1, 3], 1: [2], 2: [], 3: [2]}

```

Le sommet 2, bien que sans successeur, apparaît dans le dictionnaire avec une liste vide (conformément à la remarque du cours).

Corrigé

Q1. Une pile est adaptée car on doit toujours refermer le **plus récemment** en premier (comportement LIFO) : chaque symbole ouvrant est empilé, et chaque symbole fermant doit correspondre au sommet de la pile.

Q2.

```

1 class Pile:
2     def __init__(self):
3         self._d = []
4
5     def empiler(self, v):
6         self._d.append(v)
7
8     def depiler(self):
9         return self._d.pop()
10
11    def est_vide(self):
12        return len(self._d) == 0
13
14
15    def est_equilibre(expr):
16        p = Pile()
17        paires = {'(': ')', '[': ']', '{': '}'
18        for c in expr:
19            if c in '([{':
20                p.empiler(c)
21            elif c in ')]}':
22                if p.est_vide() or p.depiler() != paires[c]:
23                    return False
24        return p.est_vide()

```

CORRIGÉS

Q3. `est_equilibre("([{}])")` renvoie True ; `est_equilibre("([()])")` renvoie False (le `)` rencontre alors que le sommet de la pile est `[`, qui ne correspond pas).

Corrigé

```

1 class Compte:
2     def __init__(self, titulaire, solde=0):
3         self.titulaire = titulaire
4         self.solde = solde
5
6     def deposer(self, montant):
7         self.solde = self.solde + montant
8
9     def retirer(self, montant):
10        if montant > self.solde:
11            raise ValueError("solde insuffisant")
12        self.solde = self.solde - montant

```

Test : `c = Compte("Alice", 100)` ; après `c.deposer(50)`, `c.solde` vaut 150 ; après `c.retirer(30)`, `c.solde` vaut 120.

Corrigé

Q1.

```

1 arbre = Arbre(1, Arbre(2, Arbre(3, Arbre(4, Arbre(5)))))

```

Q2. L'arbre a 5 noeuds (1,2,3,4,5), donc **taille** = 5. Le plus long chemin racine-feuille passe par tous les noeuds (chaîne de fils gauches), soit 4 arêtes parcourues : **hauteur** = 4.

Q3. `taille(arbre)` renvoie 5, `hauteur(arbre)` renvoie 4 : la prediction est confirmée.

Corrigé

Q1. En lisant les 1 de la matrice : $0 \rightarrow 1, 0 \rightarrow 3, 1 \rightarrow 2, 3 \rightarrow 2$.

Q2. La ligne 2 de la matrice ne contient que des 0 : le sommet 2 n'a **aucun successeur** (c'est un puits du graphe).

Q3.

```
1 successeurs = {0: [1, 3], 1: [2], 2: [], 3: [2]}
```

Le sommet 2, bien que sans successeur, apparait dans le dictionnaire avec une liste vide (conformement à la remarque du cours).

Corrigé

Q1. Une pile est adaptée car on doit toujours refermer le symbole ouvert le **plus récemment** en premier (comportement LIFO) : chaque symbole ouvrant est empilé, et chaque symbole fermant doit correspondre au sommet de la pile.

Q2.

```
1 class Pile:
2     def __init__(self):
3         self._d = []
4
5     def empiler(self, v):
6         self._d.append(v)
7
8     def depiler(self):
9         return self._d.pop()
10
11    def est_vide(self):
12        return len(self._d) == 0
13
14
15    def est_equilibre(expr):
16        p = Pile()
17        paires = {'(': ')', '[': ']', '{': '}'
18        for c in expr:
19            if c in '([{':
20                p.empiler(c)
21            elif c in ')]}':
22                if p.est_vide() or p.depiler() != paires[c]:
23                    return False
24        return p.est_vide()
```

Q3. `est_equilibre("([{}])")` renvoie True ; `est_equilibre("([)])")` renvoie False (le `)` rencontre alors que le sommet de la pile est `[`, qui ne correspond pas).

Corrigé

```
1 class Compte:
2     def __init__(self, titulaire, solde=0):
3         self.titulaire = titulaire
4         self.solde = solde
```

```

5
6     def deposer(self, montant):
7         self.solde = self.solde + montant
8
9     def retirer(self, montant):
10        if montant > self.solde:
11            raise ValueError("solde insuffisant")
12        self.solde = self.solde - montant

```

Test : `c = Compte("Alice", 100)` ; après `c.deposer(50)`, `c.solde` vaut 150 ; après `c.retirer(30)`, `c.solde` vaut 120.

Corrigé

Q1.

```
1 arbre = Arbre(1, Arbre(2, Arbre(3, Arbre(4, Arbre(5)))))
```

Q2. L'arbre a 5 noeuds (1,2,3,4,5), donc **taille** = 5. Le plus long chemin racine-feuille passe par tous les noeuds (chaîne de fils gauches), soit 4 arêtes parcourues : **hauteur** = 4.

Q3. `taille(arbre)` renvoie 5, `hauteur(arbre)` renvoie 4 : la prediction est confirmée.

Corrigé

Q1. En lisant les 1 de la matrice : $0 \rightarrow 1, 0 \rightarrow 3, 1 \rightarrow 2, 3 \rightarrow 2$.

Q2. La ligne 2 de la matrice ne contient que des 0 : le sommet 2 n'a **aucun successeur** (c'est un puits du graphe).

Q3.

```
1 successeurs = {0: [1, 3], 1: [2], 2: [], 3: [2]}
```

Le sommet 2, bien que sans successeur, apparait dans le dictionnaire avec une liste vide (conformément à la remarque du cours).

Corrigé

Q1. Une pile est adaptée car on doit toujours refermer le symbole ouvert le **plus récemment** en premier (comportement LIFO) : chaque symbole ouvrant est empilé, et chaque symbole fermant doit correspondre au sommet de la pile.

Q2.

```

1 class Pile:
2     def __init__(self):
3         self._d = []
4
5     def empiler(self, v):
6         self._d.append(v)
7
8     def depiler(self):
9         return self._d.pop()
10
11    def est_vide(self):
12        return len(self._d) == 0
13
14
15    def est_equilibre(expr):
16        p = Pile()
17        paires = {'(': ')', '[': ']', '{': '}'

```

```

18     for c in expr:
19         if c in '([{':
20             p.empiler(c)
21         elif c in ')]}':
22             if p.est_vide() or p.depiler() != paires[c]:
23                 return False
24     return p.est_vide()

```

Q3. `est_equilibre("({})")` renvoie True ; `est_equilibre("([])")` renvoie False (le `)` rencontre alors que le sommet de la pile est `[`, qui ne correspond pas).

Corrigé

```

1 class Compte:
2     def __init__(self, titulaire, solde=0):
3         self.titulaire = titulaire
4         self.solde = solde
5
6     def deposer(self, montant):
7         self.solde = self.solde + montant
8
9     def retirer(self, montant):
10        if montant > self.solde:
11            raise ValueError("solde insuffisant")
12        self.solde = self.solde - montant

```

Test : `c = Compte("Alice", 100)` ; après `c.deposer(50)`, `c.solde` vaut 150 ; après `c.retirer(30)`, `c.solde` vaut 120.

Corrigé

Q1.

```

1 arbre = Arbre(1, Arbre(2, Arbre(3, Arbre(4, Arbre(5)))))

```

Q2. L'arbre a 5 noeuds (1,2,3,4,5), donc **taille** = 5. Le plus long chemin racine-feuille passe par tous les noeuds (chaîne de fils gauches), soit 4 arêtes parcourues : **hauteur** = 4.

Q3. `taille(arbre)` renvoie 5, `hauteur(arbre)` renvoie 4 : la prédiction est confirmée.

Corrigé

Q1. En lisant les 1 de la matrice : $0 \rightarrow 1, 0 \rightarrow 3, 1 \rightarrow 2, 3 \rightarrow 2$.

Q2. La ligne 2 de la matrice ne contient que des 0 : le sommet 2 n'a **aucun successeur** (c'est un puits du graphe).

Q3.

```

1 successeurs = {0: [1, 3], 1: [2], 2: [], 3: [2]}

```

Le sommet 2, bien que sans successeur, apparaît dans le dictionnaire avec une liste vide (conformément à la remarque du cours).

Corrigé

Q1. Une pile est adaptée car on doit toujours refermer le symbole ouvert le **plus récemment** en premier (comportement LIFO) : chaque symbole ouvrant est empilé, et chaque symbole fermant doit correspondre au sommet de la pile.

Q2.

```

1 class Pile:
2     def __init__(self):
3         self._d = []
4
5     def empiler(self, v):
6         self._d.append(v)
7
8     def depiler(self):
9         return self._d.pop()
10
11    def est_vide(self):
12        return len(self._d) == 0
13
14
15    def est_equilibre(expr):
16        p = Pile()
17        paires = {'(': ')', '[': ']', '{': '}'
18        for c in expr:
19            if c in '([{':
20                p.empiler(c)
21            elif c in ')]}':
22                if p.est_vide() or p.depiler() != paires[c]:
23                    return False
24        return p.est_vide()

```

Q3. `est_equilibre("([{}])")` renvoie True ; `est_equilibre("([)])"` renvoie False (le `)` rencontre alors que le sommet de la pile est `[`, qui ne correspond pas).

Corrigé

```

1 class Compte:
2     def __init__(self, titulaire, solde=0):
3         self.titulaire = titulaire
4         self.solde = solde
5
6     def deposer(self, montant):
7         self.solde = self.solde + montant
8
9     def retirer(self, montant):
10        if montant > self.solde:
11            raise ValueError("solde insuffisant")
12        self.solde = self.solde - montant

```

Test : `c = Compte("Alice", 100)` ; après `c.deposer(50)`, `c.solde` vaut 150 ; après `c.retirer(30)`, `c.solde` vaut 120.

Corrigé

Q1.

```

1 arbre = Arbre(1, Arbre(2, Arbre(3, Arbre(4, Arbre(5)))))

```

Q2. L'arbre a 5 noeuds (1,2,3,4,5), donc **taille** = 5. Le plus long chemin racine-feuille passe par tous les noeuds (chaîne de fils gauches), soit 4 arêtes parcourues : **hauteur** = 4.

Q3. `taille(arbre)` renvoie 5, `hauteur(arbre)` renvoie 4 : la prédiction est confirmée.

Corrigé

Q1. En lisant les 1 de la matrice : $0 \rightarrow 1, 0 \rightarrow 3, 1 \rightarrow 2, 3 \rightarrow 2$.

Q2. La ligne 2 de la matrice ne contient que des 0 : le sommet 2 n'a **aucun successeur** (c'est un puits du graphe).

Q3.

```
1 successeurs = {0: [1, 3], 1: [2], 2: [], 3: [2]}
```

Le sommet 2, bien que sans successeur, apparaît dans le dictionnaire avec une liste vide (conformément à la remarque du cours).

Corrigé

Q1. Une pile est adaptée car on doit toujours refermer le symbole ouvert le **plus récemment** en premier (comportement LIFO) : chaque symbole ouvrant est empilé, et chaque symbole fermant doit correspondre au sommet de la pile.

Q2.

```
1 class Pile:
2     def __init__(self):
3         self._d = []
4
5     def empiler(self, v):
6         self._d.append(v)
7
8     def depiler(self):
9         return self._d.pop()
10
11    def est_vide(self):
12        return len(self._d) == 0
13
14
15    def est_equilibre(expr):
16        p = Pile()
17        paires = {'(': ')', '[': ']', '{': '}'
18        for c in expr:
19            if c in '([{':
20                p.empiler(c)
21            elif c in ')]}':
22                if p.est_vide() or p.depiler() != paires[c]:
23                    return False
24        return p.est_vide()
```

Q3. `est_equilibre("({})")` renvoie True ; `est_equilibre("([)])"` renvoie False (le `)` rencontre alors que le sommet de la pile est `[`, qui ne correspond pas).

Corrigé

```
1 class Compte:
2     def __init__(self, titulaire, solde=0):
3         self.titulaire = titulaire
4         self.solde = solde
5
6     def deposer(self, montant):
7         self.solde = self.solde + montant
8
9     def retirer(self, montant):
10        if montant > self.solde:
11            raise ValueError("solde insuffisant")
12        self.solde = self.solde - montant
```

Test : `c = Compte("Alice", 100)` ; après `c.deposer(50)`, `c.solde` vaut 150 ; après `c.retirer(30)`, `c.solde` vaut 120.

Corrigé

Q1.

```
1 arbre = Arbre(1, Arbre(2, Arbre(3, Arbre(4, Arbre(5)))))
```

Q2. L'arbre a 5 noeuds (1,2,3,4,5), donc **taille** = 5. Le plus long chemin racine-feuille passe par tous les noeuds (chaîne de fils gauches), soit 4 arêtes parcourues : **hauteur** = 4.

Q3. `taille(arbre)` renvoie 5, `hauteur(arbre)` renvoie 4 : la prédiction est confirmée.

Corrigé

Q1. En lisant les 1 de la matrice : $0 \rightarrow 1, 0 \rightarrow 3, 1 \rightarrow 2, 3 \rightarrow 2$.

Q2. La ligne 2 de la matrice ne contient que des 0 : le sommet 2 n'a **aucun successeur** (c'est un puits du graphe).

Q3.

```
1 successeurs = {0: [1, 3], 1: [2], 2: [], 3: [2]}
```

Le sommet 2, bien que sans successeur, apparaît dans le dictionnaire avec une liste vide (conformément à la remarque du cours).

Corrigé

Q1. Une pile est adaptée car on doit toujours refermer le symbole ouvert le **plus récemment** en premier (comportement LIFO) : chaque symbole ouvrant est empilé, et chaque symbole fermant doit correspondre au sommet de la pile.

Q2.

```
1 class Pile:
2     def __init__(self):
3         self._d = []
4
5     def empiler(self, v):
6         self._d.append(v)
7
8     def depiler(self):
9         return self._d.pop()
10
11     def est_vide(self):
12         return len(self._d) == 0
13
14
15 def est_equilibre(expr):
16     p = Pile()
17     paires = {'(': ')', '[': ']', '{': '}'
18     for c in expr:
19         if c in '([{':
20             p.empiler(c)
21         elif c in ')]}':
22             if p.est_vide() or p.depiler() != paires[c]:
23                 return False
24     return p.est_vide()
```

Q3. `est_equilibre("([{}])")` renvoie True ; `est_equilibre("([)]")` renvoie False (le `)` rencontre alors que le sommet de la pile est `[`, qui ne correspond pas).

Corrigé

```

1 class Compte:
2     def __init__(self, titulaire, solde=0):
3         self.titulaire = titulaire
4         self.solde = solde
5
6     def deposer(self, montant):
7         self.solde = self.solde + montant
8
9     def retirer(self, montant):
10        if montant > self.solde:
11            raise ValueError("solde insuffisant")
12        self.solde = self.solde - montant

```

Test : `c = Compte("Alice", 100)` ; après `c.deposer(50)`, `c.solde` vaut 150 ; après `c.retirer(30)`, `c.solde` vaut 120.

Corrigé

Q1.

```

1 arbre = Arbre(1, Arbre(2, Arbre(3, Arbre(4, Arbre(5)))))

```

Q2. L'arbre a 5 noeuds (1,2,3,4,5), donc **taille** = 5. Le plus long chemin racine-feuille passe par tous les noeuds (chaîne de fils gauches), soit 4 arêtes parcourues : **hauteur** = 4.

Q3. `taille(arbre)` renvoie 5, `hauteur(arbre)` renvoie 4 : la prediction est confirmée.

Corrigé

Q1. En lisant les 1 de la matrice : $0 \rightarrow 1, 0 \rightarrow 3, 1 \rightarrow 2, 3 \rightarrow 2$.

Q2. La ligne 2 de la matrice ne contient que des 0 : le sommet 2 n'a **aucun successeur** (c'est un puits du graphe).

Q3.

```

1 successeurs = {0: [1, 3], 1: [2], 2: [], 3: [2]}

```

Le sommet 2, bien que sans successeur, apparaît dans le dictionnaire avec une liste vide (conformément à la remarque du cours).

Corrigé

Q1. Une pile est adaptée car on doit toujours refermer le symbole ouvert le **plus récemment** en premier (comportement LIFO) : chaque symbole ouvrant est empilé, et chaque symbole fermant doit correspondre au sommet de la pile.

Q2.

```

1 class Pile:
2     def __init__(self):
3         self._d = []
4
5     def empiler(self, v):
6         self._d.append(v)
7
8     def depiler(self):
9         return self._d.pop()
10
11    def est_vide(self):

```

```

12     return len(self._d) == 0
13
14
15 def est_equilibre(expr):
16     p = Pile()
17     paires = {'(': ')', '[': ']', '{': '}'
18     for c in expr:
19         if c in '([{':
20             p.empiler(c)
21         elif c in ')]}':
22             if p.est_vide() or p.depiler() != paires[c]:
23                 return False
24     return p.est_vide()

```

Q3. `est_equilibre("({})")` renvoie True ; `est_equilibre("([)])"` renvoie False (le `)` rencontre alors que le sommet de la pile est `[`, qui ne correspond pas).

Corrigé

```

1 class Compte:
2     def __init__(self, titulaire, solde=0):
3         self.titulaire = titulaire
4         self.solde = solde
5
6     def deposer(self, montant):
7         self.solde = self.solde + montant
8
9     def retirer(self, montant):
10        if montant > self.solde:
11            raise ValueError("solde insuffisant")
12        self.solde = self.solde - montant

```

Test : `c = Compte("Alice", 100)` ; après `c.deposer(50)`, `c.solde` vaut 150 ; après `c.retirer(30)`, `c.solde` vaut 120.

Corrigé

Q1.

```

1 arbre = Arbre(1, Arbre(2, Arbre(3, Arbre(4, Arbre(5)))))

```

Q2. L'arbre a 5 noeuds (1,2,3,4,5), donc **taille** = 5. Le plus long chemin racine-feuille passe par tous les noeuds (chaîne de fils gauches), soit 4 arêtes parcourues : **hauteur** = 4.

Q3. `taille(arbre)` renvoie 5, `hauteur(arbre)` renvoie 4 : la prediction est confirmée.

Corrigé

Q1. En lisant les 1 de la matrice : $0 \rightarrow 1$, $0 \rightarrow 3$, $1 \rightarrow 2$, $3 \rightarrow 2$.

Q2. La ligne 2 de la matrice ne contient que des 0 : le sommet 2 n'a **aucun successeur** (c'est un puits du graphe).

Q3.

```

1 successeurs = {0: [1, 3], 1: [2], 2: [], 3: [2]}

```

Le sommet 2, bien que sans successeur, apparaît dans le dictionnaire avec une liste vide (conformément à la remarque du cours).

Corrigé

Q1. Une pile est adaptée car on doit toujours refermer le **plus récemment** en premier (comportement LIFO) : chaque symbole ouvrant est empilé, et chaque symbole fermant doit correspondre au sommet de la pile.

Q2.

```

1 class Pile:
2     def __init__(self):
3         self._d = []
4
5     def empiler(self, v):
6         self._d.append(v)
7
8     def depiler(self):
9         return self._d.pop()
10
11    def est_vide(self):
12        return len(self._d) == 0
13
14
15    def est_equilibre(expr):
16        p = Pile()
17        paires = {'(': ')', '[': ']', '{': '}' }
18        for c in expr:
19            if c in '([{':
20                p.empiler(c)
21            elif c in ')]}':
22                if p.est_vide() or p.depiler() != paires[c]:
23                    return False
24        return p.est_vide()

```

Q3. `est_equilibre("({})")` renvoie True ; `est_equilibre("([])")` renvoie False (le `)` rencontre alors que le sommet de la pile est `[`, qui ne correspond pas).

Corrigé

```

1 class Compte:
2     def __init__(self, titulaire, solde=0):
3         self.titulaire = titulaire
4         self.solde = solde
5
6     def deposer(self, montant):
7         self.solde = self.solde + montant
8
9     def retirer(self, montant):
10        if montant > self.solde:
11            raise ValueError("solde insuffisant")
12        self.solde = self.solde - montant

```

Test : `c = Compte("Alice", 100)` ; après `c.deposer(50)`, `c.solde` vaut 150 ; après `c.retirer(30)`, `c.solde` vaut 120.

Corrigé

Q1.

```

1 arbre = Arbre(1, Arbre(2, Arbre(3, Arbre(4, Arbre(5)))))

```

Q2. L'arbre a 5 noeuds (1,2,3,4,5), donc **taille** = 5. Le plus long chemin racine-feuille passe par tous les noeuds (chaîne de fils gauches), soit 4 arêtes parcourues : **hauteur** = 4.

Q3. `taille(arbre)` renvoie 5, `hauteur(arbre)` renvoie 4 : la prediction est confirmée.

Corrigé

Q1. En lisant les 1 de la matrice : $0 \rightarrow 1$, $0 \rightarrow 3$, $1 \rightarrow 2$, $3 \rightarrow 2$.

Q2. La ligne 2 de la matrice ne contient que des 0 : le sommet 2 n'a **aucun successeur** (c'est un puits du graphe).

Q3.

```
1 successeurs = {0: [1, 3], 1: [2], 2: [], 3: [2]}
```

Le sommet 2, bien que sans successeur, apparait dans le dictionnaire avec une liste vide (conformement à la remarque du cours).

Corrigé

Q1. Une pile est adaptée car on doit toujours refermer le symbole ouvert le **plus récemment** en premier (comportement LIFO) : chaque symbole ouvrant est empilé, et chaque symbole fermant doit correspondre au sommet de la pile.

Q2.

```
1 class Pile:
2     def __init__(self):
3         self._d = []
4
5     def empiler(self, v):
6         self._d.append(v)
7
8     def depiler(self):
9         return self._d.pop()
10
11    def est_vide(self):
12        return len(self._d) == 0
13
14
15    def est_equilibre(expr):
16        p = Pile()
17        paires = {'(': ')', '[': ']', '{': '}'
18        for c in expr:
19            if c in '([{':
20                p.empiler(c)
21            elif c in ')]}':
22                if p.est_vide() or p.depiler() != paires[c]:
23                    return False
24        return p.est_vide()
```

Q3. `est_equilibre("([{}])")` renvoie True ; `est_equilibre("([)]")` renvoie False (le `)` rencontre alors que le sommet de la pile est `[`, qui ne correspond pas).

Corrigé

```
1 class Compte:
2     def __init__(self, titulaire, solde=0):
3         self.titulaire = titulaire
4         self.solde = solde
```

```

5
6     def deposer(self, montant):
7         self.solde = self.solde + montant
8
9     def retirer(self, montant):
10        if montant > self.solde:
11            raise ValueError("solde insuffisant")
12        self.solde = self.solde - montant

```

Test : `c = Compte("Alice", 100)` ; après `c.deposer(50)`, `c.solde` vaut 150 ; après `c.retirer(30)`, `c.solde` vaut 120.

Corrigé

Q1.

```

1 arbre = Arbre(1, Arbre(2, Arbre(3, Arbre(4, Arbre(5)))))

```

Q2. L'arbre a 5 noeuds (1,2,3,4,5), donc **taille** = 5. Le plus long chemin racine-feuille passe par tous les noeuds (chaîne de fils gauches), soit 4 arêtes parcourues : **hauteur** = 4.

Q3. `taille(arbre)` renvoie 5, `hauteur(arbre)` renvoie 4 : la prédiction est confirmée.

Corrigé

Q1. En lisant les 1 de la matrice : $0 \rightarrow 1, 0 \rightarrow 3, 1 \rightarrow 2, 3 \rightarrow 2$.

Q2. La ligne 2 de la matrice ne contient que des 0 : le sommet 2 n'a **aucun successeur** (c'est un puits du graphe).

Q3.

```

1 successeurs = {0: [1, 3], 1: [2], 2: [], 3: [2]}

```

Le sommet 2, bien que sans successeur, apparaît dans le dictionnaire avec une liste vide (conformément à la remarque du cours).

Corrigé

Q1. Une pile est adaptée car on doit toujours refermer le symbole ouvert le **plus récemment** en premier (comportement LIFO) : chaque symbole ouvrant est empilé, et chaque symbole fermant doit correspondre au sommet de la pile.

Q2.

```

1 class Pile:
2     def __init__(self):
3         self._d = []
4
5     def empiler(self, v):
6         self._d.append(v)
7
8     def depiler(self):
9         return self._d.pop()
10
11    def est_vide(self):
12        return len(self._d) == 0
13
14
15    def est_equilibre(expr):
16        p = Pile()
17        paires = {'(': ')', '[': ']', '{': '}'

```

```

18 for c in expr:
19     if c in '([{':
20         p.empiler(c)
21     elif c in ')]}':
22         if p.est_vide() or p.depiler() != paires[c]:
23             return False
24 return p.est_vide()

```

Q3. `est_equilibre("({})")` renvoie True ; `est_equilibre("([)])"` renvoie False (le `)` rencontre alors que le sommet de la pile est `[`, qui ne correspond pas).

Corrigé

```

1 class Compte:
2     def __init__(self, titulaire, solde=0):
3         self.titulaire = titulaire
4         self.solde = solde
5
6     def deposer(self, montant):
7         self.solde = self.solde + montant
8
9     def retirer(self, montant):
10        if montant > self.solde:
11            raise ValueError("solde insuffisant")
12        self.solde = self.solde - montant

```

Test : `c = Compte("Alice", 100)` ; après `c.deposer(50)`, `c.solde` vaut 150 ; après `c.retirer(30)`, `c.solde` vaut 120.

Corrigé

Q1.

```

1 arbre = Arbre(1, Arbre(2, Arbre(3, Arbre(4, Arbre(5)))))

```

Q2. L'arbre a 5 noeuds (1,2,3,4,5), donc **taille** = 5. Le plus long chemin racine-feuille passe par tous les noeuds (chaîne de fils gauches), soit 4 arêtes parcourues : **hauteur** = 4.

Q3. `taille(arbre)` renvoie 5, `hauteur(arbre)` renvoie 4 : la prediction est confirmée.

Corrigé

Q1. En lisant les 1 de la matrice : $0 \rightarrow 1, 0 \rightarrow 3, 1 \rightarrow 2, 3 \rightarrow 2$.

Q2. La ligne 2 de la matrice ne contient que des 0 : le sommet 2 n'a **aucun successeur** (c'est un puits du graphe).

Q3.

```

1 successeurs = {0: [1, 3], 1: [2], 2: [], 3: [2]}

```

Le sommet 2, bien que sans successeur, apparait dans le dictionnaire avec une liste vide (conformément à la remarque du cours).

Corrigé

Q1. Une pile est adaptée car on doit toujours refermer le symbole ouvert le **plus récemment** en premier (comportement LIFO) : chaque symbole ouvrant est empilé, et chaque symbole fermant doit correspondre au sommet de la pile.

Q2.

```

1 class Pile:
2     def __init__(self):
3         self._d = []
4
5     def empiler(self, v):
6         self._d.append(v)
7
8     def depiler(self):
9         return self._d.pop()
10
11    def est_vide(self):
12        return len(self._d) == 0
13
14
15    def est_equilibre(expr):
16        p = Pile()
17        paires = {'(': ')', '[': ']', '{': '}'
18        for c in expr:
19            if c in '([{':
20                p.empiler(c)
21            elif c in ')]}':
22                if p.est_vide() or p.depiler() != paires[c]:
23                    return False
24        return p.est_vide()

```

Q3. `est_equilibre("({})")` renvoie True ; `est_equilibre("([)])"` renvoie False (le `)` rencontre alors que le sommet de la pile est `[`, qui ne correspond pas).

Corrigé

```

1 class Compte:
2     def __init__(self, titulaire, solde=0):
3         self.titulaire = titulaire
4         self.solde = solde
5
6     def deposer(self, montant):
7         self.solde = self.solde + montant
8
9     def retirer(self, montant):
10        if montant > self.solde:
11            raise ValueError("solde insuffisant")
12        self.solde = self.solde - montant

```

Test : `c = Compte("Alice", 100)` ; après `c.deposer(50)`, `c.solde` vaut 150 ; après `c.retirer(30)`, `c.solde` vaut 120.

7 Conduire un projet informatique

7.1 Conduire un projet informatique en Terminale

THÉORÈME Contrat horaire officiel

Le programme réserve **au moins 25 %** de l'horaire total de la spécialité à la conception et à l'élaboration de projets conduits par les élèves. Sur une référence de 216 heures (6 heures hebdomadaires pendant 36 semaines), cela représente au moins 54 heures. Si l'horaire effectivement assuré diffère, le plan annuel est ajusté pour préserver la proportion minimale d'un quart.

Programme d'enseignement et Définition de l'épreuve

Cette unité met en œuvre une obligation du **Programme d'enseignement** (MENE1921247A). Elle ne constitue pas une nouvelle partie de l'examen et **ne remplace pas l'épreuve pratique**. La **Définition de l'épreuve** relève d'une autorité distincte : la note de service MENE2516123N définit, à compter de la session 2026, une partie écrite de 3 h 30 et une partie pratique d'une heure, chacune notée sur 20, avec des poids respectifs de 0,75 et 0,25. Le projet prépare des compétences utiles, mais son volume, ses livrables et sa grille ci-dessous relèvent de l'enseignement et non du format de l'épreuve.

Prérequis

Avant le cadrage, chaque équipe vérifie qu'elle sait :

- écrire, exécuter et tester un programme Python ;
- mobiliser au moins une notion du programme de Terminale NSI ;
- conserver un historique des versions et attribuer chaque contribution ;
- partager uniquement des données licites, documentées et dépourvues de données personnelles d'élèves ;
- expliquer une décision technique à l'oral et par écrit.

Question de départ et périmètre

Le projet répond à une question formulée par l'équipe et validée avec le professeur. Le thème peut provenir d'une autre discipline ou mobiliser, par exemple, une base de données, un graphe, une structure complexe, un traitement d'image ou de son, un site Web, un objet connecté ou un module d'intelligence artificielle. Le choix reste compatible avec les ressources disponibles et une réalisation complète en 54 heures de référence.

Le cahier des charges distingue :

- un **produit minimal viable**, indispensable et démontrable ;
- des fonctionnalités souhaitables, réalisées seulement après validation du socle ;
- les limites connues, hypothèses, dépendances, licences et données utilisées ;
- les critères observables qui permettront d'accepter ou de refuser chaque fonction.

ERREUR FRÉQUENTE

Accumuler des fonctionnalités sans produit minimal testable. Le programme demande une ambition raisonnable : une fonctionnalité non indispensable est reportée si elle met en danger l'aboutissement, les tests ou la documentation.

Jalons communs — plan de référence de 54 heures

Chaque jalon se termine par un point d'étape avec le professeur. L'équipe présente une preuve vérifiable, consigne la décision et met à jour les objectifs.

Jalon	Travail attendu	Durée	Preuve de passage
J0	Problème, équipe, responsabilités, contraintes et risques	3 h	Note de cadrage validée
J1	Cahier des charges, cas d'usage et critères d'acceptation	6 h	Spécification versionnée
J2	Architecture, données, interfaces et prototype exploratoire	6 h	Schéma et prototype exécutable
J3	Développement du produit minimal viable	9 h	Démonstration du socle
J4	Deuxième itération et intégration des contributions	9 h	Version intégrée et journal de décisions
J5	Tests normaux, cas limites, correction et robustesse	9 h	Rapport de tests reproductible
J6	Documentation utilisateur et technique, licences, limites	6 h	Documentation relue
J7	Stabilisation, démonstration, oral et bilan individuel	6 h	Version finale et rétrospective
Total		54 h	

À chaque point d'étape, l'équipe répond à quatre questions : qu'est-ce qui fonctionne ? Quelle preuve le montre ? Quel obstacle reste prioritaire ? Quel objectif faut-il conserver, réduire ou redéfinir ? Une modification de périmètre n'efface jamais l'historique de la décision.

Livrables obligatoires

1. une note de cadrage et un cahier des charges datés ;
2. un découpage en tâches, des responsabilités et un calendrier de jalons ;
3. les sources exécutables avec historique des versions et instructions de lancement ;
4. un jeu de tests couvrant les fonctions centrales et des cas limites ;
5. une documentation utilisateur et une documentation technique ;
6. un registre des données, bibliothèques, licences, hypothèses et limites ;
7. une démonstration finale reproductible ;
8. un bilan individuel identifiant contributions, apprentissages et améliorations.

Grille d'évaluation

La grille est communiquée dès J0. L'évaluation croise une production collective et des preuves individuelles ; elle ne se réduit ni au volume de code ni à la seule démonstration finale.

Critère	Poids	Preuve principale
Problème, périmètre et critères d'acceptation	15 %	Cadrage et cahier des charges
Architecture et choix techniques justifiés	15 %	Schémas et journal de décisions
Exactitude et fonctionnement du produit	25 %	Version exécutable
Tests, cas limites et traitement des erreurs	20 %	Rapport de tests
Documentation, reproductibilité et provenance	10 %	Guides et registre des sources
Organisation, jalons et contributions individuelles	10 %	Historique et bilans
Démonstration et argumentation orale	5 %	Présentation finale
Total	100 %	

Un produit non exécutable ne peut pas obtenir les points de fonctionnement sur la base d'une simple intention. Inversement, un projet modeste mais complet, testé, documenté et argumenté satisfait mieux le contrat qu'un prototype ambitieux impossible à reproduire.

Bilan final

Le bilan collectif compare le résultat au cahier des charges. Le bilan individuel explicite une contribution précise, un problème rencontré, la méthode utilisée pour le résoudre et une amélioration possible. Les limites connues sont documentées ; elles ne sont ni masquées ni présentées comme des fonctionnalités achevées.

CONDUIRE UN PROJET INFORMATIQUE — QCM D'AUTO-ÉVALUATION

Pour chaque question, une seule réponse est exacte.

Capacité C1

- [Q1] Le problème auquel répond un projet informatique doit être formulé :
 - de façon explicite, avec des critères permettant de constater qu'il est résolu.
 - de la manière la plus large possible, pour ne rien s'interdire.
 - seulement une fois le programme écrit.
 - par le professeur seul, et jamais par l'équipe.
- [Q2] Dans un projet mené en équipe, répartir le travail suppose avant tout :
 - que chacun écrive le même code de son côté, pour comparer ensuite.
 - de définir qui fait quoi et comment les parties s'assemblent.
 - qu'une seule personne programme et que les autres observent.
 - de tirer les tâches au sort.

Capacité C2

- [Q3] Un point d'étape est utile à condition de porter sur :
 - le nombre de lignes écrites depuis la dernière séance.
 - l'impression générale des membres de l'équipe.
 - des critères observables : ce qui fonctionne, ce qui est testé, ce qui reste à faire.
 - la seule date de rendu finale.
- [Q4] À mi-parcours, l'équipe constate qu'une fonctionnalité prévue ne sera pas terminée. La conduite adaptée est :
 - conserver le périmètre initial et rendre un projet inachevé.
 - abandonner le projet et en commencer un autre.
 - ajouter des membres à l'équipe pour rattraper le retard.
 - réduire le périmètre en accord avec le professeur, et livrer un ensemble cohérent et testé.

Clé de correction — réservée au professeur

Question	Capacité	Réponse exacte
Q1	C1	A
Q2	C1	B
Q3	C2	C
Q4	C2	D

Diagnostic des réponses erronées

► Q1

- **B** — Un énoncé trop large empêche de savoir quand le projet est terminé, et conduit à une ambition impossible à tenir dans le temps imparti. *Renvoi : C1, cahier des charges d'un projet.*
- **C** — L'élève inverse l'ordre : écrire avant de savoir ce que l'on cherche à résoudre expose à produire un programme sans destinataire. *Renvoi : C1, cahier des charges d'un projet.*
- **D** — La conception fait partie du travail évalué. Le professeur valide et recadre, mais l'équipe formule. *Renvoi : C1, conception en équipe.*

► Q2

- **A** — Dupliquer le travail gaspille le temps disponible et laisse entière la difficulté véritable : l'assemblage des parties. *Renvoi : C1, travail en équipe.*
- **C** — L'évaluation porte sur la contribution de chacun ; concentrer le développement prive les autres de tout travail à présenter. *Renvoi : C1, travail en équipe.*
- **D** — La répartition doit tenir compte des compétences et des dépendances entre tâches, ce que le hasard ignore. *Renvoi : C1, travail en équipe.*

► Q3

- **A** — Le volume de code ne mesure pas l'avancement : une fonction courte et testée vaut mieux que cent lignes qui ne s'exécutent pas. *Renvoi : C2, points d'étape et critères.*
- **B** — Une impression ne se vérifie pas et ne permet aucune décision. Un jalon doit reposer sur des faits constatables. *Renvoi : C2, points d'étape et critères.*
- **D** — Ne regarder que l'échéance finale prive de toute occasion de corriger la trajectoire pendant qu'il en est encore temps. *Renvoi : C2, points d'étape et critères.*

► Q4

- **A** — Un produit inachevé ne se démontre pas. Ajuster l'ambition fait explicitement partie de la démarche de projet. *Renvoi : C2, adapter les objectifs.*
- **B** — Repartir de zéro à mi-parcours perd tout le travail accompli et rend l'échéance encore moins atteignable. *Renvoi : C2, adapter les objectifs.*
- **C** — Renforcer une équipe en cours de route coûte du temps d'explication à ceux qui avancent, et ne rattrape pas le retard. *Renvoi : C2, adapter les objectifs.*

Apprendre, s'entraîner, se tester, progresser : la collection Nexus Réussite accompagne chaque élève jusqu'à la maîtrise.

- 📖 Un cours structuré en strates, avec la rubrique « À la machine » : chaque notion se reproduit au clavier.
- » Du code Python vérifié par exécution : aucune sortie de programme imprimée sans avoir tourné.
- 🕒 Des méthodes pas à pas avec exemples résolus commentés et pièges à éviter.
- ♦♦ Trois parcours d'exercices gradués et chronométrés, avec coups de pouce.
- ↪ QCM diagnostiques, auto-évaluation par compétences et sujets aux formats officiels.
- 🩹 Des fiches de remédiation ciblées, reliées aux erreurs réellement diagnostiquées.